TEMCO LEARN

Temco Platform

1.0 · 30 pages · built Nov 10, 2026

1. Temco Platform — architecture
2. Getting started
3. The catalog
4. Naming
5. Entry points
6. TEMCO Platform — Engineering Design (Revision 2)
7. The local assistant and its MCP server
8. Authoring parity with Dr.Explain, publishing parity with MSDN
9. Modules
10. `temco-ai`
11. Temco Docs — documentation
12. Architecture
13. Extension design
14. The registry the module reads
15. temco-editor
16. `temco-projects`
17. `temco-repositories`
18. `temco-store`
19. `temco-tools`
20. `temco-ui`
21. `temco-workbench`
22. The application flow
23. The application window
24. The application: shell, screens and flow
25. The authoring flow
26. The editor
27. The platform
28. Docs center: the redesign
29. Shell and pages, v2 — the full layout, drawn
30. The front page — the Azure portal's palette and scale

Temco Platform — architecture

Temco Platform is the **authoring** application: a desktop app built on a VS Code fork where documentation is written, organised into projects, built into a website — plus a whole-guide PDF and a Windows help file — and published to Temco Learn, the site readers open.

Everything Temco builds is either **the shell** (application behaviour) or **a module** (a feature).

TEXT

```
write  →  build  →  add  →  make the downloads  →  publish  →  GitHub Pages
```

Act	Where it happens	What it produces
write	the project's `docs/` folder, in the editor	Markdown pages and a registry
build	the project folder, by its own scripts or the built-in generator	`dist/site`
add	the site clone, by **Add This Build to the Site**	`public/imported/<id>/<version>/`
make the downloads	the site clone, by **Make the Downloads**	`public/pdf/`, `public/chm/`, the download manifest
publish	the site clone, by **Publish**	one commit on `main`; CI builds and deploys it

Start here	For
getting-started.md	**using** the platform: the flow above, step by step, with a file to look at after each step
this file	**working on** the platform: the shell, the modules, the rules that keep them apart

Repositories

Repository	Is
`temco-platform`	the application: the shell and every module
`temco-learn`	the published documentation site (Temco Learn)

The application and all its modules live in one repository. `temco-learn` is separate because it is a published artifact, not a module.

Layout

TEXT

```
temco-platform/
├── product.json                  product name, icons, default settings, policy
├── resources/                    logos, installer art, application icons
├── src/
│   ├── vs/                       VS Code — never edited
│   └── temco/                    the shell
├── extensions/
│   ├── temco-docs/               modules ([../modules/README.md](./modules/README.md))
│   ├── temco-ai/
│   └── temco-*/                  one folder per module
├── docs/                         this documentation
└── build/  cli/  scripts/  test/  remote/     VS Code — never edited
```

Where code goes

Path	Contents	Edited by
`src/vs/**`, `build/**`, `cli/**`, `scripts/**`, `test/**`, `gulpfile*`	VS Code	nobody
`product.json`, `resources/**`, `package.json`	product identity and policy	us
`src/temco/**`	the shell	us
`extensions/temco-*/**`	the modules	us
`docs/**`	documentation	us

The rule: every change Temco makes is a new file in a new folder. A change to VS Code's own files is a decision, not a habit.

The shell

```TEXT
src/temco/
├── temco.contribution.ts   the single entry point the workbench loads
├── menus/                  top-level menus and the items in them
├── branding/               About, splash, identity behaviour
├── startup/                welcome, first run, default layout
├── services/               services the modules consume
└── platform/               plumbing shared by the folders above
```

The shell owns the application's identity, its startup, its top-level menus, its default layout and the services modules share. It owns no feature. A feature that only works inside the shell belongs to a module, not to the shell.

Entry points

Two additive lines in VS Code's files: entry-points.md.

File	Addition
`src/tsconfig.json`	`src/temco/**` added to `include`
`src/vs/workbench/workbench.desktop.main.ts`	one import of `src/temco/temco.contribution.ts`

Modules

A module is a real extension. Its shape, its contract and its register: ../modules/README.md.

```text
extensions/temco-<module>/
├── README.md              purpose, identifiers, what it owns
├── package.json           id, engine range, commands, views, settings, activation
├── tsconfig.json          extends ../tsconfig.base.json
├── src/
│   └── extension.ts       the only activation entry: register, wire, dispose
├── media/                 icons, webview css and js
├── prompts/               model instructions, where a module needs them
└── scripts/               module-local gates
```

A module's documentation lives in `docs/temco-platform/modules/temco-<module>/` (../modules/README.md), never in a `docs` folder inside the module.

State	Contents	Build effect
reserved	`README.md` only	nothing is compiled and nothing ships
active	manifest, tsconfig, `src/`	compiled and shipped as a built-in extension

Only `temco-docs` is active today.

Nothing needs registering for a module to build or ship: the build picks up every `extensions/*/package.json`, the root install brings in each folder's own dependencies, and web builds skip anything that is not a web extension.

Publishing your documentation

New to the platform? Start here: from an empty folder to a page on Temco Learn, in nine steps, each with a file to look at before you go on — getting-started.md. The same document carries the rest of what a project can do — sources, the editor, validate, release, export, translate — in *Beyond the flow*.

Naming

All identifiers, and the rename map applied at the move: naming.md.

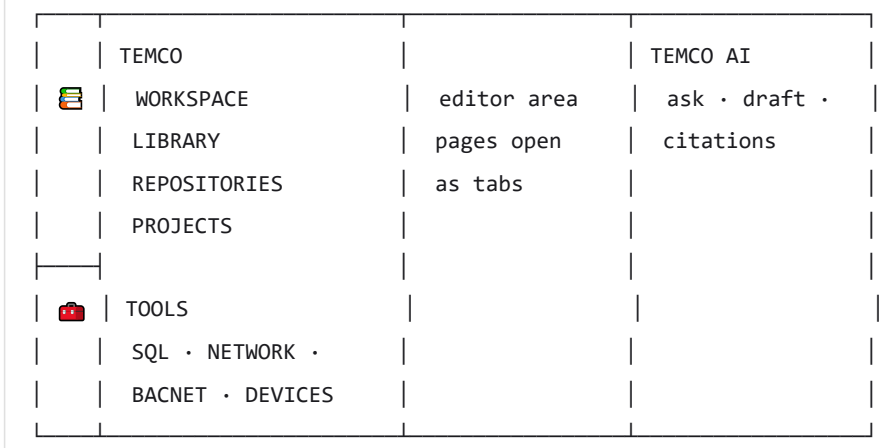
The catalog

The platform's memory — the tables `temco-store` keeps, what each one is for, and every setting a module declares: catalog.md.

UI layout

Two activity-bar icons, the editor in the middle, the AI on the right.

TEXT



Container id	Title	Declared by	Filled by
`temco`	Temco	the docs module now, the shell later	docs, repositories, projects
`temcoTools`	Tools	the first tools module	SQL, network, BACnet, devices
`temcoAi`	Temco AI	the AI module	the AI panel, secondary side bar

A container with no views is not shown, so a container may be declared before its views exist.

Surface	Contributed by	Appears as
view container	module or shell	an icon in the activity bar
view	module	a section in a sidebar
secondary-side-bar view	module	a panel on the right
status bar item	module	an item in the bottom bar
diagnostics	module	rows in Problems
webview editor	module	a tab in the editor area

Rules

1. VS Code's own source is never edited; the two entry points are the complete list.
2. Every change is a new file in a new folder.
3. A module must also work in an ordinary VS Code window — it uses the published `vscode` API only, nothing shell-specific.
4. Modules talk through commands, settings, files and events; never by importing one another's source.
5. `temco-docs` owns the catalog and every page write.
6. Read-only by default. The application writes only in the five documented cases, each behind its own command and a confirmation naming the path or the value.
7. Nothing is ever written to a source repository.

8. The only thing the application pushes is the site: a publish commits `temco-learn` and pushes `main`, and its CI deploys to Pages (getting-started.md). A project's own build is never deployed from here.
9. A build engine sits behind the content-package contract (content-package.md); nothing outside the engine's own folder names it.
10. Runtimes and models are downloaded on request into the user's own folder; the engine runs on loopback only.
11. A guide is a promise, so the docs are checked against the manifests: every `temco*.*` id in backticks must be a declared command, a registered command or a setting, every relative link and `#anchor` must land, `json` examples must parse, screenshot slots must be numbered `00, 01, 02 …`, and `## Step N` must not skip. Rename a command and the guide fails (getting-started.md is the guide; `npm run check-temco-docs` is the check).

Checks

Four commands. `check-temco` runs as part of `npm run compile`, so a stale guide or a broken button stops the build; the others are run when the thing they cover changes.

command	what it holds
`npm run check-temco`	the module boundaries, every button's command id against the manifests, the shared components, and `check-temco-docs`
`npm run check-temco-docs`	what the pages *say*: ids, links, anchors, `json`, screenshot slots, step numbers (docs.mts)
`npm run test-temco`	every suite in test/temco: the pages, the store on a real database, where a project's folder goes, the scripts a build runs, the site publish order, the publish guard, the settings rows, starters and imports, and the environment the tool server is given
`npm run test-temco-e2e`	the same application, running: every module activates, every declared command is registered, the catalog opens in the window's own profile, and a Settings-page setting round-trips through the editor's own configuration service — then **a first run, in order**: nothing configured, a folder chosen, a project made from the starter, that project's own scripts building it with a real `node`, the platform finding that build by itself, the build going onto the site, and the project still there afterwards

`test-temco` compiles the six extensions it loads, because the code it tests imports `vscode` and has to be stubbed the way the running editor would call it — one process per suite, since two suites cannot both be the stub that answers `require('vscode')`. The publish cases need a site clone; with none on the machine they skip and say so, and `TEMCO_SITE` points them at one elsewhere.

`test-temco-e2e` needs a build (`npm run compile`) and opens a real window in a throwaway profile. It is the only check that can catch what the stubs cannot: the breadcrumbs row wrote `workbench.breadcrumbs.enabled`, which this editor does not register, so the checkbox threw in a real window while every stubbed suite stayed green. Its `firstrun.case.js` is the one place the modules meet: it drives the real compiled modules in the order the guide describes, so a change in one module that breaks the next one's assumption fails there and nowhere else. `TEMCO_E2E_KEEP=1` keeps the profile for its logs, and `TEMCO_E2E_TIMEOUT_MS` bounds the run.

A suite is one file in test/temco that ends by handing its cases to `standalone()`; the runner finds the rest. A case is one `test('what must be true', () => { … })` line, and the failure it reports is the assertion's own message, so the case and the reason arrive together.

Branches

This repository is a fork of `microsoft/vscode`, so the two jobs are kept on two branches.

Branch	Is	Receives
`main`	a **mirror** of `upstream/main` — no Temco file, no Temco commit	upstream only, fast-forwarded
`t3000-platform`	**the product**: the shell, every module, the product identity	every Temco change, and each upstream sync

`main` is never edited and never carries a release: a tag on it would build an app with none of this product in it. Everything Temco is committed on `t3000-platform`, with `main` as its merge base.

Set `t3000-platform` as the repository's default branch (*Settings → General → Default branch*) — it is `main` today. GitHub only reads `workflow_dispatch` from the default branch, so while `main` is default the release workflow has no *Run workflow* button at all, and a clone lands on upstream's source rather than on the product. A ruleset blocking direct pushes to `main` is worth adding at the same time: the release guards stop a *release* from `main`, not a *commit* to it.

Syncing upstream into `main`:

```shell
git fetch origin --prune && git fetch upstream --prune
git switch main
git reset --hard upstream/main
git push --force-with-lease origin main
```

Bringing that merge into the product — a **merge**, never a rebase: the shared commits rewritten on every upstream release would make `t3000-platform` unusable for anyone who has cloned it.

```shell
git switch t3000-platform
git merge main            # conflicts land on package.json (version), product.json, resources/**
```

Releasing

A release is a **tag on `t3000-platform`**. GitHub builds the installers from it and attaches them to a draft Release named after the tag. The workflow is `.github/workflows/release.yml`.

The first release, in order

1. **Commit what is pending on `t3000-platform`** — the workflow, the root `README.md` and this page — and push it.
2. **Make `t3000-platform` the default branch** — *Settings → General → Default branch*. Until this is done the workflow has no *Run workflow* button. Add a ruleset blocking direct pushes to `main` while you are there.
3. **Leave `package.json`'s `version` alone.** It carries the VS Code base (`1.144.0` today) and is what the application reports to extensions that are not built in, where `engines.vscode` is enforced: a lower number, or a prerelease

suffix, is refused. A Temco build number is the patch digit (`1.144.1`). What an upstream sync does to the number is in the root README.

4. **Tag that commit and push the tag.**

```shell
git tag -a v<version> -m "Temco Platform <version>"
git push origin v<version>
```

Actions → Release then builds on three runners and leaves a draft Release. Check the assets, publish it.

Cutting a release

```shell
# 1. the version the installer will report, and the tag, are the same number
#    edit "version" in package.json
git commit -am "Temco Platform <version>"
git push origin t3000-platform

# 2. the tag is the release
git tag -a v<version> -m "Temco Platform <version>"
git push origin v<version>
```

Then open *Actions → Release*. Three runners build in parallel — Windows x64, Linux x64, macOS arm64 — and a **draft** Release named `Temco Platform <version>` is created with:

Asset	Is
`TemcoPlatform-win32-x64-<version>.zip`	the unpacked app: run `Temco Platform.exe`
`*Setup*.exe`	the per-user Windows installer, no admin rights needed
`temco-platform_<version>_amd64.deb`	the Debian/Ubuntu package
`temco-platform-linux-x64-<version>.tar.gz`	the unpacked Linux app
`TemcoPlatform-darwin-arm64-<version>.zip`	Apple silicon only

Check the assets, then publish the draft. Nothing is released while it is a draft.

Two guards, and why they exist

Both fail in seconds, before the 40-minute build:

1. **The tag must equal `v<version>`** in `package.json`. The installer takes its version from `package.json`, not from the tag, so a release called `v1.2.3` that installs itself as `1.144.0` is one nobody can support.
2. **The tag must be on `t3000-platform`.** `main` mirrors upstream, so a tag there would build an application with none of this product in it.

Rebuilding a tag

Run the workflow by hand (*Actions → Release → Run workflow*) with the tag's name. Nothing is re-tagged and the draft of a failed run is replaced. A failed run leaves its tag behind — delete it (`git push --delete origin v<version>`) before tagging again, or the next push is rejected as unchanged.

Two things the workflow does that packaging does not

`compile-extensions` and `build-webviews` , in that order, before the gulp packaging task. A module ships as `out/**` (the manifest's `main` is `./out/extension.js` , and `.vscodeignore` excludes `src/**` but not `out/**`), and its webview bundles are esbuild output under that same `out/` — outside VS Code's own media pipeline. A package built without them installs and runs, with an empty surface where each module's page should be.

What a release is not

Nothing is signed: there is no code-signing certificate, so Windows shows "unknown publisher" on first run and macOS needs right-click → Open (or `xattr -d com.apple.quarantine`). Nothing updates itself either — a new version is a new download, not an update. Both are later work, and both are secrets-only changes to the same workflow: a Windows certificate and an Apple Developer ID.

Phases

Phase	Work
1	Rename the repository, add the `upstream` remote, apply the naming
2	`extensions/temco-docs` — the existing extension, ids renamed, gates wired
3	The shell — `src/temco` , the two entry points, `product.json` , menus, welcome
4	Split `temco-ai` , `temco-store` , `temco-ui` , `temco-repositories` , `temco-projects` , `temco-workbench` (**all done** — every module is now one domain, and the workbench is the application)
5	`temco-learn` serves T3000 end to end, then T3001 and T3002

Getting started

Temco Platform is where product documentation is written and published. It is a desktop application, built on VS Code. In it you write pages and group them into a **project**. You then build that project into a website, with a printable PDF and a Windows help file, and publish the result to **Temco Learn**: the documentation site readers actually open, at `https://temcocontrols.github.io/temco-learn/`.

This guide walks that whole journey once, from an empty folder to a live page. It assumes you have never opened the application before, and it assumes no background in VS Code, in Git, or in the site's own code. Where one of those cannot be avoided — a commit, a catalog — it is explained at the step where you meet it.

The 30-second version

- **What it is:** you write pages in a project, build them into a site, and publish that site to Temco Learn.
- **What it needs:** the application, Node 22, a clone of the site with its dependencies installed, and permission to push to `temcocontrols/temco-learn`.
- **What you run:** `temcoDocs.buildSite`, then `temcoLearn.addBuild`, then `temcoLearn.site.downloads`, then `temcoLearn.publish`.
- **What you get:** your project on `temcocontrols.github.io/temco-learn/`, under Documentation, with one version for each build you add.

Read steps 1–9 the first time through. After that, everything you need day to day is in Running it again.

> **Screenshot — the finished result: a project card under Documentation, opened to a page, with the version selector and the download menu visible.** `media/publishing/00-published-page.png`

The two halves

Temco Platform and Temco Learn are two halves of one idea, and they meet in a single place: a copy of a project's build.

```
TEXT
you, writing a page  →  Temco Platform builds and publishes  →  the site clone one commit on main  →  Temco Learn what a reader opens
```

	Temco Platform	Temco Learn
is	the **authoring** application, a desktop app built on VS Code	the **published** site, a static React app on GitHub Pages
is for	whoever writes, reviews and owns product documentation	whoever reads it: customers, field engineers, support
what you do there	create a project, write pages, build, make the downloads, publish	read, search, switch versions, download the PDF or the help file
what it holds	your project folder, and the site clone it publishes into	one imported build per version, the catalog, the downloads
where it lives	the application on your machine	its own repository, `~/temco/temco-learn`, deployed by CI

You only need the left-hand column to finish this guide. The right-hand one is what gives the work its point, and it is also the reason there is a step that produces a PDF by hand: the site cannot make one for you.

What you see when you open Temco Platform

The activity bar on the left holds eight areas. This guide uses four of them, and it is worth knowing what the other four are before they surprise you.

Area	Holds	In this guide
Home	Overview, Projects, What needs doing	step 2: where a project is created
Documentation	Project, Pages, Publish	steps 3 and 4: the pages, and the build
Repositories	Sources: organisations, clones, the folder settings	step 1: the folders everything lands in
Temco Learn	Overview, Site, Artifacts	steps 6 to 9: add, downloads, preview, publish
AI	Draft, What it does, Server, Tools	Beyond the flow: drafts beside your text
Forum	Browse	not covered here
Tools	Studios: SQL, network, BACnet, devices	not covered here
Settings	General, Modules, AI Runtime, Branding, Publishing	step 1, and the Repositories page above

If you cannot find something in the activity bar, you can still reach it by name: the appendix lists every control together with the command id behind it.

What a reader sees on Temco Learn

It is worth seeing this once, so that you know what you are aiming at.

The reader opens	And gets
the home page	the site's sections as tiles; Documentation is where your project will appear
a section	the projects filed under it, each with its title and summary
a project	its versions — every version you add is a separate entry, never an overwrite
a version	that version's pages, in the reading order the registry gives them
a page	the page itself, a download menu for the PDF, the help file and the Markdown, and a way to say it was wrong

None of this is a separate publishing channel. The site reads one catalog, and that catalog is what step 6 writes.

Who this is for

If you are…	this guide gets you to
writing documentation for a Temco product (T3000, a controller, a protocol)	a project of your own, built and live under Documentation on the site
taking over documentation that already exists — a folder of Markdown, a Word manual, a compiled `.chm`	the same result, starting from *Import a document set* in step 2, with your existing pages converted once
working inside a product repository that already has `docs/` and its own build scripts	the same result, with your repository's own `docs:site` or `build:help:dry` scripts doing the building instead of the built-in generator
reviewing someone else's documentation	enough of the flow to read what they changed, and to publish it

You do not have to be the person who set the machine up. If somebody else did, Before the first time shows you how to check that the three settings and the site clone are right before you begin.

How to use this guide

- **Read steps 1–9 in order, once.** After that, go straight to Running it again; that is what everyday work looks like.
- **Every step has the same three parts:** what you do, **You should see** (a file or a screen to check), and **If not** (the usual cause). If the file a step promises is not there, that is the step that failed, and nothing after it is worth testing yet.
- **Buttons are named as they appear**, with the command id in backticks beside them, like `temcoDocs.buildSite`. If a button is hidden or unavailable, type that id into the Command Palette (`Ctrl+Shift+P`) and it will run the same action.
- **A `~` in a path means your home folder.** Windows paths are written with backslashes; on macOS and Linux the same path uses `/`. The only genuinely Windows-only piece is the `.chm`, which needs the help compiler.
- **Keep this guide open in a second window** while you work through it. It is meant to be read next to the application, not memorised.

What you need before you start

Need	Why
the application installed and running	this guide is about using it, not installing it
write access to the `temcocontrols/temco-learn` repository, or someone who has it	the last step pushes a commit there. Without access, everything before it still works and simply stops at *publish*
git configured with your GitHub account	a publish is a `git add`, a `commit` and a `push`, all under your name
the site clone on this machine	`~/temco/temco-learn` unless you choose otherwise. The Site page clones it for you
an hour for the first run	steps 2 and 7 account for most of it
rather less every time after	a repeat publish is four commands and about two minutes

In this guide

Section	Answers
The 30-second version	what this is, what you need, and the four commands that do it
The two halves	Temco Platform and Temco Learn, and where they meet
What you see when you open Temco Platform	the eight areas of the activity bar, and which four this guide uses
What a reader sees on Temco Learn	the site from the reader's side, so you know what you are aiming at
Who this is for	whether this guide is the right one for you
What you are making	what a project, a build and the site are, and where each one lives
The general flow	the five acts, what each writes, and what a reader notices
Words this guide uses	page id, area, chapter, version, section, catalog, guard, downloads
What you will have when you are done	the result, in five bullets
Before the first time	the setup that happens once per machine
Where things go	the three settings that decide every path
Steps 1–9	the flow, in order, each with what you should see
The editor	the ribbon, pictures, the outline and draft panel, preview, diff, and every shortcut
Running it again	the four commands for the second, third and hundredth publish
The same flow, as files	every file, and the step that wrote it
What the files look like	real snippets of the JSON you will meet
Reference: the outputs	the ticks, and which script each one runs
Two things worth doing on purpose	the tests that prove the guards really refuse
When a step goes wrong	symptom → cause → where to look
When something else goes wrong	the failures that are not about publishing
Beyond the flow	sources, the editor, validate, release, export, translate
Appendix	every button, and the command id behind it

What you are making

Five things take part. Knowing which is which is most of understanding the flow.

Thing	Is	Lives in
A project	one body of documentation: its name, its pages, and the outputs it should produce	`<projectsFolder>/<slug>/`, for example `~/temco/projects/t3000`
A page	one Markdown file, listed in the project's registry	`<project>/docs/<area>/*.md`, with the list in `docs/help/registry.json`
A picture	a screenshot or diagram a page points at — `![the trend log](media/x.png)`	`<project>/docs/media/`. The build copies everything under `docs/` that is not a page, so a picture keeps the address the page gave it
A build	the rendered site, made from those pages	`<project>/dist/site`, or `dist/docs` when your own generator wrote it
The site	the Temco Learn repository: a React app that reads a catalog of projects	`temcoLearn.siteRepo`, by default `~/temco/temco-learn`
The published page	what a reader opens on the internet	GitHub Pages, served from the site's `main` branch

The first three are yours and stay yours. The site is its own repository, and the last one belongs to GitHub. That split is why the flow has separate steps instead of one button, and why each step can fail in its own way.

The general flow

There are four acts and three places. You write and build inside **your project**. The site clone takes one version, plus its downloads. A publish puts that on **GitHub**, and the site's own CI serves it to readers.

TEXT

```
YOUR PROJECT              ──▶ THE SITE CLONE            ──▶ GITHUB, THEN READERS
write, then build             add, then the downloads       publish, then CI deploys
~/temco/projects/t3000        ~/temco/temco-learn           temcocontrols/temco-learn
```

Act	Command	Writes	What a reader would notice
write	the editor, `temcoDocs.newPage`	`docs/**/*.md`, `docs/help/registry.json`	nothing
build	`temcoDocs.buildSite`	`<project>/dist/site`	nothing yet: nobody can see it
add	`temcoLearn.addBuild`	`public/imported/<id>/<version>/`, `content/<id>/site.json`	nothing yet, though it previews at `localhost`
make the downloads	`temcoLearn.site.downloads`	`public/pdf/…`, `public/chm/…`, the download manifest	nothing yet, but the deploy would refuse without them
publish	`temcoLearn.publish`, then *Publish*	one commit, one push	a minute later, the page is live

The nine steps below are this table with the details filled in.

So the flow can stop in four places. Each is named at the step it belongs to: a build with no `index.html` to add, a site clone that is not where the setting says, a catalog that could not run, and a version whose PDF is missing. The last is the one people run into most often, which is why it has a step and a test of its own.

Words this guide uses

The platform uses a few words that mean something more specific than they do elsewhere. One pass over this table saves a lot of guessing later.

Word	Means
registry	`docs/help/registry.json`, the list of every page: its title, where it sits, whether it ships. The application reads it, and a build renders it
area	the top level of the documentation, one per product or section, declared in the registry
chapter	the folder a page sits in, inside an area
page id	the page's path under `docs/` without the `.md`: the id `t3000/commissioning/ports` is the file `docs/t3000/commissioning/ports.md`
build	the rendered site produced from the pages, in `<project>/dist/site`
version	one fixed entry of a project on the site, such as `1.0` or `1.1`. A rebuild becomes a new version rather than overwriting an old one. It becomes a folder and an address (`/t3000/1.0/`), so it is typed in digits — `1.0`, `17.4`, or a dated edition `2026.01` — never `v1.0`
section	the category a project appears under on the site: `documentation`, `training`, `tests`, `products`, `qa` or `topics`
catalog	`public/catalog.json` in the site, the one file the site's app reads to know what exists
downloads	the PDFs and help files the site serves. The deploy cannot build them, so they are made on your machine and committed
guard	the site's own check that every catalogued version has its downloads. CI runs it, and so does a publish before it commits

What you will have when you are done

- T3000 listed under **Documentation** on the site, with its own card, icon and description.
- Every version you added, each one selectable (`1.0`, `1.1`, `2.0`) with its own pages.
- A **Downloads** menu for each version: the whole-guide PDF, and a Windows help file when one could be compiled.
- Your pages in the site's search index.
- One commit on `main` in the site repository, and a green GitHub Actions run that deployed it.

Before the first time

The table above is about *you*: the application, access, an hour. This one is about the **machine**, and it happens once. If somebody else set the machine up, work down the list and confirm each line rather than assuming.

Do this once	Why
Node 22 or newer on your path (`node --version`)	the site's own scripts run with `--experimental-strip-types`
Have the site clone. The Site page (**Open the Site Page**) clones `temcocontrols/temco-learn` for you	the site is its own repository, and this application is a client of it
`cd ~/temco/temco-learn && npm ci`	the site's dependencies, once per clone
`npx playwright install chromium`	the PDFs are printed by a browser. Without it, *Make the Downloads* tells you so
Optionally, HTML Help Workshop	a `.chm` needs `hhc.exe`. Without it, the site simply has no help file

Three of those five are optional in practice: a site without PDFs still builds and previews, and a site without a help file is still a site. Node and `npm ci` are not optional. Without them the site's `catalog` cannot run, and step 6 will stop there.

Where things go

Three settings decide every path in this guide. Set them once. Every page reads them back, so you can check the answer rather than the field.

Setting	Where	Meaning
`temcoRepositories.workspaceFolder`	Settings ▸ Repositories ▸ Working folder	the working area. Clones go to `<it>/source/<repository>`
`temcoRepositories.projectsFolder`	Settings ▸ Repositories ▸ Projects folder	a project's own folder is made at `<it>/<slug>`
`temcoLearn.siteRepo`	the Site page ▸ **Choose the Site Folder**, or the setting	the site clone: `~/temco/temco-learn` unless you say otherwise

`temcoWorkbench.publishing.site` is not one of these. It is an address shown on the Publishing page, not a folder.

Step 1 — check the folders

Open **Settings** ▸ **Repositories** and read the two sentences under the fields, not the fields themselves. The page reads your saved settings, so those sentences are the honest answer to "where will this go?".

> **Screenshot — the Repositories settings page, with “Clones land in …” and “Projects are made in …” visible.**
> `media/publishing/01-folders.png`

You should see	`Clones land in …\temco\source` and `Projects are made in …\temco\projects`
If not	a value you typed but did not save still shows as the old one. Press *Choose* or leave the field, then look again. If it says `Projects are made in …\temco`, the projects folder is still empty, and new projects will land beside the clones

The third setting this guide depends on is **not** on that page. `temcoLearn.siteRepo` is the site clone; set it from the Site page with **Choose the Site Folder**. It defaults to `~/temco/temco-learn`. Check it now, because the whole right-hand half of the flow writes there.

Step 2 — create a project

Create a Project (`temcoProjects.newProject`). Give it the name `T3000` and the version `1.0`, and leave the outputs ticked: *Website*, *PDF*, *Offline help* and *Search index*.

> **Screenshot — the create-project form, with the output ticks visible and a name typed in.**
> `media/publishing/02-create-project.png`

Where the pages come from is the choice that decides what exists once the form closes:

Starting point	What it does
Start empty	records the project and nothing else. Its folder, `docs/help/registry.json` and one area come from **Create the Project's Documentation Folder** (`temcoDocs.createProjectFolder`)
From a template	writes a starter tree under the folder field — `docs/help/registry.json`, one area, two topics — named after the project, so `T3000` becomes `<projectsFolder>/t3000`
From a folder	cites a folder that already exists on this machine as the pages it reads; nothing is copied
From a repository	cites what the platform has already read — repositories, files and folders
Import a document set	converts Word, RTF, HTML, an HTML folder or a CHM **once** into `docs/` and a registry

Either way, the project is recorded under the slug of its name (`T3000` becomes `t3000`), the sources it cites are recorded with it, and the outputs become its recipe. No source is ever written to. A project is a set of references, and its own folder holds only what the project itself writes.

You should see	`~/temco/projects/t3000/docs/help/registry.json`, and T3000 in the Library
If not	the folder is the slug of the name, so look in `~/temco/projects/` for what was actually made. A project created before you set the projects folder keeps the folder it already has; only new ones move

The folder is added to the window so that its `docs/` shows in Explorer. That is expected, and nothing needs doing about it.

Step 3 — write the pages

New Page (`temcoDocs.newPage`) with T3000 selected. It opens a **form** rather than a prompt: the place is already chosen from the row you were on, and the form shows the kind of page, the file it will write, and the body it starts from. The palette offers the kinds alongside it (*New Article*, *New Task*, *New Reference*, *New Troubleshooting*, *New Release Note*), and they differ only in their starter headings.

> **Screenshot — the Pages view and the New Page form side by side, with the kind, the target file and the starter body visible.** `media/publishing/03-pages.png`

In the form	Decides
the kind	the starter headings: *Task* gives steps, *Reference* gives a table, *Troubleshooting* gives symptom, cause and fix
area and chapter	where the file is written, as `docs/<area>/<chapter>/<title>.md`
the title	the page's name in the navigation, in search and in the tree. It is **not** the file name until you accept
which source	asked only when the project cites more than one folder

You are then in the editor. Each page is a Markdown file **and** a row in `docs/help/registry.json`. Three rules explain almost every surprise you will meet later:

Rule	What it means
the **page id is its path** under `docs/` , without `.md`	`docs/t3000/commissioning/ports.md` has the id `t3000/commissioning/ports` , and the registry must name it exactly that
`ship: false` keeps a page out of the built site	a page can exist and be edited, and still never appear
`order` decides the position inside a chapter	two pages sharing a title or an `order` are a warning, not an error

A page that is registered but whose file does not exist is an **error**, and the module reports it on refresh. That is what makes the `missing` rows in `Pages` mean something.

Everything else you will want on a page lives on its row. *Rename the Page Title*, *Move Up* and *Move Down* change where it sits; *Retire* keeps the file and drops it from the site; *Create the Missing Page File* fills a `missing` row; *New Folder…* (`temcoDocs.newChapter`) adds a chapter. Right-clicking a captured screenshot in `Needs You` offers *Attach the Screenshot to a Page…*.

You should see	a Markdown file under `~/temco/projects/t3000/docs/<area>/`, and `registry.json` listing it
If not	the project has to be the **selected** project. The docs module reads the folders that project cites, not whatever folder happens to be open. A message about there being "no area to put a page in" means the project has no area yet: create a chapter, or start the project from a template

Come back to this step whenever you have more to write. Nothing downstream notices until you build. Writing itself is The editor, later in this guide.

Step 4 — build the site

Build the Site (`temcoDocs.buildSite`), or *Build the Site* on the project page. It builds the project that is **selected**, reading the folders that project cites, so the folder does not have to be open in the window.

> **Screenshot — the Build Center after a build: the four counts, one card per output this project builds, and the newest run at the head of the log.** `media/publishing/04-build.png`

Two things can produce the site, and what is in the folder decides which one runs:

In the project folder	What runs	Where the site lands
a `package.json` with `docs:site` or `docs:build`	that script, after `docs:registry` when the repository has one	wherever the script writes, commonly `dist/site` or docfx's `dist/docs`
neither	the **built-in generator**	`<project>/dist/site`

Either way, the attempt is recorded in the Build Center (`temcoDocs.builds`), and every file the build wrote is recorded against it, so Publish & Releases can list real files rather than hopes.

You should see	`~/temco/projects/t3000/dist/site/index.html`, and a row in the Build Center (`temcoDocs.builds`)
If not	a project made here has no `package.json`, so the built-in generator writes the site, and the log says what it read. If your repository does have `docs:site` or `docs:build`, that script runs instead, and it may write `dist/docs`

The exit code is never the verdict. What counts is a file that did not exist before the run and does now. A generator that exits 0 and writes nothing has failed; one that exits 1 after writing the site has not.

Step 5 — build the project's own outputs (only for a repository)

Skip this step unless T3000 cites a real repository clone under `~/temco/source/`. **Temco Projects: Build** (`temcoProjects.buildProject`) takes the outputs you ticked and runs the repository's own script for each one, **once each**:

> **Screenshot — the terminal during a project build: `docs:registry`, then one script per ticked output, then the list of files it left behind.** `media/publishing/05-project-outputs.png`

Ticked output	Script it looks for, in the repository
Website	`docs:site`, else `docs:build`
Offline help	`build:help:dry`
PDF	`build:help:dry`, else `docs:pdf`
Search index, AI index	nothing: no script here writes them

`docs:registry` runs first whenever the repository has it, because it is what the other scripts read their structure from. The build then records every file that appeared during the run, whatever the exit codes said.

You should see	the terminal run `docs:registry`, then the scripts above, and the log list the files they left behind
If not	the warning names the scripts it looked for. A repository with none of them runs nothing, which is the honest answer. A scaffolded project has no `package.json` at all, so step 4 is its build, and this step is not for it

This is the step that produces a PDF or a **`.chm` for the docs module**: the copies it records and Publish & Releases lists, under their own output. The PDF that the website serves is a different file, made in step 7 from the site's own content.

Step 6 — add the build to the site

Add This Build to the Site (`temcoLearn.addBuild`), or *Choose a build* on the Publishing page. The page that lists every build on disk is `temcoLearn.addBuilds`.

> **Screenshot — the version box, showing "project t3000" and the version field.** `media/publishing/06-add-build.png`

It finds every `dist/site` and `dist/docs` two folders below the window's folders, the projects folder and the working folder, and offers the ones that contain an `index.html` or, for a documentation project, an `index.md`. A Markdown build is added as content, so its pages become editable pages of the site; an HTML build is copied beside the catalog as it came. The box then names the project it will land in (`project t3000`) and asks one question: which version. Type `1.0`.

What happens with your answer:

copies	the whole build folder into `<siteRepo>/public/imported/t3000/1.0/`. Adding the same version again **replaces** it
records	`<siteRepo>/content/t3000/site.json`, holding the title and the section. A new project defaults to `documentation`
catalogs	runs the site's own `npm run catalog`, which rewrites `public/catalog.json`, `public/search.json`, `public/content/` and the download bundles
opens	the site page for that project and version, so the result is one click away

You should see	`~/temco/temco-learn/public/imported/t3000/1.0/index.html`, `content/t3000/site.json`, and `t3000` with version `1.0` in `public/catalog.json`
If not	the build must contain an `index.html`. The catalog is rebuilt by this step, so a failure here means the clone has no `npm ci` yet, or has no `catalog` script

The project's id is **its folder's name**, so a build in `<projectsFolder>/t3000/…` is always `t3000`, whatever its pages call themselves. Its section is `documentation` unless `site.json` says otherwise.

Repeat this step every time you rebuild. A build is copied as one version, so a changed site is a *new* version (`1.1`, `2.0`). Re-adding `1.0` replaces what is already there, which is what you want only when you are fixing that same version.

Adding a build is a local act. Nothing is published by this step.

Step 7 — make the downloads

Make the Downloads (`temcoLearn.site.downloads`) is the step between *Add* and *Publish* on the Publishing page.

> **Screenshot — the Publishing page, with the five steps visible and the downloads step running.**
> `media/publishing/07-downloads.png`

It runs the site's own scripts in this order, and stops at the first real failure:

Script	Makes	If it fails
`build:fast`	the catalog, the search index and the bundle the PDFs are printed from	fatal: nothing else can run
`pdf -- strict`	one PDF per version, and one per Markdown page, mirrored into `public/pdf/`	fatal, usually because there is no browser: run `npx playwright install chromium`
`chm`	the help files and `public/download/manifest.json`	only a warning: a machine without `hhc.exe` still has a site

You should see	`public/pdf/t3000/1.0.pdf`, `public/download/manifest.json`, and `public/chm/…` only when the help compiler is on the machine
If not	the message names the step that failed: the site build, the PDFs (browser missing), or the help files (a warning, never fatal)

These files are the one thing the deploy cannot make for you. The site's CI has no browser and no `hhc.exe`, so the PDF is printed here and committed, and the deploy **refuses to ship a version whose PDF is missing**. That refusal is the guard you will meet in step 9 if you skip this step.

Come back to this step every time you finish step 6. Each new version needs its own PDF, and the guard counts versions rather than projects.

Step 8 — look at it before publishing

Start the Site (`temcoLearn.site.start`).

> **Screenshot — the site in a browser at localhost:1025: T3000's card under Documentation, and its page open.**
> `media/publishing/08-preview.png`

It builds the site first if `dist/index.html` is not there, then serves the built pages on one port: `temcoLearn.sitePort`, 1025 by default. What you are looking at is the **clone**, with the catalog that step 6 wrote. It is what a reader will see, minus the address.

Look at	Because
the project's card under its section	the section came from `content/t3000/site.json`, so a wrong section is a one-line fix there
the version selector	every version you added is listed, and `1.0` should be there after step 6
a page's navigation and content	an empty page means the Markdown is empty, not that the build failed
the downloads menu	it lists what step 7 committed, so an empty menu means the manifest is missing
search	this is the site's own index, and a page missing from it has `ship: false`

You should see	`http://localhost:1025/` with T3000 under **Documentation**, and its page opening from the card
If not	the port is `temcoLearn.sitePort`; the site's log in the panel says whether the build or the server is the problem

If a page is missing, it is either unregistered or marked `ship: false`, so go back to step 3 rather than rebuilding. **Stop the Site** (`temcoLearn.site.stop`) when you are finished. The open port, not the process, is the proof that it is running.

Step 9 — publish

Publish (`temcoLearn.publish`) opens the publishing page: what changed, the steps of a publish in order, and the button. Press *Publish* there and confirm the message. **Publish Now** (`temcoLearn.publishNow`) does the same thing from the palette.

> **Screenshot — the Publishing page before the push: the changed paths, the five steps, and the Publish button.** `media/publishing/09-publish.png`

It does four things in order, and it stops at the first one that says no:

1	runs the site's `catalog` , so that what is committed matches what the pages say
2	runs the site's **`guard`** , the same gate its CI runs, and refuses to commit when a version is missing a download
3	runs `git add --all` and makes one commit. The message defaults to `docs: publish the site (<date>)` , and you can replace it
4	runs `git push origin HEAD:main`

Nothing is ever force-pushed. If `main` carries work this clone does not know about, the push is refused, the commit stays local, and a pull followed by another publish fast-forwards it.

Then the site's CI takes over. `pages.yml` runs its checks, builds the site, runs the same guard, and deploys `dist` to GitHub Pages at the address the log printed. A `v*` tag matching `package.json` instead packs the whole site as a zip for handing to somebody.

You should see	in the clone, `git log -1 --stat` shows the commit and `git status` is clean. The log says `pushed <commit>` , the GitHub Actions run on `main` is green, and the address the log printed loads the page a minute later
If not	*the guard refused it* means step 7 was skipped for some version. The log names the missing file, and nothing was committed. *A refused push* means the clone is behind: pull, then publish again

A publish carries everything waiting in the clone, not only the project you were working on. If that is not what you want, open the publishing page first: it lists the paths the commit would take before you press the button.

The editor

You will spend most of your time here, so five minutes learning the surface saves months of hunting for a button.

A page opens in **Temco Editor**, the editor that owns `*.md` . It has one row of controls above the text, the text in the middle, and a **Page** container on the far right. The headings you see in the text are the page's prose; its title in the navigation comes from the registry, so renaming one does not rename the other.

> **Screenshot — a page open in Temco Editor, with the ribbon groups numbered: undo/redo, Style, Lists, marks, blocks, insert, `···` , and the right group.** `media/publishing/10-editor-ribbon.png`

The ribbon, left to right

Group	Holds
undo / redo	`Ctrl+Z`, `Ctrl+Y`. This is the page's own history, not git's
Style	paragraph or heading level, and the alignment of the block
Lists	bullet, numbered and task lists
Marks	bold `Ctrl+B` · italic `Ctrl+I` · underline `Ctrl+U` · strikethrough `Ctrl+Shift+X` · inline code `Ctrl+E` · highlight `Ctrl+Shift+H`
Blocks	quote `Ctrl+Shift+B` · code block `Ctrl+Alt+C` · horizontal rule · table
Insert	**Picture**, **Screenshot**, **Annotate** (only when a picture is selected), **Link to page**
`...`	everything the row has no space for, including *Open the source* (the plain Markdown, for a paste that must not be reformatted) and *Page properties*
right group	Find `Ctrl+F` · Outline `Ctrl+Alt+O` · **Read the page**, which toggles to *Edit the page* · Preview `Ctrl+Alt+P` · Diff `Ctrl+Alt+D` · Source · Page properties

The same actions are in the editor title bar and in the right-click menu under **Temco Markdown**, so nothing is reachable only from the ribbon.

Read the page is the one people miss. It turns the editor into the rendered page, links and all, without opening Preview. Toggle it back when you want to edit again.

Pictures

Click a picture once to select it. A ring appears, with drag handles at its edges. Drag a handle to resize, and the width is written into the Markdown title as `"w=480"`, so the built site and the offline help honour it too.

> **Screenshot — a picture selected in the editor, with the drag handles and the 25% · 50% · 75% · 100% · Delete menu visible.** `media/publishing/11-editor-picture.png`

With a picture selected, the inline menu offers the widths **25%** · **50%** · **75%** · **100%** and **Delete**. `Ctrl+Z` puts a resize back. Nothing is written to the file until the editor saves, so an experiment costs you nothing until you decide to keep it.

Screenshot inserts a capture straight into the page's `media/` folder. A capture taken with no page open waits in **Needs You**, where *Attach the Screenshot to a Page…* files it later. **Annotate** draws on the selected picture, and one arrow on a screenshot is often worth a paragraph of text.

The Page container, on the right

> **Screenshot — the Page container: the page's Outline, and the Draft the AI wrote beside the page's own text.** `media/publishing/12-editor-sidebar.png`

Two things live there. The **Outline** lists the page's headings, and clicking one jumps to it. The **Draft** appears when the AI module has written one. A draft is never applied by itself: you accept it, and it becomes a single undoable edit.

Preview, Diff and Source

> **Screenshot — Preview of a page beside the editor, then the Diff view against HEAD.** `media/publishing/13-editor-preview.png`

Control	Shows	Use it when
Preview `Ctrl+Alt+P`	the page as the build renders it, in a tab of its own	you are checking a table, a callout or a picture width
Diff with HEAD `Ctrl+Alt+D`	what changed since the last commit	before publishing, especially on someone else's page
Source	the raw Markdown	a paste keeps coming out wrong
Page properties	a **read-only sheet** of the page's registry entry: title, area, chapter, order, `ship` and audience, plus the two ways to change it (the registry, or the page)	a page does not appear where you expect

Keyboard, all in one place

Shortcut	Does
`Ctrl+B` / `Ctrl+I` / `Ctrl+U`	bold / italic / underline
`Ctrl+Shift+X`	strikethrough
`Ctrl+E`	inline code
`Ctrl+Shift+H`	highlight
`Ctrl+Shift+B` / `Ctrl+Alt+C`	block quote / code block
`Ctrl+F`	find in the page
`Ctrl+Alt+O`	outline
`Ctrl+Alt+P` / `Ctrl+Alt+D`	preview / diff with HEAD
`Ctrl+Z` / `Ctrl+Y`	undo / redo
`Ctrl+S`	save the page to disk

Running it again

After the first publish, the loop is shorter than the guide above, because the folders, the project and the site clone already exist. What you re-run depends on what changed:

What changed	Build	Add	Downloads	Publish
pages only, same version	✓	✓ (same version — it replaces)	✓	✓
pages, and this is a new release	✓	✓ (new version)	✓	✓
nothing — you only want to push someone else's changes	—	—	—	✓
the site's own code or content	—	—	—	✓

The four commands, in order:

```text
Temco Docs:    Build the Site                temcoDocs.buildSite
Temco Learn:   Add This Build to the Site    temcoLearn.addBuild
Temco Learn:   Make the Downloads            temcoLearn.site.downloads
Temco Learn:   Publish                       temcoLearn.publish
```

Two habits save the most time. **Add a new version for a new release**, because re-adding the same version replaces it and readers lose the old one. And **redo the downloads every time you add a version**, because the guard counts versions, so a new version without its PDF blocks the publish rather than the deploy.

The same flow, as files: T3000

```text
~/temco/projects/t3000/                  the project, and only the project
├── docs/help/registry.json              the page registry
├── docs/commissioning/*.md              your pages
└── dist/site/index.html                 step 4: the site build

~/temco/temco-learn/                     the site clone
├── content/t3000/site.json              step 6: the title and the section
├── public/imported/t3000/1.0/index.html step 6: the build's own pages
├── public/pdf/t3000/1.0.pdf             step 7: the whole-guide PDF
├── public/chm/t3000/1.0.chm             step 7: the help file, when it compiles
├── public/download/manifest.json        step 7: what the download menu offers
├── public/catalog.json                  what the site reads, rewritten by steps 6 and 9
└── dist/                                step 9: what CI deploys
```

What the files look like

Three files carry the whole arrangement. Knowing their shape lets you tell a wrong page from a wrong build without guessing.

`~/temco/projects/t3000/docs/help/registry.json` is written by the create flow and by every page you add. It is the only list of pages; nothing else decides what exists.

JSON

```
{
  "version": 1,
  "areas": { "t3000": { "title": "t3000", "audience": "user", "noindex": false } },
  "pages": {
    "t3000/commissioning/ports": {
      "title": "Ports and wiring",
      "area": "t3000", "chapter": "commissioning", "order": 10,
      "audience": "user", "ship": true, "owner": "docs"
    }
  }
}
```

`ship: false` leaves a page out of the built site, and `order` decides its place inside the chapter. The key (`t3000/commissioning/ports`) must equal the file's path under `docs/` , or the page counts as missing.

`~/temco/temco-learn/content/t3000/site.json` is written once by step 6 and kept after that. It is how the site names the project and files it under a section.

JSON

```
{ "title": "T3000", "section": "documentation" }
```

Add `"summary"` for the sentence shown on the project's card, and change `"section"` to move the whole project to another category.

`~/temco/temco-learn/public/catalog.json` is generated and never edited. Every row is either a Markdown page the site owns or an imported page your build wrote, in one reading order.

JSON

```
{ "projects": [ { "id": "t3000", "title": "T3000", "section": "documentation",
                  "versions": [ { "version": "1.0",
                                  "pages": [ { "href": "index.html", "title": "T3000", "kind": "bui
```

That `"kind": "build"` is the giveaway: the page came from your build rather than from the site's own content. If it is missing after step 6, the catalog did not run.

Reference: what the outputs mean

The ticks on a project say what it is meant to produce, and one build is meant to produce all of them.

Output	Is	Which script runs, in your repository
Website	static pages for Temco Learn	`docs:site`, else `docs:build`, else the built-in generator
PDF	one document in a printed layout	`build:help:dry`, else `docs:pdf`
Offline help	the `.chm`	`build:help:dry`
Search index	the index the site loads	none here: the site builds it with `npm run search`
AI index	embeddings, kept on this machine only	none here: nothing is uploaded

A build writes inside your project folder. The site goes to `dist/site` (or `dist/docs`, when a repository's own generator wrote it), and the PDF and the `.chm` go wherever your scripts put them, commonly `dist/` or a `HelpDocs` folder beside the repository. Every file a build leaves behind is recorded against it, so the Builds and Downloads pages can list it.

Two things worth doing on purpose

A guard nobody has watched refuse is a guard nobody can trust.

Try this	You should see
Delete `public/pdf/t3000/1.0.pdf`, then publish	the publish **stops before committing** and names the missing file. Run *Make the Downloads* and publish again, and it goes through
Rename the project's page title, rebuild, then add the build again as `1.1`	version `1.1` lands under `t3000`: `content/` still holds one folder, and the catalog lists one project with two versions

When a step goes wrong

Step	What you see	What it usually is	Where to look
4	the build is not in the list to add	there is no `index.html` or `index.md` in it, or it sits more than two folders below a root	the folder itself. The roots are the window's folders, the projects folder and the working folder
6	adding refuses	the site clone is not where `temcoLearn.siteRepo` says	that setting, and whether the folder has `package.json` and `content/sections.json`
6, 9	the catalog did not run	there is no `npm ci` in the clone, or no `catalog` script	the site log
9	publishing refuses	the site's guard: a version has no committed PDF	the guard lines in the log
9	publishing pushes, but the site looks unchanged	the commit carried nothing, because the clone matched the last publish	`git log` and `git status` in the clone
6	the site page is wrong after a rebuild	the build was rebuilt but never added again as a new version	`public/catalog.json` and `public/imported/`

Beyond the flow

A project can do more than publish. None of the following is needed to get a page live, and all of it is available once you have one.

Bringing in material that already exists

A **source** is where pages are written from: a repository, a folder, or a document. There are two doors to the same result: the project's own page (`temcoProjects.editProject`), or the `Pages` view title → **Add a Source** (`temcoProjects.addSource` , the plug beside `+`).

If you already have a `docs/` folder, create the project with **From existing files** and choose that folder or file. A folder that already has `docs/help/registry.json` is cited as it stands, and nothing is copied. Anything else (Markdown, HTML, Word `.docx` , or a compiled `.chm`) is converted **once** into `docs/` and a registry, so from then on it is ordinary pages. A source is recorded once and can be cited by more than one project, and nothing is ever written back to it.

Checking before you build

Validate (`temcoDocs.validate`) answers one question: would a reader find a broken link, a missing page, or a page nobody registered? It reports in the Problems panel. A build does not fail on what it finds; the published site simply contains it. So run it before step 4.

Releasing a build locally

Publish on the Publish & Releases page (`temcoDocs.publishRelease`) is not publishing to the site. It copies the last build's artifacts into the application's own storage, at `published/<project>/<version>` with a manifest, behind a confirmation that names the version, the destination and the file count. `temcoDocs.openPublished` opens that copy, `temcoDocs.servePublished` serves it and `temcoDocs.stopServing` stops it. Use it to hand a build to somebody, or to keep a version that will not change. Nothing is pushed anywhere.

Exporting

Export… (`temcoDocs.export`) opens the Exports page, which tells you what you get before you get it. Every export writes into the project's `dist/` , and the notification offers *Show in Folder*.

> **Screenshot — the Exports page: the four shapes, each saying what is inside it and where it lands.**
> `media/publishing/14-exports.png`

Shape	Command	You get
the site, as a zip	`temcoDocs.exportSite`	`index.html` , `assets/site.css` , every page and each page's `media/` : the finished website before it is served from anywhere
one page, as one file	`temcoDocs.exportPage`	a single `.html` with the stylesheet inlined and every picture a `data:` URI, so it opens on a machine that has never seen the project
one page, as a Word document	`temcoDocs.exportDocx`	a `.docx` with headings, lists, tables and the pictures embedded: the format reviewers ask for
the pages, as a Markdown bundle	`temcoDocs.exportMarkdown`	every `.md` , each folder's `media/` and `registry.json` : the sources, for another tool or for an archive

Nothing is uploaded. A page with very large pictures makes a very large single file, so export the site as a zip when the pictures matter more than the convenience.

Translating a page

Translate This Page... (`temcoDocs.translatePage`) writes the translation **beside** the page, never over it: `docs/guide/setup.md` becomes `docs/guide/setup.de.md` .

> **Screenshot — the language picker, and the translated sibling page open beside the original.**
> `media/publishing/15-translate.png`

- Only the prose is translated. Code fences, inline code, table rules, link targets, image paths, front matter and the leading `#` , `-` and `>` markers are preserved byte for byte, so a translated page still builds, still links and still shows its pictures.
- **Once**: an Azure AI Translator resource. Its key goes into the OS keychain with **Set the Translator Key** (`temcoDocs.setTranslatorKey`), and the endpoint and region are the settings `temcoDocs.translator.endpoint` and `temcoDocs.translator.region` .
- **One request per page**, sent only when you run the command. Nothing is translated in the background.
- **Errors**: `401` means a key that was refused, `403` a key without a Translator resource (check the region), and `429` the free tier's rate limit. A page that is all code only says there is no prose, and costs nothing. Translating again asks before replacing the sibling file.
- A translation is an ordinary page. The registry and the review flow treat it like any other file, and a build includes it. Listing it in a language menu on the site is a registry decision, not the translator's.

When something else goes wrong

The table above covers the publishing pipeline. These are the rest:

What you see	What it means	What to do
Build the Site says *no `docs:site` (or `docs:build`) script*	the project has a `package.json` but no site script	add one, or delete the `package.json` so the built-in generator takes over
Build the Site says *has no pages marked for publishing*	no page is registered for the site, or every registered page is missing its file	open `Pages`, look for `missing` rows, and use **Create the Missing Page File**
the site builds but a page is empty	the page is registered and its file exists, but the Markdown is empty	write it, or **Retire the Page**
a page is missing from the published site	it is not registered, or it is not marked for the site	`Pages` → `…` → **Open Registry**
an export says *has no pages to export*	no registered page has a file yet	create or generate the pages first
a single-file page shows a broken picture	the picture's path in the Markdown is wrong, or it is not under the page's `media/`	fix the reference, because the same link is broken on the site
the editor shows raw Markdown	the file opened as text	in `Pages`, right-click the page → **Open the Page**; or in Explorer, **Reopen Editor With…** → **Temco Editor**
a picture is full width on the web but small in the editor	the page is older than the width support	reselect the picture, drag a handle, save
The Temco Editor surface is not built — `markdown.js`, `markdown.css` are missing	the window is running compiled extension-host code whose webview bundle was never built, or was removed by a build that cleans `out/`	`npm run compile-temco` from the repository root, then **Developer: Reload Window**. A plain `npm run compile` stops before the webviews when an earlier check fails, so the application compiles and its pages still have no surface

The output explains itself: **View** → **Output** → **`Temco Docs`**. Every build, publish, export and generator decision is written there, with the path it used.

Appendix — every button, one page

Every control the platform puts on screen, grouped by where it lives. The command id beside each one runs the same action from the Command Palette (`Ctrl+Shift+P`). Every id here was checked against the manifests, so if a row is in this table, the command exists.

Projects (activity bar)

Button	Where	Command
Create a Project	`Projects` view title `+`	`temcoProjects.newProject`
Open the Project Page	`Projects` view title	`temcoProjects.openProjectPage`
Remove All Projects	`Projects` view title	`temcoProjects.removeAllProjects`
Start (open a project)	right-click a project row	`temcoProjects.openProject`
Edit (sources of a project)	right-click a project row	`temcoProjects.editProject`
Project Detail	right-click a project row	`temcoDocs.projectPage`
Build (the project's own build)	right-click a project row	`temcoProjects.buildProject`
Stop Working on a Project	right-click the *current* row	`temcoProjects.clearProject`

Documentation (activity bar)

Button	Where	Command
New Page	`Pages` title ＋	`temcoDocs.newPage`
New Folder…	`Pages` title 📁	`temcoDocs.newChapter`
Add a Source	`Pages` title 🔌	`temcoProjects.addSource`
Capture a Screenshot	`Pages` title 📷	`temcoDocs.captureImage`
Paste Screenshot	`Pages` title 📋	`temcoDocs.pasteImage`
Refresh	every view title ⟳	`temcoDocs.refresh`
Open the Page	`Pages` … / page row	`temcoDocs.openPage`
Validate	`Pages` …	`temcoDocs.validate`
Overview	`Pages` …	`temcoDocs.overview`
Open Registry	`Pages` …	`temcoDocs.openRegistry`
Import a Word Document	`Pages` …	`temcoDocs.importDocx`
Create the Missing Page File	right-click a `missing` row	`temcoDocs.createPage`
Rename the Page Title / Move Up / Move Down / Retire	right-click a page	`temcoDocs.renamePage`, `.movePageUp`, `.movePageDown`, `.retirePage`
Attach the Screenshot to a Page…	right-click a capture in `Needs You`	`temcoDocs.attachCapture`
Build the Site	`Publish` title 🌐	`temcoDocs.buildSite`
Publish to GitHub Pages	`Publish` title ☁	`temcoDocs.publishSite`
Build the Offline Help	`Publish` title 📦	`temcoDocs.buildHelp`
Export…	Publish & Releases (opens the Exports page)	`temcoDocs.export`
Export the Site as a Zip	palette	`temcoDocs.exportSite`
Export the Page as HTML	palette, page right-click	`temcoDocs.exportPage`
Export the Pages as a Markdown Bundle	palette	`temcoDocs.exportMarkdown`
Export the Page as Word	Exports page, palette	`temcoDocs.exportDocx`
Review Center (row)	`Publish` view	`temcoDocs.review` → opens the Review Center page
Build Center (row)	`Publish` view	`temcoDocs.builds` → opens the Build Center page

Button	Where	Command
Publish & Releases (row)	`Publish` view	`temcoDocs.releases` → opens the Publish & Releases page

Editor (while a page is open)

Button	Where	Command
Bold / Italic / Link to a Page / Format…	editor title	`temcoDocs.format.bold` , `.italic` , `.linkPage` , `.pick`
Preview the Page	editor title, `Ctrl+Alt+P`	`temcoDocs.previewPage`
Diff the Page with HEAD	editor title, `Ctrl+Alt+D`	`temcoDocs.diffPage`
Page Properties	editor title	`temcoDocs.pageProperties`
Capture / Paste Screenshot	editor title	`temcoDocs.captureImage` , `temcoDocs.pasteImage`
Open the source (plain text)	editor title, ribbon `...`	`temcoEditor.openSource`
Temco Markdown (whole menu)	editor right-click	submenu with every format verb

The pages themselves

Button	On the page	Command
Build Site	Build Center, Publish & Releases	`temcoDocs.buildSite`
Build the Website / PDF / … (one card per output)	Build Center	`temcoDocs.buildOutput`
Build Offline Help	Build Center, Publish & Releases	`temcoDocs.buildHelp`
Refresh	Review Center	`temcoDocs.review`
Open folder	Build Center, Publish & Releases	`temcoDocs.revealArtifactFolder`
Open (a file)	Publish & Releases → a build	`temcoDocs.openArtifact`
Publish	Publish & Releases	`temcoDocs.publishRelease`
Publish to GitHub Pages	Publish & Releases	`temcoDocs.publishSite`
Export…	Publish & Releases	`temcoDocs.export`
Serve / Serve the Newest	Publish & Releases, release rows	`temcoDocs.servePublished`
Open (a release)	Publish & Releases → release rows	`temcoDocs.openPublished`
Stop Serving the Site	palette	`temcoDocs.stopServing`

The site

Button	On the page	Command
Start the Site	Documentation Site	`temcoLearn.site.start`
Stop the Site	Documentation Site	`temcoLearn.site.stop`
Build the Site (the site itself)	Documentation Site	`temcoLearn.site.build`
Make the Downloads	Publishing, Documentation Site	`temcoLearn.site.downloads`
Add a Build to the Site	Publishing	`temcoLearn.addBuilds`
Publish	Publishing	`temcoLearn.publish`
Publish Now	Publishing	`temcoLearn.publishNow`

Settings

Button	On the page	Command
General	Settings	`temcoWorkbench.settings.general`
Modules	Settings	`temcoWorkbench.settings.modules`
AI Runtime	Settings	`temcoWorkbench.settings.ai`
Branding	Settings	`temcoWorkbench.settings.branding`
Publishing	Settings	`temcoWorkbench.settings.publishing`
Repositories	Settings	`temcoRepositories.settings`
Check the Platform for Updates	Modules	`temcoWorkbench.settings.checkUpdates`
Test the Publish Target	Publishing	`temcoWorkbench.settings.testPublish`

The catalog

The platform's memory: one SQLite file, owned by `temco-store`, at `<globalStorageUri>/catalog/catalog.db` (with its WAL beside it). Every module reads it through `Catalog.open(context)`; no module opens its own database.

A row is written by an act — a read, a create, a build — and read by a surface. Nothing in the file is derived from the network at read time: a list from GitHub, for example, is a record of what GitHub *said*, with the time it said it.

Shape (schema version 6)

Table	What it holds
`meta`	`schema_version`, and nothing else.
`organisations`	Where repositories come from: `kind` (`organisation` | `account` | `repository`), `name`, unique `slug`, `url`, `added_at`, `last_read`, `state` (`read` | `refused` | `named`).
`repositories`	The rows of each list, `org_id` → `organisations(id)` **on delete cascade**, keyed by `(org_id, name)`. A cache of what GitHub last answered — replaced whole per organisation, kept when a read is refused. Indexed by `(org_id, pushed_at DESC)`.
`sources`	Where content actually comes from: a git remote at a branch, a folder, or an archive. Unique `slug`; `last_read`, `last_read_mode`, `last_commit`. **The truth about a repository is a source, not a cached row.**
`documents`	The projects a person manages — `name`, unique `slug`, `version`, `audience`, `owner`, `origin`, `template`, timestamps.
`doc_sources`	Many-to-many: a document cites sources, each with its own `branch`, `include_glob`, `role`.
`recipes`	One row per document: the engine (`docfx`), its `config_path`, `script`, and the `outputs` it must produce.
`builds`, `artifacts`	A build attempt and what it left behind, by document.
`commits`, `commit_pages`	What a read found in a source's history, and which pages a commit touched.
`pages`	The pages a read recorded, by source and page id.
`scans`	One row per read: mode, revision, counts.
`events`	What the platform noticed and has not shown (`seen_at` null).
`drafts`	A page's text with no file behind it: `document_slug`, `page_id` (the id it *will* have), `title`, `kind`, `body`, `state` (`draft` | `accepted` | `discarded`).
`releases`	Published versions, by document.

Versioning

`SCHEMA` is applied with `IF NOT EXISTS` on every open, then `meta.schema_version` decides whether migrations run. A version from the future is **refused in words** rather than half-read. `MIGRATIONS[n]` is what a version `n-1` file

needs to become `n`:

- 4 — `documents.origin`, `documents.template`
- 6 — drops `repo_lists`, the one-turn JSON blob that `organisations` + `repositories` replaced

Where the settings sit

Three settings decide what the repositories surfaces act on, and all are written by `temcoRepositories.settings` (the page) or by VS Code's own settings JSON:

- `temcoRepositories.organisation` — the enterprise the dashboard opens on, and the one seeded into `organisations` the first time the page opens.
- `temcoRepositories.workspaceFolder` — the root of the working area. Clones go to `source/` under it (one copy per repository, shared by every project that cites it). Empty means `~/temco`. It is outside every repository on purpose: nothing is ever written to a source, and nothing is pushed.
- `temcoRepositories.projectsFolder` — where a project's own folder is made, `<this>/<slug>/`. Empty means the working area above, which is where every project went before the setting existed; pointing it at a folder of its own keeps projects apart from the shared clones.

Naming

Every identifier in the product follows one rule: **the namespace is the module that owns the feature.**

Patterns

Kind	Pattern	Example
Repository	`temco-<name>`	`temco-platform`, `temco-learn`
Module folder	`temco-<module>`	`extensions/temco-docs`
Extension id	`temcocontrols.temco-<module>`	`temcocontrols.temco-docs`
Command id	`temco<Module>.<action>`	`temcoDocs.validate`
Command title	`Temco <Module>: <Action>`	`Temco Docs: Validate`
Setting id	`temco<Module>.<name>`	`temcoAi.model`
View id	`temco<Module>.<view>`	`temcoDocs.pages`
Container id	`temco`, `temcoTools`, `temcoAi`	titles: Temco, Tools, Temco AI
Working area	`<module>.<name>`	`temcoRepositories.workspaceFolder`

Words

Word	Used for
Temco Platform	the product, and the application's display name
Temco Learn	the published site
Temco Docs, **Temco AI**, …	a module's name in user-visible text
`temco-docs`, `temco-ai`, …	a module in paths, ids and code

One word per thing: the product is **Temco Platform**, the published site is **Temco Learn**, and a module is named after what it does — **Temco Docs**, **Temco AI**, and so on. A module's word appears in its own titles, its ids and its folder; nothing else may claim it.

Modules

Module	Extension id	Namespace
`temco-docs`	`temcocontrols.temco-docs`	`temcoDocs`
`temco-ai`	`temcocontrols.temco-ai`	`temcoAi`
`temco-repositories`	`temcocontrols.temco-repositories`	`temcoRepositories`
`temco-projects`	`temcocontrols.temco-projects`	`temcoProjects`
`temco-sql`	`temcocontrols.temco-sql`	`temcoSql`
`temco-nettools`	`temcocontrols.temco-nettools`	`temcoNettools`
`temco-bacnet`	`temcocontrols.temco-bacnet`	`temcoBacnet`
`temco-device-manager`	`temcocontrols.temco-device-manager`	`temcoDevices`

Timing

An identifier is free to change **before the first release**, and expensive afterwards: an extension id owns its storage folder, a setting id holds a user's value, and a command id is written into keybindings, menus and user scripts.

So the rule is: settle an identifier before it ships, and change it later only in a release that migrates what it holds. Storage is derived from the id — `<globalStorageUri>/catalog/catalog.db` for the catalog — so it moves with the id and is left behind when the id changes.

Entry points

The fork touches VS Code's source in exactly two places. Both are additive lines in lists. This is the complete list.

1. The compile list

File	`src/tsconfig.json`
Addition	`src/temco/**` added to the `include` list
Why	the config includes `./vs/**` and a few root files only. Without this line the shell is not compiled and not type-checked

2. The load list

File	`src/vs/workbench/workbench.desktop.main.ts`
Addition	one import of `../temco/temco.contribution.js`
Why	the workbench loads its parts, services and contributions through explicit imports. Code that nothing imports never runs

This is how VS Code's own features enter the workbench: a contribution file that is imported and registers itself. The menu bar is built the same way — it renders whatever is registered in the menu registry, and a feature such as the debugger adds its own top-level menu from its own contribution file. That is why Temco's menus are Temco's own code.

The layering rule under that import

Every file under `src/` is also governed by `local/code-import-patterns`: each one must fall in a zone, and the zone carries the list of what it may import. Two entries in `eslint.config.js` make the shell legal, and they are the reason the entry points above are the whole seam rather than the beginning of one:

Zone target	Allows
`src/temco/**`	`vs/base`, `vs/platform`, `vs/editor`, `vs/workbench`, `vs/nls.js`, and itself — the shell sits outside VS Code's layering, so its reach is stated once instead of inherited
`src/vs/workbench/workbench.desktop.main.ts`	its usual list, plus `src/temco/**` — the one place allowed to import the shell

Without the first entry a shell file is "not in a valid layer"; without the second the seam import violates the workbench zone. Neither is a relaxation for the rest of `src/vs/**`.

Not entry points

Not a seam	Why
`product.json`	the fork's own identity file: product name, icons, default settings, update and telemetry policy
`resources/**`	Temco's own images
`extensions/temco-*/**`	new folders; the build picks up any folder with a `package.json`
`docs/**`	Temco's own documentation
module build wiring	none needed: modules are compiled by the existing extension build, and each folder's dependencies are installed by the root install
`build/lib/extensions.ts`	touched only if a module ships a native npm module, which no planned module does

Gate exemptions, not seams

VS Code's gates encode policies for Microsoft's own code: a Microsoft copyright header, tab indentation, `nls` for every user-visible string, no synchronous file I/O. Temco's code follows its own conventions, so the same lines that exempt `extensions/temco-*` also exempt `src/temco/**`:

File	Addition
`build/filters.ts`	`'!extensions/temco-*/**'` and `'!src/temco/**'` in the unicode, indentation, copyright and formatting filters
`eslint.config.js`	one override for both paths, turning off the Microsoft header, sync-fs, unexternalized-strings, in-operator and bracket-notation rules
`.eslint-allowed-javascript-files`	`extensions/temco-ai/media/ai-panel.js` — a webview asset, not extension code
`build/gulpfile.extensions.ts`	the module's `tsconfig.json` in the compiled list
`package.json`	`check-temco` (the module guards, the dependency graph, the menu ids), run by `compile`

None of these is a seam into VS Code's behaviour: remove them and Temco's files stop passing the gates, but the application still builds and runs unchanged.

Rule

This list may not grow without a decision. An addition needs a written reason and a row above. Before adding one, the first question is whether the need can be met by a new folder instead — a module, a `product.json` field, or an extension contribution point.

After a VS Code update

1. Does `src/tsconfig.json` still include `src/temco/**`?
2. Does `workbench.desktop.main.ts` still import `temco.contribution`?

3. Does `src/temco/temco.contribution.ts` still exist at that path?

If all three hold, the shell is intact.

TEMCO Platform — Engineering Design (Revision 2)

Host: the VS Code fork. UI library: Fluent UI v9. Editor engine: ONLYOFFICE Docs.

Status: design for implementation. Written to be handed to a code generator. Target repository: `temcocontrols/temco-platform` (VS Code 1.144.0.0 fork, 12 `temco-*` extensions).

---

0. How to use this document

0.1 Revision history — what changed, and why it is not cosmetic

Revision 1 proposed a standalone web app (React 19 + Vite) and pushed the VS Code fork down to "an adapter". Revision 2 inverts that. The product **is** the fork; the workbench is the shell users already know; and every surface we own is built with Fluent UI v9 inside it. Three reversals:

#	Revision 1	Revision 2	Reason
1	Host = new web app	Host = the existing VS Code fork	Discards a working shell, keybindings, editor model, file watching, and the installed base for nothing
2	Editor = first-party Tiptap for most formats	Editor = ONLYOFFICE Docs for Office formats	Office fidelity is the actual requirement; re-implementing DOCX layout is not a business we are in
3	UI library = free choice	Fluent UI v9 is mandatory in the content register, forbidden in the shell register	One visual language for our screens, zero risk to the shell

Everything else — the five core concepts, the domain model, the publication state machine, the flows, the round-trip law — survives. Only where it is *hosted* changed.

0.2 Reading order for a code generator

Read in order. Do not generate UI before §2 (the two-register law) and §4 (the document server contract), because those two sections decide which library owns which pixel and which process owns which file.

0.3 Five rules that override anything else in this document

1. **The shell is not ours.** Never restyle, re-implement, or replace the title bar, activity bar, side bar trees, tabs, status bar, command palette, or settings editor. They are VS Code. They use VS Code theme variables. They do not import Fluent.
2. **Fluent does not leak.** Fluent UI v9 is imported only inside webview content (§2.2). No Fluent component is rendered into shell DOM, and no Fluent token is used to colour a shell surface.
3. **The document server is not a library.** ONLYOFFICE is a separate HTTP service. Code must treat it as an unreliable remote dependency with a health state, not as an importable editor.
4. **A save is a round trip through the storage contract.** No editor writes to the asset store directly. Every save is `editor → callback → host validates → store writes new revision`. (§4.5)

5. **Do not invent a domain concept.** Products, Assets, Editor Hosts, Publications, Knowledge. If a screen needs a sixth noun, the design is wrong — stop and re-read §5.

---

1. Vision and principles

TEMCO Platform is an engineering operating system: it holds documentation, hardware descriptions, firmware, programs, releases, and the knowledge derived from them, for products built and supported for ten or more years.

The user is a controls engineer at 2 a.m. with a T3000 that will not talk to a BACnet router. The product succeeds when that engineer finds the right page, sees the firmware version it applies to, and gets a file they can open.

1.1 Principles and what each one forbids in code

Principle	Forbids
Familiar first	A custom shell, custom tabs, a custom command palette, custom keybindings
One editor column	Side-by-side editor panes; split editors for the same asset
Structure is data	Hard-coded folder names, hard-coded product lists, "T3000" in a component
Every save is a revision	In-place file writes, "Save" without a version
Degrade loudly	A blank iframe, a silent spinner, a swallowed error
The tree is the map	A screen that shows assets but cannot be reached from the tree

1.2 Success conditions (measure these, not vibes)

- Open a 40-page `.docx` from the tree to a usable editor in **under 4 s warm**, **under 9 s cold**.
- The tree never shows a spinner for more than **300 ms** on a warm start.
- A Markdown file opened and closed without an edit produces **zero** revisions and a clean tab.
- Every asset type in §5.3 has exactly one surface that opens it, and the mapping is table-driven.
- With the document server down, every Office asset still opens **read-only with a stated reason** — never blank.

---

2. The two-register law

This is the single most important structural decision in the document. The workbench and our content are two different UI stacks, and confusing them is how this project fails.

2.1 The boundary

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ REGISTER A — THE SHELL                          owner: VS Code core      │
│ library: none (native workbench DOM + CSS)                               │
│ styling: --vscode-* theme variables only                                 │
│                                                                          │
│   title bar · activity bar · side bar trees · tabs · breadcrumbs ·       │
│   status bar · command palette · notifications · quick pick · settings   │
├──────────────────────────────────────────────────────────────────────────┤
│ REGISTER B — THE CONTENT                        owner: temco extensions  │
│ library: Fluent UI v9 + React 19                                         │
│ styling: Fluent tokens, bridged from --vscode-* (§7.2)                   │
│                                                                          │
│   every custom editor webview · every webview view panel ·               │
│   every dialog drawn inside our own surfaces                             │
├──────────────────────────────────────────────────────────────────────────┤
│ REGISTER C — THE REMOTE ENGINE                  owner: ONLYOFFICE Docs   │
│ library: ONLYOFFICE's own web app                                        │
│ styling: ONLYOFFICE's own themes, told to match ours                     │
│                                                                          │
│   the document canvas inside an <iframe>, in register B's webview only   │
└──────────────────────────────────────────────────────────────────────────┘
```

2.2 What each register may use

	Shell (A)	Content (B)	Remote (C)
Fluent UI v9	**Never**	**Always**	n/a
React	Never	Always	n/a
VS Code `TreeView`	Yes	**Never**	n/a
VS Code theme vars	Yes	Only read, to feed §7.2	n/a
Own context menu	Only via `menus` contributions	Yes (Fluent `Menu`)	Own menus
Own keyboard shortcuts	Only via `keybindings` contributions	Only non-chorded, scoped keys	Own, inside its iframe

2.3 Why Fluent cannot be "the main lib for VS Code" — and what we do instead

VS Code's workbench is hand-written DOM driven by DI, not React. Replacing its activity bar with `FluentProvider` does not produce a Fluent activity bar; it produces a broken one, and it breaks every extension that positions against those parts. What we can honestly do, and what this design does:

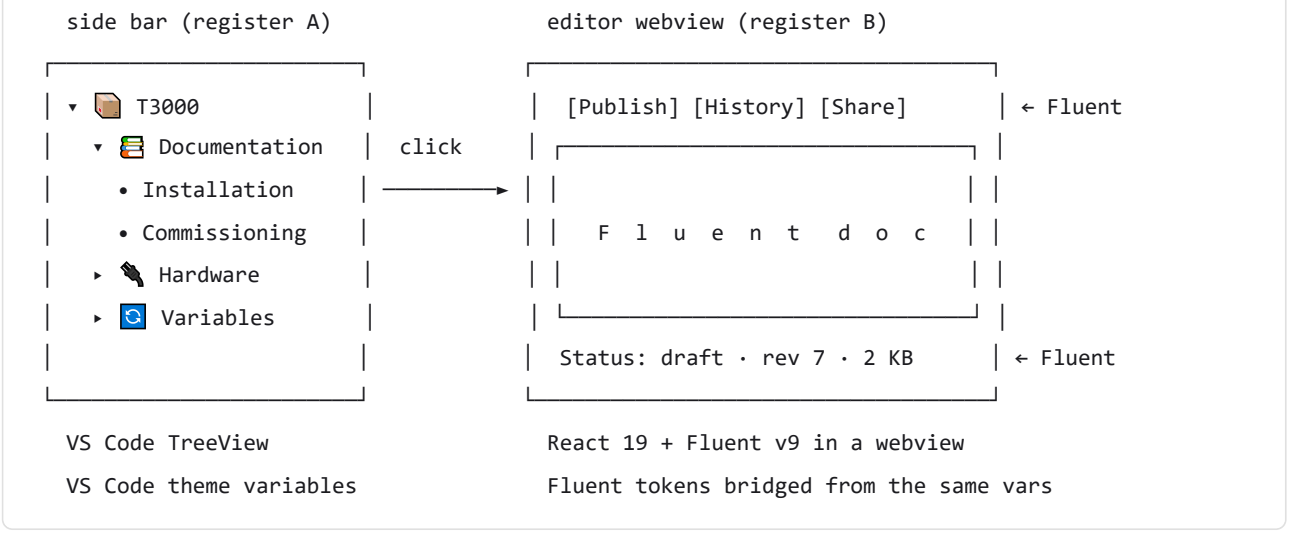
1. **Every pixel we draw is Fluent.** All our screens live in webviews, and in webviews Fluent is the only component library. Our surfaces will look and behave like one product because they share one library and one theme bridge.

2. **The bridge makes Fluent look native.** §7.2 reads VS Code's live theme and rewrites Fluent's tokens, so a Fluent `Button` in our editor panel is the same colour as a VS Code button beside it. This is what makes the seam invisible.
3. **Where a native control already exists, we use it through the host.** Adding an asset is a `TreeView` with `view/item/context` menu contributions, not a Fluent list in a webview. This is faster, accessible for free, and themable for free.

Rule of thumb: if the control is *chrome* (navigation, tabs, trees, commands), it is VS Code. If the control is *content* (a form, a table of releases, a publish checklist, a document), it is Fluent.

2.4 The seam, drawn

TEXT

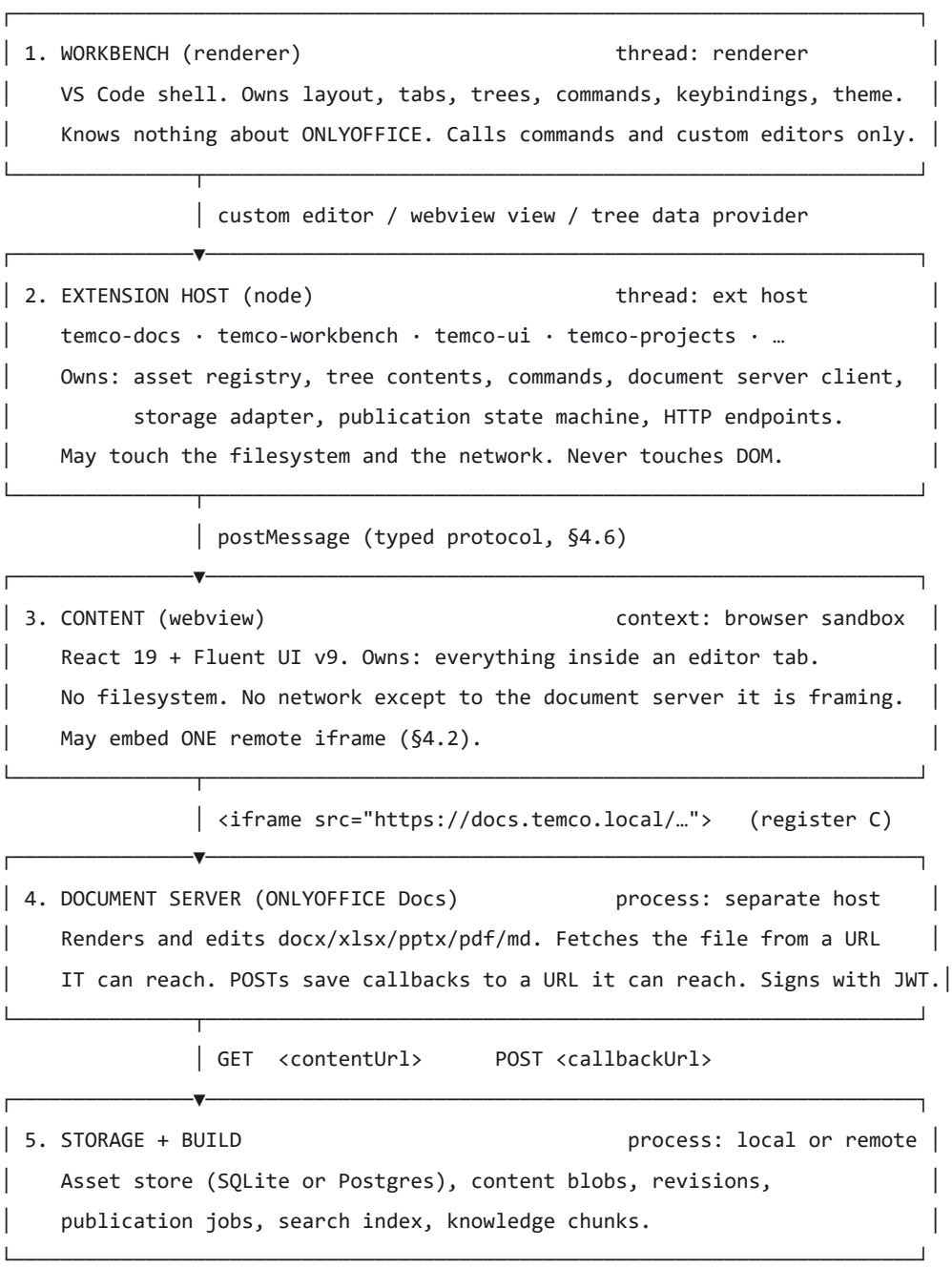


---

3. Architecture

3.1 Planes and processes

TEXT



> **Plane 4 is optional.** In the embedded topology (§4.1.1 Topology B) it collapses into plane 3: the converter runs as `x2t.wasm` inside the webview, and planes 3 and 5 talk directly. Planes 1, 2, and 5 are identical either way, which is why the transport is pluggable rather than a fork in the design.

3.2 Hard truths this architecture imposes

These are not opinions. Each one has sunk an ONLYOFFICE integration before.

#	Truth	Consequence in code
T1	The document server, not the client, downloads the document	The content URL must be reachable **from the server**, not from the user's machine. `file://` and `blob:` cannot work.
T2	That fetch is unauthenticated by design	Content URLs must be short-lived signed URLs (HMAC + expiry, single use), never session cookies. §4.4
T3	Saves arrive as a POST to a URL the server can reach	The extension host must run a small HTTP listener with a stable address and a ticket per editing session. §4.5
T4	The editor's `key` identifies a *version*, and it is cached	Reusing a key after content changes serves stale bytes. The key must be derived from `(assetId, revision, optionsHash)`. §4.3
T5	Callback status 2 must be answered and the bytes downloaded within ~10 s	Download starts immediately; the store write may finish later, but the fetch may not. §4.5
T6	vsix cannot ship the document server	Deployment is a decision (§4.9), not an implementation detail. Develop against local Docker, ship against an intranet host.
T7	A cross-origin iframe owns the keyboard while focused	Every host-side shortcut must be routed through iframe focus state (§12.3) or it will silently not fire.

4. The editor engine: ONLYOFFICE Docs

4.1 What we are integrating

ONLYOFFICE Docs (the "Document Server") is a **web application** — a separate service, typically `onlyoffice/documentserver` in Docker, listening on 80/443. It renders and edits office files in the browser and exposes them for embedding through `DocsAPI.DocEditor`. It is not a JS library. There is no `npm i` that gives us a Word editor.

TEXT

```
 we write                    they own                      they render
──────────                  ──────────                    ────────────

 config JSON  ──JWT──►  document server  ──HTTP GET──►  our content URL
  + iframe                    │                            (signed)
  mount point                 └──HTTP POST──►  our callback URL
                                    │
                                    └──────────────►  editor UI inside our iframe
```

4.1.1 Two ways to run it: server-hosted, or fully embedded

> **Correction (recorded on purpose).** An earlier revision of this document claimed the engine "cannot be embedded in a VSIX". That was wrong. It is a common belief and it is worth writing down why it is false, because it changes the architecture.

Facts, checked:

1. **The editor UI is static files.** `web-apps/` + `sdkjs/` are HTML/JS/CSS. They can be served from any origin, including a `vscode-webview://` resource root or a `file://` side-car folder.
2. **The only native piece is `x2t`**, the converter that turns `.docx` into the editor's internal model and back. Without it nothing opens.
3. **`x2t` has a WebAssembly build** maintained by CryptPad: cryptpad/onlyoffice-x2t-wasm. Conversion runs in the browser, in a Docker/Emscripten-produced `x2t.wasm`, with no native backend.
4. **A static build of the full editor exists**: cryptpad/onlyoffice-builds.
5. **CryptPad ships the complete ONLYOFFICE editor with no document server**: assets are static, conversion is WASM in the browser, collaboration is CryptPad's own CRDT layer. This is production software used by many organisations.

So the question is not *can it run without a server* — it can. The question is **which topology we can support and license.**

TEXT

```
TOPOLOGY A — server-hosted (default)

┌──────────────────────────────────────────────────────────────────────────┐
│ VS Code — webview — <iframe src="{server}/web-apps/…">                   │
│                          │                                               │
│                          ▼                                               │
│             Document Server (nginx + node + x2t native + Redis/Postgres) │
│                 GET content   ·   POST callback   ·   co-editing websockets │
└──────────────────────────────────────────────────────────────────────────┘


TOPOLOGY B — fully embedded (no server anywhere)
┌─ VS Code (Electron) ─────────────────────────────────────────────────────┐
│  extension host                                                          │
│    └ extension resource dir                                              │
│        onlyoffice/web-apps/…      static editor UI                       │
│        onlyoffice/x2t.js + x2t.wasm   converter, in-process              │
│        onlyoffice/fonts/…         fonts the renderer expects             │
│                                                                          │
│  webview (register B — Fluent host)                                      │
│    └ <iframe src="vscode-webview://…/onlyoffice/web-apps/…">             │
│         editor UI runs locally · zero network                            │
│                                                                          │
│  asset store  (bytes in, bytes out)                                      │
└──────────────────────────────────────────────────────────────────────────┘
```

Embedded open/save pipeline:

TEXT

```
.docx bytes (store)
      │
      ▼
x2t.wasm ──convert──► editor.bin  (+ fonts, theme, thumbnails)
      │                    │
      │                    ▼
      │          iframe loads the editor with editor.bin
      │                    │
      │               user edits
      │                    │
      ▼                    ▼
x2t.wasm ◄──────────── editor.bin (changed)
      │
      ▼
.docx bytes ──► store (new revision, §4.5 status "2" semantics)
```

4.1.2 What actually decides it

Constraint	A — server-hosted	B — fully embedded
Licence	AGPLv3 service, run unmodified, separate process	**AGPLv3 code redistributed inside our product**
Download size	0 MB to the client	~200–400 MB (assets + fonts + wasm)
Cold start	3–9 s (server render)	2–6 s (wasm init ~1–2 s + conversion)
Co-editing	built in	**none** — needs our own CRDT, i.e. new engineering
PDF export	server-side, reliable	via `x2t.wasm`, limited/canvas-dependent
Update path	upgrade one server	every client re-downloads
Support	official, vendored	community fork; **we own the build** (Docker + Emscripten + fs shim + font pack)
Offline	no	**yes**
Data leaves the machine	no (self-hosted)	never

The licence is the real question, not the technology. ONLYOFFICE DocumentServer is AGPLv3. Two routes:

- **Internal use (no distribution outside the company):** embedding is fine in practice. AGPL obligations attach to *conveying* the program to third parties, and to modified versions offered over a network. An internal deployment does neither. Confirm with counsel, but this is the usual industry reading.
- **Distributed product (sold or given to customers):** either buy a commercial ONLYOFFICE licence before shipping the binaries, or ship Topology A where the AGPL program stays a separate service we run.

Both routes are viable. Topology A ships sooner and keeps co-editing; Topology B removes the server entirely and is the only option if policy forbids standing one up.

4.1.3 Therefore: the transport is pluggable

Everything above the transport is identical, so we do not choose permanently. One interface, two implementations, chosen by configuration.

```typescript
interface OfficeTransport {
	readonly id: 'http' | 'embedded';
	readonly capabilities: {
		coEditing: boolean;	// true for http, false for embedded
		comments: boolean;
		exportPdf: 'server' | 'wasm' | 'none';
		offline: boolean;
		maxDocBytes: number;
	};
	/** Produces the config (and, for 'http', the signed JWT) the surface needs. */
	prepare(asset: AssetRecord, revision: number, mode: 'edit' | 'view'): Promise<OfficeConfig>;
	/** Resolves when the surface has reported a saved revision. */
	awaitSave(session: OfficeSession): Promise<Revision>;
	dispose(session: OfficeSession): Promise<void>;
}
```

Consequences, all of which keep the code honest:

- **TR-1** The host↔content protocol (§4.6) does not change. `init`, `saved`, `reload`, `error` mean the same thing in both.
- **TR-2** The `key` rule (§4.3) survives: it still identifies a version and must change when the revision changes. In embedded mode there is no server cache, but the editor still uses it to decide whether to reuse in-memory state.
- **TR-3** In embedded mode the "save callback" is a local function, not an HTTP POST. It still writes an immutable revision (§6.1 D1–D2) and it still must satisfy the 10-second rule's *intent*: convert and write off the UI thread, never block the canvas.
- **TR-4** Capability differences must be visible, not silent: with `coEditing: false` the header hides the presence avatars and the asset shows `single-user` in Properties.
- **TR-5** A test opens the same fixture through both transports and asserts the round-trip law (§14.2). If a feature only works in one, it is listed in §4.1.2 rather than discovered later.

4.1.4 Decision (recorded): both transports, embedded is the default

Chosen because a server is something the team would rather not run, and because the product may later be sold.

	Value	Consequence
Transports supported	`embedded` **and** `http`	One interface, two implementations, no fork in the design
Default	`embedded`	Works offline, no service to operate, no tickets or callbacks
`http`	opt-in per workspace	The escape hatch for co-editing and for the packaged-build licence story
Co-editing	**not available** in the default	Must be stated in the UI, not discovered (§4.1.3 TR-4)
Distribution	internal now, possibly commercial later	Forces the licence gate below

Because "may be sold later" is on the table, the embedded build needs a **build-time gate**, not a promise in a document.

4.1.5 Shipping the embedded build: the office pack and the licence gate

The 300 MB does not belong in a VSIX. It ships beside the app.

```
TEXT
```

```
┌─ what ships, and how big it is (measured on this machine) ──────────────┐
│                                                                         │
│  VS Code fork (Electron)                                                │
│   Electron runtime                        ~360 MB   already shipped today │
│   resources/app/out/…                     ~120 MB   production build    │
│   resources/office-pack/                  ~300 MB   ONLYOFFICE assets   │
│     web-apps/   sdkjs/   fonts/   x2t.wasm            → §4.1.6 to shrink  │
│     LICENSE.txt   NOTICE.txt              ← attribution, always present │
│                                                                         │
│  Extensions (temco-docs)                                                │
│   out/…                                   ~2 MB     extension code      │
│   locates the pack via officePackPath(); if absent, capability = false  │
└─────────────────────────────────────────────────────────────────────────┘
```

The gate runs in the build, so a mistake fails CI instead of becoming a legal problem:

TEXT

```
build.mjs
  │
  ├─ TEMCO_OFFICE_EMBED=1 ?  ──no──►  do not copy the pack.
  │                                    embedded transport reports
  │                                    { available: false }  → §4.10 banner
  ▼ yes
  ├─ TEMCO_OFFICE_LICENCE = ?
  │        │
  │        ├─ agpl-internal ──► allowed only when TEMCO_BUILD_CHANNEL=internal
  │        │                    CI fails if channel = public
  │        │
  │        ├─ commercial ─────► allowed in any channel;
  │        │                    NOTICE.txt records the licence reference
  │        │
  │        └─ unset ──────────► CI fails: "embedded pack requires a licence decision"
  ▼
  copy pack → write NOTICE.txt → expose in About › Third-party notices
```

Enforceable rules:

- **LG-1** A public-channel build containing `resources/office-pack/` without `TEMCO_OFFICE_LICENCE=commercial` **fails the build**. No warning, a failure.
- **LG-2** `LICENSE.txt` and `NOTICE.txt` from the ONLYOFFICE distribution are copied unmodified and are never excluded by the packager.
- **LG-3** The About dialog lists the pack version and licence mode; Diagnostics (§10.12) can export them.
- **LG-4** Missing pack is a **degraded state**, not a crash: Office assets open **read-only** with the §4.10 banner and a one-line reason, and offer Download. There is no silent fallback to the text editor — that is only ever an explicit `Open With → Text Editor` choice, because editing a `.docx` as bytes destroys it.
- **LG-5** The pack is version-pinned to the editor build (`x2t.wasm` and `web-apps` must come from the same ONLYOFFICE release); a mismatch is reported as `office-pack-mismatch`.

The internal/commercial switch is exactly the point the user raised: internal use does not trigger AGPL distribution obligations, shipping does — so the build system, not memory, decides which one is happening.

4.1.6 Size: where it comes from, and how to shrink it

Measured on the development machine, so these are facts rather than round numbers:

Part	Size	Note
Electron runtime	**360 MB**	already in the product today; not caused by ONLYOFFICE
VS Code product (`out/`)	**267 MB dev** / ~120 MB production	source maps are most of the dev figure
ONLYOFFICE office pack	**~300 MB estimate**	`web-apps` + `sdkjs` + fonts + `x2t.wasm`

Why the upstream package is so heavy: the Document Server image is **2–3 GB+** because it bundles editors for word/cell/slide/pdf, the whole `sdkjs` engine for each, dozens of locale packs, a large font collection (a major contributor — it is what makes documents look right), the `x2t` converter, and nginx/node/redis/postgres. Nothing in there is waste; it is just everything.

Shrink levers, largest first:

Lever	Est. saving	Cost
Ship **word only** (drop cell/slide/pdf editors)	~35–45 %	those formats open as download only
Prune locales to `en-US` + `zh-CN`	~10–20 %	untranslated UI for other languages
Prune fonts to a curated set (~20 families)	~30–50 %	font substitution → layout drift on documents using exotic fonts
Ship the pack as a compressed archive, extract on first run	download ↓50–60 %	install time; disk use unchanged
Download the pack on first use ("Office Pack" component)	installer **0 MB**	needs a download channel; first open needs network
Do several of the above together	**~60–120 MB plausible**	measure, do not assume

Order of preference: **word-only + two locales + curated fonts + first-run download.** It keeps the installer small and the runtime honest, and it is reversible — the pack is a versioned artifact, not code.

Licence is not MIT. ONLYOFFICE DocumentServer is AGPL-3.0 (verified from the repository `LICENSE`). The `@onlyoffice/document-editor-react` wrapper and some helper packages carry their own permissive licences, which is where the MIT impression comes from — but the editors, `sdkjs`, and `x2t` are AGPL. Nothing that does real OOXML layout is MIT; MIT-licensed packages in this space (`docx-preview`, `docx`) render or generate, they do not edit with fidelity.

If either size or licence turns out to be unacceptable, these are the engine alternatives — evaluated as engines, not as libraries:

Engine	Licence	Size	DOCX fidelity	Note
ONLYOFFICE, embedded	**AGPL-3.0**	~300 MB, reducible	excellent	we own the build
ONLYOFFICE, server	AGPL-3.0, separate service	0 MB on the client	excellent	needs a host; the only topology with co-editing
ZetaOffice (LibreOffice in WASM)	**MPL-2.0** — file-level copyleft, **no obligation on our app**; `zetajs` is MIT	large, same order	very good	commercial support from allotropia/CIB; the licence-safe swap for Topology B
Syncfusion Document Editor	commercial (community licence for small teams)	~10–20 MB	good on common documents	per-developer licence
TX Text Control	commercial	~50 MB	very good	per-developer licence
Apryse / Nutrient WebViewer	commercial	varies	good, conversion-centric	per-server licence
Tiptap / Milkdown	MIT	tiny	document model, not page layout	already rejected for Word fidelity (§4.8)

The decision rule: if AGPL blocks distribution, swap the engine for **ZetaOffice** and keep Topology B — the `OfficeTransport` interface absorbs it. If *size* is the problem, only a commercial component (Syncfusion or TX) actually solves it, and that is a budget conversation, not an engineering one. Keep this table current; re-read it before P5 when the product might ship.

4.1.7 The zero-AGPL routes, and what "mock the editor" can mean

AGPL-3.0 does not forbid *using* the software. It forbids **conveying** it without the corresponding source, and (for modified versions) offering it over a network without source. So there are four clean routes, and only two of them involve settling anything with ONLYOFFICE.

#	Route	AGPL exposure	Client size	DOCX fidelity	Cost
R1	**Separate service** (Topology A): the AGPL program runs as its own process; we only open an iframe to it	**None** — no conveyance, no derivative work	**0 MB**	excellent	a host to run
R2	**Commercial ONLYOFFICE licence** → embed Topology B legally	None (licensed)	~300 MB, reducible	excellent	licence fee
R3	**MPL engine** (ZetaOffice = LibreOffice in WASM) → embed	None on our code (file-level copyleft only)	large	very good	optional support contract
R4	**No office engine at all** — our own editor + MIT viewers	**None**	**~2 MB**	none for imported docs	engineering time

Route **R4** is the one worth taking seriously, because a VS Code fork with a Fluent UI is already a very good document authoring surface — that is what the Markdown host in §4.8 is.

4.1.7.1 The question that decides it

Not "which engine" — but **must existing `.docx` files be edited in place inside the app?**

TEXT

```
┌─ what the corpus needs ──────────────────────────────────────────┐
│                                                                  │
│  A. AUTHOR new documents          → Fluent + Tiptap (MIT)        │
│     our own .md / .temco format       zero AGPL, ~1 MB, 50 ms open │
│                                                                  │
│  B. VIEW an imported .docx        → docx-preview (MIT) or mammoth (BSD) │
│     read-only, no fidelity promise     zero AGPL, ~300 KB        │
│                                                                  │
│  C. EDIT an imported .docx        → pick one:                    │
│     exact page layout must survive                               │
│                                       1. ONLYOFFICE as a separate │
│                                          service ..... R1        │
│                                       2. commercial licence ..... R2 │
│                                       3. ZetaOffice, MPL-2.0 ..... R3 │
│                                       4. convert-on-import, edit ours, │
│                                          export .docx (lossy) .... R4 │
└──────────────────────────────────────────────────────────────────┘
```

Routing, as the editor host registry implements it (§4.11):

TEXT

```
open asset
   │
   ├─ .md / .temco ──────────► Fluent editor (MIT)                  [A]
   │
   └─ .docx
         │
         ├─ "View" ──────────► docx-preview in the webview (MIT)    [B]
         │
         └─ "Edit"
               │
               ├─ office service configured ──► ONLYOFFICE iframe   [R1]
               ├─ commercial licence ──────────► embedded pack      [R2]
               └─ neither ─────────────────────► "Convert to Temco Doc"
                                                  → editable here,
                                                  export back to .docx [R4]
```

R4 is honest about its trade: conversion is **lossy**. Tables survive, images survive, headings survive; exact pagination, floating shapes, and complex numbering generally do not. The original `.docx` is kept untouched as a revision, and the conversion is an *opt-in* action with a visible warning — never a silent import.

4.1.7.2 What "mock the editor" can and cannot mean

Three different things hide behind that phrase. Only one of them ships.

Reading	Legal	Useful	Verdict
A clickable mock — a Fluent UI that looks and behaves like the editor, with no engine behind it	Yes, no third-party code at all	Yes — for stakeholder sign-off and for designing the flow before wiring an engine	**Do it** as a design artifact, never as the product
Mimic ONLYOFFICE's chrome — our own engine behind a ribbon like theirs	Functional layouts are not protected; their **icons, logos and fonts are**	The ribbon was never the hard part	Allowed, but buys nothing
A placeholder editor that quietly replaces the real one	Yes	No	**Forbidden** — someone will lose a document (see ZA-4)

If "mock" means the first one: yes, and it is a good idea — a Fluent mock is how we validate the flow in §10 without committing to an engine. It cannot be shipped as the editor, because a mock has no save path and therefore no revision.

4.1.7.3 Rules

- **ZA-1** No AGPL code enters the VSIX or the office pack unless R2 (commercial licence) is in place. Enforced by the §4.1.5 build gate.
- **ZA-2** Under R1 we do not fork or patch ONLYOFFICE. Configuration, JWT, and an iframe only. If we ever patch it, we publish the patch — the cheapest way to stay clean is to never touch their source.
- **ZA-3** If we *ship* the Document Server image to a customer, we are conveying it and must provide the corresponding source. Simplest compliance: the customer pulls `onlyoffice/documentserver` themselves; we ship only the URL and the secret.
- **ZA-4** No surface ever presents an editable-looking document without a working save path. A placeholder that accepts keystrokes and discards them is a data-loss bug, not a mock.
- **ZA-5** Import/convert never destroys the source: the original `.docx` stays as a revision, and the converted asset records `derived-from`.
- **ZA-6** No ONLYOFFICE name or logo in our UI beyond the attribution the licence requires. "Office document" is fine; "ONLYOFFICE" as our feature name is not.

4.1.7.4 Decision (recorded): R4, with R1 kept as the escape hatch

Corpus fact: **a few dozen existing `.docx`, edited occasionally** — not weekly round-trips. That does not justify a 300 MB dependency, a licence purchase, or a service to operate.

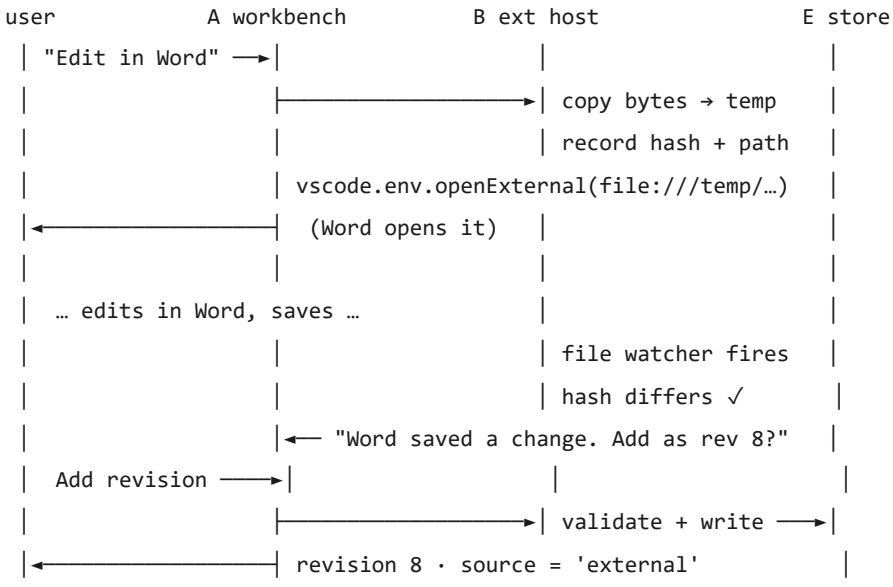
Item	Decision
Authoring	Fluent + Tiptap (MIT)
Viewing an imported `.docx`	`docx-preview` (MIT) in the webview, read-only
Editing an imported `.docx`	**Convert-on-import** (R4), lossy, opt-in, with a warning
Export	`.docx` generated from our model via the MIT `docx` package
Faithful editing of a complex original	**F12** — edit it in Word and re-import (below)
ONLYOFFICE	**not in the shipped product**; R1 becomes available the day a host exists
AGPL exposure in what we ship	**zero**

Two consequences worth stating plainly:

1. **The Markdown/Fluent editor is now the primary authoring surface**, so the caret-and-selection work already done there is the main line of investment, not a side quest. §10.5's acceptance criteria become P1 criteria.
2. **Fidelity is preserved by not competing with Word.** For the few documents that must look exactly right, the user edits in Word; we are the version store around it. That is cheaper and more honest than a lossy converter pretending otherwise.

F12 — Edit an original .docx in Word, bring it back as a revision

TEXT



Rules: the temp copy is the user's, never rewritten by us; a hash match produces no prompt and no revision (the round-trip law applies here too); the asset records `source: 'external'` so history shows it came from Word.

4.2 Composition inside a VS Code custom editor

TEXT

```
┌─ VS Code editor tab: "T3000 Manual.docx" ──────────────────────────────┐
│                                                                        │
│  ┌─ webview document (register B, our origin) ─────────────────────┐   │
│  │                                                                 │   │
│  │  ┌─ Fluent header (React) ──────────────────────────────────┐   │   │
│  │  │ [◀ Back]  T3000 Manual.docx   rev 7   ● unsaved   [Publish ▾] │   │   │
│  │  └──────────────────────────────────────────────────────────┘   │   │
│  │                                                                 │   │
│  │  ┌─ Fluent banner, only when degraded ──────────────────────┐   │   │
│  │  │ ⚠ Document server unreachable — read-only. [Retry] [Details]  │   │   │
│  │  └──────────────────────────────────────────────────────────┘   │   │
│  │                                                                 │   │
│  │  ┌─ #onlyoffice-host (100% × 100%) ─────────────────────────┐   │   │
│  │  │ ┌─ <iframe src="https://docs.temco.local/…"> ──────────┐ │   │   │
│  │  │ │                                                      │ │   │   │
│  │  │ │   ONLYOFFICE editor: ribbons, canvas, its own right panel │   │   │
│  │  │ │   (register C — we do not style anything in here)    │ │   │   │
│  │  │ │                                                      │ │   │   │
│  │  │ └──────────────────────────────────────────────────────┘ │   │   │
│  │  └──────────────────────────────────────────────────────────┘   │   │
│  │                                                                 │   │
│  │  ┌─ Fluent status bar (React) ──────────────────────────────┐   │   │
│  │  │ Page 4 of 41 · 100% · saved 14:02 · server ok            │   │   │
│  │  └──────────────────────────────────────────────────────────┘   │   │
│  └─────────────────────────────────────────────────────────────────┘   │
└────────────────────────────────────────────────────────────────────────┘
```

Only three things are ours: the header, the banner, the status bar. The document canvas is theirs. Do not attempt to inject CSS into the iframe — it is cross-origin; it will not work and it voids support.

4.3 The key — the only thing that causes corruption if you get it wrong

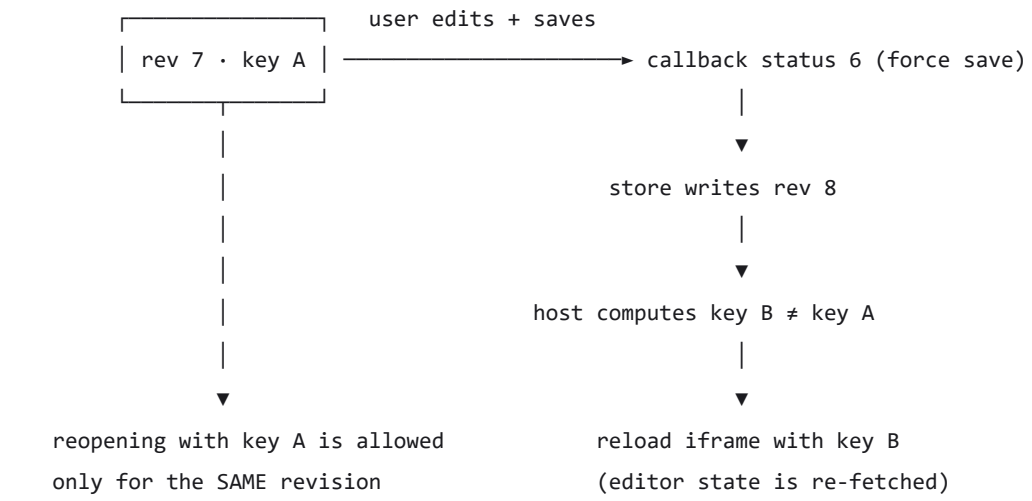
ONLYOFFICE caches a document by `key`. Same key means "same content", so it will reconnect the user to whatever it already has. Change the content without changing the key and users keep editing the old bytes and then save over the new ones.

TEXT

```
key = sha1( `${assetId}:${revision}:${optionsHash}` ).slice(0, 40)

assetId      AS-8812
revision     7                ← bumps on every accepted save callback
optionsHash  docx|word|edit   ← editing mode, language, permission set
```

TEXT



Invariants the code must enforce:

- **I1** — A key never outlives its revision. `revision` is part of the hash, so this is automatic.
- **I2** — A key is ≤ 128 chars and contains no `:`, `/`, or whitespace.
- **I3** — When two tabs open the same `(assetId, revision)`, they intentionally share a key: that is co-editing, and it is a feature. When the revision bumps, both are told to reload (§4.7 `onOutdatedVersion`).
- **I4** — Never derive the key from a timestamp or a random value. Doing so breaks co-editing and orphans the server's cache.

4.4 The content URL — reachable by the server, dead in 15 minutes *(http transport only)*

TEXT

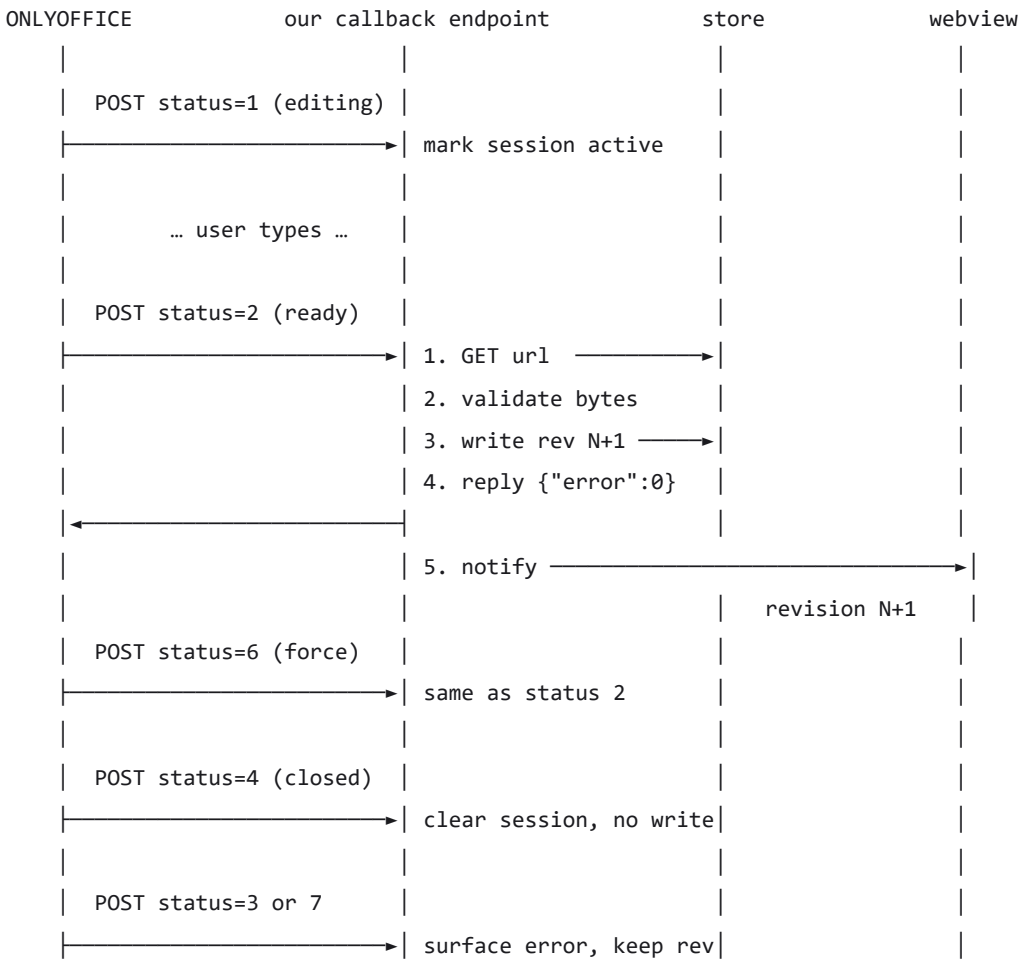
```
https://platform.temco.local/api/docs/content/AS-8812/7
    ?exp=1735660921
    &ticket=c7f3…            ← single-use, bound to asset+rev
    &sig=HMAC-SHA256(exp|ticket|asset|rev, CONTENT_SECRET)
```

Rules:

- The handler checks `exp`, checks the signature, consumes the ticket, then **streams the blob**. No session, no cookie, no `Authorization` header — the document server sends none of those.
- `Content-Type` must match the extension; the path must end in `.docx` / `.xlsx` / `.pptx` / `.pdf` / `.md` **or** `fileType` in the config must be set. Both is safer.
- Range requests must be supported (`Accept-Ranges: bytes`) — the server probes.
- The secret is per-deployment, stored in `SecretStorage`, never in settings JSON.

4.5 The save callback — status machine and the 10-second rule *(http transport only)*

TEXT



Status	Meaning	Our action
1	document being edited	record session start; no write
2	ready for saving, content changed	download, validate, write revision, reply `{"error":0}`
3	saving error	keep the old revision, raise a Fluent toast, reply `{"error":0}` so the server stops retrying
4	closed, no changes	clear session; **do not** write a revision
6	force save (autosave / version bump)	identical to 2
7	force-save error	as 3

The 10-second rule (T5). The HTTP GET of `url` must start before the callback response is written, and the response must not wait for the store write. Implement as: `await fetchBytes(url)` → reply → `await store.write(...)`. Reversing that order produces sporadic "document could not be saved" in the editor even though our store succeeded.

Validation before write (all four, or reject with status 3 semantics):

1. Non-empty body, and `Content-Length` under the asset-type limit.
2. Declared extension matches the opened format.
3. For Office types, the body starts with the correct zip magic `PK\x03\x04` (docx/xlsx/pptx are zips). A 200-with-HTML error page from a proxy is common; this catches it.
4. `assetId` and `revision` in the query match the session ticket; a replayed callback must be rejected.

4.6 Message protocol between extension host (2) and webview (3)

Typed, versioned, no free-form strings. Both directions are discriminated unions; unknown types are logged and dropped, never thrown.

TYPESCRIPT

```
// host → webview
type HostToContent =
  | { t: 'init'; asset: AssetView; server: ServerState; config: OfficeConfig | null }
  | { t: 'server'; state: ServerState }
  | { t: 'saved'; revision: number; at: string }
  | { t: 'reload'; revision: number; reason: 'outdated' | 'forced' }
  | { t: 'error'; code: ErrorCode; message: string; detail?: string };

// webview → host
type ContentToHost =
  | { t: 'ready' }
  | { t: 'forceSave' }
  | { t: 'requestReload' }
  | { t: 'openExternal'; href: string }     // host validates the URL scheme
  | { t: 'log'; level: 'warn' | 'error'; message: string };
```

`OfficeConfig` is the already-signed ONLYOFFICE config. **The JWT is signed in the extension host** and never in the webview: the secret must not enter a browser context.

4.7 The config builder

TYPESCRIPT

```typescript
// extensions/temco-docs/src/office/config.ts
interface OfficeRequest {
	asset: AssetRecord;
	revision: number;
	mode: 'edit' | 'view';
	user: { id: string; name: string };
	endpoints: { contentUrl: string; callbackUrl: string };
	theme: 'light' | 'dark' | 'contrast';
}

export function buildOfficeConfig(r: OfficeRequest, secret: string): SignedOfficeConfig {
	const fileType = r.asset.format;                          // 'docx' | 'xlsx' | …
	const config = {
		documentType: DOCUMENT_TYPE[fileType],                // 'word' | 'cell' | 'slide'
		type: 'desktop',
		document: {
			fileType,
			key: documentKey(r.asset.id, r.revision, `${fileType}|${r.mode}`),
			title: r.asset.title,
			url: r.endpoints.contentUrl,
			permissions: {
				edit: r.mode === 'edit',
				download: true,
				print: true,
				comment: true,
				review: true,
				copy: true,
			},
		},
		editorConfig: {
			mode: r.mode,
			lang: 'en',
			callbackUrl: r.endpoints.callbackUrl,
			user: r.user,
			customization: {
				uiTheme: r.theme === 'dark' ? 'theme-dark' : r.theme === 'contras
				compactHeader: true,
				compactToolbar: true,
				toolbarNoTabs: false,
				hideRightMenu: true,
				hideChat: true,
				help: false,
				about: false,
				feedback: false,
				autosave: true,
```

```
                    forcesave: true,
                },
                coEditing: { mode: 'fast', change: true },
            },
            width: '100%',
            height: '100%',
            events: { onDocumentReady: true, onError: true, onRequestClose: true, onOutdatedVe
        };
        return { ...config, token: signJwt(config, secret) };
}
```

`DOCUMENT_TYPE`: `docx|doc|odt|rtf|txt|md → 'word'`, `xlsx|xls|ods|csv → 'cell'`, `pptx|ppt|odp → 'slide'`, `pdf → 'pdf'`.

4.8 Format matrix — who edits what

Format	Editor host	Why
`.docx`, `.doc`, `.odt`, `.rtf`, `.txt`	**ONLYOFFICE word** (R1/R2/R3) or **convert-on-import** (R4, §4.1.7)	Fidelity, tracked changes, comments, page layout — or a lossy but AGPL-free path
`.xlsx`, `.xls`, `.ods`, `.csv`	**ONLYOFFICE cell**	Formulas, pivots, formatting
`.pptx`, `.ppt`, `.odp`	**ONLYOFFICE slide**	Master layouts
`.pdf`	**ONLYOFFICE pdf** (view)	Signatures later; never round-trip
`.md`, `.mdx`	**Both — per asset, remembered** (§4.8.1)	Default is the Fluent editor (50 ms open, frontmatter safe, offline). ONLYOFFICE when the asset is marked `office`.
`.json`, `.yaml`, `.xml`, `.bacnet`, `.ini`	**VS Code text editor**	Already perfect; leave it
`.drawio`	**Diagram editor** (phase 3)	Remote iframe, same host pattern
`.md` **preview**	Fluent Markdown editor, preview pane	Same surface
images	VS Code image preview	Already perfect

4.8.1 Per-asset host memory

Explicit choice beats a global preference: the same `.md` file may be prose for one author and tabular for another, and nobody should have to remember a setting.

TEXT

```
resolve(asset, explicitOverride?)
  │
  ├─ 1. explicitOverride          ← "Open With…" right now
  ├─ 2. asset.preferredHost       ← remembered from the last explicit choice
  ├─ 3. temco.docs.defaultHost    ← user setting ('auto' = skip to 4)
  └─ 4. registry priority         ← the table in §4.11
```

TYPESCRIPT

```typescript
// field on AssetRecord — see §6 for the full record
preferredHost?: 'office' | 'markdown' | 'text';
```

Rules:

- **H1** Choosing a host in *Open With…* writes `preferredHost` on the asset, so the next open is the same — this is the flow the user asked for ("when you choose … at the default md editor to change").
- **H2** `preferredHost` is metadata, not content. Changing it writes **no revision** and does not dirty the tab.
- **H3** `Open With… → Markdown editor (default)` clears the field rather than storing `'markdown'`.
- **H4** If the remembered host cannot serve the current format (e.g. `.xlsx` marked `markdown`), fall through to rule 4 and log one warning — never fail to open.
- **H5** The tree shows a small marker on assets whose host differs from the format default, so a `.md` that opens in ONLYOFFICE is not a mystery.

The cost is honest and accepted: **two engines to keep tested for `.md`**. §14.2 adds a matrix test that opens the same fixture through both hosts and asserts the round-trip law holds for each.

> **On Markdown and ONLYOFFICE.** ONLYOFFICE Docs 9.0 added native `.md` editing, and **the decision is to support both**, chosen per asset and remembered (§4.8.1). Fluent remains the *default* for four measurable reasons: frontmatter and non-standard fences do not round-trip through a document model; a 2 KB file pays a full editor session; there is no cursor/selection API we can drive for diagnostics; and offline authoring stops working. The escape hatch is real, not decorative: `Open With… → Office` on any markdown asset, then the choice sticks. If a future ONLYOFFICE release makes `.md` a first-class text round trip, the default can change — this row, not the architecture.

4.9 Deployment and licensing — decide before writing code

Option	Setup	Use for	Cost
Intranet Document Server (recommended)	One Docker host at whatever address the team can resolve (internal DNS, a VM IP, or a cloud URL — `docs.temco.local` is only a placeholder), TLS, JWT secret per environment	Production. All clients, one address, one co-editing session	Community edition is free with concurrency limits; white-label/branding needs a commercial licence
`localhost` Document Server	`docker run -i -t -d -p 8080:80 onlyoffice/documentserver` on each machine; the callback must then be reachable at `host.docker.internal`	Development, and single-user offline work	Docker required per machine; 10–20 s cold start; **no co-editing and no shared review**, because each user has an isolated server
ONLYOFFICE DocSpace / cloud	Their hosted API	If we never want to run servers	subscription
Embedded in the app (Topology B, §4.1.1)	Ship static `web-apps` + `x2t.wasm` as extension resources; no server at all	When policy forbids running a service, or offline is required	~200–400 MB download; **no co-editing**; AGPLv3 redistribution (fine internally, needs a commercial licence if the product is distributed); we own the build
Bundle Desktop Editors	A separate Electron app; cannot be embedded as a component	—	—

Two configuration values decide whether it works at all: `TEMCO_OFFICE_SERVER_URL` (a `https://` origin, no trailing slash) and `TEMCO_OFFICE_JWT_SECRET` (in `SecretStorage`). A missing or mismatched secret means every editor open fails with ONLYOFFICE's generic "The document security token is not correctly formed" — surface it in diagnostics (§11.9) with the words "*JWT secret mismatch between platform and document server*" so the cause is one search away.

4.10 Degraded states — never a blank frame

TEXT

```
server state    what the user sees                                   what still works
──────────────  ──────────────────────────────────────────────────   ──────────────────
unknown         skeleton · "Connecting to document server…"          tree, other assets
reachable       the editor                                           everything
unreachable     MessageBar · "Read-only — server unreachable."        download, history,
                [Retry] [Health check]                              publish existing rev
auth-failed     "Token mismatch. An administrator must check the      download, history
                JWT secret." + Diagnostics link, no Retry loop
too-old         "This needs ONLYOFFICE 9.0+ (server is X)."           download
server-error    "Document server returned 500." + correlation id      download
no-secret       "Document server is not configured."                  all non-Office
```

Health probe: `GET {server}/healthcheck` with a **2 s** timeout, cached for 30 s, and **never** on the tree-loading path.

In embedded mode (Topology B, §4.1.1) there is no probe to run. It is replaced by a **capability check** — `x2t.wasm` loaded, fonts present, `SharedArrayBuffer` available (chromium needs cross-origin isolation for the threaded build) — and the same states apply: `unknown` while loading, `ready`, `unavailable` with the banner. Failure text names the missing piece rather than "server unreachable".

4.11 The editor host registry

One table decides everything. A new format is a new row, not a new code path.

```typescript
interface EditorHost {
        readonly id: 'office' | 'markdown' | 'text' | 'diagram' | 'pdf';
        readonly priority: number;                      // higher wins a tie
        supports(asset: AssetRecord): boolean;
        readonly kind: 'remote-iframe' | 'webview' | 'vscode-native';
        open(asset: AssetRecord, ctx: OpenContext): Promise<void>;
}

// resolution order (§4.8.1): explicit Open With → asset.preferredHost → setting → highest priority
```

Registry contents at v1:

id	kind	matches	surface
`office`	remote-iframe	docx/xlsx/pptx/pdf/odt/ods/odp/csv	`temcoDocs.office` custom editor
`markdown`	webview	md/mdx	`temcoDocs.markdown` custom editor
`text`	vscode-native	everything else, and any explicit "Open as text"	VS Code text editor
`diagram`	remote-iframe	drawio	phase 3
`pdf`	remote-iframe, view-only	pdf	`temcoDocs.office`

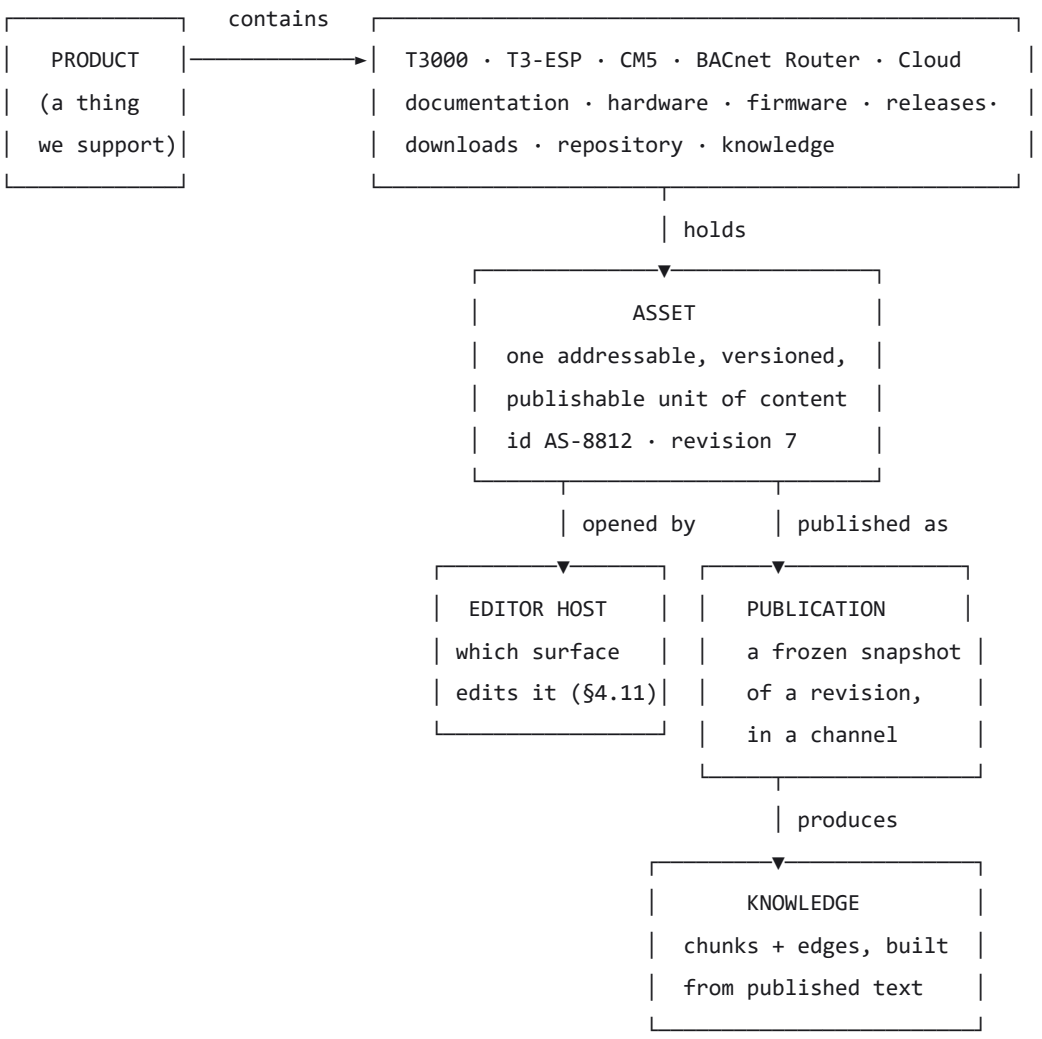
The "Open With…" entry on an asset's context menu lists these by display name, so a `.md` can be forced into Office and a `.docx` can always be forced into the text editor when a document is corrupt.

---

5. The five core concepts

Nothing else. If a requirement introduces a sixth noun, it is one of these five wearing a hat.

TEXT



Concept	One sentence	Never
Product	A thing we build and support for years	A folder
Asset	One addressable, versioned, publishable unit	A path on disk
Editor Host	The surface that opens an asset	A file extension check scattered in UI code
Publication	A frozen revision shipped to a channel	"Publish" that mutates in place
Knowledge	Derived index over published content	A hand-maintained second copy of the docs

5.1 The tree is the product, rendered by VS Code

TEXT

```
T3000                          ← Product (register A: VS Code TreeView)
  Documentation                ← Area
    Installation               ← Document asset
    Commissioning              ← Document asset
    Troubleshooting/
      No comms with router     ← Document asset
  Hardware
    Inputs (12)                ← Asset collection, opens a Fluent table
    Outputs (8)
    Variables (240)
  Firmware
    13.0 · Latest
    12.9 · LTS
  Releases
    13.0 · 2026-09
  ⬇ Downloads
```

Deterministic derivation, never hand-curated: `Product → areas(assetKind) → assets(area, parent)`. The provider is a `TreeDataProvider` in the extension host; it renders in register A and must never contain Fluent.

---

6. Domain model

TYPESCRIPT

```typescript
type AssetId = string;      // 'AS-8812'
type Revision = number;     // 1..n, monotonic, never reused

type ProductId = string;    // 'T3000'

type AssetKind =
        | 'document'      // docx/md/odt — the documentation body
        | 'spreadsheet'   // xlsx/csv
        | 'presentation'  // pptx
        | 'diagram'       // drawio/svg
        | 'pdf'
        | 'firmware'      // binary + version metadata
        | 'repository'    // link to a git remote
        | 'device'        // a physical controller model
        | 'input' | 'output' | 'variable' | 'program' | 'schedule' | 'graphic'
        | 'template'      // a starting point for a new asset
        | 'release'       // a named bundle of other assets
        | 'download';     // a file offered to customers

type Format =
        | 'docx' | 'doc' | 'odt' | 'rtf' | 'txt' | 'md' | 'mdx'
        | 'xlsx' | 'xls' | 'ods' | 'csv'
        | 'pptx' | 'ppt' | 'odp'
        | 'pdf' | 'drawio' | 'svg' | 'png' | 'jpg'
        | 'json' | 'yaml' | 'xml' | 'bin' | 'zip';

interface AssetRecord {
        id: AssetId;
        productId: ProductId;
        kind: AssetKind;
        format: Format;
        title: string;
        slug: string;                    // stable within (productId, kind, parentId)
        parentId: AssetId | null;        // tree shape for documentation
        area: string;                    // 'guides' | 'tutorials' | 'api' | … (§6.2)
        revision: Revision;              // current
        byteSize: number;
        updatedAt: string;
        updatedBy: string;
        /** Set only when the user picked a non-default host through Open With… (§4.8.1). */
        preferredHost?: 'office' | 'markdown' | 'text';
        /** Set when a publishable revision exists; null when never published. */
        publishedRevision: Revision | null;
}
```

```
interface RevisionRecord {
        assetId: AssetId;
        revision: Revision;
        bytes: Uint8Array;
        hash: string;                    // sha256 of bytes
        createdAt: string;
        createdBy: string;
        source: 'editor' | 'import' | 'restore' | 'template';
        message: string;                 // "Saved from Word editor", "Restored rev 5"
}
```

6.1 Invariants the store enforces

- **D1** — `revision` increases by exactly 1 per accepted save. Concurrent saves that would produce the same number fail with `ConflictError`.
- **D2** — Bytes are immutable once written. "Editing" a revision means appending a new one.
- **D3** — `publishedRevision` is always ≤ `revision`.
- **D4** — Deleting an asset deletes its revisions and removes it from every publication. It never deletes a published revision still referenced by a live publication.
- **D5** — `slug` is unique within `(productId, kind, parentId)` and, once a publication references it, changing it requires a redirect entry.

6.2 Areas — the documentation taxonomy

Mirrors MSDN, because the audience already knows it:

area	Holds	Example
`guides`	Task-oriented procedures	Commissioning a T3000
`tutorials`	Learning sequences	Your first BACnet schedule
`architecture`	Why the system is shaped this way	Polling model
`api`	Reference, one page per symbol	`SetOutput()`
`release-notes`	One page per version	13.0
`troubleshooting`	Symptom → cause → fix	No comms with router
`standards`	Internal conventions	Naming
`references`	Tables, pinouts, registers	Modbus register map

7. Fluent UI v9 inside VS Code

7.1 Pins and placement

Package	Version	Where
`@fluentui/react-components`	`^9.x` (pin exact, record it in `temco-ui/package.json`)	**`extensions/temco-ui`** — the only package that imports Fluent
`@fluentui/react-icons`	matching	`temco-ui`
`react` / `react-dom`	19.x (already in `temco-docs`)	`temco-docs`, `temco-ui`

`temco-ui` is currently an empty stub (no dependencies, no views). It becomes the design system package:

```
TEXT
```

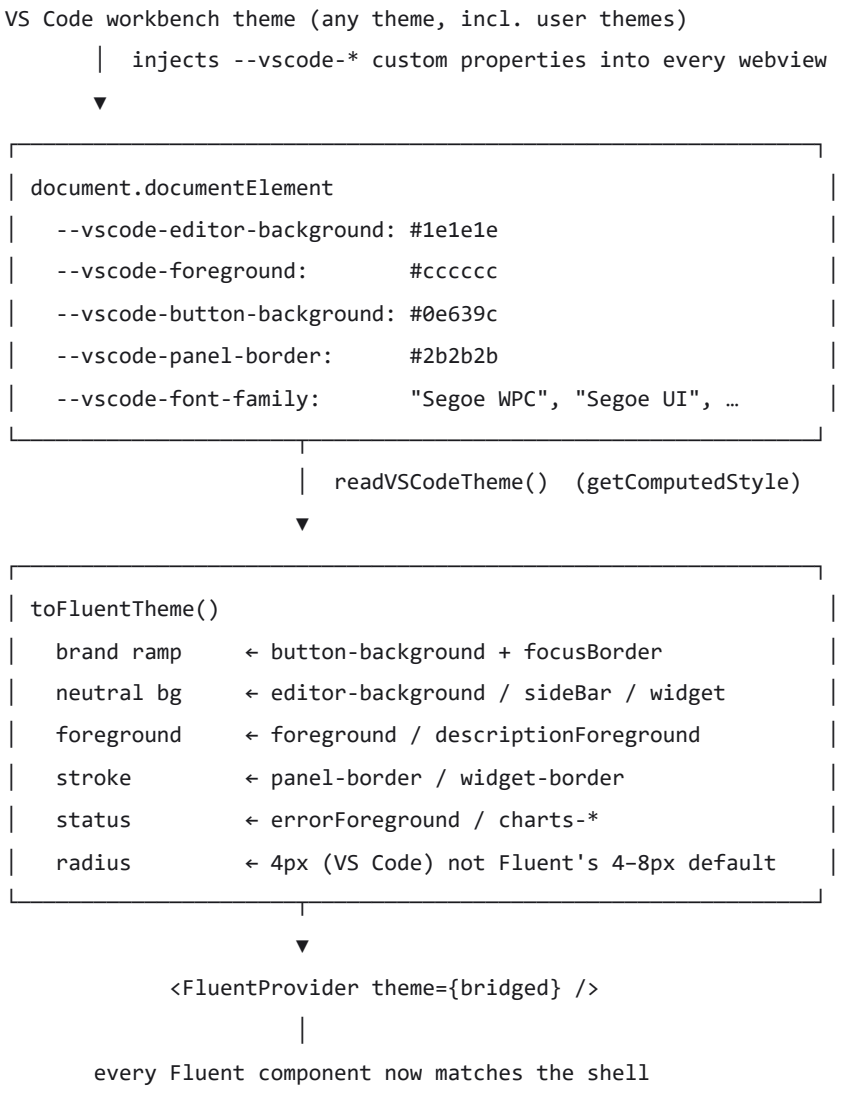
```
extensions/temco-ui/
  src/
    index.ts                ← the ONLY public entry point
    provider/TemcoProvider.tsx    ← FluentProvider + theme bridge + density
    theme/readVSCodeTheme.ts      ← §7.2
    theme/toFluentTheme.ts        ← §7.2
    density/compact.ts            ← §7.4
    components/                   ← wrappers, not replacements
    patterns/                     ← EmptyState, ErrorState, StatusBar, DataTable
```

Import rule (lint-enforced): `@fluentui/*` may appear only in `extensions/temco-ui/src/**`. Every other extension imports `temco-ui`. This keeps one Fluent version in the bundle and one place to swap it.

7.2 The theme bridge — how Fluent stops looking foreign

VS Code publishes its colours as CSS custom properties (`--vscode-*`) on `:root` in every webview. Fluent publishes its own tokens in a JS theme object. The bridge reads the former and writes the latter.

TEXT


VS Code workbench theme (any theme, incl. user themes)
injects --vscode-* custom properties into every webview
document.documentElement
--vscode-editor-background: #1e1e1e
--vscode-foreground: #cccccc
--vscode-button-background: #0e639c
--vscode-panel-border: #2b2b2b
--vscode-font-family: "Segoe WPC", "Segoe UI", …
readVSCodeTheme() (getComputedStyle)
toFluentTheme()
brand ramp ← button-background + focusBorder
neutral bg ← editor-background / sideBar / widget
foreground ← foreground / descriptionForeground
stroke ← panel-border / widget-border
status ← errorForeground / charts-*
radius ← 4px (VS Code) not Fluent's 4–8px default
<FluentProvider theme={bridged} />
every Fluent component now matches the shell

TYPESCRIPT

```typescript
// extensions/temco-ui/src/theme/toFluentTheme.ts  (shape, not final numbers)
import { createLightTheme, createDarkTheme, type BrandVariants, type Theme } from '@fluentui/react-

function read(name: string, fallback: string): string {
        const v = getComputedStyle(document.documentElement).getPropertyValue(name).trim();
        return v === '' ? fallback : v;
}

export function brandRamp(): BrandVariants {
        const base = read('--vscode-button-background', '#0e639c');
        // ramp[10..160]; index 80 is the token Fluent uses for the accent
        return { 10: shade(base, -0.55), /* … */ 80: base, /* … */ 160: tint(base, 0.6) } as BrandV
}

export function toFluentTheme(kind: 'light' | 'dark' | 'hc'): Theme {
        const brand = brandRamp();
        const theme = kind === 'dark' || kind === 'hc' ? createDarkTheme(brand) : createLightTheme(
        return {
                ...theme,
                fontFamilyBase: read('--vscode-font-family', 'Segoe UI'),
                fontSizeBase300: read('--vscode-font-size', '13px'),
                borderRadiusMedium: '4px',
                colorNeutralBackground1: read('--vscode-editor-background', theme.colorNeutralBack
                colorNeutralBackground2: read('--vscode-sideBar-background', theme.colorNeutralBac
                colorNeutralForeground1: read('--vscode-foreground', theme.colorNeutralForeground1
                colorNeutralForeground3: read('--vscode-descriptionForeground', theme.colorNeutral
                colorNeutralStroke1: read('--vscode-panel-border', theme.colorNeutralStroke1),
                colorPaletteRedForeground1: read('--vscode-errorForeground', theme.colorPaletteRed
        };
}
```

Re-bridge when the theme changes. VS Code sets `document.body.dataset.vscodeThemeKind` (`vscode-light` | `vscode-dark` | `vscode-high-contrast` | `vscode-high-contrast-light`) and mutates `--vscode-*`. One `MutationObserver` on `body[data-vscode-theme-kind]` plus a one-frame settle is enough; debounce to 50 ms because a theme switch rewrites hundreds of variables.

Do not use `createLightTheme` / `createDarkTheme` alone. Unbridged Fluent is immediately recognisable as "a web page in a panel": different greys, different corner radius, different accent.

7.3 What we never do with Fluent

Forbidden	Why	Instead
`FluentProvider` wrapping shell DOM	Shell is not React	Provider is the root of each webview's React tree only
Fluent `Tree` for the side bar	Loses native accessibility, drag/drop, theming	VS Code `TreeView`
Fluent `Menu` for the tab context menu	That menu is owned by the shell	`menus` contribution
Fluent `Dialog` for a modal that blocks the whole window	Two modal systems, focus traps fight	VS Code command with `showQuickPick` / `showInputBox`, or a Fluent dialog **inside our webview only**
Fluent `Persona` with a photo	No avatar service; broken images	Initials chip
Fluent's default `fontFamilyBase`	Segoe UI ≠ VS Code's font stack	Bridged from `--vscode-font-family`

7.4 Density

VS Code rows are 22 px. Fluent defaults to 32–40 px. In `TemcoProvider`, apply a compact density for editor chrome and normal density for forms:

TYPESCRIPT

```
// toolbar / status bar / property lists
<FluentProvider theme={theme} style={{ fontSize: '12px' }}>
        {/* size="small" on every input, Button, Dropdown, Tag, MenuButton */}
</FluentProvider>
```

Rules: toolbar controls `size="small"`; form controls `size="medium"`; all rows **22 px**; all icons **16 px**; no Fluent shadow on in-content surfaces (VS Code has no elevation inside editors except menus and tooltips).

7.5 CSP and Griffel

Fluent v9 uses Griffel, which injects `<style>` at runtime. The webview CSP must allow it, and the document server needs a frame slot:

HTML

```
<meta http-equiv="Content-Security-Policy" content="
  default-src 'none';
  img-src ${webview.cspSource} data: blob:;
  font-src ${webview.cspSource};
  style-src ${webview.cspSource} 'unsafe-inline';
  script-src ${webview.cspSource} 'unsafe-inline' ${officeOrigin};
  frame-src ${officeOrigin};
  connect-src ${webview.cspSource} ${officeOrigin};
  worker-src ${webview.cspSource} blob:;">
```

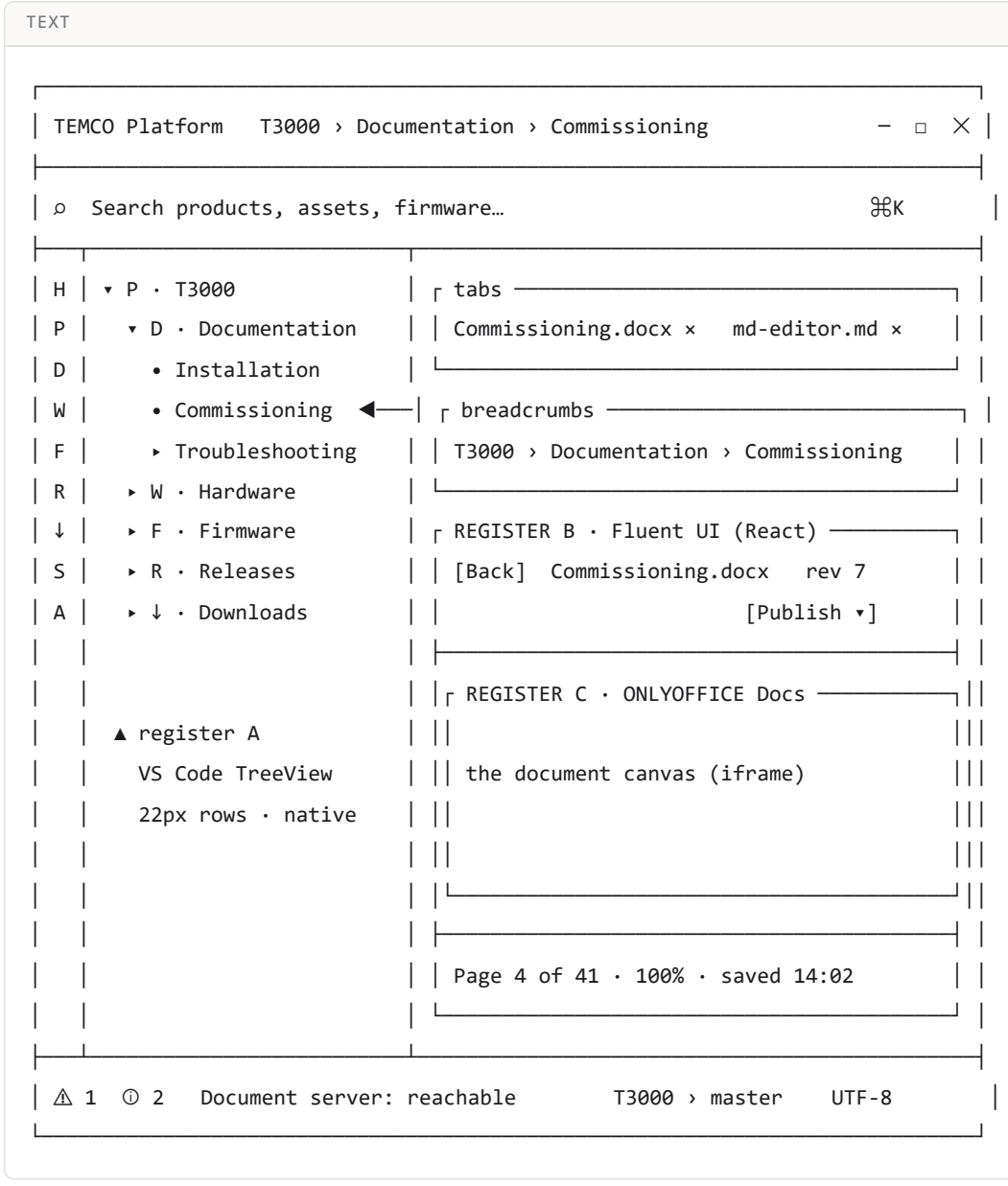
`${officeOrigin}` is the one configured document server origin — never `https:`. A `.docx` from a customer must not be able to reach a host we did not name.

---

8. The workbench

8.1 Full layout

TEXT



Activity bar legend: `H` Home · `P` Products · `D` Documentation · `W` Hardware · `F` Firmware · `R` Releases · `↓` Downloads · `S` Search · `A` Administration. Single-cell glyphs on purpose — emoji are double-width in some renderers and every border in this figure would drift.

The ONLYOFFICE status item is a **status bar item contributed by `temco-docs`** (§9.4) — register A, so it uses codicons and theme classes, not Fluent.

8.2 Activity bar — the whole platform, nine items

Order	Icon	id	Opens	Register
1	`home`	`temcoWorkbench.home`	Home webview view (recent, drafts, health)	B
2	`package`	`temcoWorkbench.products`	Product tree (the main explorer)	A tree
3	`book`	`temcoWorkbench.docs`	Documentation tree, filtered	A tree
4	`circuit-board`	`temcoWorkbench.hardware`	Hardware objects tree	A tree
5	`server`	`temcoWorkbench.firmware`	Firmware versions	A tree
6	`tag`	`temcoWorkbench.releases`	Releases	A tree
7	`cloud-download`	`temcoWorkbench.downloads`	Download catalogue	A tree
8	`search`	`workbench.view.search`	VS Code search, extended with an asset filter	A
9	`settings-gear`	`temcoWorkbench.admin`	Publication queue, diagnostics, health	B

Below the separator: AI (`sparkle`) from `temco-ai` , then Settings (built-in). **No custom activity bar rendering** — these are `viewsContainers.activitybar` contributions, so drag/drop, hide/show, and keyboard navigation come free.

8.3 Panel (bottom)

The existing VS Code panel hosts three contributed views: **Problems** (markdown lint), **Output → Temco Docs** (document server calls, one line per open/save), and **Publication log**. Nothing custom is drawn there.

---

9. From routes to VS Code constructs

Revision 1 had URLs. In a desktop host there are no URLs — there are **view ids, custom editor ids, commands, and a URI scheme**. This table is the contract; code must not invent a second way to reach a surface.

Concept (v1 route)	VS Code construct	id
`/` Home	webview view	`temcoWorkbench.home`
`/products`	tree view	`temcoWorkbench.products`
`/products/:id`	webview view, set by selection	`temcoWorkbench.product`
`/products/:id/docs`	tree view	`temcoWorkbench.docs`
`/doc/:assetId` (docx…)	**custom editor**	`temcoDocs.office`
`/doc/:assetId` (md)	**custom editor**	`temcoDocs.markdown`
asset properties	webview view in the *secondary* side bar	`temcoWorkbench.properties`
`/releases`	tree view	`temcoWorkbench.releases`
`/downloads`	tree view	`temcoWorkbench.downloads`
`/search`	built-in search + filter	`workbench.view.search`
`/review`	webview view	`temcoWorkbench.review`
`/publish`	webview view + command	`temcoWorkbench.publish`
`/admin`	webview view	`temcoWorkbench.admin`
`/ai`	webview view (`temco-ai`)	`temcoAi.chat`

9.1 The asset URI

Every asset needs an addressable identity so tabs restore, links work, and `Open With…` can be forced.

```text
temco-asset:/AS-8812?rev=7&host=office

scheme   temco-asset          registered by temco-docs; read-only, never on disk
path     /AS-8812             assetId
rev      revision (optional)  absent = latest
host     editor host id       absent = resolve via registry (§4.11)
```

Custom editors register for `viewType: 'temcoDocs.office'` and resolve through `CustomTextEditorProvider`-style `CustomEditorProvider` with `supportsMultipleEditorsPerDocument: false` (one editor column, principle §1.1).

9.2 Commands (25 for phase 1)

Command id	Title	Where
`temco.asset.open`	Open	tree item, `Enter`
`temco.asset.openWith`	Open With…	tree item context, `⌘⇧O`
`temco.asset.new`	New Asset…	tree view title
`temco.asset.newFromTemplate`	New From Template…	tree view title
`temco.asset.rename`	Rename	context, `F2`
`temco.asset.delete`	Delete…	context
`temco.asset.history`	Show History	context
`temco.asset.restoreRevision`	Restore Revision…	history
`temco.asset.download`	Download Copy…	context
`temco.asset.revealInTree`	Reveal in Tree	editor title
`temco.docs.forceSave`	Save Now	editor title, `⌘S`
`temco.docs.reload`	Reload From Server	editor title
`temco.docs.healthCheck`	Document Server Health Check	command palette
`temco.docs.diagnostics`	Show Document Server Diagnostics	command palette
`temco.publish.start`	Publish…	asset context, editor title
`temco.publish.queue`	Open Publication Queue	admin view, status bar
`temco.publish.cancel`	Cancel Publication…	queue item
`temco.product.add`	Add Product…	products view title
`temco.search.inAsset`	Find in Asset…	editor, `⌘F`
`temco.import.file`	Import Files…	tree view title
`temco.template.saveAs`	Save As Template…	asset context
`temco.docs.refresh`	Refresh	tree views
`temco.docs.signOutOffice`	Forget Document Server Session	diagnostics
`temco.ai.explainAsset`	Explain This Asset	editor title (`temco-ai`)
`temco.ai.findRelated`	Find Related Assets	context (`temco-ai`)

9.3 Settings

Key	Type	Default	Meaning
`temco.docs.serverUrl`	string	`""`	Document server origin, e.g. `https://docs.temco.local` (http transport only)
`temco.docs.transport`	enum	`embedded`	`embedded` | `http` | `auto` — `auto` uses http when `serverUrl` is set **and** reachable, else embedded
`temco.docs.officePackPath`	string	`""`	Override for the embedded office pack; empty = beside the app (§4.1.5)
`temco.docs.editorTheme`	enum	`auto`	`auto` | `light` | `dark` | `contrast` — forwarded to ONLYOFFICE `customization.uiTheme`
`temco.docs.autosaveSeconds`	number	`30`	0 disables; only affects ONLYOFFICE `autosave` hints
`temco.docs.markdown.engine`	enum	`fluent`	`fluent` | `office` — the §4.8 escape hatch
`temco.docs.defaultHost`	enum	`auto`	`auto` | `office` | `markdown` | `text`
`temco.docs.maxRevisionBytes`	number	`52428800`	Reject larger saves with a clear message

The **JWT secret is not a setting.** It lives in `SecretStorage` and is set by an administrator through `temco.docs.diagnostics`.

9.4 Status bar items

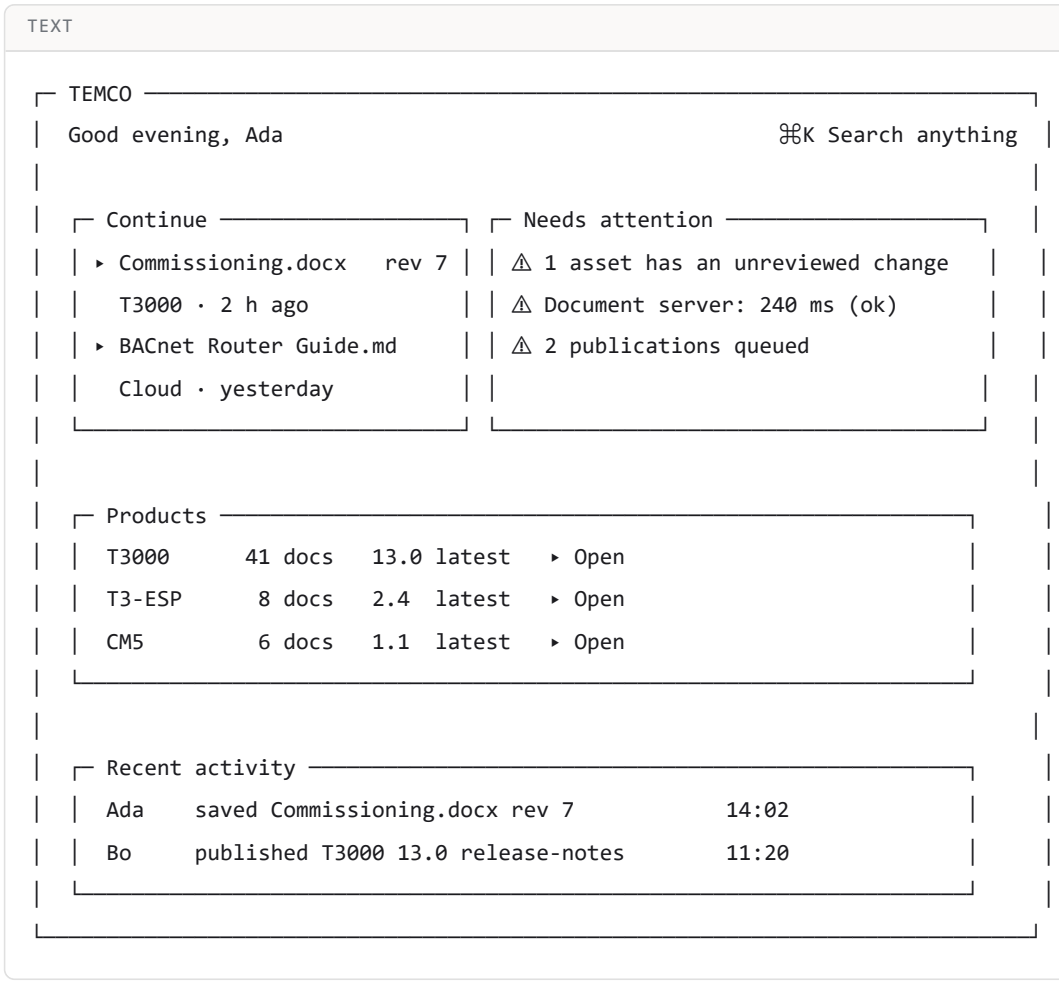
Item	Text	Click
Server	`$(plug) Document server` / `$(debug-disconnect) Document server` + tooltip with latency	`temco.docs.healthCheck`
Save	`$(cloud-upload) Saving…` → `$(check) Saved 14:02` — only while an Office editor is open	`temco.docs.forceSave`
Queue	`$(tag) 2 queued` — only when publications are pending	`temco.publish.queue`

10. Screens

Twelve surfaces. Each is either a **tree** (register A) or a **webview** (register B). None is drawn with a third stack.

10.1 Home — `temcoWorkbench.home` (webview)

TEXT



Fluent: `Text` (display), `Card`, `Badge`, `Button`, `MessageBar`, `Divider`, `Tag`. States: loading → 6 skeleton rows; empty (no products) → `EmptyState` with *Add Product*; server down → `MessageBar intent="warning"` above Products only.

Acceptance: first meaningful paint from a cold extension host under 600 ms; no document server call on this screen.

10.2 Product tree — `temcoWorkbench.products` (register A)

VS Code `TreeDataProvider`. Rows are 22 px, icons are codicons, selection drives the secondary side bar. Every row has a stable `id` so expansion survives refresh.

TEXT

```
▾ T3000                    product · 41 docs · firmware 13.0
    ▾ Documentation        area · guides (18 assets)
        Installation       docx · rev 7
        Commissioning      docx · rev 7
      ▸ Troubleshooting    area · 4 assets
    ▸ Hardware             12 inputs · 8 outputs · 240 variables
    ▸ Firmware             13.0 latest · 12.9 LTS
    ▸ Releases             3
    ▸ Downloads            14
▸ T3-ESP                   product · 8 docs
▸ CM5                      product · 6 docs
```

Selection drives the secondary side bar (§10.6); `Enter` opens the asset through F1.

Acceptance: expanding a product with 300 assets stays responsive (< 100 ms per expansion); no network call per row (batch-load children).

10.3 Product overview — `temcoWorkbench.product` (webview, secondary)

TEXT

```
┌─ T3000 ──────────────────────────────── [Open Docs] [New ▾] [Publish ▾] ─┐
│ 41 documents · 12 inputs · 8 outputs · firmware 13.0 · last published 09-14 │
├──────────────────────────────────────────────────────────────────────────┤
│ Tabs: Overview │ Documentation │ Hardware │ Firmware │ Releases │ Downloads  │
├──────────────────────────────────────────────────────────────────────────┤
│ Documentation by area          │ Recently changed                        │
│  guides            18          │  Commissioning.docx        rev 7  2 h   │
│  tutorials          6          │  Troubleshooting.md        rev 3  yesterday│
│  api               11          │                                         │
│  troubleshooting    4          │ Health                                  │
│  release-notes      2          │  ○ Document server     240 ms           │
└──────────────────────────────────────────────────────────────────────────┘
```

10.4 Office editor — `temcoDocs.office` (webview + iframe)

Drawn in full at §4.2. Acceptance criteria:

- **OE-1** Opening never marks the tab dirty and never writes a revision.
- **OE-2** Opening a second time with the same revision reuses the same key and joins the same co-editing session.
- **OE-3** With the server unreachable, the surface shows the read-only banner within 2 s and offers Download and History.
- **OE-4** A save produces exactly one new revision; the revision number in the header updates without reloading the tab.
- **OE-5** Closing with unsaved changes prompts once, and *Save* forces a save (§4.5 status 6) before closing.
- **OE-6** Keyboard: `⌘S` triggers `temco.docs.forceSave`; while the iframe has focus, the host must not steal any key except `⌘S` and `⌘W` (§12.3).

10.5 Markdown editor — `temcoDocs.markdown` (webview, Fluent)

TEXT

```
┌─ Commissioning.md ──────────────────── rev 7 · draft — [Preview] [Publish] ┐
│ B I U H1 H2 • 1. " </> 🔗 ▦ —  ≡  (Fluent Toolbar, size=small, 22px)        │
├──────────────────────────────────────────┬──────────────────────────────────┤
│ ## Commissioning the T3000               │  (preview pane, toggleable)      │
│                                          │  Commissioning the T3000         │
│ 1. Connect power to terminals 24 V.      │  ──────────────────────────      │
│ 2. Open the T3000 app and select…        │  1. Connect power…               │
│                                          │                                  │
│ | Input | Range |                        │                                  │
│ |-------|-------|                        │                                  │
│ | AI1   | 0–10V |                        │                                  │
├──────────────────────────────────────────┴──────────────────────────────────┤
│ Ln 12, Col 4 · 1,204 words · Markdown · saved 14:02                         │
└─────────────────────────────────────────────────────────────────────────────┘
```

Rules carried over from the incidents that produced this design:

- **MD-1** The document is never replaced while the editor is focused. Outside changes are queued and applied on blur.
- **MD-2** A message that does not change the text must not touch the document (no caret jump, no undo entry).
- **MD-3** Opening never dirties the tab.
- **MD-4** Paste of an image inserts and saves an attachment asset; paste of HTML is sanitised to the allowed schema.
- **MD-5** The context menu is ours (Fluent `Menu`), so a right-click inside a selection keeps the selection.

10.6 Properties — `temcoWorkbench.properties` (webview, secondary side bar)

TEXT

```
┌─ Properties ──────────────────────────┐
│ Title      [Commissioning.docx      ] │
│ Slug       [commissioning           ] │
│ Area       [guides                ▾ ] │
│ Product    [T3000                 ▾ ] │
│ Format     docx                 (read)│
│ Revision   7 · 41 KB · 2026-10-23 14:02│
│ Published  rev 6 → T3000 13.0        │
│ [Show History]  [Download Copy…]     │
└──────────────────────────────────────┘
```

10.7 Publish — `temcoWorkbench.publish` (webview)

TEXT

```
┌─ Publish T3000 13.0 ──────────────────────────────────────────────────┐
│ Channel  ( ) Internal  (•) Public         [Publish 3 assets]          │
│                                                                       │
│ ┌─ Check ─────────────────────────────────────────────────────────┐   │
│ │ ✓ All 3 anchors resolve                                         │   │
│ │ ✓ 1 code block language missing  → warning                      │   │
│ │ ✓ No asset is open in an editor                                 │   │
│ │ ✓ 2 assets already published at this revision (will be skipped) │   │
│ └─────────────────────────────────────────────────────────────────┘   │
│                                                                       │
│ Queue: Commissioning.docx · Troubleshooting.md · Release-notes.docx   │
└───────────────────────────────────────────────────────────────────────┘
```

10.8 Publication queue — `temcoWorkbench.publish` (webview, admin)

TEXT

```
┌──────────────────────┬─────┬──────────┬────────────┬─────────┬────────┬────────┐
│ Asset                │ Rev │ Channel  │ State      │ Started │ Time   │        │
├──────────────────────┼─────┼──────────┼────────────┼─────────┼────────┼────────┤
│ Commissioning.docx   │ 7   │ public   │ live       │ 14:02   │ 1.8 s  │ log    │
│ Troubleshooting.md   │ 3   │ public   │ building   │ 14:02   │ 0.4 s  │ log    │
│ Release-notes.docx   │ 9   │ internal │ failed     │ 13:58   │ 2.1 s  │ retry  │
└──────────────────────┴─────┴──────────┴────────────┴─────────┴────────┴────────┘
```

Columns: asset · revision · channel · state (`queued | building | live | failed`) · started · duration · actions (*Open log*, *Retry*). Failed rows keep their log and offer Retry; retry re-enqueues the **same** `assetId@revision`, never "latest".

10.9 Releases and Downloads — tree views (register A)

TEXT

```
Releases                                      Downloads
  13.0    2026-09-14  latest  verified          T3000-Setup.exe          48 MB
  12.9    2026-03-02  LTS     shield            T3000-Setup.zip         112 MB
  12.10   2025-11-20  beta    beaker            T3000-Firmware.bin        9 MB
  12.8    2025-06-11  EOL                       T3-ESP-Firmware.bin       2 MB
▸ T3-ESP                                      ▸ SDK
▸ CM5                                         ▸ Documentation packs
```

Rows show version, date, and a stability marker as a codicon (`$(verified)` latest, `$(shield)` LTS, `$(beaker)` beta). Opening a release opens a small Fluent detail view listing its assets; downloading writes through `vscode.workspace.fs` after a save dialog.

10.10 Search — built-in (register A)

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ asset:docx product:T3000 output                          [Aa] [.*] [ab]  │
├──────────────────────────────────────────────────────────────────────────┤
│ Commissioning.docx  ›  guides  ›  T3000                    4 matches     │
│   12   …connect power to terminals 24 V, then open the…                  │
│   31   …output calibration is described in chapter 4…                    │
│ Troubleshooting.docx  ›  troubleshooting  ›  T3000         1 match       │
│    8   …no output on terminal 5: check the schedule…                     │
├──────────────────────────────────────────────────────────────────────────┤
│ 5 results in 2 assets      filters: asset:docx product:T3000 output      │
└──────────────────────────────────────────────────────────────────────────┘
```

`workbench.view.search` with a contributed `search` context filter: `asset:docx product:T3000`. Do not build a second search UI; extend the one that already has regex, replace, history, and the tree.

10.11 Review — `temcoWorkbench.review` (webview)

TEXT

```
┌─ Review ─────────────────────────────────────────────────────────────────┐
│ Awaiting review (2)                          diff: rev 6 → rev 7         │
│  ▸ Commissioning.docx   Ada   2 h   [Approve] [Request changes]          │
├──────────────────────────────────────────────────────────────────────────┤
│ Text diff (for md/docx-as-text).  For Office files: open both revisions. │
└──────────────────────────────────────────────────────────────────────────┘
```

Honest limitation: a visual diff of `.docx` inside our own UI is out of scope for phase 1. The surface offers *Open rev 6* and *Open rev 7* side by side is forbidden (one editor column), so it offers **Open rev 6**, then **Open rev 7**, and records which the reviewer saw.

10.12 Diagnostics — `temcoWorkbench.admin` (webview)

A health matrix, the single place to debug a broken document server:

TEXT

```
┌─ Diagnostics ────────────────────────────────────────────────────────────┐
│ Server URL     https://docs.temco.local                    (configured)  │
│ JWT secret     •••••••• set 3 days ago                      (ok)         │
│ Health check   200 in 240 ms                                (ok)         │
│ Version        ONLYOFFICE Docs 9.0.4                                     │
│ Reachability   platform → server ✓      server → platform ✓              │
│ Last error     none                                                      │
│ [Run health check]  [Edit secret]  [Copy diagnostics JSON]               │
└──────────────────────────────────────────────────────────────────────────┘
```

Reachability runs a real probe: the platform asks the server to fetch a fixture URL (`/api/docs/ping/<token>`) and reports whether it saw the hit. This single row diagnoses T1, the failure that costs hours.

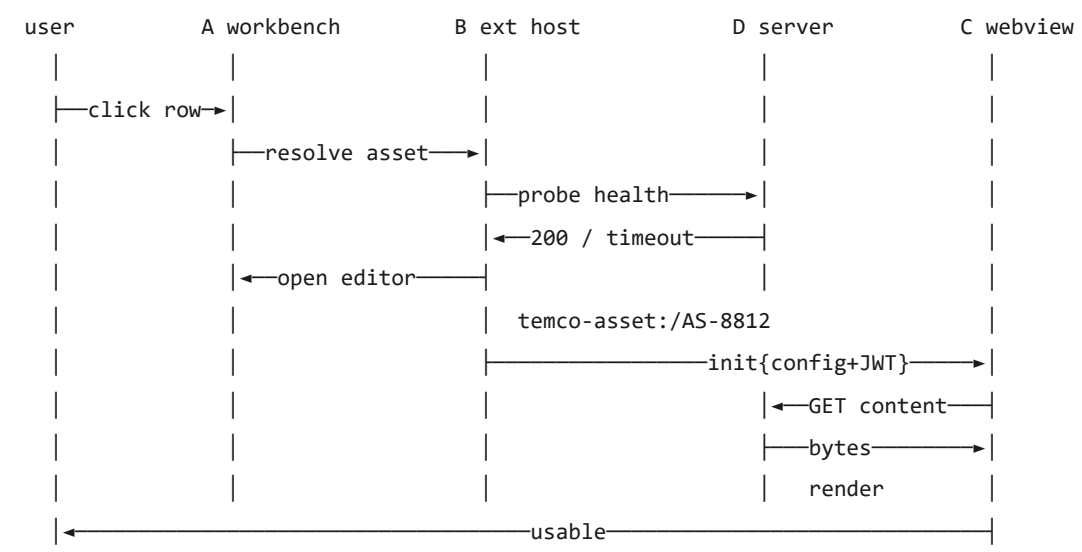
---

11. Flows

Each flow names its participants by plane: **A** workbench, **B** extension host, **C** webview, **D** document server, **E** store.

F1 — Open a Word document from the tree (the primary flow)

TEXT

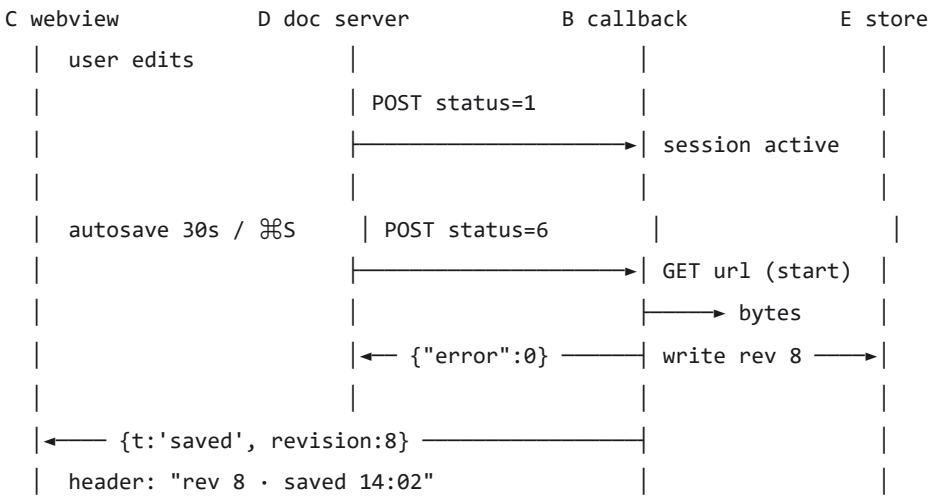


1. **click row** — the `TreeDataProvider` resolves the element to an `AssetId` ; no network yet.
2. **resolve asset** — read the `AssetRecord` from the store; `format: docx` selects the `office` host (§4.11).
3. **probe health** — `GET /healthcheck` , 2 s timeout, cached 30 s. On timeout jump to F5.
4. **open editor** — `vscode.commands.executeCommand('vscode.openWith', temcoAssetUri, 'temcoDocs.office')` .
5. **temco-asset:/AS-8812** — the tab opens immediately with a skeleton, before any network.
6. **init{config+JWT}** — the host mints a single-use ticket (§4.4), builds and **signs** the config (§4.7), and posts it to the webview.
7. **GET content** — the document server fetches the signed URL **itself**. The client never sends the bytes.
8. **bytes** — streamed from our content handler; range requests are supported.
9. **render** — `onDocumentReady` → the webview posts `{t:'ready'}` → the host clears the skeleton.
10. **usable** — the user can type; focus is now owned by the iframe (§12.2).

Failure branches: health timeout → banner, no iframe (§4.10). Content 403 because the ticket expired → `{t:'error', code:'content-expired'}` → one automatic retry with a fresh ticket, then banner.

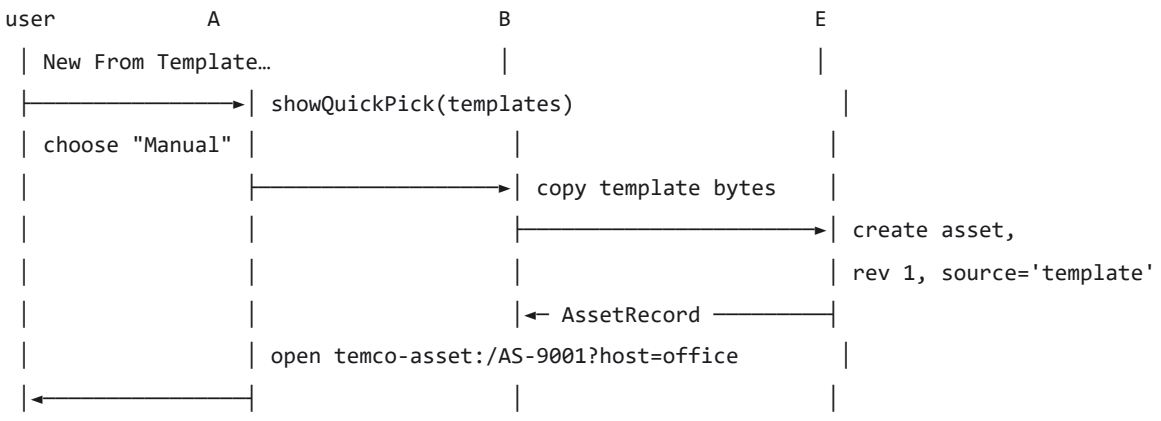
F2 — Edit, force save, revision bump

TEXT



F3 — New document from a template

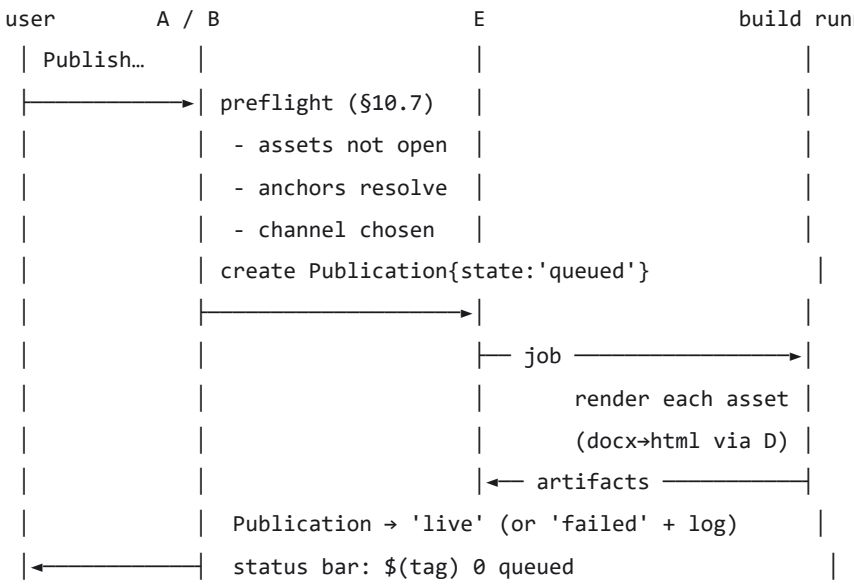
TEXT



Why copy instead of "Save As": ONLYOFFICE has no template API. A template is a plain asset marked `kind: 'template'`; instantiating is a byte copy plus a new asset row. This also means templates are versioned like everything else.

F4 — Publish a product version

TEXT



Publications are **frozen**: they store `assetId@revision` tuples. Publishing never reads "latest" at build time.

F5 — Document server unreachable

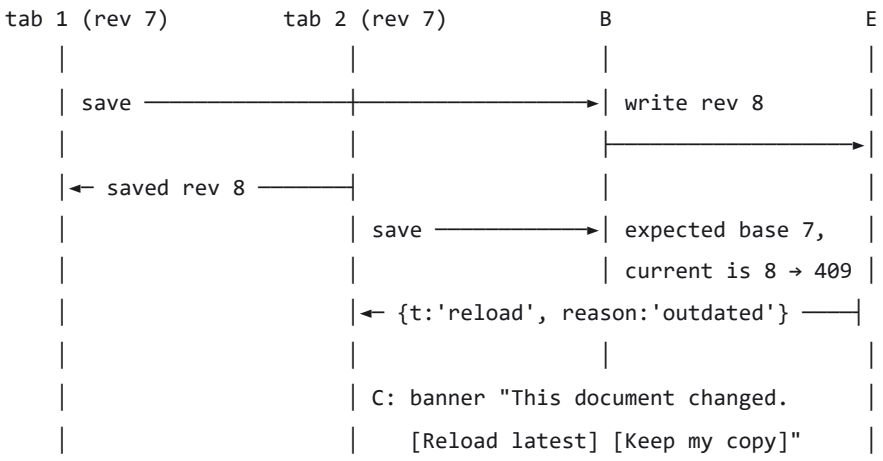
TEXT



Never: a blank iframe, an infinite spinner, or a silent fallback to the text editor (that would let a user edit a `.docx` as bytes and destroy it).

F6 — Two editors, one asset, revision conflict

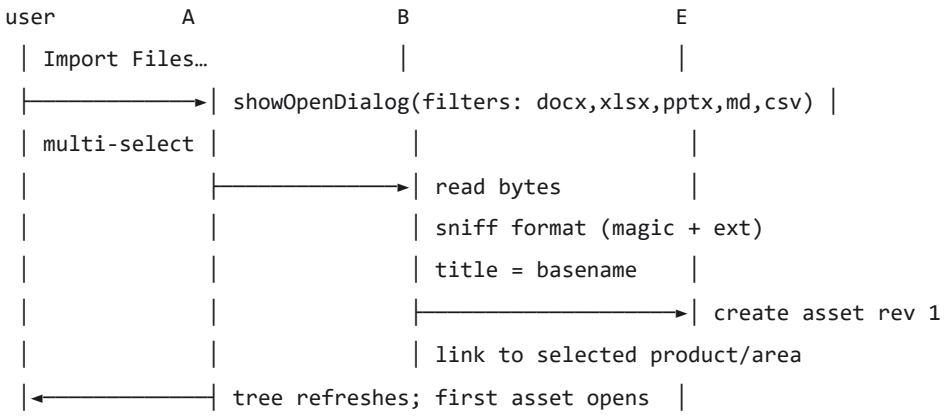
TEXT



Both revisions survive (D1, D2). "Keep my copy" writes the user's bytes as rev 9 with `message: 'from conflicting edit'`, so nothing is lost.

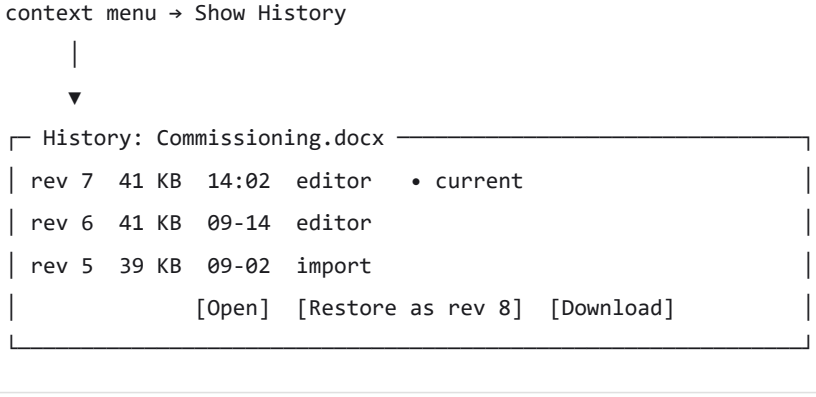
F7 — Import files

TEXT



F8 — History and restore

TEXT



Restore never rewinds; it copies old bytes forward as a new revision (D2).

F9 — Open With (forcing a host)

TEXT

```
context menu → Open With…  →  QuickPick, ordered by the registry
     │
     ▼
┌──────────────────────────────────────┐
│ Office (Word)          ● default     │
│ Markdown editor                      │
│ Text Editor                          │
└──────────────────────────────────────┘
     │ choose "Text Editor"
     ▼
temco-asset:/AS-8812?host=text  →  opens in the native text editor (read/write)
```

This is the escape hatch when a document is corrupt, and the mechanism the user asked for when they said *"when you choose … at the default md editor to change"*.

F10 — Search to open

TEXT

```
⌘K → type "commissioning"            (command center, register A)
     │
     ▼
results: assets · commands · settings · files
     │ Enter on an asset
     ▼
temco.asset.open → F1
```

F11 — Open and save with no server (Topology B)

TEXT

```
user          A workbench (Fluent)      B ext host + x2t.wasm          C editor iframe
  │             │                         │                              │
  ├──open──────►│                         │                              │
  │             ├──transport=embedded────►│                              │
  │             │                         │ store: GET .docx rev 7       │
  │             │                         │ x2t.wasm: docx → editor.bin  │
  │             │◄──init{editor.bin}──────┤                              │
  │             │                         ├─load iframe─────────────────►│
  │             │                         │ no network                   │
  │             │                         │◄──────────────────forceSave──┤
  │             │                         │ x2t.wasm: editor.bin → docx  │
  │             │                         │ validate + write rev 8       │
  │             │◄──────────saved rev 8─────────────────────────────────────┤
```

Same host protocol as F1/F2, so nothing above the transport changes. The differences are all in `B`: conversion runs in a Worker (~0.4–2 s depending on document size), there is no health probe, no ticket, no signed URL, and no callback POST. `forceSave` therefore means "convert now" rather than "ask the server to flush".

---

12. Keyboard, focus, accessibility

12.1 Shortcut ownership

Scope	Owner	Examples
Global chords	A (VS Code)	`⌘K`, `⌘⇧P`, `⌘⇧E`, `⌘W`, `⌘,`
Asset actions	A	`⌘S` → `temco.docs.forceSave`, `F2` rename, `⌘⇧O` open with
Inside our webview	B/Fluent	`⌘F` find-in-asset, `Esc` close popover, `Tab` traversal
Inside the document iframe	D (ONLYOFFICE)	Everything, including `⌘B`, `⌘F`, `Enter`

12.2 The focus contract (the rule that prevents the "typing goes somewhere else" class of bug)

TEXT

```
┌──────────────────────────────────────────────┐
│  A user is typing in a document              │
│  → B must not steal focus                    │
│  → B must not replace the document           │
│  → B must not re-render the surface          │
└──────────────────────────────────────────────┘
```

Implementation rules:

1. **FR-1** Any host→content update is queued while the content surface reports `focused: true`, and applied on `blur` (§10.5 MD-1).
2. **FR-2** An update that does not change content is dropped, not applied (§10.5 MD-2).
3. **FR-3** Never call `webview.postMessage` on a timer while the user is typing, except the save heartbeat, which carries no document bytes.
4. **FR-4** Never re-create the iframe element to "refresh" it. Change its `key` (React) only when the ONLYOFFICE key changes (§4.3).
5. **FR-5** When the iframe has focus, the host registers no `keydown` capture handler except for the allowlist in §12.3.

12.3 Routing `⌘S` while the iframe has focus

A cross-origin iframe swallows key events. VS Code's `keybindings` still fire because they are evaluated in the host document **before** the webview, but a keybinding that only makes sense inside our editor must be scoped:

JSONC

```jsonc
{
        "command": "temco.docs.forceSave",
        "key": "ctrl+s",
        "mac": "cmd+s",
        "when": "activeCustomEditorId == 'temcoDocs.office' || activeCustomEditorId == 'temcoDocs.m
}
```

Track iframe focus in the webview (`window.blur` + `document.activeElement === iframe`) and post `{t:'focus', focused}` so the host can enable iframe-only commands (e.g. `Enter` still belongs to the document, so it is never claimed).

12.4 Accessibility requirements

Requirement	Implementation
Every Fluent control has a label	`Field label={…}` ; no placeholder-only inputs
The header and status bar are reachable by keyboard	Plain DOM order; skip-link to the canvas
The iframe has a title	`title="Document editor: Commissioning.docx"`
Server state is announced	`MessageBar` in an `aria-live="polite"` region, once per state change
Colour is never the only signal	Save state uses icon + text (`● unsaved` / `✓ saved 14:02`)
High contrast	Bridge maps `vscode-high-contrast` → ONLYOFFICE `theme-contrast-dark` ; Fluent theme built with `createHighContrastTheme`
Focus visible in our chrome	Fluent's `:focus-visible` outline, never removed
Screen reader can tell registers apart	The webview root gets `aria-label="Document content"` , the side bar is native

13. Performance budgets

Budget	Target	Measured where
Tree first paint (warm)	< 300 ms	`performance.mark` in the provider
Any tree expansion (300 children)	< 100 ms	provider timer
Webview boot (React + Fluent, cold)	< 400 ms	`ready` message timestamp
Office editor: server probe	< 2 000 ms timeout, cached 30 s	health client
Office editor: open → `onDocumentReady` (warm)	< 4 000 ms	`onDocumentReady` event
Office editor: open → ready (cold server)	< 9 000 ms	same
Save callback: GET started	< 500 ms	callback handler
Save callback: revision visible in UI	< 1 500 ms	`saved` message
Webview bundle (each surface, gzip)	< 350 KB	build report
Fluent import: one copy only	verified by build	bundle analysis

Cold-start is dominated by the document server, not by us. Pre-warming on extension activation is allowed **only** after 3 s idle and **only** the `/healthcheck` probe — never a document load.

---

14. Testing

14.1 The pyramid, adapted to a two-register product

Level	What	Tool	Runs
Unit	key derivation, config builder, JWT signing, callback validation, snapshot rules, tree derivation	`node --test` against compiled `out/`	every commit
Contract	callback handler against recorded ONLYOFFICE payloads (status 1/2/3/4/6/7)	in-process HTTP	every commit
Webview render	each surface mounts, no throw, key elements present	jsdom (`check-webview.mts`)	every commit
Real-browser editing	typing, caret, selection, paste, undo on the Markdown editor	Playwright (`check-editor.mts`)	every commit
End-to-end	open a real `.docx` in a real VS Code window against a real document server	`validate-ui-scenario` scenario	nightly / on demand
Accessibility	keyboard-only pass, high contrast, screen-reader labels	manual checklist + axe on webviews	before each phase exit

14.2 Non-negotiable regression tests

1. **Key test** — same `(assetId, revision)` ⇒ same key; revision bump ⇒ different key; key matches `/^[A-Za-z0-9_-]{1,128}$/`.
2. **Round-trip law** — open then close with no edit ⇒ zero new revisions, tab not dirty. Applies to both Office (status 4) and Markdown.
3. **Callback replay** — a second POST with the same ticket is rejected and does not write.
4. **Truncated body** — a callback whose GET returns HTML instead of a zip is rejected, no revision written, error surfaced.
5. **Caret** — for the Markdown editor, a host `set` that does not change text must leave `selection.head` untouched. (This test exists and has caught the bug before.)
6. **Degradation** — with the probe forced to timeout, the Office surface renders the banner and no iframe within 2 s.
7. **Dual-host markdown** — the same `.md` fixture opened through the Fluent host and through the Office host each satisfies the round-trip law (open, close, zero revisions), and `preferredHost` survives a reload (§4.8.1 H1, H2).
8. **Host fallback** — an asset whose `preferredHost` cannot serve its format still opens, with exactly one warning logged (H4).
9. **Licence gate** — a build with `TEMCO_BUILD_CHANNEL=public` and the office pack present but `TEMCO_OFFICE_LICENCE` unset **fails**, and the same build with `commercial` passes (LG-1).
10. **Pack missing** — with the pack removed, Office assets open read-only with the banner, no crash, and no silent text-editor fallback (LG-4). `x2t.wasm` / `web-apps` version mismatch reports `office-pack-mismatch` (LG-5).

14.3 What cannot be tested in CI

The ONLYOFFICE canvas is cross-origin. We cannot assert on its DOM. We assert on **our** state machine (config sent, callback received, revision written) and screenshot the canvas for human review. Do not write a test that reaches into the iframe.

---

15. Security

Risk	Control
Content URL reused by someone else	Single-use ticket + HMAC + 15 min expiry; bound to `(assetId, revision)`
JWT secret leak	`SecretStorage` only; never a setting, never logged, never sent to the webview
Malicious document	Documents are data, not code — but the *editor* runs them in a sandboxed origin. Never render a document in our own origin
SSRF via content URL	The URL host must equal the configured platform origin; reject anything else
Callback spoofing	Per-session ticket in the callback URL; unknown tickets are 404 + logged
Oversized save	`temco.docs.maxRevisionBytes` enforced before read; stream with a cap
Path traversal in asset ids	`assetId` matches `/^AS-[0-9]{4,8}$/`; never used as a filename
Iframe framing our origins	`frame-src` limited to `${officeOrigin}`; `X-Frame-Options` on the platform API
Clipboard of a pasted image	Written through the extension host into the asset store as a new asset (§10.5 MD-4), never as a data URL in the document

16. Repository shape and dependencies

```text
temco-platform/                     (VS Code 1.144.0.0 fork)
  src/vs/…                          untouched
  extensions/
    temco-ui/                       ★ NEW: the design system
      src/index.ts
      src/provider/TemcoProvider.tsx
      src/theme/readVSCodeTheme.ts
      src/theme/toFluentTheme.ts
      src/density/compact.ts
      src/components/…
      package.json                  deps: @fluentui/react-components, @fluentui/react-icons, react
    temco-docs/                     the docs module
      src/
        asset/store.ts              revisions, invariants D1–D5
        asset/tree.ts               TreeDataProvider
        office/config.ts            buildOfficeConfig, documentKey, jwt
        office/client.ts            health probe, version probe
        office/callback.ts          the HTTP listener (§4.5)
        office/content.ts           signed streaming handler (§4.4)
        office/transport/http.ts    OfficeTransport — server-hosted (§4.1.1 A)
        office/transport/embedded.ts OfficeTransport — x2t.wasm in-process (§4.1.1 B)
        office/transport/index.ts   choose by config; capability report
        hosts/registry.ts           EditorHost table (§4.11)
        publish/state.ts            Publication machine
        commands/*.ts               25 commands (§9.2)
      webview/
        office/                     React host: header, banner, status, iframe
        markdown/                   React + Tiptap, Fluent chrome
      package.json                  contributes: customEditors, commands, settings, views
    temco-workbench/                shell contributions (exists: 4 views, 37 commands)
    temco-projects/ temco-repositories/ temco-store/ temco-ai/ …   (existing)
```

Boundary lint (add to `eslint`):

Rule	Enforced
`@fluentui/*` importable only from `extensions/temco-ui/src/**`	`no-restricted-imports` with `overrides`
`temco-ui` must not import `vscode`	keeps it renderable in tests and in a browser
`temco-docs/src/**` must not import React or Fluent	the extension host has no DOM
`temco-docs/webview/**` must not import `vscode`	webviews only speak `postMessage`
No `document.` in `src/**`, no `require('fs')` in `webview/**`	same boundary, caught early

17. Roadmap

Phase	Ships	Exit criterion
P1 — Office spine	`temco-ui` + Fluent bridge; `temcoDocs.office` custom editor on the **embedded** transport; office pack + licence gate (§4.1.5); asset store with revisions; product/documentation trees; Open With	A real `.docx` opens, edits, saves, and reopens in the real VS Code window **with the network off**; round-trip law passes; a public-channel build without a licence fails CI
P2 — Authoring	Markdown editor on Fluent; properties panel; history/restore; import; templates	An author writes a whole guide set without leaving the platform
P3 — Publish	Publication machine, preflight, queue, DocFX build, downloads	Publishing T3000 13.0 produces a site and a zip from frozen revisions
P4 — Breadth	Excel/powerpoint surfaces; diagram host; review flow; search filters; **`http` transport for co-editing**	Every asset kind has exactly one host and one acceptance test; co-editing works when a server is configured
P5 — Knowledge	Chunking, embeddings, knowledge edges, AI Q&A over published text	"How does Output 5 work?" answers with citations to revisions

18. Definition of done, and open decisions

18.1 Done means

- [ ] Every §10 screen has its acceptance criteria met, checked in a real VS Code window.
- [ ] Every §14.2 regression test exists and fails when its fix is removed.
- [ ] No Fluent import outside `temco-ui`; the lint rule is enabled and green.
- [ ] No `--vscode-*` variable is used to colour a Fluent surface directly (the bridge does it).
- [ ] Document server down produces §4.10 states, verified by hand.
- [ ] The extension packs and installs as a VSIX; the document server address is configured, never bundled.
- [ ] `docs/temco-platform/modules/temco-docs/extension-design.md` updated with what was actually built.

18.2 Decisions still open (recommendation in brackets)

1. **Transport** — **decided:** support both, `embedded` by default, `http` opt-in (§4.1.4). Co-editing is therefore *not* in the default build, and the UI must say so (TR-4).
2. **Distribution scope** — **decided:** internal now, may be sold later. This is exactly the case that needs the build gate in §4.1.5, because the AGPLv3 question only becomes urgent at the moment of conveyance. Next action: legal confirm of LG-1's wording before P1 ships, not before it starts.
3. **Document server edition** — Community (free, concurrency limits, no white-label) or commercial. This decides whether the ONLYOFFICE logo appears in our editor. [Start Community; budget for commercial when branding matters.]
4. **Where the document server runs** — one shared intranet host, or `localhost` Docker on every machine (§4.9). Shared is recommended: co-editing and shared review only exist with a shared server, and per-machine Docker is

more setup per person, not less. [Shared intranet host; `localhost` for development only.]

5. **Where the asset store lives** — local SQLite for a single workstation, or a shared Postgres for a team. The content and callback endpoints imply a reachable host, which argues for shared. [Shared Postgres + a small HTTP service; SQLite only for a personal profile.]
6. **Markdown host — decided:** support both, remembered per asset (§4.8.1), Fluent by default. Remaining sub-question: whether `temco.docs.markdown.engine` should exist at all now that `preferredHost` covers it. [Keep it; it is the default for assets that have no explicit choice.]
7. **Whether `.md` and `.docx` may hold the same page** — i.e. one asset, two formats. [No. One asset, one format. A conversion creates a new asset with a `derived-from` edge.]
8. **Engine fallback if AGPL blocks distribution** — four routes in §4.1.7: R1 separate service, R2 commercial licence, R3 MPL engine (ZetaOffice), R4 no engine at all. [Start on **A + B + R4** (author in Fluent, view via `docx-preview`, convert-on-import with a warning) because that ships with zero AGPL exposure; add **R1** the day a host exists. Re-evaluate at the point the product is sold.]
9. **How much existing `.docx` must be edited in place** — this, not preference, decides between R4 and R1/R2/R3 (§4.1.7.1). [Answer it with a corpus count: how many documents, and how many of them are round-tripped weekly.]

18.3 One-paragraph brief for the generator

The brief is §23, at the end of this document, so it can name the feature under construction.

> Build TEMCO Platform inside the existing VS Code fork. The shell is VS Code and must not be touched; every screen we own is a webview built with React 19 and Fluent UI v9 from `extensions/temco-ui`, themed by bridging `--vscode-*` variables into a Fluent theme. Office documents are edited by ONLYOFFICE, behind a pluggable `OfficeTransport`: either a Document Server embedded as a single cross-origin iframe (JWT-signed config, `key` derived from `(assetId, revision, options)`, short-lived signed content URL, save callback that writes an immutable revision), or a fully embedded build with static `web-apps` plus `x2t.wasm` where conversion happens in-process and no server exists at all. Markdown has two hosts — a Fluent editor by default, the same Office host when the asset remembers that choice in `preferredHost` — resolved by `explicit override → preferredHost → setting → format priority`. The domain is five concepts — Product, Asset, Editor Host, Publication, Knowledge — and no sixth noun may appear. Every save is a revision; nothing edits in place; nothing ever shows a blank iframe.

---

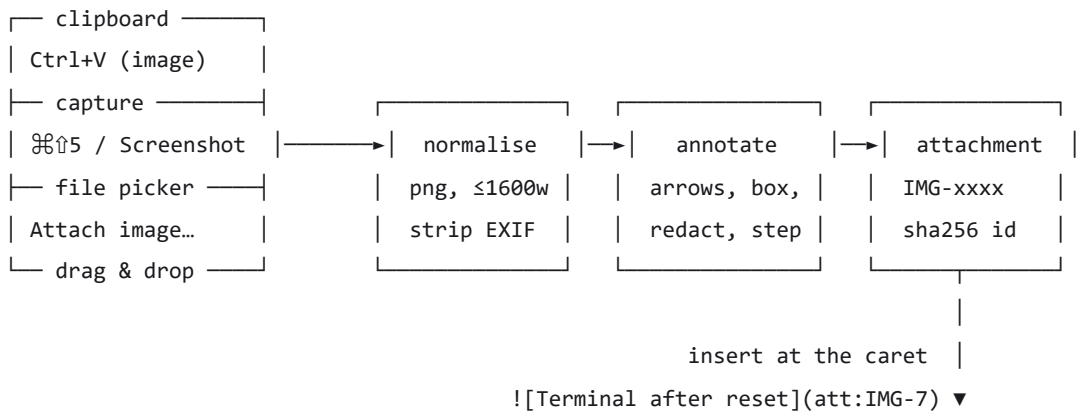
22. The document feature — the whole flow

Scope decision: **this is the only feature we build now.** Products, hardware, firmware and releases stay as data the document feature can reference; their screens come later. Everything below is needed for an author to sit down and write a page with screenshots in it, and for the page to stay correct across versions.

22.1 One pipeline for every image

Three entrances, one path. The author never picks a format, a folder, or a filename.

TEXT



Rules that make this worth building:

- **P1** An image is never a data URL in the document. It becomes an attachment with an id, and the document holds `att:IMG-7`. A 400 KB base64 blob inside a markdown file is how a docs repo becomes uncloneable.
- **P2** Capture, paste and file all end in the same annotation surface — the author does not get a worse experience because of where the bitmap came from.
- **P3** The attachment id is the **content hash**, so pasting the same screenshot twice reuses one attachment and one file on disk.
- **P4** Insertion is at the caret, never appended. The editor's selection survives, which is the whole point of "quick".
- **P5** Failure is visible: no clipboard permission, no capture permission, unsupported type — each gets a sentence naming the cause and the fallback.

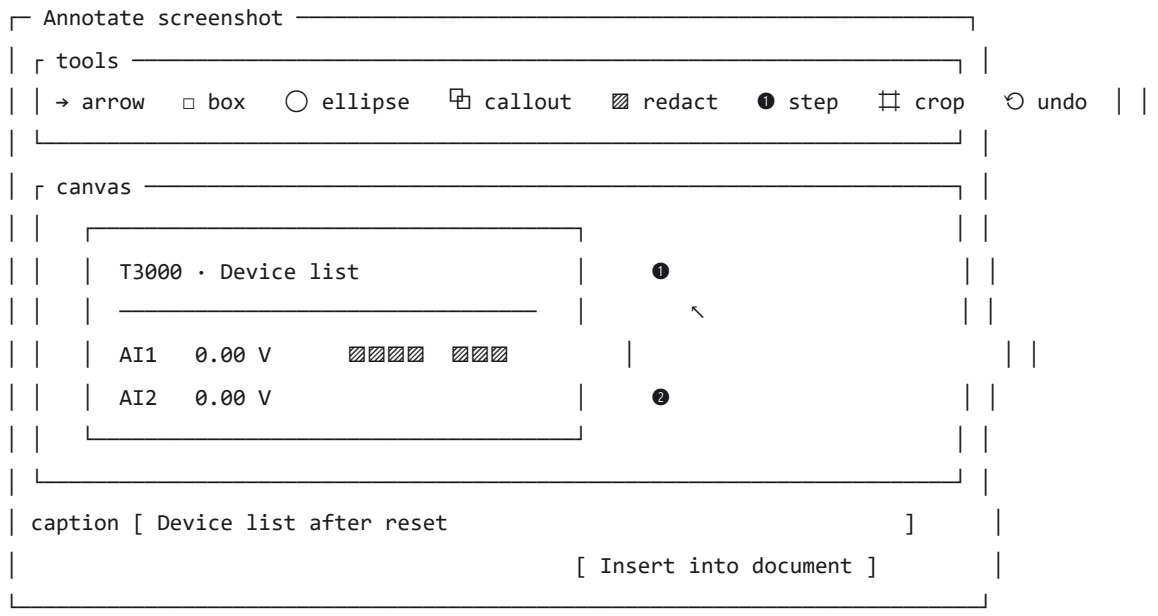
22.2 The paste matrix — what each clipboard flavour becomes

Clipboard holds	Result	Why
PNG / JPEG / DIB bitmap	attachment + `![…](att:IMG-n)`	the common case: a screenshot of the T3000 app
HTML with a `<table>`	a markdown table, cells flattened	pasting a table from Excel must stay a table
HTML with `<ul>/<ol>`	a markdown list	
HTML with headings/paragraphs/bold	markdown with those marks kept, colours and fonts dropped	we do not carry Word's styling into markdown
HTML with `<img src>` that is reachable	downloaded and re-hosted as an attachment	a link to someone's intranet breaks in a year
Plain text	plain text, at the caret	
Files (`CF_HDROP`)	each file attached; images go through §22.1	
A URL on its own	offered as a link, not fetched	

The one rule behind the table: **format follows the source's structure, not its styling.**

22.3 The annotation surface

TEXT



- **A1** Every tool works with one drag. No property panels, no selection handles.
- **A2 Redact is not blur.** Blur is shown as a convenience, but the redact tool *replaces* pixels with a solid block and the UI says so, because a blurred licence key is a published licence key.
- **A3** Undo undoes one operation, not the session. The author keeps working after a mistake.
- **A4** Annotations are baked into the PNG on insert. Nothing downstream has to understand them, and a reader cannot remove them.
- **A5** The caption becomes the `alt` text — accessibility and search for free, and it stops "image1.png" reaching a customer.
- **A6 Callout** magnifies the dragged region (≈2.2×) into an inset connected by leader lines, so a small terminal value can be read on a 900 px screenshot. It samples the pixels underneath, so an arrow drawn first also appears magnified.
- **A7 Crop** bakes everything drawn so far and is undoable once. After a crop the canvas is the new truth; the author is never asked to confirm a destructive step they can reverse.

22.4 Attachments age; documents do not

A screenshot is a photograph of a version. When firmware 13.1 ships, every screenshot of the 13.0 device list is a lie that still looks true.

TEXT

```
attachment IMG-7
├── data         sha256 → stored once
├── caption      "Device list after reset"
├── appliesTo    "13.0"            ← what the picture shows
├── referencedBy AS-8812 rev 7     ← who points at it
└── state        fresh | stale | orphan
```

- **S1** `appliesTo` is set when the image is inserted (the document's version at that moment), and it is editable.
- **S2** A version change (a release, or the automation in §22.5) marks every attachment whose `appliesTo` is the old version **stale**, in the document and in a review queue.
- **S3** Stale is a warning on the page, not a block. Publishing a stale screenshot is allowed and recorded.
- **S4** An attachment referenced by nothing is shown as orphaned rather than deleted silently.

22.5 Automation: an agent proposes, a person approves

The stakeholder ask is "MCP / GitHub actions generate changes automatically as we do new versions". The honest shape of that is a **proposal loop**, not a write-through.

TEXT

```
firmware tag v13.1 pushed
       │
       ▼
GitHub Action: docs-drift
       │  1. run the docs tooling in CI
       │  2. ask the MCP server for the affected assets
       ▼
┌──────────────────────────── MCP tools ────────────────────────────┐
│ docs.find_version_refs(from, to)      → assets + line hits        │
│ docs.get_asset(id)                    → current revision + body   │
│ docs.propose_revision(id, patch, why) → a draft revision, not a save │
│ docs.attach_image(id, png, caption)   → attachment + reference    │
│ docs.list_stale_attachments(version)  → screenshots that lie      │
│ release.notes_from_commits(range)     → a draft release note      │
└──────────────────────────────────────────────────────────────────┘
       │
       ▼
Pull request (or a review item in the app) containing:
  · proposed revisions with a diff
  · the reason ("13.0 appears 7 times; 3 are firmware gates")
  · stale screenshots listed
       │
       ▼
a person approves  →  publication  (never the agent)
```

- **G1** Automation **never** writes to a published revision and never publishes. It proposes.
- **G2** Every proposal carries its evidence: the commit range, the matching lines, the MCP call that produced it.
- **G3** The same loop runs without CI — the editor has an "Automation" surface that shows the proposals and applies them as ordinary revisions, so the feature is useful on day one.
- **G4** An agent may attach an image (a generated diagram, a re-captured screenshot) only through `docs.attach_image`, which applies the same size and format rules as §22.1.

Shipped: `.github/workflows/docs-propose.yml`. It runs on any `v*` tag (or by hand), derives the version pair from the tag, scans the configured folder for the old version, uploads a report and a markdown summary, applies the

replacements on a branch and opens a **draft** pull request. It has no path that writes to the default branch, and no path that publishes. The scanner is inline in the workflow because this repository forbids new `.js` files — a scanner outside the tree is a scanner that rots silently.

22.6 What "a very good editor" means here, concretely

Not a feature list — a set of properties that can be tested:

Property	Test
Typing never loses the caret	type 20 characters in the middle of a paragraph; selection is where the author left it
Paste is one keystroke	a screenshot goes from clipboard to annotated and inserted without a dialog chain longer than one
Save is one keystroke	`⌘S` writes a revision; the tab never shows a dirty dot it cannot clear
Nothing is lost by opening	open, close, zero revisions
Every image has words	no attachment reaches a publication with an empty `alt`
The reader can tell truth from history	a stale screenshot is labelled where it is read, not only in an admin screen
A callout shows what it claims	drag over a region; the inset is that region magnified, not a stock box
Nothing irreversible is one click away	crop is undoable; nothing is deleted, only superseded

22.7 What is built

§22 is no longer only a design. The editor it describes ships as the *Temco Markdown* surface of `extensions/temco-docs` — a custom text editor on Fluent UI v9, default for `*.md`, with capture, annotation and the paste matrix in the surface and the file still the document. The implementation record — the protocol, the caret rules, the defects that only a real window showed, and the deliberate deviations from this section — is `modules/temco-docs/extension-design.md`, and the real-window evidence is the `temco-markdown-editor` scenario (nine steps, all passing).

Of this section's rules, the ones that now ship:

Rule	Where it lives
§22.1 one pipeline for every image	`src/attachments.ts` — one landing path for a paste, a capture and a file picked from disk
§22.2 the paste matrix	the surface: an image opens the annotator, HTML tables and lists become Markdown, plain text stays text
§22.3 the annotation surface	`webview/docs/annotate.tsx` — arrow, box, ellipse, callout, redact, blur, numbered step, crop, undo
§22.4 S1 `appliesTo` at insert time	recorded from the release workflow's `version-report.json`
§22.4 S2 a version change marks pictures stale	`staleImages()` over the record, reported by `temcoDocs.staleScreenshots`
§22.4 S3 stale is a warning, not a block	the report records; nothing refuses a publish
§22.6 typing never loses the caret	the three rules in `main.tsx`, asserted by `check-docs` and by the scenario
§22.6 the editor a person actually writes in	outline, find and replace, link to a page, page properties, list continuation, indent, colour behind the words, a draggable split and three view modes — the features the text editor had and the default editor did not

Three parts of this section remain design only: the attachment **state** machine (fresh / stale / orphan), the publish **preflight** that reads the record inside the build rather than beside it, and §22.5's proposal loop, which is shipped as a workflow (`.github/workflows/docs-propose.yml`) but has not yet run against a real release.

---

23. One-paragraph brief for the generator

> Build TEMCO Platform inside the existing VS Code fork. The shell is VS Code and must not be touched; every screen we own is a webview built with React 19 and Fluent UI v9 from `extensions/temco-ui`, themed by bridging `--vscode-*` variables into a Fluent theme. Office documents are edited by ONLYOFFICE, behind a pluggable `OfficeTransport`: either a Document Server embedded as a single cross-origin iframe (JWT-signed config, `key` derived from `(assetId, revision, options)`, short-lived signed content URL, save callback that writes an immutable revision), or a fully embedded build with static `web-apps` plus `x2t.wasm` where conversion happens in-process and no server exists at all. Markdown has two hosts — a Fluent editor by default, the same Office host when the asset remembers that choice in `preferredHost` — resolved by `explicit override → preferredHost → setting → format priority`. **The feature under construction is documents (§22): a markdown editor where a screenshot goes from clipboard to annotated and inserted in one keystroke, every image is an attachment with a caption and a version it depicts, and version changes arrive as proposals from CI that a person approves.** The domain is five concepts — Product, Asset, Editor Host, Publication, Knowledge — and no sixth noun may appear. Every save is a revision; nothing edits in place; nothing ever shows a blank iframe.

The local assistant and its MCP server

The assistant runs a model on the user's machine. This document specifies the tool server it calls, the protocol between them, the code on both sides, the build that produces the server, and the way the two processes find each other inside the packaged application.

It is organised in three parts:

1. **What exists** — the current call path, measured, with file and line references (§1–§2).
2. **What will be built** — the C# server and the TypeScript client, in full (§4–§6).
3. **Build and run** — how the server is compiled, where it is installed, and how it is started from inside the running application (§7–§8).

Everything in §3 marked *proven* was compiled and executed while this document was written — the C# server on .NET 10.0.400 with `ModelContextProtocol` 2.3.1, the TypeScript client typechecked with `tsc` 5 against `@modelcontextprotocol/client` 2.4.0 and the declarations in `src/vscode-dts/vscode.d.ts`. The probe project is described in §3.4 so the result can be reproduced.

---

1. What runs today

One funnel. Every AI feature in the product ends at `complete()`.

```
TEXT
```

```
webview (temcoAi.panel)                aiPage.ts
   │  postMessage {type:'ask', text}
   ▼
pipeline                               ai/panel.ts
   │  retrieve(docs) -> prompt -> complete()
   ▼
complete(context, messages, options)   ai/runtime.ts:104
   │  POST http://127.0.0.1:<port>/v1/chat/completions
   ▼
llama-server                           spawned at ai/runtime.ts:144
   │  --model <gguf> --host 127.0.0.1 --port <n> --ctx-size 8192
   ▼
Qwen2.5-Instruct (1.5B / 3B / 7B, Q4_K_M)   pinned in ai/catalog.ts
```

The request body is built at `ai/runtime.ts:115`:

```
JSON
```

```
{ "messages": [ ... ], "temperature": 0.2, "stream": false, "max_tokens": 640 }
```

There is no `tools` key, the answer is read as a single string, and `llama-server` is started without `--jinja`. The assistant answers from retrieved text; it cannot read a point, query the catalog, or run a program.

The state it already keeps, which the server reuses rather than replaces:

Thing	Where	Shape
Runtime and model pins	`extensions/temco-ai/src/ai/catalog.ts`	llama.cpp `b11577` per platform, Qwen2.5 1.5B/3B/7B; `ArtifactKind = 'runtime' | 'model'`, entries carry `platform`, `architecture`, `sha256`
Download, checksum, unpack	`extensions/temco-ai/src/ai/install.ts`	`install(context, entry)`, `installedArtifacts(context)`, `modelRoot(context)`
The install root	`install.ts:41`	`temcoAi.modelFolder`, else `context.globalStorageUri.fsPath`; each artifact unpacks into a folder named `entry.id`
The one HTTP call	`ai/runtime.ts:104`	`complete()`
Prompt versions	`ai/prompts.ts`	`ai-v5`
Citation logging	`ai/store.ts`	`temcocontrols.temco-store` API
Catalog database	`extensions/temco-store/src/catalog/store.ts:70`	`globalStorage/catalog/catalog.db`, `node:sqlite`, WAL

2. What the model could not do, and what changed

Missing piece	Consequence	State
`--jinja` on the `llama-server` argument list	The server never applies the model's chat template, so it never emits `tool_calls` and rejects a `tools` payload	done
`tools` in the request body	The model is never told a tool exists	done
Reading `choices[0].message.tool_calls`	A reply that asks for a tool is treated as final text	done
Something that executes a call	The extension host could, but then the tool surface is private to the assistant	done: `tools/temco-mcp`
A loop over `tool_calls`	One turn is one request, so a call cannot be answered	done: `ai/agent.ts`
A service for the tools to ask	The device, repository and project tools need one HTTP surface	not built (open decision 1)
The release step that publishes the server	There is nothing to download until a release carries it	done: section 7.4

3. What is proven, and what is not

> This section is the record of the first end-to-end run, over `stdio`. The application no longer uses that transport: it starts the server in `http` mode and manages the process (§6.5, §8.4). The transcripts below are kept because they are what established that the server, the tool list and the schema are right — and because the `stdio` mode still exists and still behaves this way.

3.1 The server, and the client, over stdio

Built, published, and driven over stdio. Transcript from the actual run:

```
TEXT
```

```
-> {"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-06-18","capabilit
<- {"result":{"protocolVersion":"2025-06-18","capabilities":{"logging":{},"tools":{"listChanged":tru

-> {"jsonrpc":"2.0","method":"notifications/initialized"}

-> {"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}
<- {"result":{"tools":[{"name":"device.list","title":"List devices","description":"Every T3000 devic

-> {"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"device.read_point","arguments":{
<- {"result":{"content":[{"type":"text","text":"pointType must be INPUT, OUTPUT or VARIABLE, not 'NO
```

What that establishes:

- The attributes on a plain class, discovered by `WithToolsFromAssembly()`, produce a `tools/list` whose `inputSchema` is generated from the C# signature, and whose `annotations` carry `readOnlyHint`, `destructiveHint`, `idempotentHint` and `openWorldHint`. The extension keeps its own approval list anyway (section 6.4): a server is not the authority on whether its tools are safe.
- A parameter with a default (`string? nameContains = null`) has `"default": null` and is absent from `required`; without the default it is `["string","null"]` listed as required.
- A tool that returns `CallToolResult` with `IsError = true` sends its message to the client verbatim. A tool that throws does not: the same call once threw `ArgumentException` and the client received only `"An error occurred invoking 'device.read_point'."`. That is why `ToolResults.Guard` exists and why no tool throws.

The whole round trip was then run through the compiled extension code (`out/ai/mcp.js`, plain Node, no editor) against the built server, with a stub platform service on `127.0.0.1:3210`:

```
TEXT
```

```
log [mcp] 23 tools from .../temco-mcp.exe
tools: 23, first: device.list readOnly=true
device.list -> [{"Serial":1234,"Name":"AHU-1","ProductType":"T3-BB","Address":"192.168.1.50","Online
                {"Serial":1235,"Name":"VAV-2","ProductType":"T3-LB","Address":"192.168.1.51","Online
catalog.tables -> no catalog database is available on this machine
bad argument -> pointType must be INPUT, OUTPUT or VARIABLE, not 'NOPE'.
disposed
```

3.2 The tool list, as built

Twenty-three tools, each with the hints that describe it:

Tool	Reads / writes
`device.list`, `device.describe`, `device.read_point`	reads
`device.trendlog_list`, `device.trendlog_query`	reads
`device.alarm_list`	reads
`device.write_point`, `device.alarm_acknowledge`	writes
`docs.search`, `docs.read`, `docs.outline`, `docs.stale`	reads
`repo.list`, `repo.show`, `repo.status`, `repo.log`, `repo.diff`	reads
`project.list`	reads
`project.create`	writes
`catalog.tables`, `catalog.query`	reads, read-only connection
`learn.search`, `learn.post_status`	reads

Fifty is the ceiling in practice: a local 7B model loses accuracy as the list grows and the arguments deepen, so every argument here is a scalar.

3.3 The extension half, and what remains unproven

#	Link	Status
1	The extension host may spawn a child process	proven: `ai/runtime.ts` already spawns `llama-server`
2	The client loads in a CommonJS extension with no dependency	proven: `ai/mcp.ts` requires only `child_process` and `fs`; the compiled `out/ai/mcp.js` ran the round trip above
3	A downloaded executable runs from the install root	proven by the same mechanism the runtime uses: `install()` unpacks into `modelRoot(context)` and records `artifact.path`
4	Two processes can share the catalog	proven: `catalog/store.ts` sets `journal_mode = WAL`; the server opens the file `ReadOnly` with a `busy_timeout`
5	The server never writes next to itself	proven: state goes to `TEMCO_MCP_STATE` and the catalog
6	`llama-server` emits `tool_calls` when started with `--jinja`	vendor-documented; the runtime is not installed on this machine. Section 8.5 step 2 settles it
7	A turn completes end to end inside the app	needs 6, a model, and the platform service. Section 8.5 step 3
8	The packaged application runs the installed server	follows from rows 1 and 3, but has not been run from an installed build

3.4 Findings the implementation now depends on

- **`Microsoft.Extensions.Hosting` is a required explicit reference.** `ModelContextProtocol` does not flow the assembly that contains `Host.CreateApplicationBuilder`; without it the build fails with `CS0103: The name 'Host' does not exist in the current context`.
- **`Microsoft.Data.Sqlite` 10.0.0 pulls a flagged native SQLite.** Restore raises `NU1903` (high severity, `SQLitePCLRaw.lib.e_sqlite3` 2.1.11). 10.0.14 restores and builds with zero warnings.
- **A tool with an optional parameter must default the token too.** C# requires optional parameters after required ones, so a tool ends with `(..., string? optional = null, CancellationToken cancellationToken = default)`, or the compiler rejects the order with `CS1737`.
- **stdin must stay open.** Driving the server from a file that ends immediately produced a full log, executed tool calls, and zero responses: the transport saw end-of-input and finished.

4. Change 1 — let the runtime call tools

4.1 The argument list

TYPESCRIPT

```typescript
const server = spawn(
  runtime.path,
  [
    '--model',
    model.path,
    '--host',
    '127.0.0.1',
    '--port',
    String(port),
    '--ctx-size',
    '8192',
    // Without this the server never applies the model's chat template, so a `tools` payload is
    // rejected and a reply never carries `tool_calls`.
    '--jinja',
  ],
  { windowsHide: true }
);
```

4.2 The types

TYPESCRIPT

```typescript
/** One tool as the model sees it: a name, a sentence, and a JSON Schema for the arguments. */
export interface ToolSpec {
  name: string;
  description: string;
  parameters: Record<string, unknown>;
}

/** One call the model asked for, answered by whoever owns the tool. */
export interface ToolCall {
  id: string;
  name: string;
  arguments: Record<string, unknown>;
}

export interface Completion {
  text: string;
  toolCalls: readonly ToolCall[];
}
```

`ChatMessage`, at `ai/runtime.ts:38`, gains the third role that carries a tool result and the two fields that pair a call with its answer:

TYPESCRIPT

```typescript
export interface ChatMessage {
  role: 'system' | 'user' | 'assistant' | 'tool';
  content: string;
  tool_calls?: { id: string; type: 'function'; function: { name: string; arguments: string } }[];
  tool_call_id?: string;
}
```

4.3 `complete()` gains `tools` and returns the calls

TYPESCRIPT

```typescript
export async function complete(
  context: vscode.ExtensionContext,
  messages: readonly ChatMessage[],
  options: {
    onToken?: (text: string) => void;
    token?: vscode.CancellationToken;
    maxTokens?: number;
    /** Omitted for the plain question-and-answer path, which stays exactly as it is. */
    tools?: readonly ToolSpec[];
  } = {}
): Promise<Completion> {
  const server = await startAiServer(context);
  const payload = JSON.stringify({
    messages,
    temperature: 0.2,
    stream: options.onToken !== undefined,
    max_tokens: options.maxTokens ?? DEFAULT_MAX_TOKENS,
    ...(options.tools?.length
      ? {
          tools: options.tools.map((tool) => ({
            type: 'function',
            function: { name: tool.name, description: tool.description, parameters: tool.parameters
          })),
          tool_choice: 'auto',
          parallel_tool_calls: true,
        }
      : {}),
  });
  const response = await post(server.port, '/v1/chat/completions', payload, options.onToken, option
  touchIdle();
  return readCompletion(response, options.onToken !== undefined);
}

/**
 * A streamed turn is text-only: a half-rendered call cannot be answered, so a turn that may call a
 * never streamed.
 */
function readCompletion(raw: string, streamed: boolean): Completion {
  if (streamed) {
    return { text: raw, toolCalls: [] };
  }
  const parsed = JSON.parse(raw) as {
    choices?: {
      message?: {
        content?: string | null;
```

```
        tool_calls?: { id: string; function: { name: string; arguments: string } }[];
      };
    }[];
  };
  const message = parsed.choices?.[0]?.message;
  const toolCalls = (message?.tool_calls ?? []).map((call) => ({
    id: call.id,
    name: call.function.name,
    arguments: parseArguments(call.function.arguments),
  }));
  return { text: message?.content ?? '', toolCalls };
}

function parseArguments(text: string): Record<string, unknown> {
  try {
    const value: unknown = JSON.parse(text || '{}');
    return typeof value === 'object' && value !== null ? (value as Record<string, unknown>) : {};
  } catch {
    return {};
  }
}
```

`complete()` changes shape, so all four call sites read `.text` instead of the string: `ai/panel.ts:219`, `ai/panel.ts:308`, `extension.ts:128`, `extension.ts:140`. With `tools` omitted they behave exactly as they do now.

---

5. The server — `tools/temco-mcp`

5.1 Layout

TEXT

```
tools/temco-mcp/
  Temco.Mcp.csproj
  Program.cs                host, logging to stderr, stdio and http transports
  Platform/
    IPlatformApi.cs         the outside world, as one interface
    HttpPlatformApi.cs      the implementation: base URL from the environment
  Tools/
    DeviceTools.cs          [McpServerToolType]
    DocsTools.cs
    RepoTools.cs
    CatalogTools.cs
  Store/
    CatalogDb.cs            read-only SQLite over the extension's catalog
    Audit.cs                one row per call
  tests/
    Temco.Mcp.Tests.csproj  drives the real host over an in-memory transport
```

It lives at the repository root rather than under `extensions/` because it is not an extension: it is a program the extension starts.

5.2 `Temco.Mcp.csproj`

XML

```xml
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <InvariantGlobalization>true</InvariantGlobalization>
    <SatelliteResourceLanguages>en</SatelliteResourceLanguages>
    <!-- A trimmed build breaks WithToolsFromAssembly(); a single-file build extracts e_sqlite3.dll
         time. The publish output is a plain folder instead. -->
    <PublishSingleFile>false</PublishSingleFile>
    <PublishTrimmed>false</PublishTrimmed>
    <AssemblyName>temco-mcp</AssemblyName>
    <RootNamespace>Temco.Mcp</RootNamespace>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="ModelContextProtocol" Version="2.3.1" />
    <!-- Explicit: ModelContextProtocol does not flow the assembly that declares Host. -->
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="10.0.14" />
    <!-- 10.0.0 restores SQLitePCLRaw.lib.e_sqlite3 2.1.11, which NuGet flags NU1903 (high). -->
    <PackageReference Include="Microsoft.Data.Sqlite" Version="10.0.14" />
  </ItemGroup>
</Project>
```

5.3 `Program.cs`

CSHARP

```csharp
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using Temco.Mcp;
using Temco.Mcp.Tools;

var builder = Host.CreateApplicationBuilder(args);

// stdout is the protocol. Every log line must go to stderr, or the first write breaks the stream.
builder.Logging.ClearProviders();
builder.Logging.AddConsole(options => options.LogToStandardErrorThreshold = LogLevel.Trace);

builder.Services.AddSingleton<IPlatformApi>(_ => new HttpPlatformApi(
    Environment.GetEnvironmentVariable("TEMCO_PLATFORM_API") ?? "http://127.0.0.1:3210",
    TimeSpan.FromSeconds(20)));
builder.Services.AddSingleton(_ => new Audit(
    Environment.GetEnvironmentVariable("TEMCO_MCP_STATE")
        ?? Path.Combine(Path.GetTempPath(), "temco-mcp")));

builder.Services
    .AddMcpServer(options =>
    {
        options.ServerInfo = new() { Name = "temco-mcp", Version = "1.0.0" };
    })
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

await builder.Build().RunAsync();
```

The two `using Temco.Mcp…` lines are load-bearing: top-level statements sit in the global namespace, so the project's own types are not in scope without them.

5.4 A tool, in full

The attribute defaults are the unsafe ones — `ReadOnly = false`, `Destructive = true`, `OpenWorld = true` — so every tool states its own intent.

CSHARP

```csharp
using System.ComponentModel;
using System.Text.Json;
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;

namespace Temco.Mcp.Tools;

[McpServerToolType]
public sealed class DeviceTools(IPlatformApi platform, Audit audit)
{
    [McpServerTool(
        Name = "device.list",
        Title = "List devices",
        ReadOnly = true,
        Destructive = false,
        Idempotent = true,
        OpenWorld = false)]
    [Description("Every T3000 device known to the panel: serial, name, product type, address, and wl
    public async Task<CallToolResult> ListDevices(
        [Description("Optional text; only devices whose name contains it are returned.")] string? n
        CancellationToken cancellationToken = default)
    {
        try
        {
            var devices = await platform.GetDevicesAsync(nameContains, cancellationToken);
            audit.Record("device.list", JsonSerializer.Serialize(new { nameContains }), devices.Cou
            return Text(JsonSerializer.Serialize(devices));
        }
        catch (Exception error) when (error is HttpRequestException or InvalidOperationException)
        {
            return Fail($"the platform service did not answer: {error.Message}");
        }
    }

    [McpServerTool(
        Name = "device.read_point",
        Title = "Read a point",
        ReadOnly = true,
        Destructive = false,
        Idempotent = true,
        OpenWorld = false)]
    [Description("Read one input, output or variable of a device: its value, units, and range.")]
    public async Task<CallToolResult> ReadPoint(
        [Description("Device serial number.")] int serial,
        [Description("Point type: INPUT, OUTPUT or VARIABLE.")] string pointType,
```

```
    [Description("One-based point index, as the point list shows it.")] int pointIndex,
    CancellationToken cancellationToken = default)
{
    if (pointType is not ("INPUT" or "OUTPUT" or "VARIABLE"))
    {
        return Fail($"pointType must be INPUT, OUTPUT or VARIABLE, not '{pointType}'.");
    }
    try
    {
        var point = await platform.ReadPointAsync(serial, pointType, pointIndex, cancellationTol
        audit.Record("device.read_point", JsonSerializer.Serialize(new { serial, pointType, poi
        return Text(JsonSerializer.Serialize(point));
    }
    catch (Exception error) when (error is HttpRequestException or InvalidOperationException)
    {
        return Fail($"device {serial} has no {pointType} point {pointIndex}: {error.Message}");
    }
}

[McpServerTool(
    Name = "device.write_point",
    Title = "Write a point",
    ReadOnly = false,
    Destructive = true,
    Idempotent = false,
    OpenWorld = true)]
[Description("Write one point. Changes equipment state, so it is never called without an approva
public async Task<CallToolResult> WritePoint(
    [Description("Device serial number.")] int serial,
    [Description("Point type: OUTPUT or VARIABLE.")] string pointType,
    [Description("One-based point index.")] int pointIndex,
    [Description("The value to write.")] double value,
    CancellationToken cancellationToken = default)
{
    try
    {
        var result = await platform.WritePointAsync(serial, pointType, pointIndex, value, cance
        audit.Record("device.write_point", JsonSerializer.Serialize(new { serial, pointType, po
        return Text(JsonSerializer.Serialize(result));
    }
    catch (Exception error) when (error is HttpRequestException or InvalidOperationException)
    {
        return Fail($"the write did not reach device {serial}: {error.Message}");
    }
}

private static CallToolResult Text(string text) =>
```

```
        new() { Content = [new TextContentBlock { Text = text }] };

    private static CallToolResult Fail(string message) =>
        new() { IsError = true, Content = [new TextContentBlock { Text = message }] };
}
```

Three rules follow from §3.1 and hold for every tool:

1. **Return `CallToolResult`; do not throw.** An exception reaches the model as `"An error occurred invoking '<tool>'."` with no reason, so a model that passed a wrong argument cannot correct it. That includes infrastructure failures: a refused connection to the platform service is the tool's error to report, not an exception to leak.
2. **Give an optional parameter a default, and default the token with it.** `string? nameContains = null` yields `"default": null` and keeps the name out of `required`; without it the schema lists it as required. Because C# requires optional parameters to come after required ones, a tool with any optional argument ends its list with `(..., string? optional = null, CancellationToken cancellationToken = default)` — the token needs the default too, or the compiler rejects the order (`CS1737`).
3. **State the four hints on every tool.** `ReadOnly`, `Destructive`, `Idempotent`, `OpenWorld` default to `false`, `true`, `false` and `true` respectively, and reach the client as `annotations`; leaving them out claims the tool destroys things and reaches outside the machine.

As built, the two result constructors and the guard live in `Tools/ToolResults.cs` so that no tool repeats them: `Text(value)`, `Fail(message)` and `Guard(body)`, which turns a refused connection, a missing record or a cancellation into a sentence. The device tools are typed because their answers have a shape; the read-only rest go through `IPlatformApi.ReadJsonAsync(path)` and return the service's own JSON rather than growing a method per route.

5.5 The platform interface

The tools do not speak T3000 protocols. They call one interface, so the transport can change without touching a tool.

CSHARP

```
namespace Temco.Mcp;

public interface IPlatformApi
{
    Task<IReadOnlyList<DeviceInfo>> GetDevicesAsync(string? nameContains, CancellationToken cancell
    Task<PointInfo> ReadPointAsync(int serial, string pointType, int pointIndex, CancellationToken
    Task<WriteResult> WritePointAsync(int serial, string pointType, int pointIndex, double value, C
}

public sealed record DeviceInfo(int Serial, string Name, string ProductType, string? Address, bool
public sealed record PointInfo(string Label, double Value, string Units, string Description);
public sealed record WriteResult(bool Accepted, string Message);
```

CSHARP

```csharp
using System.Net.Http.Json;
using System.Text.Json;

namespace Temco.Mcp;

public sealed class HttpPlatformApi : IPlatformApi
{
    private readonly HttpClient _http;
    private static readonly JsonSerializerOptions Json = new(JsonSerializerDefaults.Web);

    public HttpPlatformApi(string baseUrl, TimeSpan timeout)
    {
        _http = new HttpClient { BaseAddress = new Uri(baseUrl), Timeout = timeout };
    }

    public async Task<IReadOnlyList<DeviceInfo>> GetDevicesAsync(string? nameContains, CancellationT
    {
        var url = nameContains is null ? "/devices" : $"/devices?nameContains={Uri.EscapeDataString
        var devices = await _http.GetFromJsonAsync<List<DeviceInfo>>(url, Json, cancellationToken);
        return devices ?? [];
    }

    public async Task<PointInfo> ReadPointAsync(int serial, string pointType, int pointIndex, Cance
    {
        var point = await _http.GetFromJsonAsync<PointInfo>(
            $"/devices/{serial}/points/{pointType}/{pointIndex}", Json, cancellationToken);
        return point ?? throw new InvalidOperationException($"Device {serial} has no {pointType} po
    }

    public async Task<WriteResult> WritePointAsync(int serial, string pointType, int pointIndex, do
    {
        var response = await _http.PutAsJsonAsync(
            $"/devices/{serial}/points/{pointType}/{pointIndex}", new { value }, Json, cancellationT
        response.EnsureSuccessStatusCode();
        var result = await response.Content.ReadFromJsonAsync<WriteResult>(Json, cancellationToken)
        return result ?? new WriteResult(false, "The service returned no answer.");
    }
}
```

Which service answers on `TEMCO_PLATFORM_API` is open decision 1 in §11. Only `HttpPlatformApi` is affected by the answer.

5.6 The catalog database, read-only

`temco-store` owns the file; the server only reads it.

CSHARP

```csharp
using Microsoft.Data.Sqlite;

namespace Temco.Mcp;

public sealed class CatalogDb
{
    private readonly string _connectionString;

    public CatalogDb(string path)
    {
        // The extension opened this file with journal_mode=WAL, so a reader here never blocks its w
        _connectionString = new SqliteConnectionStringBuilder
        {
            DataSource = path,
            Mode = SqliteOpenMode.ReadOnly,
            Pooling = true,
        }.ToString();
    }

    public async Task<IReadOnlyList<IReadOnlyDictionary<string, object?>>> QueryAsync(
        string sql, CancellationToken cancellationToken)
    {
        await using var connection = new SqliteConnection(_connectionString);
        await connection.OpenAsync(cancellationToken);
        await using (var pragma = connection.CreateCommand())
        {
            // Read-only, but still a shared file: wait rather than fail while a writer holds the lo
            pragma.CommandText = "PRAGMA busy_timeout = 5000;";
            await pragma.ExecuteNonQueryAsync(cancellationToken);
        }
        await using var command = connection.CreateCommand();
        command.CommandText = sql;
        await using var reader = await command.ExecuteReaderAsync(cancellationToken);
        var rows = new List<IReadOnlyDictionary<string, object?>>();
        while (await reader.ReadAsync(cancellationToken))
        {
            var row = new Dictionary<string, object?>(StringComparer.Ordinal);
            for (var i = 0; i < reader.FieldCount; i++)
            {
                row[reader.GetName(i)] = await reader.IsDBNullAsync(i, cancellationToken) ? null : r
            }
            rows.Add(row);
        }
        return rows;
```

```
    }
}
```

The path comes from the extension, which uses the same expression as `catalog/store.ts:1473`: `vscode.Uri.joinPath(context.globalStorageUri, 'catalog', 'catalog.db').fsPath`.

5.7 The tool catalogue

Names are the contract; the model sees them verbatim. Twenty-three is deliberately short — a local 7B model loses accuracy as the list grows and the arguments deepen — and every argument is a scalar.

Tool	Reads / writes	Backed by
`device.list`	reads devices	`IPlatformApi.GetDevicesAsync`
`device.describe`	reads one device	`IPlatformApi`
`device.read_point`	reads one point	`IPlatformApi.ReadPointAsync`
`device.write_point`	writes one point	`IPlatformApi.WritePointAsync`
`device.trendlog_list`	reads trendlog names	`IPlatformApi`
`device.trendlog_query`	reads history over a range	`IPlatformApi`
`device.alarm_list`	reads alarms, optionally active only	`IPlatformApi`
`device.alarm_acknowledge`	acknowledges one alarm	`IPlatformApi`
`docs.search`	full-text over the published documentation	`docs/` in the repository
`docs.read`	reads one page by path	`docs/`
`docs.outline`	reads the table of contents	`docs/README.md`
`docs.stale`	counts pages whose code moved	`docs/`
`repo.list`	reads repositories	`IPlatformApi`
`repo.show`	reads one repository	`IPlatformApi`
`repo.status`	reads the working tree of one clone	`IPlatformApi`
`repo.log`	reads commits	`IPlatformApi`
`repo.diff`	reads a diff; never applies one	`IPlatformApi`
`project.list`	reads projects	`IPlatformApi`
`project.create`	creates one project	`IPlatformApi`
`catalog.query`	runs one read-only statement	`CatalogDb`
`catalog.tables`	lists tables and columns	`CatalogDb`
`learn.search`	searches the site's content	`IPlatformApi`
`learn.post_status`	reads the publication state of a topic	`IPlatformApi`

5.8 Audit

One row per call, in the extension's storage, so what the assistant did can be read back.

CSHARP

```
using Microsoft.Data.Sqlite;

namespace Temco.Mcp;

public sealed class Audit
{
    private readonly string _path;

    public Audit(string appDataDirectory)
    {
        _path = Path.Combine(appDataDirectory, "temco-mcp", "audit.db");
        Directory.CreateDirectory(Path.GetDirectoryName(_path)!);
        using var connection = new SqliteConnection($"Data Source={_path}");
        connection.Open();
        using var command = connection.CreateCommand();
        command.CommandText = """
            CREATE TABLE IF NOT EXISTS calls (
                at TEXT NOT NULL,
                tool TEXT NOT NULL,
                arguments TEXT NOT NULL,
                rows INTEGER NOT NULL
            );
            """;
        command.ExecuteNonQuery();
    }

    public void Record(string tool, string arguments, int rows)
    {
        using var connection = new SqliteConnection($"Data Source={_path}");
        connection.Open();
        using var command = connection.CreateCommand();
        command.CommandText = """
            INSERT INTO calls (at, tool, arguments, rows) VALUES ($at, $tool, $arguments, $rows);
            """;
        command.Parameters.AddWithValue("$at", DateTimeOffset.UtcNow.ToString("O"));
        command.Parameters.AddWithValue("$tool", tool);
        command.Parameters.AddWithValue("$arguments", arguments);
        command.Parameters.AddWithValue("$rows", rows);
        command.ExecuteNonQuery();
    }
}
```

The policy gate is in the loop (§6.4), not in the tool: a call to something in the `DESTRUCTIVE` set is only made after the panel asks the person.

---

6. The client inside `temco-ai`

6.1 Nothing to install

The client is written in `ai/mcp.ts` rather than pulled in as a dependency. `@modelcontextprotocol/client` would be the first runtime dependency in an extension that installs nothing, and this extension is compiled with `tsc` rather than bundled, so a dependency would also have to be packaged and kept in step with every release. The protocol the assistant needs is four messages long, and the server is ours.

`temco-ai` therefore keeps `"dependencies": {}`. The version that used the SDK typechecks, but it would have been the only reason this extension needed a `node_modules`.

6.2 `ai/mcp.ts`

Two functions over two messages, section 8.4:

Export	What it does
`fetchTools(endpoint, token, log)`	Posts `tools/list` and answers with `ToolSpec[]` built from each tool's `name`, `title`, `inputSchema` and annotations
`runTool(endpoint, token, name, args, token)`	Posts `tools/call` and joins the `content[].text` it gets back. A failed call is not special-cased: the server already returns its reason as text, and that text is what the model should read

Both go through one `post()`: an `http.request` to the endpoint with `Authorization: Bearer <token>`, `Accept: application/json, text/event-stream`, and the answer read to the end. The transport frames the answer either way — the ASP.NET implementation streams it as a server-sent event with one `data:` line — so `unwrap()` takes the `data:` lines when the content type says `text/event-stream` and the whole body otherwise, then reads `result` or throws the server's `error.message`.

There is no session to open and nothing to dispose: the server runs in its default stateless mode, so each POST carries everything and a request that arrives while another is in flight is unaffected. That is why the client has no class any more — the two things that made one necessary, a pipe to write to and a handshake to replay, are gone.

Timeouts are 15 s for the list and 30 s per call, both enforced by the socket's own `timeout`, and a `CancellationToken` destroys the request it belongs to. A `401` is reported as it is: the token belongs to the window's process, and a token the server refuses means the process was restarted behind the caller's back.

6.3 `ai/binPath.ts` — where the binary is

The server is an installed artifact, exactly like the runtime and the models: a catalog entry, a checksum, a folder under `modelRoot(context)`, recorded in the manifest by `install()`.

TYPESCRIPT

```typescript
// Where the tool server is, and what it needs to know.
//
// The server is an installed artifact, exactly like the runtime and the models: a catalog entry, a
// a folder under `modelRoot(context)`. Nothing here hard-codes an install path, so `temcoAi.modelFo
// moves the server with everything else.

import * as vscode from 'vscode';
import { CatalogEntry, toolServerFor } from './catalog';
import { install, installedArtifacts } from './install';

/** The catalog entry for this machine, if the catalogue has one for it. */
export function serverEntry(): CatalogEntry | undefined {
  return toolServerFor(process.platform, process.arch);
}

/** The installed executable, or `undefined` when it has not been installed on this machine yet. */
export function serverBinary(context: vscode.ExtensionContext): string | undefined {
  const entry = serverEntry();
  if (!entry) {
    return undefined;
  }
  return installedArtifacts(context).find((artifact) => artifact.id === entry.id)?.path;
}

/** Installs it if it is missing, and answers with the executable path. */
export async function ensureServerBinary(context: vscode.ExtensionContext): Promise<string> {
  const entry = serverEntry();
  if (!entry) {
    throw new Error(
      `No tool server is published for ${process.platform}-${process.arch}, so the assistant cannot
    );
  }
  const artifact = await install(context, entry);
  return artifact.path;
}

/** What the server is told about this machine. Nothing else is inherited. */
export function serverEnv(context: vscode.ExtensionContext): Record<string, string> {
  const env: Record<string, string> = {};
  for (const key of ['PATH', 'HOME', 'USERPROFILE', 'SystemRoot', 'TEMP', 'TMP', 'TMPDIR', 'LANG'])
    const value = process.env[key];
    if (typeof value === 'string') {
      env[key] = value;
    }
  }
```

```
  env.TEMCO_CATALOG_DB = vscode.Uri.joinPath(
    context.globalStorageUri,
    'catalog',
    'catalog.db'
  ).fsPath;
  env.TEMCO_MCP_STATE = context.globalStorageUri.fsPath;
  env.TEMCO_PLATFORM_API = platformApi();
  return env;
}

/** The address the device, repository and project tools ask. Empty means they are not usable here.
export function platformApi(): string {
  return vscode.workspace.getConfiguration('temcoAi').get<string>('platformApi', '').trim();
}
```

Three consequences, all of them wanted:

- The installer stays the size it is; the server arrives on first use, the same way a model does, and asks once before it does.
- `ArtifactKind` in `catalog.ts:7` becomes `'runtime' | 'model' | 'tool-server'`, and one `install()` serves all three. `modelRoot(context)` stays the single root, so `temcoAi.modelFolder` moves the server too.
- A missing binary is an ordinary state, not an error: the Tool Server page's **Start** downloads and starts it, and until then the plain answer path runs unchanged.

6.4 `ai/agent.ts`, the loop

The loop keeps its shape — ask, run what it asked for, ask again, with a six-step budget and a set of calls that need a person's yes. What changed is where the server comes from: it is not opened and disposed per turn any more, because it is a process the window owns and the Tool Server page shows.

TYPESCRIPT

```typescript
import { platformApi, serverBinary } from './binPath';
import { ChatMessage, ToolSpec, complete } from './runtime';
import { callThroughServer, toolServer } from './server';

/** The tools the server offers, without asking the model anything. Starts the server when it is not
export async function listTools(context: vscode.ExtensionContext): Promise<readonly ToolSpec[]> {
  return toolServer().tools(context);
}

export async function callTool(
  context: vscode.ExtensionContext,
  name: string,
  args: Record<string, unknown>,
  options: { log?: (line: string) => void; approve?: Approval; token?: vscode.CancellationToken } =
): Promise<string> {
  if (needsApproval(name)) {
    const yes = options.approve ? await options.approve(name, args) : false;
    if (!yes) {
      options.log?.(`[mcp] ${name} declined`);
      return 'The person at the keyboard declined this call.';
    }
  }
  return callThroughServer(context, name, args, options.token);
}

export async function runAgent(context, messages, options): Promise<AgentResult> {
  // No platform service is no longer fatal: the catalog tools answer from this machine.
  if (platformApi().length === 0) {
    options.log('[mcp] no platform service is configured (temcoAi.platformApi); tools that need one
  }

  const tools = await listTools(context);
  for (let step = 0; step < MAX_STEPS; step++) {
    const reply = await complete(context, thread, { tools, token: options.token });
    // ... unchanged: push the assistant turn, gate each call, run it through callThroughServer, pus
  }
}
```

Three properties are worth naming, because each was a decision:

- **Both entry points still pass one gate.** `callTool()` asks before a write, and `runAgent` asks before a write, and both are in this file so a third caller cannot arrive without one.
- **The approval set stays in the extension.** It is not read from the server's `destructiveHint`, which a tampered server could set to `false`.

- **A missing platform service is a sentence, not a throw.** It used to stop the turn; now the catalog tools answer and the rest say the service did not answer, which is something the model can pass on to a person.

6.5 Lifecycle

`ai/server.ts` owns the process. It is the one place that starts it, so the panel, the Tools page's runner and the Tool Server page all talk to the same thing.

Event	What happens
Nothing needs the server	It is not running. Opening a page never starts it, and `status()` says `stopped`
A turn, or the Tools page, needs the catalogue	`toolServer().tools(context)` calls `ensure()`, which starts it if it is not up and reuses the process if it is
First start on a machine with no server	`start()` downloads it through `install()`, then starts it — "Start" is the button that installs
Port	`temcoAi.serverPort`, or a free port found by binding one and releasing it. Never a fixed default: two windows must not collide
Token	24 random bytes per start, passed in `TEMCO_MCP_TOKEN`, required on every request but `/health`. It is not a secret to be stored: it dies with the process
Readiness	`/health` is polled every 250 ms for up to 30 s. Only a 200 counts; a process that is up but silent is not running
The process exits on its own	State `failed`, with the last three lines it wrote, so the page can say what happened instead of "stopped"
`temcoAi.serverIdleMinutes`	`0` (the default) keeps it for the window. Anything else stops it after that long without a call
The window closes	`deactivate()` calls `disposeToolServer()`, which kills the child. Nothing is left behind
No artifact for this platform	`serverEntry()` is undefined, `start()` fails with that sentence, and the plain answer path still runs

7. Building it

7.1 Locally

POWERSHELL

```
winget install Microsoft.DotNet.SDK.10   # once per machine; .NET 10 is the current LTS

dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r win-x64 --self-contained true `
  -p:PublishSingleFile=false -o artifacts/temco-mcp-win-x64
dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r linux-x64 --self-contained true `
  -p:PublishSingleFile=false -o artifacts/temco-mcp-linux-x64
dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r osx-arm64 --self-contained true `
  -p:PublishSingleFile=false -o artifacts/temco-mcp-osx-arm64
```

The three RIDs are the three rows of the release matrix in `.github/workflows/release.yml`:

Release matrix	`process.platform` / `process.arch`	RID	Catalog entry id
`win32` / `x64`	`win32` / `x64`	`win-x64`	`temco-mcp-win-x64`
`linux` / `x64`	`linux` / `x64`	`linux-x64`	`temco-mcp-linux-x64`
`darwin` / `arm64`	`darwin` / `arm64`	`osx-arm64`	`temco-mcp-osx-arm64`

`serverEntry()` matches on `process.platform` and `process.arch`, so the table is the only place the mapping is written down; the folder name comes from the entry id, which `install()` uses as the unpack folder.

7.2 What it produces

344 files and 108 MB for one platform, because a self-contained publish carries the runtime with it — and, since the HTTP transport arrived, ASP.NET Core with it. The whole folder is the artifact: apphost, `temco-mcp.dll`, `e_sqlite3.dll`, and the framework. Nothing is written next to it at run time, so it may sit in a read-only store.

7.3 Into an archive, and its checksum

`install()` unpacks with the platform's own `tar`, which reads `.tar.gz` everywhere (GNU tar does not read `.zip`), then finds the executable by name anywhere inside. The archive is the unit that is downloaded and verified, exactly as a model is:

POWERSHELL

```
tar -czf artifacts/temco-mcp-win-x64.tar.gz -C artifacts temco-mcp-win-x64
(Get-FileHash artifacts/temco-mcp-win-x64.tar.gz -Algorithm SHA256).Hash.ToLowerInvariant()
```

The hash and the byte count go into the `TOOL_SERVERS` entry in `catalog.ts`, with `kind: 'tool-server'`, `platform`, `architecture` and `licence`. **They are not pinned yet**, and until they are, `install()` asks once before it downloads: a modal naming the file, the exact URL, the size (or "not published in the catalog") and the folder it lands in, with one **Download** button. The checksum the file turns out to have is recorded in the manifest either way, so a later change is still noticed. `temcoAi.allowUnpinned` skips the question for a machine that installs unattended — it no longer *is* the question. The unpacked folder `temco-mcp-win-x64` is what `install()` records, and `artifact.path` is the executable inside it.

That is the whole first run, and it was exercised end to end against a local HTTP server serving the real archive: cancel installs nothing; **Download** fetches 49,248,480 bytes, writes the manifest (`id, kind, file, sha256, bytes, url, path, installedAt`) and leaves `temco-mcp.exe` where `serverBinary()` looks; a second install asks nothing; `allowUnpinned` asks nothing.

`.gitignore` carries `tools/temco-mcp/bin/` , `tools/temco-mcp/obj/` and `extensions/temco-ai/bin/` : build output of the machine, never of git.

7.4 In CI

In `.github/workflows/release.yml` , in the build job. The SDK is installed next to Node, and the publish happens after the application is packaged and before the platform's staging step, so the archive lands in the same `staging/` folder the release collects:

YAML

```yaml
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 10.0.x

      - name: Stage the tool server
        shell: bash
        run: |
          set -euo pipefail
          case "${{ matrix.platform }}-${{ matrix.arch }}" in
            win32-x64) rid=win-x64 ;;
            linux-x64) rid=linux-x64 ;;
            darwin-arm64) rid=osx-arm64 ;;
            *) echo "::error::No tool server is published for ${{ matrix.platform }}-${{ matrix.arch
          esac
          dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r "$rid" --self-contained true
            -p:PublishSingleFile=false -o "tool-server/$rid"
          mkdir -p staging
          tar -czf "staging/temco-mcp-$rid.tar.gz" -C "tool-server/$rid" .
          ls -l "staging/temco-mcp-$rid.tar.gz"
```

The existing `softprops/action-gh-release` step already collects `release/*.tar.gz` , so the three archives ride in the release next to the installers without a second upload step.

Two properties matter:

1. It runs on the same runner as the build, so the RID matches the artefact the release publishes. There is no cross-compilation and no RID guessing.
2. It is a separate step with its own exit code, so a broken tool server fails the release instead of hiding behind a later command — the failure mode that the Windows `pwsh` steps in this workflow were split to avoid.

The catalogue does not pin the tag. `TOOL_SERVER_RELEASE` in `catalog.ts` is `` `v${vscode.version.split('-')[0]}` `` , because every release attaches its own three archives: a pinned tag would send a newer application to an older build, or to a tag that never carried one.

7.5 During development

```powershell
dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r win-x64 --self-contained true `
  -p:PublishSingleFile=false -o extensions/temco-ai/bin/win-x64
```

That folder is the third place `serverBinary()` looks, and it is in `.gitignore`. A source checkout therefore runs the server from its own tree, before any release carries an artifact; a packaged application has it only if the installer put one there, which it does not. The three places are tried in order: what the installer recorded (`installedArtifacts()`), `<modelRoot>/<entry.id>` — which is what `temcoAi.modelFolder` moves, and where a manifest entry makes `installedArtifacts()` see a hand-placed copy — and `extensions/temco-ai/bin/<rid>` beside this extension.

The server answers two ways, and both are testable without the application:

```powershell
# stdio: one request at a time on a pipe, for an inspector or a script
npx @modelcontextprotocol/inspector extensions\temco-ai\bin\win-x64\temco-mcp.exe

# HTTP: the mode the application manages
$env:TEMCO_MCP_TRANSPORT='http'; $env:TEMCO_MCP_PORT='7391'; $env:TEMCO_MCP_TOKEN='abc'
extensions\temco-ai\bin\win-x64\temco-mcp.exe
curl.exe http://127.0.0.1:7391/health
#   -> {"ok":true,"name":"temco-mcp","version":"1.0.0"} without a token
curl.exe -X POST http://127.0.0.1:7391/mcp -H "Content-Type: application/json" `
  -H "Accept: application/json, text/event-stream" `
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}'
#   -> 401 without the header, and the 23 tools with -H "Authorization: Bearer abc"
```

---

8. Running it inside the application

8.1 The process tree

```text
temco-platform.exe                    the window
 └─ extension host (Node, type: commonjs)
     ├─ llama-server                   ai/runtime.ts:144    one model, loopback HTTP
     └─ temco-mcp                      ai/server.ts         one per window, loopback HTTP, managed
         └─ (HTTP) the platform service on 127.0.0.1:3210
```

Neither child starts at activation. `llama-server` starts on the first `complete()`; `temco-mcp` starts the first time something needs the catalogue — a turn, the Tools page, or the Start button — and then stays until the idle setting or the closing window says otherwise. Both are stopped explicitly, so a closed window leaves no processes behind.

8.2 Where the binary is, per layout

Layout	Install root	Executable
Development	`<repo>/extensions/temco-ai` `globalStorage`, or `temcoAi.modelFolder`	`<root>/temco-mcp-win-x64/temco-mcp.exe`
Packaged	`<userData>/User/globalStorage/temcocontrols.temco-ai`	`<root>/temco-mcp-win-x64/temco-mcp.exe`
Source checkout	`extensions/temco-ai/bin/<rid>` beside the extension	`extensions/temco-ai/bin/win-x64/temco-mcp.exe`

The binary is *not* inside `extensions/temco-ai`, and no `.vscodeignore` question arises: it is installed by `install()`, the same call that installs a model, and resolved by `installedArtifacts()`. The install root is the same root the models use, so a user who moved `temcoAi.modelFolder` onto a larger disk moved the tool server with them.

The published file keeps its executable bit through `Compress-Archive` / `unzip`, and the manifest records the path that was verified. If a copy loses the bit, the first `open()` fails with `EACCES`; the one-time repair belongs next to the spawn:

TYPESCRIPT

```typescript
if (process.platform !== 'win32') {
  try {
    fs.chmodSync(binary, 0o755);
  } catch {
    // Read-only install; the file most likely already carries the bit.
  }
}
```

8.3 What the application passes in

Variable	Value	Why the server needs it
`TEMCO_CATALOG_DB`	`temco-store`'s `databasePath()`	The catalog, opened `ReadOnly`, WAL already on. Asked of the module that owns the file, not derived from this extension's own storage
`TEMCO_MCP_STATE`	`<globalStorage>`	Where `audit.db` is created
`TEMCO_PLATFORM_API`	`temcoAi.platformApi`, default `http://127.0.0.1:3210`	Where the device, repo, and project tools get their answers
`TEMCO_MCP_TRANSPORT`	`http`	Which of the two transports to serve. Absent means `stdio`
`TEMCO_MCP_PORT`	`temcoAi.serverPort`, or a free port the supervisor found	Where to listen, on `127.0.0.1` only
`TEMCO_MCP_TOKEN`	24 random bytes per start	The bearer token every request but `/health` must carry
`PATH`, `HOME` / `USERPROFILE`, `SystemRoot`, `TEMP*`, `LANG`	copied from the extension host	Nothing else is inherited; a curated list is auditable

Everything else is a setting read with `vscode.workspace.getConfiguration('temcoAi')` in `serverEnv()` and `server.ts`, never a file next to the binary. `TEMCO_MCP_LOG` is honoured if it is set (`information` puts the server's own per-call logging back); the extension leaves it alone, so the channel carries calls rather than the framework's chatter.

8.4 The wire contract

The server speaks both transports, chosen by `TEMCO_MCP_TRANSPORT`; the application uses `http`.

`stdio` — newline-delimited JSON-RPC on stdin/stdout, stderr for logs. One rule follows from a failure observed while writing this document: **the client must keep stdin open for the life of the session.** Driving the server from a file that ends immediately produced a full stderr log, executed tool calls, and *zero* responses on stdout. `StdioClientTransport` holds the pipe open; a hand-written probe must too. The server also never writes to stdout outside the protocol, which is why §5.3 sends logs to stderr.

`http` — `WebApplication` on `http://127.0.0.1:<TEMCO_MCP_PORT>`, with `WithHttpTransport()` in its default *stateless* mode:

Route	Behaviour
`GET /health`	`{"ok":true,"name":"temco-mcp","version":"1.0.0"}`. No token: it is how the supervisor knows the process is up, and it says nothing worth hiding
`GET /` and `GET /mcp`	The same facts in words: what this process is, that the protocol is `POST /mcp`, and that the application holds the token. No token, because someone who pasted the address into a browser was going to see *something*, and a bare 401 taught them nothing
`POST /mcp`	JSON-RPC. `tools/list` and `tools/call` work as the first request on a connection, because there is no session to establish
Any other path	401 unless `Authorization: Bearer <TEMCO_MCP_TOKEN>`

Three details the client depends on, each verified against the running server:

1. **No `Mcp-Session-Id`.** Stateless mode rejects the header outright (`400 … not supported in stateless mode`), so the client keeps no session state and one POST is one question.
2. **Answers arrive framed as events.** The content type is `text/event-stream` and the JSON is on one `data:` line, even for a plain request. A client that only tries `JSON.parse(body)` sees an empty answer.
3. **`/health` is not the same thing as ready.** It answers before the tool list does, which is why the supervisor polls it rather than the catalogue.

The listening socket is bound to `127.0.0.1` by name, never to `0.0.0.0`, and the token is generated per start: a loopback port is reachable by every process on the machine, including the three tools that write.

Pasting the address into a browser used to give 401 — correct, and useless. It now answers with the description above (`GET /` and `GET /mcp`, verified: 200 with the JSON, while `POST /mcp` without the header still answers 401 and with it answers the 23 tools). The Tool Server page says the same thing in one sentence, so nobody has to discover it from a status code.

8.5 Verifying it end to end

POWERSHELL

```powershell
# 1. the server answers MCP at all, without the application
npx @modelcontextprotocol/inspector <installRoot>\temco-mcp-win-x64\temco-mcp.exe
#    -> the inspector lists the tools from §5.7 and can call one

# 2. the runtime can call a tool at all. Both paths are printed by the extension on start:
#    "[ai] starting <label> as <runtime.path> on 127.0.0.1:<port>" in the "Temco AI" output channel
Set-Content -LiteralPath probe.json -Encoding UTF8 -Value '{"model":"local","messages":[{"role":"us
& "<runtime.path>" --model "<model.path>" --host 127.0.0.1 --port 8099 --jinja
#    in a second window, once the model has loaded:
curl.exe http://127.0.0.1:8099/v1/chat/completions -H "Content-Type: application/json" --data-binar
#    -> a working setup answers with choices[0].message.tool_calls, not prose

# 3. the whole turn, in the application
scripts\code.bat
#    -> ask in the Temco AI panel: "read input 1 of device 1234"
#    -> the "Temco AI" output channel shows [mcp] 23 tools from …, then the call, then the answer
```

Step 2 decides whether `--jinja` alone is enough for the chosen model. If the reply is prose with no `tool_calls`, the model's template is the reason, and the answer is `--chat-template-file` or a model whose template the runtime already knows.

8.6 The second exposure: the same server for Copilot Chat

The server is a plain MCP server, so the application can also declare it to the language model that ships with the product — same binary, same environment, started by the editor instead of by `temco-ai`.

Two ways to declare it, and they are different products: `McpStdioServerDefinition` gives the editor its own `stdio` process (the mode is still there, and needs no token because nothing else can reach a pipe), while `McpHttpServerDefinition` points at the managed process — which is only right while it is running, and is why the definition provider below would have to be re-registered when the state changes. The `stdio` one is the simpler contract for a second consumer.

TYPESCRIPT

```typescript
export function registerToolServer(context: vscode.ExtensionContext): vscode.Disposable {
  return vscode.lm.registerMcpServerDefinitionProvider('temcoAi.tools', {
    provideMcpServerDefinitions: () => {
      const binary = serverBinary(context);
      return binary
        ? [new vscode.McpStdioServerDefinition('Temco Platform Tools', binary, [], serverEnv(contex
        : [];
    },
  });
}
```

`McpStdioServerDefinition(label, command, args?, env?, version?)`; `version` is what makes the editor notice a changed tool list and offer a refresh. The provider is called eagerly and must not prompt — anything interactive belongs in `resolveMcpServerDefinition`.

8.7 When it does not start

Symptom	Cause	Fix
`spawn … ENOENT`	The artifact is not installed, or the manifest has no entry for this platform	Press Start on the Tool Server page; it installs for this RID (§7.1)
`spawn … EACCES` on macOS or Linux	The executable bit was lost in a copy	The `chmod` guard in §8.2
`Failed` with `address already in use` in the log	`temcoAi.serverPort` names a port something else holds	Set it back to `0`, or pick another
`did not answer … within 30s`	The process started and never served — a port it could not bind, or a crash before `app.RunAsync()`	The three lines the page shows are the end of stderr; the whole log is behind **Show the log**
The page says `failed` with `stopped (exit 1)`	The server exited on its own; the message carries the last lines it wrote	Read them first — an ASP.NET host failure names itself
`the tool server refused this window's token`	The process was replaced while a request was in flight (a restart)	Press Restart, or just ask again: the next call starts it
`SQLITE_BUSY`	A writer holds the catalog longer than 5 s	The `busy_timeout` in §5.6; a long write in `temco-store` is the real cause
The model reads `An error occurred invoking …`	The tool threw instead of returning `CallToolResult` — including a refused connection to the platform service	§5.4 rule 1
Tools listed, calls return prose	The model answered instead of calling	Tighten the tool descriptions; the prompt's instruction to call is the other half
`tool_calls` never appear	`--jinja` is missing	§4.1, then §8.5 step 2

9. Left panel and pages

The activity-bar group stays `AI` (`temcoAiHub`, owned by `temco-workbench`); `temco-ai` contributes four group views into it and keeps its own secondary-sidebar view.

Surface	Id	Content
Activity bar, `AI`	`temcoAiHub`	Group container
- Draft	`temcoAi.draft`	The turn in progress, and what is waiting to be settled
- What it does	`temcoAi.whatItDoes`	The acts, and the pages the answers leant on
- Server	`temcoAi.server`	The model: runtime, model, and what it is told
- Tools	`temcoAi.mcp`	What the assistant can reach outside this window
Secondary side bar	`temcoAi.panel` (`Temco AI`)	The conversation
Editor page	`temcoAi.page`	Today's settings page: the runtime and its prompts
Editor page	`temcoAi.tools`	The catalogue of calls, and the box that runs one
Editor page	`temcoAi.connection`	The tool server: its state, its address, Start/Stop/Restart, the port, the platform address, the log
Editor pages	`temcoAi.actions` , `temcoAi.citations` , `temcoAi.stale`	Unchanged

The `Tools` group is the way in. Three rows: the conversation, the connection, and the calls. Two of them open pages of their own, one each.

Row	Command	What it does
Open the AI panel	`temcoAi.openPanel`	Reveals the conversation; **the same row puts it away.** A plain open toggles, because the row a reader opens a thing with is the row they reach for again. A caller that names a page never closes anything
The Tool Server	`temcoAi.connection`	The program behind every call: whether it is running, where it answers, how to stop it, and the two settings it reads
The tools it can call	`temcoAi.tools`	The catalogue itself

The title bar carries the same two ideas as buttons, so the group is useful without opening a page. The log is **not** a fourth row: it is a button on the Tool Server page, where the rest of the program's state is.

Two pages rather than one, because a status and a list are different questions and the first one is asked when something is wrong. A page that answered both put a **Start** button under a catalogue that had to be loaded before the button could be seen — which is exactly the state (nothing is running) in which the catalogue cannot load.

`temcoAi.tools` is a page, not a tree view, because a flat list of rows cannot show an argument schema. Two parts:

- **The catalogue**: tabs, one per part of the API, taken from the first word of each call's name and kept in the order the server lists them, with a count beside each: `Device 8` , `Docs 4` , `Repo 5` . The strip is a plain row of words with their counts: twelve-pixel labels in the section's own face at six hundred weight, the first letter capitalised, no background under the pointer or under the open tab, and a single hairline in the theme's primary colour along

the bottom of the tab that is open. The strip wraps rather than scrolling, so no tab is ever hidden past its right edge. The calls of the open tab run across the row as cells: `device.list` over `Every T3000 device…`, `device.describe` over `One device in full…`, as many per row as fit, each description cut to three lines with the whole sentence on hover, so one long sentence cannot set the height of every row. The name and its sentence share one face — a name in monospace read as code, and these are not code. The name is a **link** that selects that call in the runner, seeds its arguments and scrolls there, which is the only thing a reader clicking a name could have wanted. The open tab shows six cells and carries the way to the rest at the right end of the strip: `Show 2 more`, which names what is held back, and `Show less` once it is out. There is no reads-and-writes split; the calls that stop for a person are named in one line under the strip instead of being a heading. Fetched through `tools/list` when the page opens — which starts the server if it is not up — so the list cannot drift from what the model is offered.

- **Run one**: the picker the links drive, the arguments as JSON seeded from the chosen tool's schema, and the answer. It calls the same `callTool()` the loop uses, so a write still asks the person first: the page is a second way in to the same gate, not around it.

When the catalogue cannot be read, the page says why **and offers the Tool Server page** rather than an install button: the reason is almost always that nothing is running, and starting it is a decision that belongs where its state is shown.

`temcoAi.connection` is the other half:

- **Status**: a word with a dot beside it — `Running`, `Starting`, `Failed`, `Stopped` — then the address it answers on (`http://127.0.0.1:<port>/mcp`), how many tools it offers, its process id, and how long it has been up. Under them **Start**, **Stop**, **Restart**, and **Show the log**. The buttons disable themselves while a start is in flight and when they are the wrong verb for the state, so nothing offers "Stop" to something that is not running.
- **The program**: installed or not, where, the platform, and the artifact id — with the `dotnet publish` line when there is nothing installed, because a source checkout is the case that has no release to download from.

Both **Where it answers** and **Platform API** were removed from this page, and each for the same reason: a page about one program should describe that program, not ask the reader to configure something they cannot yet use.

Removed	Why it was there	Where it lives now
Port field	A fixed port is only needed by something *outside* this window that has to reach a known address — an inspector, a script. Everyone else gets a free port per start, which is what stops two windows colliding	`temcoAi.serverPort`, in Settings. The address it produces is still shown, read-only, in **Status**
Platform API field	It names the HTTP service the device, repository and project tools ask. **No such service exists yet** (open decision 1), so the field asked a person to configure an address that could not answer	`temcoAi.platformApi`, in Settings; the `Temco AI: Platform API` command now opens the setting instead of a page

Four things the page does not do, each a decision:

1. **Nothing starts on open.** A status page that changes what it reports is not a status page.
2. **Nothing is guessed.** A stop and a crash both end in a process that is gone; only the exit itself separates them, which is why the failure branch carries the last lines the process wrote.
3. **The log is a button, not a row.** It is a page about one program, and the log is one of the things that program produces.

4. **The state is repainted from the module, not re-rendered.** Start, stop and restart each answer with a `status` message that the page's own script paints into the four facts it owns, so a field halfway through being typed is never thrown away.

Three older dialogs were removed in favour of these pages: a status toast that said only that the server was ready, an input box that asked for an address without saying what it was for, and a row that opened a page with no status on it.

10. Security

Concern	Rule
The model decides what to call	The loop is the boundary, not the model's goodwill: `DESTRUCTIVE` is a set in the extension, not a property read from the server
A write reaches equipment	Only after an explicit yes in the panel, and the yes is recorded with the arguments
The server is reachable from the network	It is not: the socket is bound to `127.0.0.1` by name, never `0.0.0.0`
Another process on this machine calls it	Every request but `/health` needs the bearer token this window generated at start. The token lives in memory, is passed in the child's environment rather than its arguments, and dies with the process
The catalog is modified	Opened `ReadOnly` (§5.6); writes stay in `temco-store`
A tool result is code	Results are text and are rendered as text, never evaluated
A downloaded server is tampered with	Pinned sha256 per platform where the catalogue has one; where it does not, the download is confirmed once with the file's origin and size in front of the person, and the checksum it had is recorded
Secrets	None are passed but the per-start token; the environment is a curated list, not a copy of the host's

11. Decisions

Settled, with the reason each one was settled that way:

1. **The server is HTTP, and the window manages it.** `TEMCO_MCP_TRANSPORT=http` is what the application starts, on loopback, with a per-start token. The reason is not throughput: a pipe has no address and no state, so a page could only ever say "ready" or nothing. A program that writes needs a stop button, and a stop button needs a process someone owns. `stdio` stays as the second mode, because it is what an inspector and a script want, and it is one `switch` in `Program.cs`.
2. **The server is downloaded, not bundled.** 108 MB and 344 files per platform in every installer is worse than a verified download on first use through the machinery that already installs models. A release attaches `temco-mcp-<rid>.tar.gz` for each of its three platforms, and the catalogue asks for the one that matches the running application's version. Bundling stays available for an air-gapped build by adding the artifact to the catalogue with a `file://` URL.

3. **Two pages, not one.** The catalogue and the connection were one page for a while. They are two questions asked at different times, and the state in which the catalogue cannot be read is the state in which the connection is the only thing worth showing.
4. **The log is a button on the Tool Server page, not a row in the panel.** The panel names things a person goes to; a log is what a person goes to *after* something on a page has gone wrong.
5. **The supervisor starts the server, not the loop.** `runAgent` used to own the process because it was the only consumer. It has three now — the loop, the runner, the catalogue — and the page that shows the address has to be looking at the process the others use.
6. **A missing checksum is a question, not a refusal.** The entry used to demand a setting be found before the first download, which made "Start" a dead end on a machine that had never enabled it. The confirmation carries more information than the setting ever did — the exact URL, the size, the destination — and the observed checksum is recorded either way, so nothing is lost by asking instead of refusing. Pinning the hashes stays the goal; it is a data task that needs one build to exist first, and `allowUnpinned` now means only "do not ask".

Still open:

1. **Who answers `TEMCO_PLATFORM_API`.** The device, repo, and project tools need one HTTP surface. Either a service in the product answers it, or `IPlatformApi` gains an implementation that speaks T3000 protocols directly and the URL becomes a connection setting. Only `HttpPlatformApi` changes either way.
2. **Where the audit lives.** The server owns `audit.db` in this document, while the extension already owns the store schema. One has to be authoritative before the first release.
3. **Whether the 7B model may act by default.** The 1.5B model may not; the catalogue can carry a per-model `maxTools` and a `mayCallDestructive` flag.
4. **Whether one process should serve several windows.** The token is per window and the port is per window, so two windows run two processes. A shared process would need a shared token and a supervisor outside the extension host; nothing needs it yet.
5. **Whether `/health` should need the token.** It does not, so the supervisor can poll it before it knows anything. It exposes the name and version of a program whose binary is on the same disk.

---

12. Where the code is

Section	File
4, the runtime	`extensions/temco-ai/src/ai/runtime.ts`
5.1 to 5.8, the server	`tools/temco-mcp/**` (both transports: `Program.cs`)
6.1, 6.2, the client	`extensions/temco-ai/src/ai/mcp.ts`
6.3, where the binary is	`extensions/temco-ai/src/ai/binPath.ts`
6.4, the loop	`extensions/temco-ai/src/ai/agent.ts`
6.5, the process, its port and its token	`extensions/temco-ai/src/ai/server.ts`
7.2, the catalogue entries	`extensions/temco-ai/src/ai/catalog.ts`
7.3, unpacking a tool server	`extensions/temco-ai/src/ai/install.ts`
8.3, the environment	`extensions/temco-ai/src/ai/binPath.ts`, `extensions/temco-ai/src/ai/server.ts`
9, the groups and the two pages	`extensions/temco-ai/src/aiPages.ts`, `extensions/temco-ai/package.json`
9, the commands	`extensions/temco-ai/src/extension.ts`
the one output channel	`extensions/temco-ai/src/ai/log.ts`
7.4, the release step	`.github/workflows/release.yml`

Authoring parity with Dr.Explain, publishing parity with MSDN

The product direction is two references at once, and they answer different questions:

- **Dr.Explain** is the *authoring* reference: how a person starts a project, brings existing material in, and edits topics with their properties in front of them.
- **MSDN** is the *output* reference: what a reader gets. The website is the product; CHM and PDF are the same tree in other clothes.

Where the two disagree, MSDN wins — a surface that looks like Dr.Explain but does not produce an MSDN-shaped site has failed the requirement.

1. Where we stand today

Everything below is what the code does, not what the mock draws.

Thing	Where	Reality today
A project	catalog `documents` row (`extensions/temco-store/src/catalog/schema.ts`)	name, slug, version, audience, owner, timestamps
What it reads	`doc_sources` (many-to-many, per-source branch + include glob)	created by the wizard's **Sources** step
What it builds	`recipes` (`engine`, `config_path`, `script`, `outputs`)	the wizard writes `engine: 'docfx'` and the ticked outputs
Outputs offered	`extensions/temco-projects/src/projectFlow.ts`	website · PDF · offline help (CHM) · search index · AI index
Creation flow	`src/projectFlow.ts`	4 steps: **Basics** (name, version, owner, audience) → **Sources** → **Outputs** → **Review**
Content	each product repository's `docs/help/registry.json` + Markdown (`extensions/temco-docs/src/registry.ts`)	the repository is the source of truth; the platform reads it
Page metadata already modelled	`RegistryPage`	`title, area, chapter, order, audience, ship, owner, tags, keywords, help_ids, code, reviewed`
Authoring already implemented	`newPagePage.ts` (3-step new page), `pageOps.ts` (`writePage` — the only disk write), `registryEdit.ts` (field writes)	a page is created from the location + template + draft, and registry fields survive regeneration
Builds	`extensions/temco-docs/src/builds.ts`	the **repository's own scripts** (`docs:site`, `docs:build`, `build:help:dry`) run as VS Code tasks, and success is judged by the artifacts
Checks	`diagnostics.ts`, `sections.ts`	structure, staleness, unreviewed pages

Two consequences worth stating plainly:

1. **We already have the Dr.Explain page properties.** `ship` is its "export conditions", `help_ids` is its "help context ID", `keywords` is its index, `order` is its TOC position, and `owner` / `audience` / `reviewed` are its bookkeeping. Nothing in the data model is missing for authoring.
2. **We are missing the front door and the import.** Creation starts at "name, version, owner" — a project is assumed to have content already. There is no way to begin from a Word file, an HTML folder, a CHM, a template, or a repository.

2. Dr.Explain's flow, from the surfaces

Start page:

- four ways in — **Create a local project**, **Create a shared project on the server**, **Checkout project from the server**, **Open an existing project**;
- six importers — **Microsoft Word**, **RTF**, **Compiled HTML Help (CHM)**, **HLP**, **Standalone HTML**, **folder of HTML/TXT/XML**;
- **recent projects** and **templates in all languages**.

Editor:

- topic tree with a synthetic *Title* / *Table of contents* root and numbered topics;
- **properties panel** per topic: title, filename, export conditions, status, header/footer, page numbering, help context IDs;
- central page with **Menu / Index / Search** tabs;
- toolbar previews: **Content editor**, **HTML preview**, **CHM preview**, **PDF preview**;
- AI assistant rail with the model/provider setting.

3. Gap analysis

Capability	Dr.Explain	Us today	What it means
Start from nothing	local project	**have it** (Basics step)	—
Start from a template	templates in all languages	no	a template is a repository-shaped skeleton; cheap and high value
Start from existing material	Word, RTF, CHM, HLP, HTML, folder	no	**the biggest gap**
Shared / checkout	server + checkout	git remote + clone exists in `temco-repositories`, not in the project flow	map to git, do not build a server
Topic tree editing	drag, renumber, nest	tree view + `order` field	add reorder → `order` write
Per-topic properties	full panel	registry fields, but no form	**form over `registryEdit`**
Export conditions	per topic	`ship`	expose it
Context IDs / index	per topic	`help_ids`, `keywords`	expose them
WYSIWYG body editing	HTML editor inside the tool	Markdown in the editor + rendered preview	do **not** copy; MSDN topics are Markdown
Preview before build	HTML · CHM · PDF	HTML preview exists; CHM/PDF only after a build	add CHM/PDF preview after first build
MSDN-shaped site	—	the site is the repository generator's; our output choice is `website`	make the contract explicit (§5)

4. Decisions

D1 — Creation asks *where the content comes from*, not a cosmetic "type". The origin decides which steps follow and what gets written on *Create*. A dropdown that changes nothing is worse than none.

Origin	What it does on Create	Writes
Blank project	creates the project; the first page is authored here	`documents` + `recipes`
From template	copies a template folder into the chosen location, then the project cites it	template tree + `documents` + `doc_sources`
From a repository (Dr.Explain's "shared")	clones/attaches a remote at a branch and reads its `docs/help/registry.json`	`sources` + `doc_sources`
Import a document set	converts the material into pages, then the project cites the folder	Markdown tree + registry entries + `doc_sources`
Open existing	picks a folder that already holds a registry	`sources` only

D2 — Importers write the repository's contract, never a second content store. An importer is a converter: source → `docs/help/*.md` + `docs/help/registry.json`, exactly the shape the product's own generator writes. The platform database stays what it is (a register of projects, sources, builds, releases) — otherwise the same text would live twice and drift.

D3 — MSDN is an output contract, not a theme. See §5.

D4 — Page properties are ours already; add only what MSDN needs. Expose `ship`, `order`, `keywords`, `help_ids`, `audience`, `owner` through a form in the page editor. Two new optional fields are worth having: `kind` (concept · how-to · reference · tutorial — MSDN's `ms.topic`) and `applies_to` (the product versions a page is for). Header/footer and page numbering are **template** settings for PDF, not per page.

D5 — Collaboration is git, not a server. "Shared project" = a project whose source is a repository with a remote; "checkout" = clone + branch; review is a pull request. `temco-repositories` already clones, pulls and records branches. Building a Dr.Explain server would duplicate what the repository host already does better.

D6 — The website is the product; CHM and PDF derive from it. One tree, three targets: website (primary), CHM (same topics, `help_ids` → HH context IDs, `keywords` → index), PDF (one print profile).

5. The MSDN contract (what "similar to MSDN" means concretely)

Structure. `area → chapter → topic`, the shape the registry already has. A topic is one Markdown file; `order` decides TOC position; a page not in the registry is not published.

URL. `/{area}/{chapter}/{slug}` — stable, lowercase, no build hash. Renaming a slug writes a redirect entry, because links outlive builds.

Every topic page carries:

- breadcrumb `Area / Chapter / Title`;
- left **TOC** of the whole set with the current branch open;
- an **on-page TOC** built from `##` / `###`;
- **Applies to** (from `applies_to`, or the project version);
- **previous / next** in TOC order;
- **feedback** and a "this page is stale" signal from `reviewed`;
- a permalink for every heading.

Index and search. `keywords` feeds the site index; the search index (already a wizard target) is built from titles, headings and keywords, so a reader can reach a page by a controller's name as well as by its title.

Version selector. A released version is a `releases` row; the site serves the newest plus archived versions. The version selector is a list of those rows, not a rebuild.

CHM. The same tree compiled with `help_ids` as context IDs, `keywords` as the index, and the chapters as the TOC levels. A topic that is `ship: false` is in the site's draft preview and not in the CHM.

Where the furniture is produced — and why Temco Learn is not it. A topic page is rendered by the product repository's own site generator (`docs:site`), and Learn's *Article Preview* opens the published file rather than drawing a second copy of it (`extensions/temco-workbench/src/learnPages.ts` : *"a preview that can disagree with the shipped page is worse than no preview"*). So the platform's half of the contract is **the metadata that makes the furniture possible** — `order` for the TOC, `keywords` for the index, `ship` for what is published, `help_ids` for the CHM, `title` for the heading and the breadcrumb — and the generator's half is drawing it. Adding a page-properties form (phase 2, below) is what closes the loop between them.

6. The project flow, redesigned

Five steps; step 2–4 are the steps we have, unchanged in content.

1. **Origin** — Blank · Template · Repository · Import · Open existing.
2. **Basics** — name, version, owner, audience (today's step 1).
3. **Bring in** — *shape follows the origin*: template picker · repository picker (remote + branch) · importer picker (Word/RTF/HTML/HTML folder/CHM) with a destination folder · folder picker.
4. **What it builds** — the targets we already offer, with **Website** ticked for a new project.
5. **Review** — what will be written, and where the files land. Nothing is written before this step.

Two rules stay: nothing is written until the last button, and the project you create becomes the selected project.

7. Phases

Phase	Deliverable	Status
1	`documents.origin` + `documents.template` (schema 4, with a migration); wizard step **Origin** (blank · repository · folder · template · import), where the origin decides what the third step asks; a template writes a starter `docs/help` tree and refuses to overwrite anything; folder and template projects record their folder as a source	**done**
2	Page-properties form (`temcoDocs.pageProperties` , Markdown title-bar action) over `registryEdit` : title, owner, audience, order, ships, keywords, tags, `help_ids` , with the confirmation naming every value it will write	**done**
3	Importers (`temcoDocs.import` , called by the wizard's *Import a document set* origin): **HTML folder / page** and **Markdown folder** natively; **Word / RTF / ODT** through pandoc; **CHM** through `hh.exe -decompile` (Windows). All four write the repository contract and refuse a folder that already has a registry	**done**
4	The platform's half of the MSDN contract: the metadata itself (`title` , `order` , `keywords` , `ship` , `help_ids` , `audience`), which phase 2 made editable and which the module's existing validation already reports on. A separate readiness page was built and then removed: it re-tested the same fields without adding a decision, and the Pipeline group already has Review Center and Build Center. The drawing half — breadcrumb, on-page TOC, prev/next, permalinks, redirects, version selector — belongs to each product repository's `docs:site` generator, which is not in this repository	**platform half done**
5	CHM/PDF previews after a build (`temcoDocs.openArtifact` , from the Build Center's artifact rows); drag reorder → `order` (already in `pageOps.handleDrop`); the repository origin cites recorded sources and links to the Sources page, where `temcoRepositories.fetchSource` clones	**done**

Phase 1 is the answer to "do we add more types?" — yes, five origins, and each one changes what the next step asks.

8. What we deliberately do not copy

- **Its WYSIWYG HTML editor.** MSDN topics are Markdown; the editor is VS Code's, and the page renders beside it.
- **Its project database.** The repository keeps the text; the platform keeps the register.
- **Its server and checkout.** Git and pull requests.
- **Its single-file exports as the main path.** The site is the product; CHM is one of its outputs.

Modules

A module is a real VS Code extension inside the application. Namespaces follow the module (../architecture/naming.md).

Register

Module	Purpose	Namespace	State
`temco-workbench`	the application's own surfaces: the Library, the dashboard, what needs doing	`temcoWorkbench`	**active**
`temco-ui`	where a page of ours opens, what a command argument means, and the shared page components (hero, figures, sections, cards)	`temcoUi`	**active**
`temco-store`	the platform's own memory: the catalog database, the selection, the naming rules	`temcoStore`	**active**
`temco-docs`	the registry, pages, checks, and the writes	`temcoDocs`	**active**
`temco-learn`	the published site: its app, the site folder, the panel, the preview, the downloads the deploy cannot make, and the push to `gh-pages`	`temcoLearn`	**active**
`temco-editor`	the editor for a page: the Markdown surface, its ribbon, its engine seam	`temcoEditor`	**active**
`temco-repositories`	the organisation, local copies, the working area, reading a source	`temcoRepositories`	**active**
`temco-ai`	the local runtime and models, the prompts, the panel, drafts and answers	`temcoAi`	**active**
`temco-projects`	projects, sources, outputs, builds	`temcoProjects`	**active**
`temco-sql`	registers, queries, export	`temcoSql`	reserved
`temco-nettools`	devices on the wire	`temcoNettools`	reserved
`temco-bacnet`	objects, points, trends	`temcoBacnet`	reserved
`temco-device-manager`	fleets, firmware, health	`temcoDevices`	reserved
`temco-tools`	the Tools container until those four modules exist: Devices, I/O Studio, BACnet Studio, Network and SQL Studio, as pages over a sample	`temcoTools`	**active**

States

State	Contents	Build effect
reserved	`README.md` only	the build ignores the folder: nothing is compiled, nothing ships, no container appears
active	`package.json`, `tsconfig.json`, `src/`	compiled and shipped as a built-in extension

A module becomes active on the day it has a first real feature. Reserving a module with a manifest would ship an extension with nothing in it, which is why a folder becomes a module only when it has something to do.

Ownership

`temco-store` — the platform's own memory, and only that: one SQLite file per window (sources read, projects, their sources, the pages a read found, builds, artifacts, commits, events), the **selection** (which project the window is on, and the page it last opened), and the naming rules the tables are keyed by (a slug). It has one command, no views and no settings. It is the base every other module stands on, and it is deliberately *not* `temco-sql`: that module is about the databases a **product** uses, and it neither owns nor knows about this file.

`temco-docs` — the content side: the registry of areas, chapters and pages, the structural checks, the **Workspace tree** (its own view), the pages, the Markdown surfaces, the reports and the builds, and the five writes. It owns the only write a page ever gets, and it publishes the documentation model other modules read (`src/api.ts`) — the Library and the dashboard are the workbench's, and they read that model rather than a copy of it.

`temco-ui` — the shared UI conventions, and nothing else: the editor column a page of ours opens in (one group, never a split), what a command argument names (a slug, a row, or a row's own command argument), and the page components a first screen is built from — the banded hero, the band of figures, a section heading, and a row of cards. It has no command, no view and no setting.

`temco-workbench` — the application's own surfaces, and the only module that is not a domain: the **Library** (the index of what every module has; each row's action is that module's own command), the **dashboard** (the application talking about itself), the **needs page**, which tab opens at startup, and which sidebar section comes forward when the project changes. It writes nothing and owns no data: its rows come from the store and from the documentation module's published model.

`temco-ai` — the local model: the runtime and models downloaded on request into the user's own folder, the prompts, the AI panel, drafts and their diffs, and the citations that open the page they name. It reads the documentation through `temco-docs` and never writes a page.

How two modules may talk

Only one shape is allowed, and it runs in one direction:

- **A provider publishes, a consumer declares.** `temco-ai` requires `temcocontrols.temco-docs` (`extensionDependencies`), so that module activates first and its exports — the sets, the pages, the selection, the editor column, one change signal — are there when the consumer's `activate()` runs. The shapes are spelled twice, once in each module (`src/api.ts` in the provider, `src/docs.ts` in the consumer), because an extension cannot import another extension's sources.

- **A consumer never requires a provider that requires it.** That is a cycle, and a cycle is a load-order bug waiting to happen. The second direction is always a *command* the first module calls and may find missing, and the caller gates what it shows on a context key it sets itself (`temcoDocs.aiInstalled`).

`temco-docs` and `temco-ai` are the worked example: one declared dependency, one optional command surface. `temco-store` is the other shape a module can have — a base with no surface of its own, which every other module requires and none calls back into.

`npm run check-temco` refuses the mistakes this section warns about: a dependency graph with a circle in it (VS Code disables *every* extension in a circle, so the application comes up with a silent sidebar), a menu, submenu or welcome link that names an id no module declares, a view contributed under a container no module declares (the workbench shows nothing at all — no icon, no panel, no error), a container id two modules both claim, and a module-level binding that collides with the CommonJS wrapper (`module` , `require` , …), which throws before the extension runs.

`temco-repositories` — the organisation and the local copies: the repository list, a source's files, cloning one branch with full history, `pull --ff-only` , removing the local copy, reading a file's text, **reading a source** — the registry a repository declares, from a folder or over the API, recorded into the store — and the **vocabulary of the repository's own documentation scripts** (which scripts exist and what each is for). It owns the working area (`temcoRepositories.workspaceFolder` , default `~/Temco`). It reads the platform's memory through `temco-store` and the shared page rules through `temco-ui` , and it needs nothing of `temco-docs` : what a registry *means* is that module's business, what a repository *has* is this one's.

`temco-projects` — a project as a first-class thing: name, version, audience, owner, the sources it is made of, the outputs it produces, and what building one left behind. Its pages are the list, the hub and the edit form. It reads sources and the repository's own scripts through `temco-repositories` , the platform's memory through `temco-store` , and it needs nothing from `temco-docs` : the pages a project *has* were recorded in the store by a read.

`temco-sql` — registers, queries and export against the databases the products use. It owns its own connections and storage.

`temco-nettools` — discovery, addressing, capture and inspection. The transport layer the device modules build on.

`temco-bacnet` — BACnet objects, points and trends, read through `temco-nettools` transports. It owns its object and point models and its storage.

`temco-device-manager` — fleets and health: inventory, firmware, status and the actions a person takes on a device. It reads devices through `temco-bacnet` and `temco-nettools` .

Folder

TEXT

```
extensions/temco-<module>/
├── README.md              purpose, identifiers, what it owns — the only file a reserved module has
├── package.json           id, engine range, commands, views, settings, activation
├── tsconfig.json          extends ../tsconfig.base.json
├── src/
│   ├── extension.ts       the only activation entry: register, wire, dispose
│   └── <feature>/
├── media/                 icons, webview css and js
├── prompts/               model instructions as text, where a module needs them
└── scripts/               module-local gates
```

Its documentation is `docs/temco-platform/modules/temco-<module>/` **in this repository**, not a `docs` folder inside the module. There is no `.vscodeignore`: the platform build packages a built-in module, `vsce` does not.

Contract

1. Modules talk through commands, settings, files and events. No module imports another module's source.
2. The catalog belongs to `temco-docs`. Other modules read it through commands.
3. Each module owns its own storage under its own extension id — never inside a repository, never committed.
4. One module, one namespace. A command id is registered exactly once.
5. A module is a built-in extension. It uses the published `vscode` API only and never the shell's internals, so it still runs in an ordinary VS Code window.
6. A module never assumes the shell.

Adding a module

1. Create the folder with the shape above.
2. Write `package.json` with its id, commands, views, settings and activation events.
3. `tsconfig.json` extends `../tsconfig.base.json`.
4. Store everything under its own id's storage.
5. Add its gate to the release list: `tsc`, plus its own behaviour check where it has one.
6. Write its page in `docs/temco-platform/modules/temco-<module>/`: commands, settings, storage, and what it must not own.
7. Compile it with the platform build (`npm run gulp compile-extension:temco-<module>`) and check it loads.

Nothing else is required: the build picks up any folder with a `package.json`, the root install brings in its dependencies, and web builds skip it automatically.

`temco-ai`

The local-AI module: it runs a model on this machine and answers from the documentation it is given.

Folder	`extensions/temco-ai`
Extension id	`temcocontrols.temco-ai`
Namespace	`temcoAi`
State	active
Container	**Temco AI** (`temcoAi`) in the right side bar — the panel (`temcoAi.panel`)
Requires	`temcocontrols.temco-docs`
Settings	`temcoAi.modelFolder` , `temcoAi.allowUnpinned` , `temcoAi.runtime` , `temcoAi.model` , `temcoAi.idleMinutes`

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, the panel, and the model the rest of the platform reads (`status` , `facts` , `modelLabel` , `analyse` , `draft`)
`src/docs.ts`	what this module reads of `temco-docs` — the contract in one file, and the only place that crosses modules
`src/surface.ts`	the AI page's view type
`src/aiPage.ts`	the page: the runtime, the model, the folder they live in, the downloads, and the log
`src/ai/catalog.ts`	what can be downloaded: the pinned llama.cpp build with the checksums GitHub publishes, the models on offer, their licences. No network, no filesystem
`src/ai/install.ts`	the model folder, the resumable download and its checksum check, the unpack, and the manifest of what is installed
`src/ai/runtime.ts`	the local server process: started on demand on a loopback port, health-checked, stopped when idle — and the single `complete()` call everything else uses
`src/ai/machine.ts`	what this machine can run, and which catalog entry suits it
`src/ai/history.ts`	the commits a draft is about: `git log` over the page's `code` globs since its last review, metadata and paths only. Never throws
`src/ai/prompts.ts`	loads `prompts/*.md` and fills them in: the house rules, the question, the draft, the area, a new page. The only text that may reach the model
`src/ai/citations.ts`	what an answer pointed at: the ids that resolve, the ids that do not, and the plain term search behind a low-confidence fallback
`src/ai/panel.ts`	the panel: chat about the open page, `Draft update` as a diff, `Ask the area` , and the model line. Decides nothing itself
`src/ai/commands.ts`	the list a person picks from: install, use, remove, open the folder, and the AI's status
`media/`	the panel's script and styles (`ai-panel.js` is a browser asset, not extension code) and the module's icon
`prompts/`	the templates the module sends to the model, one file per task

Commands

Command	What it does
`temcoAi.page`	opens the AI page, or one of its sections (`downloads`)
`temcoAi.openPanel`	focuses the panel in the right side bar
`temcoAi.models`	the list of runtimes and models: what is installed, what can be downloaded, which one is in use
`temcoAi.removeModel`	deletes an installed runtime or model, and clears it from the settings when it was the one in use
`temcoAi.openModelFolder`	opens the folder the downloads live in
`temcoAi.dockRight`	shows the AI in the right side bar
`temcoAi.status`	which runtime and model are in use, whether the server is running, and how to stop it or read its log

How it reads the documentation

One declared dependency, in one direction:

- this module **requires** `temcocontrols.temco-docs` , so that module is activated first and its exports — the sets, `collectPages` , `pageForUri` , the selection, the editor column, one change signal — are ready when this module's `activate()` runs. `src/docs.ts` is where they are declared and where a call reaches them.
- the documentation module **requires nothing** of this one. It calls `temcoAi.page` and `temcoAi.models` for what a person opens, and asks this module's exports for the two things that are work rather than navigation: a body to draft, and a source to analyse. It gates the rows and menu entries it shows on `temcoDocs.aiInstalled` , so an install without this module shows no door that does not open.

The tool server

The assistant can call tools: a C# MCP server lives at `tools/temco-mcp` and this module is its client. The protocol between them, the tools it exposes, how the server is built, and how the application starts and stops it are specified in `design/ai-assistant-and-mcp-server.md` .

What it never does

- Writes a page. A draft goes back to the page that asked for it, and a person accepts it as one undoable edit.
- Leaves the machine. The runtime listens on loopback, models arrive only from a command a person ran, and the only thing that ever leaves is an API read the documentation module asked for.
- Ships native code. A compiled engine is per-OS, so it is downloaded on request into the user's own folder.

Temco Docs — documentation

This folder documents the **Temco Docs module** (`extensions/temco-docs`): what it does, how it is built, what it assumes, and where it is going. The platform-wide design is design/temco-learn/.

The product documentation itself is not here: the pages and the registry live in the product repositories, because that is where the content has to be.

Document	Read it when
`architecture.md`	you need the whole picture: the repositories, the flow, how it ships
`registry-schema.md`	you touch anything that reads or writes registry data
`extension-design.md`	you add a view, a command, a check or a diagnostic
`platform-design.md`	you work on the platform itself: sources, the documents a person creates, builds and artifacts, the AI over history, and the catalog
`editor-design.md`	you work on editing: the toolbar, creating a page, images, drafts, and the write rule they wait on
`app-flow.md`	you work on the left toolbar, a panel, or where a row leads: the four containers, what each view asks, and the page behind every row
`authoring-flow.md`	you work on screenshots, the side bar, or when a page must be re-illustrated: the capture stage, the two views, and what still waits on a decision
`app-layout.md`	you need the target UI: the application window, the site's generated dashboard, and the seam between them
`design/temco-learn/`	you need the platform design itself: content model, registry, search, the offline CHM, editing, feedback, AI, governance

The three names, and why they are different

Name	What it is
Temco Platform	the application: the branded build of VS Code, with these modules built in
Temco Docs	this module — the documentation features, in the application and in an ordinary VS Code window
Temco Learn	the published site: the documentation people read

One name for the tool, one for the module, one for the artifact. The module maintains content; Temco Learn displays it. Keeping them apart is what stops "put it in Temco Learn" from being ambiguous.

The guide, and why a module has one

`temcoDocs.overview` opens **Documentation** (`src/overview.ts`): the tree answers "where is this page", and this page answers "what am I looking at" — pages, areas, how much is behind its source, which faults the checks found, and the three checks that can be run from here. Every number is computed from the same registry the tree reads, so

the page and the tree cannot disagree. It is also the **one owner of per-area coverage**: a *Coverage & Health* page used to exist on the workbench dashboard, drawn from this same registry, and was removed rather than left to drift.

How To Use was removed: it was a mock guide whose numbers were typed in ("13.0 is current", "17 pages are waiting"), and a guide that quotes stale figures teaches the wrong thing.

The rest of the container is the mock's Documentation module, page for page: **Projects**, a repository, **Sources**, **Documents** and a page, a **Source Explorer**, **Review Center**, **Build Center** and **Releases** (`src/docsPages.ts`). The side bar holds the mock's sections and rows — Pages, Projects, Sources, Areas, Pipeline, Working Set — plus the page tree itself, which is why there is no separate *Docs Explorer* activity-bar icon: one icon browses one object.

The Review Center is a queue and the draft it opens: a vertical tab list on the left (each row the draft's name, the id it proposes, how long it has waited, and the source it read), and on the right exactly one draft in full — its kind, its line count, the text exactly as it will be written, and the two acts that settle it. Accept and Discard live in the draft rather than on the queue's rows, because an act that writes a page belongs beside the text it writes and a button on a row invites settling a draft nobody has read; the queue's rows are a tab list, so a keyboard reaches them the way it reaches the tabs on Publish & Releases. Only the newest is marked `Latest` , and choosing another row is what makes it the one on the right.

The Build Center is two answers, in the order a person needs them. **What this project builds** — a card per output the project was created with (Website, PDF, Offline help, Search index, AI index), drawn in a row like the create form's own ticks: what the output is for, the script it would run *here*, the files it has now, and the sentence saying why when this repository has no script for it. Only a card with a script carries a **Build** button, and the card itself is never the button — reading a target must not start a build, and a target that cannot be made is reported rather than approximated with a different one. **Builds** — the runs, newest first, as a record rather than as a stack of cards: one list, a hairline between two runs, `#12` , `Latest` , when it started, how long it took counted from the two timestamps, and the verdict as a word. It folds after five runs. Above them, four figures state the project at a glance (builds · files · written · last build), and the files themselves are not reprinted here: they are listed on **Publish & Releases**, one block per output, with the fold a hundred and twenty pages need. A build history is a sequence, and a box around every run says each one is an unrelated object.

The title block carries **only the two acts this page owns** — `Build the Site` and `Build the Offline Help` — plus the per-output `Build` buttons on the rows below them (`temcoDocs.buildOutput`). `Refresh` was removed (reopening a page re-reads it) and `Publish & Releases` became a link in the files section it belongs to: three of four buttons in a title block were navigation, which is what made the page read as a menu rather than a record.

Every one of them is drawn with the shared page vocabulary from `temco-ui` (`page.css` + `page()`), so they are the pages whose look is the mock's design rather than a private `<style>` block. The pattern they establish is the one the module's other pages move to: a page describes what it draws, and `temco-ui` decides how that looks and how a click becomes a command.

Publish & Releases reads the same list of outputs, in the same order, and shows them as the tabs of "The files a reader can be given": a row of tabs (built, with its file count and size; or the line saying what makes it, since the search index is written by the site when a build is imported; or dashed and greyed when the create form did not tick it), and under the row exactly one panel — the files of the chosen output, the first six and a native **Show N more files** / **Show less** fold, or the callout saying why there is nothing to list. Choosing a tab is presentation only, answered by a small script the page carries: every panel is rendered, the first is open, and a window with scripts off still shows the files. The tabs are a real tab list — `aria-selected` , a roving `tabindex` , Arrow/Home/End — and the categories are never derived from whatever happens to be in the artifacts table: a target a person asked for is shown whether or not a build has produced it yet.

Where the platform design lives

`temco-learn/design/` — 17 documents covering the content model, the registry, search, the offline CHM, the editing and git flow, feedback, the AI features, the implementation plan and the governance rules. `16-overview.md` is the one-page version.

`design/authoring-and-msdn-output.md` is the other half: what an author gets, and what a reader gets. Its §5 is the **MSDN contract** — the concrete meaning of "looks like Microsoft Learn" that a surface is judged against — and §6 is the project flow that follows from it.

They describe the platform, and their paths belong to the product repository. `docs/t3000/**`, `docs/help/registry.json`, `docs/docfx.json`, `scripts/docs-*.mjs` and `dist/docs/` are the *product* repo's paths, not this one's: the content and the publishing pipeline stay there by design.

Where the two disagree, the design package wins — and the disagreement is a bug worth fixing in one place or the other.

Architecture

The repositories

TEXT

```
C:\QN\temcocontrols\
│
├─ T3000Webview-EEZ-Studio        the product repo — the content lives here
│   ├─ docs/t3000/**              the pages, one folder per chapter
│   ├─ docs/help/registry.json    the structure: areas, pages, owners, order
│   ├─ docs/help/index.json       generated: headings + git provenance
│   ├─ docs/toc.yml  **/toc.yml   generated navigation
│   ├─ docs/templates/t3000/      the site template: brand, page-meta bar
│   ├─ dist/docs/                 the rendered copy, and the package the site reads
│   └─ scripts/docs-*.mjs         the build side: registry, gates, changelog, media
│
├─ temco-platform                 this repository — the fork of VS Code: the shell, the modules, the
│   ├─ extensions/temco-docs/     the module — every documentation feature lives here
│   │   ├─ src/registry.ts        reads the registry, runs the structural checks
│   │   ├─ src/tree.ts            the Content view: areas, chapters, pages, gaps
│   │   ├─ src/diagnostics.ts     problems, in the file that has to be fixed
│   │   ├─ src/extension.ts       view, status bar, commands, file watcher
│   │   └─ src/…                  builds, report, scaffold, feedback, ai/ — the module table is in e
│   │   └─ media/docs.svg         the view icon
│   ├─ docs/temco-learn/design/ the platform design (17 documents)
│   └─ product.json               branding of the application: name, icon, protocol, gallery
│
└─ docfx                          the publishing engine — upstream, pinned
                                  never modified; a version bump only
```

The flow

TEXT

```
open a product repo in the editor
     │
     ▼
read     docs/help/registry.json   →  areas · chapters · pages · owners · order
     │
     ▼
show     Content view              →  pages, chapters, issues, never reviewed
     │                                Gaps → only the pages with a real fault
     ▼
check    file exists · id agrees with area/chapter · title unique · order unique
     │
     ▼
fetch    the runtime and a model are downloaded on request, into the user's own folder
     │
     ▼
serve    the local server starts on demand, on loopback only, and stops when it is idle
     │
     ▼
draft    the local AI reads a commit range and proposes page edits
     │
     ▼
approve  a named human accepts; nothing else ever writes
     │
     ▼
publish  the engine         →  the package  →  the site
         the help build     →  CHM          →  ships with the installer
```

How it ships

```text
the application   →  TemcoStudio-Setup.exe
                     ├─ the branded build of VS Code  ("Temco Platform")
                     └─ + the module, built in — one installer, one update path

plain VS Code     →  the same module, run from the platform's sources
                     (it uses the published `vscode` API only — nothing shell-specific)

same module · same views · same checks · same AI — the only difference is who builds it

what a customer installation gets

T3000
  └─ T3000_Help.chm      the only artifact from this platform
                         no extension, no editor, no AI, no registry
```

What it publishes (target)

```text
/                        the home page: what is inside, what changed recently
/t3000/latest/…          a product, current — the pages exactly as they are today
/t3000/13.0/…            the same pages for a pinned version
/t3000/13.0/downloads    the PDF, the help file and the offline copy
/_admin/                 health: owners, gaps, staleness — noindex
/firmware/               a second product later, from the same pipeline
```

The three boundaries that must not blur

1. **Content never moves.** Pages stay at `docs/t3000/**` in the product repo, which is why the application, the MCP tools and the F1 help keep working untouched. This extension reads them, and writes to one only when a maintainer asks and confirms the exact path: the file for a page the registry declares, a draft they applied, or an empty registry where a folder they have open has none. Nothing under `_admin/` or `_edit/` is ever written — those are generated pages — and nothing else in the folder is touched. Nothing is ever committed. One deliberate exception: the platform **design** lives at `docs/temco-learn/design/`, because the design of a thing belongs with the thing. The product repo keeps the content and the pipeline; it does not register or publish the design pages.
2. **The application is packaging, not features.** The shell holds the branding, the default layout and the build. A module must never depend on something only the shell provides, or it stops working in plain VS Code.
3. **One artifact crosses into a customer installation.** The CHM. No editor, no registry, no AI runtime.

Decisions already made, so they are not re-litigated

- The publishing engine is **pinned, never patched** unless a real need appears; the fork of it, if it ever happens, lives as commits in the project's own repo, not as loose patch files.

- One repository for the whole product. No submodules in either direction — the module is built in, not consumed as a package.
- The AI drafts and a named human approves. Nothing publishes itself.

Extension design

> **Layout superseded.** The views, the Pages tree behaviour and the flow are defined in ../../ui/redesign.md. Files such as `home.ts` and `views.ts` mentioned below do not exist; the code is under `extensions/temco-docs/src/`.

Responsibilities, one file each

Module	Owns
`src/registry.ts`	reading `docs/help/registry.json` and the generated `index.json` / `page-meta.json`, the page id → file rule, the structural checks, the counts. No VS Code UI beyond `Uri`, `WorkspaceFolder` and the filesystem
`src/tree.ts`	the Content view: which nodes exist, what they show, what a click does
`src/diagnostics.ts`	which issues become Problems, and on which file
`src/extension.ts`	activation, the view and status bar, the commands, the file watcher, and the model it publishes to other modules (`src/api.ts`)
`src/store.ts`	what this module reads of `temco-store` — the catalog, the selection, the slug rule — in one file, and the only place that crosses modules
`src/ui.ts`	what it reads of `temco-ui` — where a page of ours opens, and what a command argument names
`src/projects.ts`	what it knows of `temco-projects` — one signal, so the Library's rows come back when a project changes
`src/repositories.ts`	what it reads of `temco-repositories` — a source's code files, the material for a draft, and the repository's own scripts
`src/api.ts`	what this module publishes: the documentation model `temco-ai` reads (the sets, the pages, the selection, the editor column, one change signal)
`src/aiModule.ts`	what this module knows of `temco-ai` : whether it is installed, its commands, and the two calls it makes when it is (a draft, an analysis)
`src/builds.ts`	the product repo's own builds, invoked as VS Code tasks and judged by the artifact they leave — never by an exit code, and never the form that deploys
`src/report.ts`	the staleness report: what needs attention, grouped by owner, as Markdown. Counts rather than lists for the pages that have merely never been reviewed
`src/scaffold.ts`	starting a registry in an open folder that has none: the smallest valid file, on a confirmation, with the seeding left to the repository's own generator
`src/home.ts`	the dashboard: the one page that is not about a folder — identity, how the application works, what exists now, how to start, and every feature with its state
`src/views.ts`	the other two sidebar sections: **Work** (what needs doing, and what changed) and **Library** (what it reads and produces)
`src/feedback.ts`	feedback rows routed to their page's owner, against a proposed file shape, with the aging called out
`src/previewPanel.ts`	the page as a reader sees it, in its own editor tab beside the source: one panel per document, redrawn as the page is typed. Rendering is the built-in Markdown pipeline, so the preview and the editor's own idea of the page cannot disagree; the DocFX `> [!NOTE]` markers are dressed as the alerts the site publishes

Module	Owns
`src/diffPage.ts`	the page against its last commit, in `vscode.diff`. HEAD's text is read with `git show`, written under the extension's storage, and deleted when the tab closes. The right-hand side is the editor's buffer, so an unsaved change is part of the diff it shows
`src/outline.ts`	the **Page** pane: the page's headings as a nested jump list, and its word count in the status bar. Fenced blocks are skipped, because a `#` inside a code sample is a comment
`src/fluentEditor.ts`	**Temco Markdown**: the authoring surface — the page as a document on Fluent UI, with capture and annotation in the surface. The default editor for `*.md`; it owns the view type and the open command, and nothing of the protocol
`src/markdownEdits.ts`	what bold, a list, a heading, a table and an image **mean**, as pure transforms over text and offsets — plus the writing behaviours (`continueList`, `indentLines`) and the search (`findMatches`, `replaceAll`). Both editors call it — Monaco through `src/format.ts`, the Fluent surface directly — so the two cannot drift into producing different files
`src/attachments.ts`	where an image lands: the `media/` folder, the `NN-slug` name, the size limit, the width warning, and the record beside it. One path for the Monaco command, the surface's paste, a capture and a file picked from disk
`src/attachmentRecord.ts`	that record as data — parse, serialise, add a caption, decide what is stale — with no `vscode` import, which is what makes the rule testable
`webview/docs/main.tsx`	that surface's shell, its caret rules, its paste matrix, its caption question, its outline, its find bar and its view modes; `annotate.tsx` is the canvas annotator beside it, `highlight.ts` the colour behind the text, `find.tsx` and `outline.tsx` the two panels, `theme.ts` the `--vscode-*` → Fluent token bridge, `markdown.ts` the page renderer
`scripts/check-fluent.mts`	the content-library boundary: only `webview/docs` may import `@fluentui`. `npm run check-fluent`
`scripts/check-docs.mts`	the Fluent surface's bundle loaded into a DOM and driven the way the host drives it — it renders, `ready` is posted **once**, the formatting actions produce the right Markdown, an identical `set` disturbs nothing, an outside change still arrives. `npm run check-docs`
`scripts/check-record.mts`	the attachment record through a table of cases, including the one that matters: a picture with no recorded version is never called stale. `npm run check-record`
`tsconfig.webview.json`	the webview sources type-checked on their own, since `tsconfig.json` covers the extension host only. `npm run check-types`
`scripts/check-learn.mts`	the Temco Learn portal: what a `manifest.json` means, and what the page it writes says. The first coverage of a file written on every publish and read by nobody. `npm run check-learn`
`src/learnPage.ts`	the portal page and the manifest rules, as pure functions — no `vscode`, so the publish path has something to test
`src/webviewText.ts`	the bridge every webview surface shares: one `WorkspaceEdit` per save, line endings preserved, the echo guard, the capture handshake, and the bar actions (preview / diff / source / properties). Nothing that could corrupt a file is decided twice

Module	Owns
`src/csv.ts`	CSV as a grid can hold it and back again: RFC 4180 quoting, delimiter and end-of-line detected, and the *quoting style* preserved per cell so a one-cell edit is a one-line diff. No VS Code here at all
`src/gridEditor.ts`	**Grid mode**: a CSV edited as cells (Univer). Same bridge as `webviewText.ts`; the file stays line-per-row because `csv.ts` does the writing
`scripts/build-webview.mts`	the one bundler step: `webview/grid.ts` → `out/webview/grid/` and `webview/docs/main.tsx` → `out/webview/docs/`. `tsc` cannot do this — the surfaces are browser code with npm dependencies

It no longer has a `src/catalog/` or a `src/selected.ts`: the database, the selection and the slug rule are `temco-store`'s, reached through `src/store.ts`.

`registry.ts` is deliberately free of presentation: it is the part that would survive replacing the view, and the part any other module reuses to know what pages exist — `temco-ai` reads it through the exported model rather than through this folder.

The tree

TEXT

```
Content                        (view container: Temco Docs)
├ Gaps (12)                    only when something is wrong, across all open repos
├ T3000Webview-EEZ-Studio      only when more than one folder is open
│  ├ t3000                     title · audience · noindex
│  │  ├ api-reference          chapter, with a page count
│  │  │  ├ Data Points API      owner · ship · reviewed, plus its issues as children
│  │  │  └ …
│  │  └ …
│  └ legacy
└ …
```

- Counts live on the **parents**, because "which area needs attention" is the real question.
- A page with issues is collapsible and its children are the reasons.
- A page's tooltip carries what the generator knows and the registry does not: its last change, its headings, and — marked in the description and the icon — whether its `code` moved on since the last review. With `index.json` / `page-meta.json` absent, that block is simply missing and nothing else changes.
- Clicking a page opens the file; if the file does not exist, the click offers to create it.
- Areas and chapters are never empty: an area declared in `areas{}` with no pages still appears, which is itself information.

The page in the editor

The editor of record is **VS Code's own**, deliberately. `15-online-editor.md` rejects a rich editor for these files on a reason that survives checking: the pages are written one paragraph per source line (measured: 95 characters on

average, 168 lines over 200), so an editor that re-wraps rewrites every paragraph it touches and a one-word change becomes a whole-file diff. What was missing was never the engine — it was the chrome around it.

Surface	What it is	Why not something new
the toolbar	`src/format.ts`, on `editor/title` and in the context menu	Monaco has no formatting bar; the buttons edit the buffer and undo like typing
Preview	`previewPanel.ts`, a tab beside the source (`preview:<page>.md`)	VS Code shows a page, the build shows an alert; the panel dresses the markers so the two agree
Diff	`diffPage.ts`, `vscode.diff` against `HEAD`	the editor's own diff, so the hunks seen are the hunks the pull request carries
the **Page** pane	`outline.ts`: headings as a nested jump list, plus a word count in the status bar	line, column, encoding and end-of-line are already the status bar's, so nothing repeats them
Grid	`gridEditor.ts` + `webview/grid.ts`: a CSV as cells, reached by *Open in the Grid*	the file stays CSV, one row per line, so a one-cell edit stays a one-line diff
Temco Markdown	`fluentEditor.ts` + `webview/docs/`: the page as a document on **Fluent UI**, with capture and annotation built in — the **default** for `*.md`	see *The Fluent Markdown surface* below
Temco Editor	not this module any more: the page editor is `extensions/temco-editor` (`temcoEditor.markdown`), and **Open in Temco Editor** asks for it by command	see temco-editor

One bridge, two surfaces

> **Record.** This is the period when `temco-docs` shipped two editing surfaces at once. It ended: `Temco Markdown` is gone, the page surface became its own module, and the shared bridge went with it (temco-editor). What is left here is only the grid.

Two editors over one file is a decision, not an accident: **what you see while editing** is a real choice, and it cannot be both. A source-visible editor (`Temco Markdown`) is the only way to be sure `git diff` will show what you did; a WYSIWYG editor (`Temco Editor`) is the only way to write a table without thinking about pipes. Everything *else* about them is shared, and it is shared in code, not by convention:

Shared, in the bridge (`src/webviewText.ts` , now `temco-editor/src/bridge.ts`)	Why it cannot be per-surface
the ordered-write guard (a set of texts asked for, not just the newest)	a write that lands late looks like an outside change, and answering it moves the caret to the end of the page
the file's own line endings	a browser only has `\n` ; writing that into a CRLF file is a whole-file diff for one word
one `WorkspaceEdit` over the whole file	so `Ctrl+Z` undoes an edit rather than a keystroke, and the dirty dot means the same thing
the capture pipeline (`image` , `pastedImage` , `capture` , `commitCapture` , `caption` , `linkPage`)	the picture lands beside the page with its words and its version; two copies of that would drift
the surface-failure reporter in the HTML	an editor that cannot start must say so in its own tab — an empty one is the failure a person cannot act on

`PageEditor` is therefore 30 lines: it names the bundle (`page`) and nothing else. Two surfaces, one set of rules. The **ribbon** is the second half of that: `.ribbon` lives in `temco-ui/media/page.css` , so both editors wear the same strip.

A surface that asks a question draws the question itself. A webview frame is sandboxed without modals, so `window.prompt` returns `null` there — the Link button was dead for exactly that reason, and the page now asks in the vocabulary's own box (`.page-surface__ask`), where Enter accepts and Escape cancels.

The controls both surfaces need live in `webview/shared/` : `find.tsx` (the bar; replacement is optional, because a rich document cannot truthfully "replace text" across formatting), `outline.tsx` (a jump list whose target is called `at` — a line for a source surface, a document position for a rich one), `annotate.tsx` (the canvas annotator) and `surface-controls.css` . They were inside `webview/docs/` until the page needed them; a copy in each surface is how the two would drift.

The page's chrome: a document application, not a card

The rich page is arranged the way a document application is, because that is what it is: `Home · View` at the top, groups under each, a sheet on a canvas, and a status strip. Two tabs and not three: there are about twenty controls, and the insert ones (link, picture, screenshot, table) are part of writing, so they sit on `Home` beside the marks and the block commands. `View` is what reads the page or hands it to another surface. A third tab for six controls was a tab nobody visits, and it made every control a guess about which of the two tabs held it.

Element	Rule	Why
the ribbon	flush to the tab: no border on three sides, no corners, `border-bottom` only	a card floating in a window is a page; a document's ribbon is part of the window
the controls	**30 px icon buttons, 20 px glyphs, 44 px panel**, no group captions	a writing bar should cost as little vertical space as possible; the captions are dropped because a caption under every group is a teaching aid a narrow editor pane cannot afford — the hairline between groups and each control's tooltip carry the same information. Only an icon-only control gets a square (`btn--icon`); a word button keeps its own width, which is what stops two of them drawing on top of each other
density	`full` ≥ 940 px, `compact` below — measured, not guessed	905 px is what the whole `Home` tab needs (83 history + 215 marks + 128 style + 128 lists + 149 blocks + 182 insert + hairline and padding). A compact bar keeps nine controls and folds five into `⋯`. Every group declares its controls **once**, with a tier, and `⋯` is the same table with the tier-2 rows taken out — an overflow menu written by hand is a second copy that drifts
the sheet	`.page-surface__paper` : **no border, no radius**, on `.page-surface__canvas` , `min-height: calc(100% - 16px)`	a sheet is a sheet; the canvas is a plain scroll container, **not** a flex row — a flex item is stretched to the container's height, which cut the sheet off at the fold and let the rest of the document spill onto the grey
one paragraph style	the **Text** menu names the block the caret is in and sets it; there is no second row of `Normal` / `Heading 1–3` buttons	two controls for one thing made the bar teach two ways to do it, and the buttons pushed the insert group off a narrow pane
zoom	`– 100% +` in the status strip, a view only	it never reaches the file, which is why it can exist at all
Find / Outline	Find is on the **View** tab; the outline is a **rail on the right** with its toggle at the far right of **Home** and `Ctrl+Alt+O` anywhere	navigating a document is not editing it — but a table of contents is *used* while writing, so its toggle lives where the writing is. Right, not left: the rail is the TOC the published page shows, so the structure you edit is the structure you ship
the outline as a tree	nesting from `src/outlineModel.ts` (`scripts/check-outline.mts` proves it): a heading hangs off the nearest one above it with a lower level, **however many levels are skipped**; the row the caret is in is marked current; folding is per row and remembered; the filter keeps the branch that leads to a match	a list of indented headings cannot say "this belongs to that", and a `###` after a `#` is a real document, not an error. The model is pure so the three rules that fail silently — a skipped level, a branch that springs open while typing, a caret with no current row — are checked without a browser

Element	Rule	Why
the outline rail	**300 px by default**, docked on the right, **dragged by its divider** (180–560 px, arrow keys too) and remembered in the webview state; never floated	the width is a preference, so it is set by hand once rather than guessed at; the two limits are what stop a drag from eating the page. It is `Ctrl+Alt+O` away if it is in the way
a toggle that is on	the glyph takes the **accent**, a hairline ring says pressed — never a filled block	a block of the selection colour reads as a button that was just clicked, not as a mode that is *on*. Bold, italic, Find, Outline and the rail all use it, so an "on" looks the same wherever it is
the rail opens itself	on a page with **three or more** headings, once, and never again for someone who closed it (`outline: undefined` means "never asked")	a TOC of one line is furniture; a rail that reopens itself after being closed is a fight
Read mode	View → *Read the page*: a second engine (`reading.tsx`) over the same seam, no caret, no editing tabs	reading a page should not be one keystroke from changing it; and it is the proof that the seam is real
Link · Picture · Screenshot · Table	the **Home** tab, beside the marks: a link and a picture are what writing a page is made of	Word's `Insert` tab exists for the fifty things Word can insert; six do not earn a tab. The horizontal rule, which used to be in two tabs, is now in one place
Annotate	**Home**, enabled only when a picture is selected, and it **replaces** that picture	the page keeps one figure, not a pair

What is deliberately **not** in this ribbon: font, font size, text colour, alignment, spacing, margins. Markdown cannot hold them, and a control whose effect disappears on the next save is a lie — the same reason the alignment group was removed from the source surface.

The editor's three files, and the seam between them

```text
webview/page/
  main.tsx       409  the mount: host protocol, the document, find/outline/zoom/status
  ribbon.tsx     311  the bar: tabs, groups, gallery, overflow      ← imports no Tiptap
  document.tsx   297  the engine: extensions, usePageDocument, actionsFor, stateOf
  reading.tsx     50  a second engine that cannot edit             ← proves the seam
```

> **Where this is now.** The three files moved to `extensions/temco-editor/webview/markdown/` when the editor became its own module, and the seam was separated from the engine: the two types below live in `contract.ts` (no package named in it), and `document.tsx` sits under `engine/tiptap/` with `bubble.tsx`, `handle.tsx` and `taskLists.mts`. Everything this section says about the seam still holds; only the paths changed. `temco-docs` keeps `webviewText.ts` ' share of the work as `src/webview.ts`, which is now just `webviewAssets` for the grid.

The seam is two types, both in `document.tsx` :

`PageState`	what is true at the caret — `block`, `marks`, `lists`, `inTable`, `onImage`, `link`, `canUndo`, `canRedo`, `words`
`PageActions`	named operations — `setBlock`, `toggleMark`, `setLink`, `insertImage`, `replaceImage`, `goto`, `load`, `content`, `node` …

`grep -r "@tiptap" webview/page/ribbon.tsx webview/shared/` returns nothing, which is the property worth keeping: the bar can be re-skinned without touching the engine, and the engine replaced without touching a button. Read mode is the proof — it hands the same `PageRibbon` a frozen `READING_ACTIONS` and static markup instead of an editor, and the bar draws itself exactly as before with its editing tabs removed.

One trap the split exposed, recorded because it will catch the next person: an actions facade built on every render makes the host-message effect re-subscribe on every keystroke, and the document that arrived while the effect was detached is lost — the page then opens with stale text. The facade is one object per editor (`useMemo` on the editor instance).

The three optional surfaces are webviews, so they need a bundler — the only one in this extension:

```
TEXT
webview/grid.ts       ──esbuild──▶  out/webview/grid/grid.js      11.9 MB  Univer (Apache-2.0)
```

The page surface has its own bundler in its own module (`extensions/temco-editor`, `out/webview/markdown/`), because a module builds what it owns.

Where this belongs: the extension, and what moving it into core would take

The surfaces are an extension today. The question "should this be product code instead" has a factual answer in this fork, so it is recorded here rather than argued:

	Extension (`extensions/temco-docs`)	Core (`src/vs/workbench/contrib/…`)
how a surface is registered	`customEditors` contribution + `registerCustomTextEditorProvider`	an `EditorPane` on `EditorExtensions.EditorPane` with its own `EditorInput` + `IEditorSerializer`; the in-tree templates are `contrib/imageCarousel` (pane + webview) and `contrib/update/releaseNotesEditor` (a webview editor)
what it can reach	the public API: `vscode.*`	every service: `IEditorService`, `IStorageService`, `IFileService`, `IContextKeyService`, `INotificationService`, the chat/agent services, `MenuRegistry`, `ViewsRegistry`
tab restore	`retainContextWhenHidden`, a webview kept alive in a hidden tab	the editor input serializes itself, so a tab comes back without the hack
iteration	edit, `npm run build-webview`, reload the window	edit, client compile, repo gates
price of a change	local and quick	`valid-layers-check`, layering rules, a full client compile

Two things decide it. **A core surface cannot `import` React, Tiptap or Fluent** — `src/vs` has no React and no JSX toolchain — so the *content* would still be a webview bundle whatever we do; only the *host* moves. And the fork already contains the pattern for that split (`imageCarousel` : DOM in core, a webview for what needs one).

The target, when the surfaces stop being experimental:

```
TEXT
```

```
src/vs/workbench/contrib/temcoDocs/
  browser/
    temcoDocs.contribution.ts        registers the panes, commands, menus, views
    markdownPageEditor.ts            EditorPane   ┐
    markdownPageEditorInput.ts       EditorInput  ├ the authoring surface
    pageEditor.ts                    EditorPane   ┐
    pageEditorInput.ts               EditorInput  ├ the document surface
    media/docs.js · media/page.js    built bundles (esbuild, not committed)
  common/                            the round trip, the page model, the registry
  test/browser/                      what `check-page` and `check-docs` do today
```

The migration is worth doing when one of these is true: a surface needs a workbench service an extension cannot see (the editor group layout, the chat context, a real `IStorageService`), tab restore has to be exact, or the module needs its own keybindings and menus without the `when` -clause traps. Until then the extension is not a limitation being tolerated — it is the cheapest place to keep a surface that is still being designed, and everything above it (the round trip, the capture pipeline, the page model) is already plain TypeScript that moves without a rewrite.

The output goes to `out/webview/` , beside the extension host JavaScript and inside the directory this repository already treats as build output, so it is ignored by git. It is deliberately **not** committed: the `local/code-no-new-javascript-files` rule rejects a newly added `.js` , and only a CODEOWNERS edit can allowlist one. `vscode:prepublish` runs the build for a packaged extension, so a source tree needs `npm run build-webview` once.

What was tried and removed: the rich editor

The page was once offered twice: the source surface, and a **Rich mode** built on the official Tiptap UI Components template — 161 vendored files, 21 @tiptap/* packages and a 1.6 MB bundle. It was **removed**, because:

- **The file is the source of truth, and it is Markdown.** The site, the offline help and the AI context all read it, and review is a git diff of one paragraph per line. A ProseMirror document has to be serialised back, and a round trip that re-wraps a paragraph turns a one-word edit into a whole-file diff — which is the failure this whole document exists to avoid.
- **It cost more than it gave.** Four extensions had to be removed because they do not survive that round trip (TextAlign, Subscript/Superscript, Typography, colour highlight), each one a standing risk that the repairs in `tidy()` cannot keep up. Two editors was also two sets of interactions to keep right, and the one that showed the document *as a document* was never the one that showed the bytes.
- **It did not look like the product.** It wore the template's own chrome rather than `page.css` , so the two editors did not resemble each other, let alone the rest of the application.

What it taught is kept, because it is the reason the shipped surface has the caret rules it has: a set that replaces the document under the caret moves the caret to the end of the page, an echo of the editor's own write must be dropped rather than applied, and `applyEdit` landing out of order makes a late write of ours look like an outside

change. Those three lessons are enforced in `webview/docs/main.tsx` and asserted in `check-docs` and in the scenario below.

The rich page: Tiptap, and the round trip that makes it safe

> **Where it lives now.** This surface became its own module: `extensions/temco-editor`, view type `temcoEditor.markdown`, and it is the **default** for `*.md` today. The paths below are the ones it had while it lived here; the reasoning, the rules and the checks are unchanged and still apply — see temco-editor for the current tree. The two-surface comparison this section describes is over: the winning affordances (a selection menu, a block handle, a placeholder) are in that module, the losing prototype was deleted, and the ribbons of the two surfaces became one.

The rich editor is back, and it is the same engine — Tiptap, then 3.30.3 and now 3.32, twelve packages, its own bundle at 983 KB (1457 KB today, with Fluent, Radix and the drag handle).

Part	What it was	Where it is now
`src/pageMarkdown.ts`	the whole contract: `toHtml` (`marked`), `toMarkdown` (`turndown` + GFM), `roundTrip`	`temco-editor/src/markdown.ts`
`webview/page/main.tsx`	the Tiptap editor and its ribbon; it never sees a file, only Markdown over the message port	`temco-editor/webview/markdown/main.tsx`
`webview/page/taskLists.mts`	Tiptap's `TaskList` / `TaskItem`, extended to accept the shape `marked` writes as well as their own	`temco-editor/webview/markdown/engine/tiptap/taskLists.mts`
`pageEditor.ts`	`temcoDocs.page`, a `CustomTextEditorProvider` on the same text document the source surface edits	`temco-editor/src/surfaces/markdown.ts` (`temcoEditor.markdown`)
`scripts/check-page.mts`	thirteen cases, one of them a whole page that must come back byte-identical	`temco-editor/scripts/check-page.mts`
`scripts/check-task-lists.mts`	the two task-list shapes, through a real editor in jsdom: open, save, and the lists that must not be claimed	`temco-editor/scripts/check-task-lists.mts`

The rules the conversion needs, each one a case in `check-page`:

- Tiptap wraps a table in `<colgroup>`; the GFM rule only recognises a table whose first child is a row, so a table would serialise as raw HTML and the file would lose it. The colgroup is dropped — column widths are the reader's.
- A paragraph inside a list item or a table cell emits **no** blank line. With one, a tight list becomes loose and every bullet is rewritten, and a table row breaks into fragments.
- Turndown pads a bullet marker to the width of the longest marker in the list (`- one`), so markers are collapsed back to one space. A task item's checkbox is followed by one space, not two.
- `*italic*` is written `*italic*`, not `_italic_`; the GFM plugin's strikethrough writes a single `~`, which is not strikethrough at all, so it is overridden to `~~`.
- Underline and highlight have no Markdown spelling. They are kept as the inline HTML Markdown already allows (`<u>`, `<mark>`), because a button whose effect disappears on the next save is a lie.
- **A task list is two shapes, and both are accepted.** `marked` writes `- [x] done` as `<ul><li><input type="checkbox" checked> done</li></ul>`; Tiptap renders it as `ul[data-type="taskList"] > li[data-type="taskItem"]` with the checkbox inside a `<label>`. Without a rule for each direction the checkbox was lost twice — the page opened as a plain bullet list, and an item that did survive saved as `- done`. `engine/tiptap/taskLists.mts` adds the second shape to the parser, and the `taskCheckbox` rule in `src/markdown.ts` reads the checkbox wherever it sits. `check-task-lists.mts` drives a real editor over both shapes; `check-page.mts` keeps the serialised result byte-identical.
- **Alignment and colour are not in the toolbar.** A paragraph's alignment lives in a `style` attribute Markdown cannot hold, which is the same reason table widths are not kept. The reader decides both.

The two surfaces are peers over one file, not two products: the tab's menu reaches the other one (`temcoDocs.page.open`, *Open With…*), and both write the same document, so save, the dirty dot, `Ctrl+Z` and `git diff` mean the same thing in either. The Markdown source surface stays the default for `*.md`; a workspace that wants the page instead changes one `priority` in `package.json`, or a person picks it once in *Open With…*.

The two editors also share one ribbon: `.ribbon` and its groups live in `temco-ui/media/page.css`, so the strip a writer recognises in the source surface is the strip in the page, drawn from the same buttons and badges.

The engine and the design language: both stay

Two questions come back often enough to answer once.

Tiptap is not the editor; it is the document model. It owns the schema, the text, the caret, the undo history and the HTML — `engine/tiptap/document.tsx` (the schema and the editor) and `engine/tiptap/taskLists.mts` (the two task-list shapes) are the only files in the product that import `@tiptap/*`. Everything that a person sees or presses is ours: `ribbon.tsx`, `main.tsx`, `outline.tsx`, `find.tsx`, and the round trip in `src/markdown.ts`, which converts HTML ⇄ Markdown with `marked` / `turndown` and has no Tiptap import at all. The contract between the two is `PageState` (a snapshot) and `PageActions` (named operations), and `reading.tsx` runs the same ribbon, outline and status strip behind an implementation with **no editor**, which is the seam being real rather than drawn. Since the move, the contract is its own file (`webview/markdown/contract.ts`) and names no package at all.

Swapping the engine is therefore possible and pointless: it would rewrite those 315 lines, change no button, and re-prove the behaviour `check-page` already proves. Tiptap earns its place at roughly a third of the bundle, because a rich-text model with tables, task lists, images, marks and undo is not a weekend's work.

Fluent is not the design language either. It supplies the **icons** (plain SVGs, no provider needed, five files) and **one dialog** — the annotator, ten Fluent components plus the `FluentProvider` that gives it the editor's light or dark

theme. The ribbon, the menus (Radix, for positioning), the outline tree, the sheet and the status strip are the vocabulary's own HTML and CSS. `scripts/check-fluent.mts` fails the build if anything outside `webview/markdown`, `webview/shared` (in `temco-editor`) imports it, so a second design language cannot creep in unnoticed. Removing Fluent entirely would mean replacing those glyphs and one dialog — worth doing when the annotator next needs a change, not before.

What was removed: ONLYOFFICE

`src/office/` (sign, config, bridge, desktop paths, the custom editor), its four commands, its settings block and `scripts/check-office.mts` were deleted. ONLYOFFICE Docs does edit `.md` natively and the integration worked, but the licence and the deployment each decide the question on their own:

- **AGPL-3.0.** The Document Server is a network service; serving it to our own users is exactly the case the licence covers, and embedding it in a desktop product does not avoid that.
- **A server nobody has.** It is not a library — it is a Docker or WSL deployment with its own storage, callback URLs and JWT signing. No such server exists in this repository or on the machines this product runs on, and a document editor that needs one before it can open a file is not a desktop product.

Tiptap answers the same need — a document, in the product, offline, on the file — with a licence we can ship.

Importing Word documents

`src/docx.ts` + `src/docxConvert.ts`: `mammoth` reads the `.docx` into semantic HTML and `turndown` turns that into Markdown, in a child process so the extension host never blocks. The result is a *starting point* — Word's fonts, colours, spacing and text boxes have no Markdown equivalent and are dropped rather than faked — which is why it writes a new page beside the document instead of replacing anything.

The converter is TypeScript, compiled to `out/docxConvert.js` and spawned from there. It is deliberately not a `.mjs` beside the sources: the repository's pre-commit hook rejects a newly added JavaScript file, and `scripts/**` is excluded from the packaged extension, so a script there would convert Word documents from a source tree and fail from an installed one.

Tiptap's own DOCX extensions were considered and are not used: both need a paid subscription and a private registry, and *import* runs on Tiptap's cloud with a JWT.

Both are `iife` and self-contained: a webview has no module loader and there is no CDN, so everything a surface needs travels with it. `npm run build-webview` writes both (`esbuild` + Sass) into `out/webview/`, which is ignored by git — so a source tree needs that one command before the editor or the Grid surface will open, and `vscode:prepublish` runs it for a packaged extension. No network and no `npm install` are needed *after* that build.

What neither surface owns: the document. Both send the whole file text to the extension, which applies it as a single `WorkspaceEdit` — so `Ctrl+Z`, the dirty dot, save, *Diff* and the git gutter keep working exactly as they do in the source editor.

Not built: a grid for `.xlsx`. It is a binary file that git cannot diff and that needs an `.xlsx` parser of its own (the maintained options are either deprecated on npm or drag in a pro tier), so a workbook keeps opening as what it is — a binary — rather than behind a grid that pretends it can be reviewed.

The Grid surface: a spreadsheet over CSV

`webview/grid.ts` is Univer, bundled the same way the page is; `src/gridEditor.ts` is its host. The contract is deliberately narrow: the host sends `{ type: 'set', rows }` with what `parseCsv` read, the surface sends back `{ type: 'change', rows }` once an edit settles, and the host writes that through `writeCsv` as one `WorkspaceEdit`. The cells are plain strings and the quoting is the extension's, so a one-cell change stays a one-line diff.

Four things about it are worth keeping written down, because each one cost an afternoon and none of them fails loudly:

- **The event is the workbook's.** In Univer 1.2.0 `onCommandExecuted` belongs to `FWorkbook`, not to the application. The first version guarded `typeof api.onCommandExecuted === 'function'`, which is false in every build, so the surface never reported anything: the grid accepted a cell edit, displayed it, and saved nothing.
- **A sheet is 1000 × 20 unless told otherwise.** `getRange` past the sheet's own size throws `Range is out of bounds`, so the read is clamped to `getMaxRows()` / `getMaxColumns()` and the workbook is created at least as large as the file it is given. Reading `Math.max(200, maxRows)` columns was asking for a range that did not exist.
- **Univer runs commands continuously** — selection, scroll, render. A debounce that clears its own timer on every command therefore never fires; the timer is armed once and allowed to run, and `flush()` decides whether anything really changed by comparing against the last thing it sent.
- **A throw inside a timer is invisible.** Reading the cells happened in the settle callback, so its exception became an unhandled page error and the surface simply stopped reporting. Both the read and the flush say so with `console.error`.

`src/cells.ts` holds the parts that need no browser — `asRows`, `widthOf`, `trimCells` — so `scripts/check-grid.mts` can prove the round trip without a window: a file opened and not edited reproduces itself byte for byte, and one edited cell stays one line.

The preview and the diff are **tabs of the editor group, not tabs inside one pane**. A custom editor that owned the document would have to bundle its own Monaco, and every copy of Monaco is worse than the one this window is already running: no git gutter, no search results landing on the right line, no other extension's decorations. The `Source | Preview | Diff` strip in the design is therefore three sibling tabs, which is also what VS Code's own Markdown preview does.

Which editor opens what:

File	Opens in
`.md`	**Temco Markdown**, by default — one *Open With… → Text Editor* away from the source, and the file never changes meaning
`.csv`, `.tsv`	the text editor — or the Grid, on request
`.xlsx`	whatever VS Code opens a binary with; there is no grid for it (see above)
everything else	untouched — the ordinary VS Code editor

One editor per kind, on purpose. There is no second rich-text surface for `.md`: the file is the source of truth and it is already readable in the surface's `Page` view, so an engine between the person and the bytes buys formatting they cannot review in a diff. The Office question is settled the same way, in `../../platform-design.md`: `.docx` / `.xlsx` / `.pptx` are *not* edited in the product — Word documents come **in** through `src/docx.ts`, and an OOXML editor would make the source of truth binary, un-diffable, and (ONLYOFFICE being AGPL-3.0) unshippable.

The Markdown surface — removed, and where its features went

> **This surface is gone.** `temcoDocs.markdown`, `src/fluentEditor.ts` and `webview/docs/` were deleted, and `temcoDocs.page` became the default for `*.md`. What follows is kept as the record of what it did and why it went; the parts that survive live in `src/webviewText.ts` (the bridge) and `webview/shared/` (find, outline, the annotator).
>
> Why it went: two editors over one file is only justified when they *see* something different, and after the port they did not — the page had every authoring feature the Markdown surface had, so the second surface was a second place for a bug to live. Two things were lost with it and are recorded as lost: the `Editor | Split | Page` split view, and the "In this article" rail in preview. Neither is replaced by a fake: the native text editor *is* the source view (`View → Open source`), and navigation is the outline rail plus the Page pane.

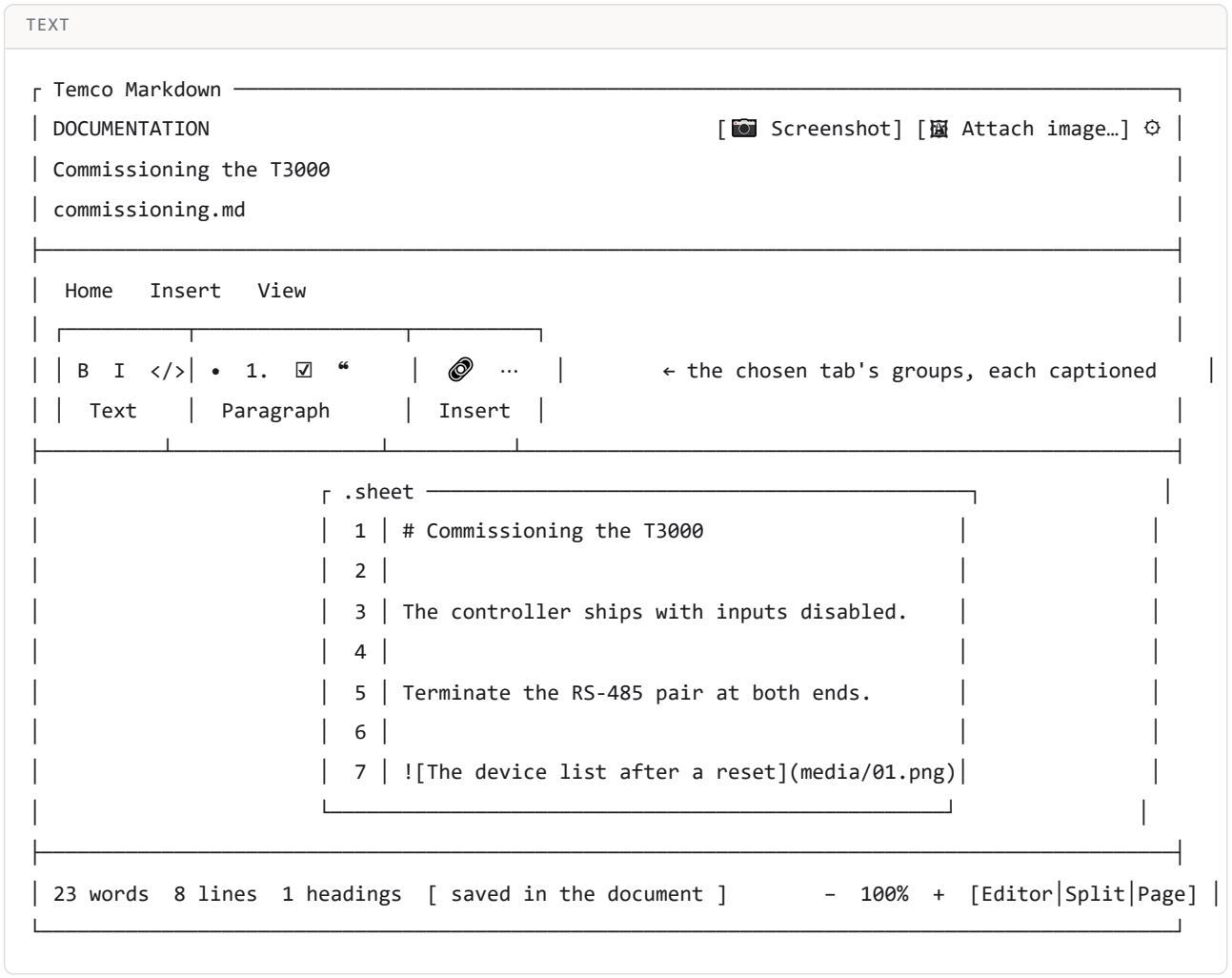
Feature it had	Where it is now
capture, attach, paste, caption, link-to-page	`src/webviewText.ts` — shared by the page, unchanged
annotate a picture	`webview/shared/annotate.tsx`, opened from the page's Insert tab
find	`webview/shared/find.tsx`, on the page's View tab (no replace: formatting boundaries)
outline	`webview/shared/outline.tsx` as a rail; `headingsIn()` in the page surface
preview / diff / source / properties	the page's View tab, through the same host actions
`Editor | Split | Page`	**lost** — the text editor is the source view
"In this article" rail	**lost** — the outline rail covers navigation

The original description is kept below, because the reasoning about chrome and vocabulary still applies to the page.

The page as a document, in **the application's own vocabulary**. It is the default editor for `*.md`, because authoring is what a page is for and the chrome around it is what was missing — not the engine. The document is still the file: the surface writes Markdown through the document, so `Ctrl+Z`, the dirty dot, save, *Diff* and the git gutter behave exactly as they do in the source editor, and *Open With… → Text Editor* shows the same bytes.

The layout is not invented here. `temco-ui/media/page.css` already carries a Markdown editor — `.md-editor` with a bar, a numbered gutter, `.md-line` colours and a status strip — and every other Temco page is drawn from the same file. This surface imports `page.css` into its bundle and renders `.view`, `.page-head`, `.md-editor`, `.badge` and `.segmented`, so it looks like the product instead of like an editor that happens to be inside it. Fluent supplies the popups only (the `⋯` menu, the outline popover, the prompt dialog); nothing on the page itself is a Fluent control.

TEXT



The arrangement is a document editor's, not a web page's: a head that says which document this is, a ribbon of named tabs whose chosen tab shows captioned groups of commands, the document itself on a **sheet** centred in the surface, and a status bar along the bottom with the counts on the left and the zoom and the view mode on the right.

Tab	Groups
Home	*Text* — bold, italic, inline code · *Paragraph* — bullet, numbered, task, quote · *Insert* — link, and `⋯` for every block
Insert	*Blocks* — table, code block, rule, `⋯` · *Links* — link to a page · *Page* — page properties
View	*Open* — the preview tab, the diff, the source editor · *Find* — the outline, find and replace

The tabs answer to `←` / `→` / `Home` / `End` and are a proper `tablist`: one tab in the tab order, `aria-selected` on the chosen one, `aria-controls` to its panel. A tab's controls exist **only** while its tab is chosen, which is why `check-docs` visits all three before it says every action is reachable.

Two sheets, or one. `Editor` shows the sheet alone; `Page` shows a sheet of `.article__body`; `Split` shows both with a divider you can drag. The reader **does not repeat the title** — the head already says it, and in `Split` the source says it too, so three copies of one line is two too many. Both sheets carry the same inset, so their first lines sit on the same pixel row. One pane keeps a 900px measure instead of running the width of the tab.

Zoom is a font size, not a transform. `− 100% +` in the status bar sets the *same* inline font size on the source and on its colour mirror, which is the only reason the colour stays under the words: two layers, one value. `MD-16` grows the text and measures both, because a size applied to one of them would look correct until the first character.

Every icon button carries an `aria-label` and a tooltip.

Action	How it is reached
bold · italic · inline code	button, and `Ctrl+B` / `Ctrl+I`; a backtick over a selection makes it code
heading 2 · 3 · 4	`···` — pressing the level again removes it
bullet · numbered · task · quote	one button each; every selected line is marked, and pressing again unmarks
code block · table · rule · five callouts	`···` — the code block asks for a language, the callout for one of the five DocFX renders
link	button, and `Ctrl+K` — the address is asked for, the selected words become the label
link to a page	`···` — the host offers the pages the registry declares and inserts a **relative `.md` path**, the only link the build can resolve
screenshot · attach image	the two buttons in the page head
page properties	the ⚙ in the page head, or `···` — the right-hand panel, for the page in front
outline	the outline button on the ribbon's View tab — the page's headings, indented by level, and one click puts the caret on one
find and replace	`Ctrl+F`, or the search button on the View tab: literal or pattern, a count, next/previous, and **replace all** as one edit
source · split · preview	the segmented control at the right of the status bar; the divider between the panes **drags**
preview · diff · source	the three icons beside it; each asks the host to run a command **with this document's URI**, and each is also in the tab menu's *Temco Docs* submenu

Writing, not just Markdown

Four behaviours exist because a page of instructions is written in a list, edited in place and read back:

Behaviour	Why
`Enter` inside a list starts the next item, numbering included, and `Enter` on an empty item ends the list	every step is typed as an item; making each one by hand is the most repeated keystroke in the page
`Tab` / `Shift+Tab` indent and outdent the selected lines	a nested step is two spaces, and nothing else should move
the colour behind the words	see below — the input stays a textarea, so none of the caret rules are put at risk for colour
the caret is revealed after a jump	the outline moves the caret into a line that may be off screen, and a caret you cannot see is a caret you do not trust

Colour without a second editor engine

The source pane is a `<textarea>`, and it has to stay one: every caret guarantee in this surface rests on the native element's own selection behaviour. So the highlighting is a **mirror `<pre>` behind the text**, and the textarea's own

text is transparent with only its caret and selection visible:

```
TEXT
```

```
┌──────────────────────────────────────────────┐
│ <pre class="editor__mirror" aria-hidden>  ← colour, tokens, identical font  │
│ <textarea class="source">                 ← transparent text, real caret     │
└──────────────────────────────────────────────┘
```

Two things make or break it, and both are asserted: the mirror must hold **exactly** the same characters (one dropped space slides the colour out from under the words), and the control must **not paint its own background** — Fluent gives the textarea a surface, which is precisely what hides the colour, so that one declaration is `!important` on purpose. `MD-10` in the scenario measures both, because neither is visible in a diff and both look fine in a type check.

File	What it owns
`src/fluentEditor.ts`	`MARKDOWN_VIEW_TYPE`, the `MarkdownEditor` subclass, `openMarkdown` — no protocol of its own
`src/webviewText.ts`	the bridge every webview surface shares: `set` / `change` / `pastedImage` / `capture` / `commitCapture` / `caption`, the echo guard, EOL handling, the capture command
`webview/docs/main.tsx`	the shell, the caret rules, the paste matrix (image → annotate, HTML table/list → Markdown)
`webview/docs/annotate.tsx`	the canvas annotator: arrow, box, ellipse, callout, redact, blur, numbered step, crop, undo
`webview/docs/markdown.ts`	`renderMarkdown` for the page, `htmlToMarkdown` for what is pasted in
`webview/docs/theme.ts`	`--vscode-*` → Fluent brand ramp, neutrals, radius and font; `watchTheme` repaints on a theme change
`webview/docs/docs.css`	geometry, the paper, the parts of a document Fluent has no concept of

The protocol

TEXT

```
webview                                host (WebviewTextEditor)
   │  {type:'ready'}                      │
   ├──────────────────────────────────────▶│  answers with `set` — the document's own text
   │  {type:'change', text}               │
   ├──────────────────────────────────────▶│  one WorkspaceEdit over the whole file, 350 ms after typi
   │  {type:'action', action:'capture'} │
   ├──────────────────────────────────────▶│  takes the screenshot and holds it (nothing is written ye
   │  ◀──────── {type:'captureReady'} ──┤
   │  {type:'action', action:'commitCapture', caption}
   ├──────────────────────────────────────▶│  writes media/NN-slug.png, records caption + version
   │  ◀── {type:'image', src, alt} ──────┤  answers with a page-relative address and the words
   │  {type:'action', action:'caption', src, caption}   (an image that landed without words)
   ├──────────────────────────────────────▶│  records the words; nothing moves
   │  {type:'action', action:'preview' | 'diff' | 'source'}
   ├──────────────────────────────────────▶│  runs the command **with this document's URI**
```

The capture is taken **before** the caption is asked for, and that order is deliberate: the screen a person wants is the one behind this window, and a dialog raised first would photograph the question. The bytes stay on the host while the surface asks, because a full screen is megabytes of base64 and it would cross the boundary twice for nothing.

`action` carries the document, because inside a custom editor there is no `activeTextEditor`: a command that fell back to "whatever editor is in front" would find nothing and quietly do nothing.

The caret, in the surface that ships

The three rules learned from the rich editor's round trip hold here too, and they are enforced in the webview rather than hoped for:

Rule	Why
a `set` exactly equal to what is on screen is dropped	the common case is an echo of our own write; redrawing identical text only moves the caret
a `set` that is an outside change while the surface has focus is **queued**, applied on blur	a replacement under the caret collapses the selection being dragged
an inserted image goes in **at the caret**, never appended	"quick" means where you are, not the end of the page
`ready` is posted **exactly once**, ever	the host answers `ready` with its own copy of the document; posting it again mid-write is what puts older text back under the caret

Validating this in real VS Code found a fourth defect that the prototype could not show, and it was the worst of the set: the message listener was re-subscribed whenever the status state changed, and each subscription posted `ready` again. `ready` makes the host answer with its own copy of the document — an answer that can arrive while a write of ours is still in flight, which puts the older text back **under the caret**. That is exactly the "the cursor comes to another place" report, arriving through a channel that only exists in the real host. The listener is now installed once, and the current closure is carried by a ref.

`npm run check-docs` now guards it headlessly: the bundle is loaded into a DOM, a `set` is delivered, the surface is typed into, and the run fails if `ready` was posted more than once. Removing the fix makes it fail (`the surface reported ready`, `typing did not post a second ready`), which is the check earning its place — no screenshot and no type check can see this.

The status badge is part of the same fix. The host never echoes a change it applied itself, so a badge that compared the page against "the text the document opened with" stayed on *unsaved* forever. It now means what it says — `writing…` while the bridge is behind, `in document` once the edit is in the document — and it deliberately does not duplicate "is the file saved", which is the tab's dirty dot.

Screenshots, and why the capture is host-side

`getDisplayMedia` is not dependable inside a webview — the frame is not granted display capture in every build — so the capture happens where the OS window is. `captureScreen()` runs a small platform command (`System.Windows.Forms.Screen` `System.Drawing` on Windows, `screencapture -x` on macOS), writes a PNG to the temp folder, and **holds the bytes**. Linux reports that capture is unsupported and points at the paste.

Nothing else about the pipeline is different from a paste: the bytes are written beside the page and the surface gets a relative address back.

Concern	Decision
where an image lives	`media/` beside the page — one folder per page folder, so a page moved with its folder keeps its images
what the Markdown holds	`media/NN-slug.png`, relative — what the build resolves, so a published page needs no rewriting
the file name	`NN-slug`, the number continuing the folder's own sequence and the slug from the page — never `image.png`, and never a name that collides
two screenshots with the same name	impossible: the sequence number is read from the folder
what the file receives	nothing at capture time; the surface inserts the reference and the normal save path carries it in
the words	**one string**, used as the alt text and as the visible caption. A screen reader and a reader see the same sentence, and there is no second field to fall out of step
what a picture depicts	`appliesTo`, recorded at insert time (design §22.4, S1)

The pictures a version leaves behind

`media/attachments.json` sits beside the images and says what each one is:

JSON

```
{ "version": 1,
  "images": {
    "01-commissioning.png": {
      "caption": "The device list after a reset",
      "appliesTo": "13.0",
      "added": "2026-10-24"
    } } }
```

`appliesTo` is the version the release workflow's `version-report.json` names at the moment the image is inserted, so nothing has to be remembered by a person. When the report moves to a newer version, every picture still claiming the old one is **stale** — a warning, never a block (S3), and `temcoDocs.staleScreenshots` writes the report that records which ones were known about:

TEXT

```
# Screenshots a newer version has left behind — 2026-10-24

The documentation describes **13.1**. These 2 pictures were taken of something else,
and they still look true.

| Page | Image | Shows | Documentation |
| --- | --- | --- | --- |
| t3000/commissioning | 01-commissioning.png | 13.0 | 13.1 |
| t3000/data-points | 02-scaled.png | 13.0 | 13.1 |

Stale is a warning, not a block. Publishing one of these is allowed, and this report is what records
```

Two silences are deliberate. An image with **no** `appliesTo` is never called stale — "we do not know" is a different claim from "this is old", and a false accusation on every page written before the record existed would make the report worthless. And no version report means no report at all, because there is nothing to compare against.

What was proven in a real window

`.build/vscode-playwright-mcp/temco-markdown-editor.cjs` drives a dev build and passes **sixteen** steps: the surface opens, the source pane holds the file from disk, typing reaches the **file** after `Ctrl+S`, the caret is still where typing left it, re-opening the page costs **zero bytes**, the surface fills the editor area, a screenshot is taken and then asks what it shows, the caption becomes both the reference and the record, the preview renders it as a figure whose alt text is the same words, the colour behind the text lines up with the words, the outline lists the headings and jumps to one, find counts its matches, the view modes show one pane or both, and the tab menu's *Temco Docs* submenu offers the three views for this surface. The last one chooses each ribbon tab and proves the document is untouched by the choosing, because a tab is a place commands live, not a place text does. The sixteenth zooms in and measures the source against its colour mirror, then resets, because a zoom that moves one layer and not the other is invisible in every other check.

A second scenario, `.build/vscode-playwright-mcp/temco-page-editor.cjs`, drives the **rich page** and passes **three** steps: the page opens on the file that was in front and holds its text, typing marks the tab dirty and only `Ctrl+S`

writes the new line to the file as Markdown, and **Bold** from the ribbon produces `**` in the file while the heading it did not touch stays byte-identical. A fourth thing is proved outside the window: `npm run check-page` renders a whole page to HTML and back and requires it to come back unchanged, and the built bundle is loaded in a plain browser with a stubbed host to prove that the first answer to `ready` is not lost.

Three defects were found by running that scenario, and all three are fixed in the product:

Symptom	Cause	Fix
the editor tab was there and the area inside it was empty, with no webview at all	the bundle threw at load: `ColorBackground24Regular` was used but never imported, and `webview/page` was not in `tsconfig.webview.json`, so nothing type-checked it	the import is restored and the surface is type-checked like the other one
the page opened blank although the host had sent the text	the host answers `ready` at once, and the answer arrived while Tiptap was still creating its editor, where the handler dropped it	the answer is held in a module-level inbox and handed over when there is an editor to accept it
the scenario clicked an editor that was not on screen	two tabs carried the same file name — the source surface and the page — and a hidden webview still answers a selector	the page is left as the only tab, and "in front" is checked by the editor's own header naming *Temco Editor*

The first one is the lesson worth keeping: a webview that fails to start looks exactly like a webview that has not been caught up yet, and neither looks like an error. The `try` / `catch` in `resolveCustomTextEditor` exists because of it — a surface that cannot start now says so in its own tab.

A third scenario, `.build/vscode-playwright-mcp/temco-docs-page-link.cjs`, opens a page nested inside a documentation set. Its second step — **link to a page** and **page properties** — is reported as **blocked, not passed**: both read the project catalog, and the harness cannot register a project, so a folder holding `docs/help/registry.json` is still "Repositories with a registry: 0". Both menu items are reachable in the surface; the picker and the panel behind them are the unverified part, and registering a project in the harness is the missing capability.

Three defects were found by looking at that run's screenshots and logs rather than by its assertions, and all are fixed:

Symptom	Cause	Fix
the whole surface sat in a band at the top of the editor	`FluentProvider` renders its own `div` inside `#root`; with no height every `height: 100%` below it collapses to nothing	`#root > div { height: 100% }`
the source pane was 260 px tall in an 828 px pane	Fluent's `Textarea` is a **wrapper** around the real `<textarea>`, and the control it draws is capped by its own `max-height`	both levels fill: the wrapper is a flex box, the control takes `height: 100%` with `max-height: none`
a run failed with an empty source pane	the bar paints before the document arrives, so "the editor is open" was being decided by the first paint rather than by the text	the scenario waits for the file's text, which is what a person sees

The layout one is now an assertion (`MD-06`), because a screenshot alone does not keep it fixed.

Three harness facts are worth writing down, since they cost the most time: a webview is **two frames deep** (`iframe.webview` serves the platform's `index.html`, and the extension's document is the nested `#active-frame`); `retainContextWhenHidden` keeps a webview alive in a **background tab**, so finding its frame proves nothing about what a person can click; and on a loaded machine a cold dev build can take **two minutes** to activate, which is a timeout to raise rather than a bug to chase.

When a menu entry is gated on the editor, key it on the editor. Retiring the rich editor left three commands — *Open in Source Mode*, *Preview the Page*, *Diff the Page with HEAD* — gated on `activeCustomEditorId == 'temcoDocs.richTiptap'`, an id that no longer existed, so they were hidden and nothing failed. Two rules came out of it, and `MD-14` now enforces both:

Rule	Why
`activeCustomEditorId` names **this** surface; `editorLangId` does not	`activeCustomEditorId` is empty unless the active input *is* a custom editor, while `editorLangId` follows the active **text** editor — and a webview is not one. The built-in markdown preview keys its `editor/title/context` entry on `resourceLangId` for exactly this reason, and that difference is why the tab menu's Temco Docs submenu was invisible until it was corrected.
a `when` that can never be true looks exactly like a working feature	Nothing errors and nothing logs. Only an assertion that the command is *present* tells the two apart, which is why the step opens the menu instead of trusting the manifest.

The three now live in that submenu rather than in `editor/title`, because the surface already draws them in its own head — as icons, next to the view modes — and a title-bar twin both repeated them and squeezed the breadcrumb. `editor/title` keeps only the source-mode twins keyed on `editorLangId`, so each action appears once in each mode.

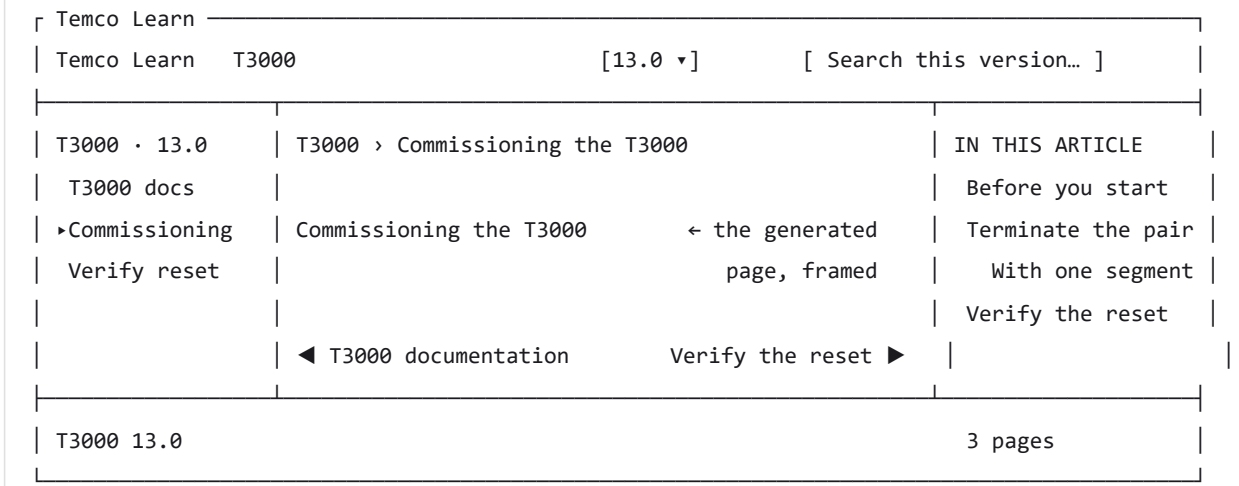
Two deliberate deviations

Deviation	Why
Fluent lives in `extensions/temco-docs/webview/docs/`, not in `temco-ui`	`temco-ui` has no React runtime and no JSX toolchain; extracting means giving it both. It is confined rather than unowned: `scripts/check-fluent.mts` fails the build if any other file imports `@fluentui`. The trigger to extract is a **second** Fluent surface.
`.md` opens in this editor by default	it is the authoring surface. The risk of a custom editor is that it changes what the file means; this one writes Markdown through the document, so it cannot. Every other surface here stays `option`.

The reader: what a published version looks like

Authoring is one half of documentation; this is the other half. A published version gets a page beside the files the generator produced, and **the reader is not a second renderer**: the article is the generated page in a frame, so DocFX (or MkDocs, or Docusaurus) keeps rendering its own Markdown, and the reader supplies the shape a documentation site has.

TEXT



Decision	Why
one self-contained file, model embedded as JSON	it works from a folder, from the loopback server, and inside a CHM alike — no fetch, no build step, no second file to keep in step
the article is a **frame**	a Markdown renderer here would be a second implementation of the generator's job, and it would drift
the sidebar is built from the **pages in the folder**	a doc set never has to be told about the reader, and the sidebar cannot disagree with what was published
"In this article" is read from the frame	it needs same-origin, which `siteServer` gives; from `file://` a browser refuses and the panel says nothing rather than guessing
written as `temco-reader.html`, never `index.html`	the generator's own home page is not ours to overwrite
the deep link is `#<href>`	a page can be sent to a colleague, and a reload lands on it

`src/learnReader.ts` is pure and `scripts/check-learn.mts` renders it: three columns, every page in the sidebar in order, the first page framed and marked, the version picker, no markup from a title, an empty set that still renders, and the hash deep link. Twenty checks, no browser.

The same shape inside VS Code. `Page` mode puts the article's own headings beside it — the rail a documentation site has, because a long page is crossed by its headings and not by scrolling. Headings are addressable: `renderMarkdown` writes `id` from `slugOf`, so a rail link lands on the section, `levels` indent, and the rail is hidden in `Split`, where the pane is already half a tab. `check-docs` asserts the links and their anchors, not the pixels.

The AI panel

TEXT

```
Temco Docs                                    (panel: the AI, docked on the right)
├─ header        the page's title (· stale) and the model in use · offline
├─ transcript    your question, the answer as it streams, notes, errors
├─ Draft update  → opens a diff: the page on the left, the draft on the right
│                  Apply replaces the whole page — one Ctrl+Z takes it back
├─ Ask the area  → the question, with the id, title and headings of the pages around it
└─ Ask           → a question about the page on screen
```

- **It writes nothing by itself.** A draft lands in a diff and in the panel; putting it into the page is a button, and the change is an ordinary undoable edit.
- **What reaches the model is fixed.** The page's text and metadata, and for an area question the ids, titles and headings of its neighbours. No other file, no repository sweep, no credential.
- **Every draft names its prompt version**, so a draft can be traced to the instructions that produced it.
- **An answer's citations are checked.** A page it names becomes a chip that opens it and is dashed when the page is stale; a name that is not a page is reported rather than believed; and an area answer that named nothing is replaced by search results instead of a synthesis.
- **With nothing installed the panel still opens** and says what to download, so the extension never looks broken — the AI is optional the whole way down.

The dashboard, and why this repository does not have one

`00-architecture.md` puts the dashboard on the **site**, not in the editor: a **generated static page** at `/_admin/` — coverage · stale by owner · docs-owed · unowned · unreviewed · broken links · feedback · health score — noindex, out of the PDF, the CHM and the offline zip, built by the product repository's own pipeline (`13-implementation-plan.md` step 1.19). Its rows deep-link **into** the editor, and that hand-off is this extension's half of it.

An earlier phase built an organisation read here instead — a repository list over the GitHub API and a sidebar view — which answered the dashboard's own question more weakly (repository-level facts, none of the page-level health) and in a second place. Both are deleted, along with the token setting they needed. The rule is the same one this document already carries for checks: **the extension never computes an answer the product repository already owns.**

Commands

Command	Does
`temcoDocs.showStatus`	refreshes, writes the full report to the *Temco Docs* output channel, offers **Show Problems**
`temcoDocs.validate`	same, and is what the status-bar item and the view title button call
`temcoDocs.refresh`	re-reads the registries
`temcoDocs.openRegistry`	opens `registry.json` for the folder the node belongs to
`temcoDocs.openPage`	opens the page, or offers to create the missing file
`temcoDocs.markdown`	opens the page in **Temco Markdown**, the Fluent authoring surface — the way back in from the source editor
`temcoDocs.staleScreenshots`	the report of pictures a newer version has left behind, from the record beside each page's images. Nothing is written and nothing is refused (design §22.4, S3)
`temcoDocs.reveal`	shows the page's file — or a chapter's, an area's or a source's folder — in the operating system's file manager (`revealFileInOS` , not the VS Code view of a similar name)
`temcoDocs.openTerminal`	opens an integrated terminal at the folder that row stands for
`temcoDocs.copyPath` · `copyRelativePath`	the row's path on the clipboard, absolute or relative to the open folder
`temcoDocs.newPage` · `newChapter`	a page (kind, title, file name) or a chapter — a folder with its first page, since an empty chapter does not exist
`temcoDocs.newPage.article` · `.task` · `.reference` · `.troubleshooting` · `.releaseNote`	the same as `newPage` with the kind already named, so the *New Page of a Kind* submenu is one click per kind
`temcoDocs.renamePage`	the title as a registry value, and the page's first `#` with it — never the file name
`temcoDocs.movePageUp` · `movePageDown`	a two-number swap of `order` in `registry.json`
`temcoDocs.retirePage`	`deprecated` and `superseded_by` ; the file stays, so old links keep redirecting
`temcoDocs.buildSite`	invokes the repository's own `docs:site` as a task and reports the files it produced
`temcoDocs.buildHelp`	invokes the non-deploying help build (`build:help:dry`) and reports the CHM it produced
`temcoDocs.stalenessReport`	opens the report of what is owed as an untitled Markdown document — nothing is written
`temcoDocs.newPage` · `newChapter` · the five kind commands	open the **New page page** (`src/newPagePage.ts`) with the row's place and, for a kind, its kind already chosen. The page writes through

Command	Does
	`writePage()` — one write, one confirmation that names the file
`temcoDocs.feedback`	reads each repository's `docs/help/feedback.json`, routes the rows to their owners, and calls out the aging

Where they surface: the **Workspace view's title bar** carries *New page*, *New chapter* and *Refresh* inline, and its `…` holds the page acts (open · reveal · terminal · copy a path · rename the title · move up · move down · retire · the *New Page of a Kind* submenu · the registry) plus Home and Validate — the same list the row's own right-click menu shows, which is the Library's shape and the Explorer's. A row's **left click** opens a page or expands a folder; a **right click** (or the keyboard's menu key) opens the menu; the hover carries nothing. Everything else is in the palette, where it costs no room — and `startRegistry` is a row in the view's empty state, so the view and the button that fixes it are never apart. There is no organisation-wide surface here by design: that question belongs to the site's generated dashboard, which this extension hands off to rather than re-implements.

What refreshes the view

- activation, and any workspace-folder change;
- a change to `**/docs/help/registry.json` (created, changed, deleted) — it changes the shape of the tree;
- a change to `**/docs/help/index.json` or `**/docs/help/page-meta.json` — they change the provenance the items display;
- an explicit refresh.

Everything else, including saving a page, does not refresh: the tree describes structure, not prose.

Rules the code follows

- **Two writes, each asked for.** Reading is what the extension does; it writes in exactly two places, and only on a command that names what it will create: the empty file for a page the registry already declares, and an empty registry where a folder you have open has none. Both are files the registry's own generator already expects, and neither is ever an existing file. **Nothing under `_admin/` or `_edit/` is written at all** — the design's rule is that those are generated pages, not documentation. What it downloads — the runtime, a model, the manifest that records them — goes to its own storage folder, or to the folder a setting names, and never into a repository.
- **A diagnostic lands where the fix is** — the page for a page problem, `registry.json` for a missing file or an unreadable registry.
- **Counts are not problems.** `reviewed: null` and empty tag lists are normal in a 335-page registry; as warnings they would bury the faults that matter.
- **No module knows about the shell.** Nothing reaches into the application's own code, by design.
- **Nothing is fetched unasked.** The only network calls are the runtime and model downloads the user starts from *Models and Runtime*: no telemetry, no update check, no background refresh, no organisation scan. No credential is stored at all.
- **Nothing native ships.** A compiled engine is per-OS; a downloaded one is not, which is what keeps one `.vsix` valid on every platform — the application, plain VS Code, Windows or macOS.
- **Building is not shipping.** The build commands invoke the repository's own scripts and report what those scripts produced. The form that deploys into a T3000 installation is never run, and nothing is uploaded or committed.
- **A generated page is never overwritten.** Which now means: nothing under `_admin/` or `_edit/` is written at all. The dashboard is the product repository's generated page; this extension is the deep-link target it hands off to.

- **An empty state is actionable.** A view that only explains itself is a dead end, and the folder with no registry is exactly where a new product repository starts. The Content view's empty state is a row that runs *Start a Registry* rather than a sentence telling the reader to go and find one.
- **A process, only while it is wanted.** The local server starts when something asks the AI for an answer, never at activation; it listens on loopback only; and it is stopped when idle, when its model is removed, and when the window closes. The extension holds no model in its own process, so nothing to load or unload blockingly.

Guardrails, in code

`12-ai-features.md`, *Guardrails*, lists them; this is where each one actually lives, so it can be checked rather than trusted.

Guardrail	Where it is enforced
No shipped page is edited by a model	A draft opens in a diff; the page changes only when **Apply draft** is pressed, and then as one undoable `WorkspaceEdit`
No secrets, no customer data	The only thing a prompt can take is a `PageContext` — page text, path, registry metadata — plus ids, titles and headings for an area question. No parameter exists that could carry another file, a token or a sweep of the repository
Traceable	`PROMPT_VERSION` is printed on every draft and in the panel's model line, and a draft also names its commit range (`temco-ai/src/ai/history.ts`)
Degradable	The panel opens with nothing installed and says what to download; the server never starts without a runtime and a model; a missing prompt file is a named error, not a stack trace; a page too large for one draft is refused rather than half-drafted
Deterministic first	Staleness is read from `index.json` , never recomputed; the low-confidence fallback is plain term overlap; citations are resolved against the registry
Cost bounded	6 turns of history, 40 area candidates, 40 commits, 100k characters of page text, 1024 tokens per answer and 2048 per draft, `--ctx-size 8192`
Offline	The runtime is a local process on `127.0.0.1` ; the only other network calls are the downloads the user starts
One content library	`scripts/check-fluent.mts` fails if anything outside `webview/docs` imports `@fluentui` — a second Fluent surface is the trigger to extract it, not a stray import
A surface cannot corrupt a file	every webview editor shares `WebviewTextEditor` , which owns the echo guard, the line endings and the single- `WorkspaceEdit` write; a surface only names its bundle, and `scripts/check-grid.mts` proves the CSV round trip keeps the file byte-for-byte when nothing is edited
One meaning per action	every formatting action is a pure transform in `src/markdownEdits.ts` , called by both editors — there is no second copy to drift
The bar cannot teach a key that does nothing	`scripts/check-shortcuts.mts` reads every `shortcut:` the ribbon advertises and traces it to an engine keymap, a `package.json` keybinding or a named shell listener; `Ctrl+F` and `Ctrl+Shift+X` were both taught and unbound when it was written
The page in front is a page, not a text editor	`frontUri()` reads the active tab as well as `activeTextEditor` , because a custom editor has none; without it Properties, the capture target and every row command silently saw nothing
Every image has words, and a version	`landImage()` is the only writer into `media/` ; it records the caption and the version the picture depicts, and `check-record` proves a picture with no recorded version is never called stale

Roadmap

Phase	Adds
1	**the Content view** — registry, checks, gaps, status bar
2	**the AI** — model download, runtime, draft page updates from a commit range, ask-the-area; a local model, offline, drafts only, a human accepts
3	**publishing** — invokes the product repo's site build and offline help build and hands over the artifacts; it orchestrates, it does not reimplement the pipeline
4	*(not a module of its own)* — the documentation health report is the platform's generated page at `/_admin/`
5	**feedback and health** — intake, and the report of what is owed; the report's cadence stays in the product repo's CI

The module's design is what this document and `registry-schema.md` hold; the platform-wide design is `design/temco-learn/`.

The AI is the reason the module exists. Everything before it is what the AI needs in order to know what it is talking about.

The registry the module reads

`docs/help/registry.json` in each product repository is the single description of that product's documentation set. This extension is a **reader** of it. The schema is owned by the generator in the product repo (`scripts/docs-registry.mjs`) — if a field ever needs to change, it changes there first and this reader follows.

Shape

JSON

```
{
  "version": 1,
  "areas": {
    "t3000": { "title": "t3000", "audience": "user", "noindex": false }
  },
  "pages": {
    "t3000/api-reference/data-points": {
      "title": "Data Points API",
      "area": "t3000",
      "chapter": "api-reference",
      "order": 20,
      "audience": "user",
      "ship": true,
      "owner": "docs",
      "tags": [],
      "keywords": [],
      "help_ids": [],
      "code": [],
      "reviewed": null
    }
  }
}
```

Fields, and what they decide

Field	Where	Meaning
`areas`	registry	the top level of the site: one entry per product area, with its default audience and whether it is excluded from the published site
`pages`	registry	every page, keyed by **page id**
`title`	page	the page's name, used for search, navigation and the tree
`area` , `chapter`	page	where the page sits; both must agree with the page id
`order`	page	position inside its chapter. Pages are ordered by it, then by title
`audience`	page	`user` , `developer` or `contributing` ; falls back to the area's audience
`ship`	page	`false` keeps the page out of the shipped help build
`owner`	page	who is answerable for the page's accuracy
`tags` , `keywords` , `help_ids` , `code`	page	search facets, context-help ids, and the code globs a change should make the page stale. Empty is normal
`reviewed`	page	ISO date of the last human review, or `null` . `null` is a **count**, not a warning

Page id → file

The page id is the path under `docs/` , without the extension:

Page id	File that must exist
`t3000/api-reference/data-points`	`docs/t3000/api-reference/data-points.md`

That single rule is what makes a page "missing" checkable, and it is why this extension never guesses at paths: the id is the path.

Generated neighbours

The product repo also emits, and this extension may read:

- `docs/help/index.json` — per page: its headings, the last commit that touched it (author, sha, date), and the generator's `stale` verdict.
- `docs/help/page-meta.json` — the same provenance in the smaller shape the published page reads.

Neither is required. Absent, the extension works exactly as before; only provenance goes missing. A malformed one is a **warning on that file** — the fix is to regenerate it in the product repo — never an error, because it hides nothing else.

What the extension shows from them: a page's last change and headings in its tooltip, a `stale` marker on the page (description and icon), and a stale count in the repository and area tooltips plus the status report. The page's `code` globs are read from the same place, because that is what a `Draft update` 's commit range is read against.

`stale` **is read, never computed here.** The verdict — a commit touched a path the page claims in `code`, after its last review — belongs to the product repo's generator (`05-content-and-registry.md`, *Staleness*). Two answers to that question would be worse than none, so the extension only repeats the generator's answer.

Structural checks

Run on every refresh, cheap enough to run on 335 pages without thinking about it:

Check	Severity	Reported on
the page has no title	error	the page
`area` is not declared in `areas{}`	error	the page
the id disagrees with `area` / `chapter`	error	the page
registered, but the file does not exist	error	`registry.json`
no `order`	warning	the page
two pages share one title	warning	both pages
two pages share one `order` inside a chapter	warning	both pages
the registry declares no pages	warning	`registry.json`
the registry is not valid JSON	error	`registry.json`

Deliberately not checked here: prose quality, links, images and staleness. Those belong to the product repo's own gates (`npm run docs:verify`, the media audit, `docs-owed`) and to the AI phase, where a commit range can be read. Duplicating them in the extension would create two answers to one question.

The feedback file — a proposal, not a contract

Where a feedback row comes from is not settled: the feedback design (`09-feedback-subscriptions-notifications.md`) assumes an endpoint on the T3000 server, while the management overview (`16-overview.md`) describes a different hosting model. This module reads the smallest shape that carries what routing needs, per repository:

```JSON
{
  "version": 1,
  "rows": [
    { "page": "t3000/api-reference/data-points", "kind": "wrong", "note": "…", "at": "2026-09-01",
  ]
}
```

`page` is a page id, `at` is an ISO date, and `state` is `open` or anything else for handled. A row whose page is not in the registry is still reported — with its id, as plain text rather than a link — because that is a finding about the row, not a reason to drop it. With no such file, **Feedback by Owner** says the source is not agreed rather than reporting no feedback.

This shape is the extension's proposal, and the docs owner accepts or replaces it. It is the only part of the feedback design that is not settled.

temco-editor

The editor for a Markdown page: one module, one editor, three layers. It owns the surface and nothing about what a page *is* — that stays in `temco-docs`.

Identifiers

Extension id	`temcocontrols.temco-editor`
View type	`temcoEditor.markdown` — **Temco Editor**, `priority: "default"` for `*.md`
Commands	`temcoEditor.open` · `temcoEditor.openSource` · `temcoEditor.preview` · `temcoEditor.diff` · `temcoEditor.pageProperties`
Namespace	`temcoEditor`
Storage	none
Depends on	`temcocontrols.temco-docs` (its model, through `src/docs.ts`)

Why it exists

Writing a page in a text editor means thinking about pipes and asterisks; a WYSIWYG surface means writing the page. The editor used to live inside `temco-docs` as `temcoDocs.page`, next to the registry, the tree, the builds and the release report — 494 KB of documentation domain beside ~250 KB of editor. This module is that 250 KB, given its own front door.

It also replaced its own prototype: for a while there were two editors to compare. The comparison is over, the winning affordances are here — a **selection menu** for the marks, a **drag handle** beside a block, a **placeholder** on an empty page — and the prototype is deleted.

The three layers

```
TEXT
the surface      main.tsx · ribbon.tsx · reading.tsx      asks for named operations, reads a snapsho
   ↕ contract    webview/markdown/contract.ts             PageState · PageActions · Mark · ListKind
the engine       webview/markdown/engine/tiptap/          the only place a Tiptap package is named
```

The rule is enforced by reading, not by trust: `grep -r "@tiptap" webview/markdown/*.tsx webview/shared/` returns nothing, `scripts/check-shortcuts.mts` reads the shortcuts out of `engine/tiptap/document.tsx` rather than assuming them, and replacing Tiptap means deleting one folder.

The host is the same shape from the other side: `src/` knows a document, a panel, an edit and a line ending, and no `Editor`, `Node`, `Mark`, `PluginKey` or `Transaction`. What a page *is* — its media folder, the version it describes, the pages a link may reach — it asks `temco-docs` for, through `src/docs.ts`.

What it owns

- the view type, the provider, and the commands that reach it;
- the bridge to the file: one `WorkspaceEdit` per save, the file's own line endings, and the ordered-write guard that keeps a caret still while two writes land out of order;
- the Markdown contract (`src/markdown.ts`) — the rules that make an unedited page open and save to the same bytes;
- the surface: the ribbon, the selection menu, the block handle, find, the outline rail, read mode, zoom, capture;
- its bundle and the one bundler step that produces it.

What it must not own

- **the registry, the pages model, the builds, the release report** — `temco-docs` ;
- **where an image lands.** `media/` , the file name and the attachment record are the model's; this module supplies the bytes and the words and asks (`landImage`);
- **a second Markdown dialect.** The round trip is one implementation. If Tiptap's own serialiser would be better, that is a change to the contract, proved by `check-page` first.

What was learned the hard way

- **A task list was destroyed twice** — on open, because `marked` writes `<ul><li><input type="checkbox">` and Tiptap wants `ul[data-type="taskList"]` ; on save, because `turndown-plugin-gfm` wants the `<input>` as the item's first child and Tiptap renders it inside a `<label>` . Both shapes are now accepted on the way in (`engine/tiptap/taskLists.mts`) and the checkbox is read wherever it sits on the way out (`src/markdown.ts`).
- **Tiptap merges attribute parsers over a rule's `getAttrs`** , so `checked` had to be parsed in `addAttributes` , not in the parse rule.
- **A missing bundle is a blank tab, not an error**: `bridge.ts` asks for `markdown.js` and `markdown.css` by name, a 404 on a script fires no `onerror` , and `compile-extension:<name>` cleans `out/` before `tsc` — which deletes the bundle. `check-surface.mts` loads the built file, so a stale or missing bundle fails the gate instead of the tab.

Gates

`npm run vscode:prepublish` , in order: `check-commands` · `check-fluent` · `check-types` (host and webview, separately) · `compile` · `check-page` (13 round-trip cases) · `check-task-lists` (a real editor, both shapes) · `check-outline` (29 cases, no browser) · `check-shortcuts` (every key the bar teaches is bound) · `build-webview` · `check-surface` (the built bundle in jsdom: it starts, draws the page, writes one edit back, keeps its checkboxes, and survives a round trip through Read mode).

From the repository root, `npm run compile-temco` builds the extensions **and** both webview bundles — the only correct way to rebuild after a `tsc` run, which cleans `out/` first.

`temco-projects`

A documentation project as a first-class thing: its name, its sources, its outputs, its builds, and the pages it produces. One page — the edit form — and one command that opens the project's pages.

Folder	`extensions/temco-projects`
Extension id	`temcocontrols.temco-projects`
Namespace	`temcoProjects`
State	active
Pages	`temcoProjects.project` (the form) — opening a project shows its pages through `temcoDocs.documents`
Settings	none
Requires	`temco-store` (memory), `temco-ui` (page columns), `temco-repositories` (sources and their scripts)
Required by	`temco-docs` — for the one signal that brings the Library's project rows back

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, the output channel a build writes to, and the change signal
`src/projectHub.ts`	~~one project in depth~~ — removed: it read the same counts the Library and the dashboard already show, and clicking a project now selects it and opens its pages
`src/projectPage.ts`	the form: name, version, audience, owner, sources, outputs
`src/project.ts`	the commands that act on a project, and the argument tolerance they need to be called from a row
`src/removal.ts`	removing a project's folder with it: the question that names the folder, the delete, and what happens to the sources that recorded a path inside it
`src/projectBuild.ts`	building a project's outputs: the generator first when the repository has one, then one script per ticked output, judged by the files they left behind
`src/surface.ts`	the two view types
`src/store.ts` · `src/ui.ts` · `src/repositories.ts` · `src/aiModule.ts`	what this module reads of the store, the UI rules, the repository scripts and the AI — each in one file, because an extension cannot import another extension's sources

Commands

`temcoProjects.newProject` · `openProject` · `editProject` · `removeProject` · `selectProject` · `clearProject` · `addSource` · `buildProject` — eight, each doing one thing to a project.

Boundary rules

- **A project is a name and a set of relationships**, not a copy of anything. Its sources stay in their repositories, its pages stay where they are written, and the store holds the rows that tie them together.
- **The pages a project has come from the store.** A read recorded them; this module does not parse a registry, which is why it needs nothing from `temco-docs`.
- **A build is invoked, never reimplemented.** The script is the product repository's own; it runs in that repository's folder, its exit code is reported and never trusted, and the verdict is the file that did not exist before and does now. Every ticked output that has a script runs, once each — asking for a site and a PDF runs both. The deploying form of the help build is never invoked.
- **Removing a project takes its folder with it.** The row and everything recorded against it go, and so does the folder the project owns — `<projectsFolder>/<slug>/` (`temcoRepositories.projectsFolder`, or the working area when it is empty) — holding its `docs/`, its build and what that build produced. The confirmation names that folder before it is deleted. No origin is ever written to and nothing is ever pushed; a clone in the shared `source/` folder is untouched.
- **The AI is optional.** This module declares `temco-store`, `temco-ui` and `temco-repositories`, never `temco-ai`: its pages ask the AI for facts when that module is installed and say so when it is not.
- **The scripts vocabulary is the repositories module's.** Which of the repository's own scripts exist, and what each is for, is a fact about a repository — this module reads the list (`src/repositories.ts`) rather than keeping a second copy.

`temco-repositories`

The organisation and the local copies of it: the repository list, the working area, and reading a source into the platform's memory.

Folder	`extensions/temco-repositories`
Extension id	`temcocontrols.temco-repositories`
Namespace	`temcoRepositories`
State	active
Page	**The Repositories** (`temcoRepositories.page`) — a webview tab, opened by its own command
Settings	`temcoRepositories.organisation` , `temcoRepositories.workspaceFolder` , `temcoRepositories.projectsFolder`
Requires	`temco-store` (the platform's memory), `temco-ui` (where a page opens)
Required by	`temco-docs` — for the code files of a source and the material a new page is drafted from

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, the page command, the output channel, the change signal, and the API
`src/repository.ts`	the organisation's repositories as an API read (paged, with a token when there is one), and a source's code files — from a working tree or from `git/trees`
`src/read.ts`	reading a source: the registry it declares, parsed for pages, recorded in the catalog with what the read concluded; the token in secret storage; opening a file over the API
`src/workarea.ts`	the working area: where a clone goes, whether one is there, how big it is, and clone / pull / remove
`src/repositoriesPage.ts`	the page: what the organisation has, what has been read, what has a local copy, and the buttons for each
`src/surface.ts`	the page's view type (`temcoRepositories.page`)
`src/store.ts` · `src/ui.ts`	what this module reads of `temco-store` and `temco-ui` — the copies of their shapes

Commands

`temcoRepositories.page` · `readSource` · `readAllSources` · `removeSource` · `revealSource` · `setSourceBranch` · `openFile` · `setToken` · `fetchSource` · `pullSource` · `openClone` · `removeClone` · `changeWorkspaceFolder` · `openWorkFolder` — the working area and reading, each with its own command rather than a mode.

Two of those are for other modules rather than for the palette, and are deliberately not declared: `projectWorkFolder` answers where a project's folder is and whether it exists, and `deleteProjectFolder` deletes it. `temco-projects` asks both when a project is removed, so the confirmation can name the folder before it goes — the working area is this module's, and the project row is that one's.

Boundary rules

- **Nothing is written to a repository, and nothing is ever pushed.** Every write this module makes is to the catalog; git is used for `clone` and `pull --ff-only` and for reading history, and never for a change to a tracked file.
- **The working area is outside every repository** (`temcoRepositories.workspaceFolder` , default `~/temco`), so a clone can never be mistaken for a source. A project's own folder is placed by a second setting — `temcoRepositories.projectsFolder` , empty meaning "beside the clones" — so projects can be kept apart from the shared `source/` without moving it.
- **A token lives in secret storage**, keyed `temcoRepositories.githubToken` . It is never written to a setting, a file or the catalog.
- **Reading is explicit.** No read, clone or pull happens on its own: the first read offers a local copy, and every act after that is a command a person ran.
- **What a registry means is not decided here.** This module reads the file and records the rows; the registry's shape, its checks and its areas are `temco-docs` ', which is why this module needs nothing from it.
- **The page opens where our pages open.** That rule is `temco-ui` 's — one group, never a split — and it is read here rather than re-decided (`src/ui.ts`).

`temco-store`

The platform's own memory: one SQLite file per window, the selection, and the naming rules the tables are keyed by. It is the base every other module stands on.

Folder	`extensions/temco-store`
Extension id	`temcocontrols.temco-store`
Namespace	`temcoStore`
State	active
Surface	one command (`temcoStore.status`) — no view, no container, no settings
Requires	nothing
Required by	`temco-docs` (and, through it, `temco-ai`); `temco-repositories` and `temco-projects` when they are split out

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, and the API below
`src/api.ts`	the published contract: `available()` , `catalog()` , `CatalogUnavailable` , `selection` , `slugOf`
`src/catalog/schema.ts`	the tables, one per idea, and `SCHEMA_VERSION` — a bump is a migration, and a newer database is refused in words
`src/catalog/store.ts`	`node:sqlite` : the open, the pragmas, the schema, and every statement the platform runs — sources, projects (`documents` in the tables), their sources, recipes, builds, artifacts, commits, pages, scans, events
`src/selection.ts`	which project the window is on and the page it last opened from it, in `workspaceState` , with the one signal the views and the AI listen to
`src/names.ts`	`slugOf` — the name → key rule the tables assume
`src/commands.ts`	the one command: what the store is, where it is, what it holds

The catalog, in one paragraph

A **source** is somewhere documentation was read from: a repository, a folder, an archive, with the branch and the local copy when there is one. A **project** (the table is still `documents` ; renaming it buys a migration and nothing else) is a name, a version, an audience, an owner, the sources it is made of, and the outputs it produces. A **read** records what a scan found — the page ids, their paths, their owners — and what it concluded, so a list can say "read, 42 pages" without re-reading anything. **Builds** and **artifacts** are what the project's own pipeline produced, keyed by path so a second build of the same site does not turn one output into two rows. **Events** are what happened, and

`unseenEvents()` is how a surface says "something moved". **Commits** are the history a draft is about, per source and per page.

Boundary rules

- **It has no domain opinions.** It stores and returns rows; it does not decide what a project is, what to read, or what to build. Those are `temco-projects`' and `temco-repositories`' questions.
- **It knows no view.** No webview, no tree, no icon: the command prints to an output channel and that is all.
- **It is not `temco-sql`.** That module is about the databases a *product* uses (registers, queries, export). This is the platform's own file, and the reserved module keeps its meaning.
- **It is never a repository's storage.** Nothing is written into a product repository: the database lives in this module's `globalStorageUri` (`<storage>/catalog/catalog.db`) and the materialised trees in a `sources/` cache beside it.
- **A caller passes no context.** `Catalog.open(context)` exists for compatibility at the call sites, and the argument is ignored — the file is the store's, opened with the store's own storage.

Callers

A consumer mirrors what it needs in its own `src/store.ts` (an extension cannot import another extension's sources) and declares this module in `extensionDependencies`, so the exports exist before its own `activate()` runs. `extensions/temco-docs/src/store.ts` is the copy to follow: it re-presents the same names — `Catalog`, `CatalogUnavailable`, `Selection`, `slugOf` — so a call site reads as it did before the split.

`temco-tools`

The device and data surfaces, in the `Tools` activity-bar container.

Folder	`extensions/temco-tools`
Extension id	`temcocontrols.temco-tools`
Namespace	`temcoTools`
State	active
Views	**Devices**, **I/O Studio**, **BACnet Studio**, **Network**, **SQL Studio** — one view per studio, one row per surface
Pages	one per studio (`temcoTools.devices` , `temcoTools.io` , `temcoTools.bacnet` , `temcoTools.network` , `temcoTools.sql`), each a stack of folded surfaces
Requires	`temco-ui`

Why one module

The platform documents a `temcoTools` container "declared by the first tools module", and four separate modules (`temco-device-manager` , `temco-bacnet` , `temco-nettools` , `temco-sql`) are reserved for the day each has a backend. Until then this module is their container: the surfaces exist, are navigable, and are drawn with the same page kit as the rest of the platform, so the design is decided now and the backends fill it rather than redesign it.

What is real, and what is sample

Real	Sample
the container, the five views, the side-bar rows, the pages and their sections	the rows: `src/data.ts` is the mock's sample — devices, points, objects, hosts, tables, queries
the navigation: a row opens its section on the studio's page; a head button opens another section of the same page	the numbers and states
the vocabulary: `temco-ui` 's `page.css` — page head, group rules, tables, badges	—

The sample lives in exactly one file. When a backend arrives (a socket, a stack, a driver), it replaces `data.ts` and no page has to change.

Files

File	Owns
`src/extension.ts`	the five views, the `temcoTools.open` command, and the studio page that reveals or re-renders
`src/pages.ts`	the surfaces of each studio: tables, sections, and the page that stacks them
`src/data.ts`	the sample behind every row, in one place
`src/ui.ts`	what it reads of `temco-ui`: the page shell and the click-to-command script

Boundary rules

- **A page belongs to a studio, not to a surface.** The side bar names a surface, the page opens it; five pages per studio would be five places to keep the same head in step.
- **Nothing is reported as live.** A page that showed a sample value without saying so would be worse than an empty page; the module README and the code say it, and the day a backend lands the sample file is deleted.
- **No writes.** Nothing here sends a command to a controller, a stack or a database.

`temco-ui`

The application's shared UI conventions, and nothing else.

Folder	`extensions/temco-ui`
Extension id	`temcocontrols.temco-ui`
Namespace	`temcoUi`
State	active
Surface	none — no command, no view, no setting
Requires	nothing
Required by	every module that opens a page (`temco-repositories` , `temco-docs` , …)

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, and the API below
`src/columns.ts`	`pageColumn()` and `fileColumn()` — the one tab group our pages open in, and the same group for the files they open beside them
`src/rows.ts`	`rowSlug()` — what a command argument names: a slug, a row, or a row's own command argument
`src/page.ts`	`page()` — the document around a page, `escapeHtml()` , and the click-to-command script a page runs
`media/page.css`	the page vocabulary: what a module draws inside a page tab

The page vocabulary

The mock site (`docs/mock`) is where this application's pages were designed, and `media/page.css` is that design shipped to the product: the same class names and shapes as `docs/mock/css/views.css` , so a page can be designed in the mock and built here without being re-designed.

What changes between the two hosts is only colour. The mock owns literal colours; the product reads the theme the person chose. The top block of `page.css` is that whole difference — every mock colour role (`--fg` , `--border` , `--accent` , `--ok` …) is mapped onto a `--vscode-*` variable, and every rule after it means the same thing in both places. Sizes follow `--vscode-font-size` , so raising the editor font size makes a page larger rather than ignored.

`page(body, { title, script })` returns a complete document: the stylesheet inlined (a webview that links a file needs `localResourceRoots` and a URI that survives the server case; a page needs neither), a content-security policy, and the body inside `.view > .view__inner` , which is the width rule the mock already uses.

A page runs a command by message, not by link. Anything carrying `data-command` posts `{ type: 'command', command, args }` back to its module when clicked; `commandScript` is that script and `runCommand(message)` is the

half that runs it. A page therefore never decides what a command means, and a module can audit the whole of what its pages can do in one place. It also means `enableScripts` buys exactly one behaviour instead of a small application per page.

The column rule, and why it is a rule

Our pages are tabs in one group. The dashboard, a project, the AI page, the repositories page and the documentation pages they open share a single tab strip, so whichever one is in front fills the editor area. Never a split: a page in half the window is not "shown", and two groups means one of them is always too narrow.

The rule was settled on 2026-10-14 after two wrong answers, both of which asked "is the tab in front one of ours?" and fell back to `Beside` — which is what produced the split, because a webview tab does not report its view type the way that check assumed. It asks nothing about what is in front now: it finds the group that already holds a page of ours and opens there, so our pages can only ever occupy one group, whatever is clicked, from wherever.

The prefix it matches is `temco`, so every module's view type counts as one of ours — `temcoDocs.*`, `temcoAi.page`, `temcoRepositories.page`.

Why a module and not a helper inside one of the others

The rule lived in `temco-docs/src/surface.ts`, which was right while every page was that module's. Once pages spread across modules, the rule had to belong to something *none* of them owns: a page opened from `temco-ai` and a page opened from `temco-repositories` must land in the same strip, and a copy of the rule per module is a copy that will diverge — which is exactly how the split it exists to prevent came back the first two times.

Consumers

Declare it in `extensionDependencies` and read it through a bridge of your own — `extensions/temco-repositories/src/ui.ts` is the copy to follow — because an extension cannot import another extension's sources.

`temco-workbench`

The application's own surfaces. It is the only module that is not a domain: it owns the **Library** (the index of what every module has), the **dashboard**, the **needs page**, and the three layout decisions that come with them — which side panel and which tab open at startup, and which sidebar section comes forward when the project changes.

Folder	`extensions/temco-workbench`
Extension id	`temcocontrols.temco-workbench`
Namespace	`temcoWorkbench`
State	active
View	**Library** (`temcoWorkbench.library`), contributed into the `temco` activity-bar container that the documentation module declares
Pages	`temcoWorkbench.overview` (the dashboard), `temcoWorkbench.activity` (what the platform recorded), `temcoWorkbench.openNeeds` (what wants a person), `temcoWorkbench.welcome` (the page a new window opens on)
Setting	`temcoWorkbench.start.open` — `welcome` · `project` · `page` · `home` · `none` (default `welcome`) · `temcoWorkbench.start.panel` — the left panel's item revealed at startup (default `temcoDashboard`) · `temcoWorkbench.updateChannel` and the `branding` · `ai` · `publishing` keys the Settings pages write
Requires	`temco-store` , `temco-ui` , `temco-repositories` , `temco-docs`
Required by	`temco-docs` — nothing; the dependency runs one way, and the workbench reads the documentation module's model

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation: builds the three surfaces, wires their signals, and decides the first tab
`src/sidebar.ts`	the side-bar groups that are not a data view — Platform, Needs Work, Site, Delivery, Profile, Learn, Support — registered here because a container cannot declare its order in the activity bar
`src/dashboard.ts`	the dashboard's two pages: **Overview** (what the platform holds, what is waiting, the pipeline, the activity) and **Recent Activity**
`src/helpPages.ts`	Welcome, Help, Keyboard Shortcuts, Logs, Report an Issue, About — and the one preference Welcome carries
`src/settingsPages.ts`	the five settings pages (General, Modules, Branding, AI Runtime, Publishing). Every row is a declared setting, so a write lands in `settings.json` and the window obeys it at once; the page reads the configuration back and never keeps a value of its own
`src/needsPage.ts`	what wants a person, gathered from every module into one list with the reason and where the fix is
`src/learnPages.ts`	Temco Learn as a reader sees it: Home, T3000, an article, Downloads, Learning Paths
`src/surface.ts`	the page view types
`src/docs.ts` · `src/store.ts` · `src/ui.ts` · `src/repositories.ts` · `src/aiModule.ts`	what it reads of the documentation module, the store, the UI rules, the repository scripts and the AI — each in one file, because an extension cannot import another extension's sources

Commands

`temcoWorkbench.home` (the dashboard) · `temcoWorkbench.openNeeds` (the needs page) · `temcoWorkbench.welcome` (the welcome page) · `temcoWorkbench.toggleWelcomeStartup` (the checkbox in its footer, which writes `temcoWorkbench.start.open`) · `temcoWorkbench.settings` (the settings page) · `temcoWorkbench.setStartOpen` (the one command a row on it runs) · `temcoWorkbench.help` (the help page). Everything the Library's rows and menus do is another module's command — the menus are here, the work is not.

What the pages refuse to invent

The mock has surfaces whose content this product cannot honestly produce, and none of them were copied across:

Mock page	Here
Help › Keyboard Shortcuts, Help › Logs	Not built. VS Code publishes neither list to an extension, and a page that guessed would be a page that lies. The Help page carries what is real instead: the build, the module register, the platform string, and the issue tracker.
Workspaces	Removed. A workspace was the mock's folder-above-the-project level, drawn over a fixed sample (`T3000`, `T3-BACnet`, …) with counts nothing recorded, and a "create" that wrote only into the window. The product's unit is the **project**, which the catalog holds; its sources are the folders, and New Project is the wizard the sample stood in for.
Devices, I/O Studio, BACnet, Network, SQL Studio	Not built. Those modules are reserved — a page over a module that does not exist would be a mock inside the product.

One home: Welcome, the VS Code way

A window that opens on a page you did not ask for has to earn it, so the welcome page carries only what a person can act on: six ways in (each a button running another module's command), three walkthroughs that list their steps instead of claiming progress, and a footer whose single checkbox writes the setting that decides the first tab. There is no picture of the product on it.

The arrangement is VS Code's, deliberately:

- **Welcome is the first tab.** `start.open` defaults to `welcome`, so a launch answers "what is this and where do I begin" before it shows any work.
- **The page owns the switch.** Its footer checkbox *Show the welcome page at startup* writes `temcoWorkbench.start.open` — there is no second place to turn it off, and the checkbox and the setting cannot disagree.
- **Help ▸ Welcome brings it back**, as does the palette, every time.
- **The Dashboard is not a second home.** It is the activity-bar container you go to *after* Welcome — Overview (what the platform holds and what is waiting), Recent Activity, Needs Work. VS Code's equivalent is Explorer: state you click into, never a page that opens itself beside the welcome page.

The two only ever fought because either could be the first tab. Now one is the default and the other is a click, and each has one question: Welcome asks *where do I start*, the Overview asks *what is in it and what needs me*. With the checkbox off the chain never ends at the welcome page — `project` → the page you were last on → the overview — so turning it off means what it says; an empty window gets the overview, which has its own empty state and a *New Project* button. **Coverage & Health** is gone as a third dashboard page: the documentation module draws that report from the same registry with the same figures, so the Overview's Knowledge card links to it instead of duplicating it.

Where a settings page writes

Every row on the five Settings pages is a declared setting, so everything a page decides is in the profile's `settings.json`:

Page	Keys
General	`workbench.colorTheme` · `temcoWorkbench.start.panel` · `workbench.breadcrumbs.enabled` · `telemetry.telemetryLevel` · `temcoWorkbench.updateChannel`
Modules	none — the rows describe what is loaded, and every button runs another module's command
Branding	`temcoWorkbench.branding.productName` · `.accent` — the name the pages draw, and the accent written into `workbench.colorCustomizations`
AI Runtime	`temcoAi.model` · `.runtime` · `.modelFolder` · `.allowUnpinned` · `.idleMinutes` · `.serverPort` · `.serverIdleMinutes` · `.platformApi` — the AI module's own keys, so the page writes exactly what that module reads
Publishing	`temcoWorkbench.publishing.site` · `.path` · `.channel` · `.website` · `.pdf` · `.offlineHelp` · `.searchIndex` · `.aiIndex`

Two facts are not settings and are not written there: **when a row last changed** and **what Test Connection last read back**. Both are kept in the profile's own state rather than in the file, because they record what happened rather than what a person chose. `Restore Defaults` removes the value instead of writing a copy of today's default, so the manifest stays the only place a default is stated.

Boundary rules

- **The Library is an index, not a domain.** It shows what the store recorded and what the documentation module published; it owns no data of its own and keeps no copy.
- **A row's action belongs to the module that owns the thing.** *Read a source* is `temcoRepositories.readSource`, *Build the project* is `temcoProjects.buildProject`, a page's *Rename* is `temcoDocs.renamePage`. The Library names them; it never re-implements one.
- **Nothing here writes.** The only state it changes is the project being worked on, and that is the store's own command.
- **The AI is optional for it too.** The Library's AI row appears only when that module is installed — asked, never assumed.
- **The Workspace is not its view.** The tree of pages is the documentation module's own view; the workbench only *brings it forward* by id when the project changes.

The application flow

> **Superseded for the Docs center.** The window layout, the Pages tree, the journey and the Publish flow are defined in redesign.md. Where this page disagrees with it, `redesign.md` wins.

The information architecture as built: the left toolbar, the four panels, the page in the middle, and the one right-hand panel. This is the confirmed design — `authoring-flow.md` is the capture stage inside it.

Left toolbar

```
TEXT
```

```
┌──────┐
│  ⧉   │  Explorer          VS Code's own
│  ⌕   │  Search            VS Code's own
│  ▦   │  Dashboard         projects · overview · needs you · recent activity
│  ▤   │  Documentation     pages · project · pipeline · working set
│  ⧉   │  Repositories      org → repository → code files
│  ✦   │  AI                models · drafts waiting
│  ◫   │  Learn             site · delivery
│  ▥   │  SQL Studio        query & browse · schema & maintenance
│  ⚙   │  Settings          profile · platform
│  ?   │  Help              learn · support
└──────┘
```

The shell's `ORDER` holds the eight module containers (`temcoDashboard · temco · temcoRepos · temcoAiHub · temcoLearn · temcoSql · temcoSettings · temcoHelp`); Explorer and Search are VS Code's and come first.

Dashboard — four views

View	What the panel shows	The page behind it
Projects (`temcoProjects.projects`)	a row per project: sources · built · current, with ✎ and ⓘ	clicking a row opens the **project page** (`temcoDocs.projectPage`): counts, sources, outputs, build buttons
Overview (`temcoWorkbench.platformView`)	**the facts as rows** — Pages · Reviewed · Stale · Issues · Sources · Outputs · Build	each row opens its page (the tree, the report, Problems, the project form, Build Center)
Needs You (`temcoDocs.queue`)	the queue's groups; a group opens its page	the **queue page** (`temcoDocs.needsPage`): screenshots to place, retake these, never reviewed — each with its act
Recent activity (`temcoWorkbench.activityView`)	the activity row	the **activity page** (`temcoWorkbench.activity`): every recorded event, newest first

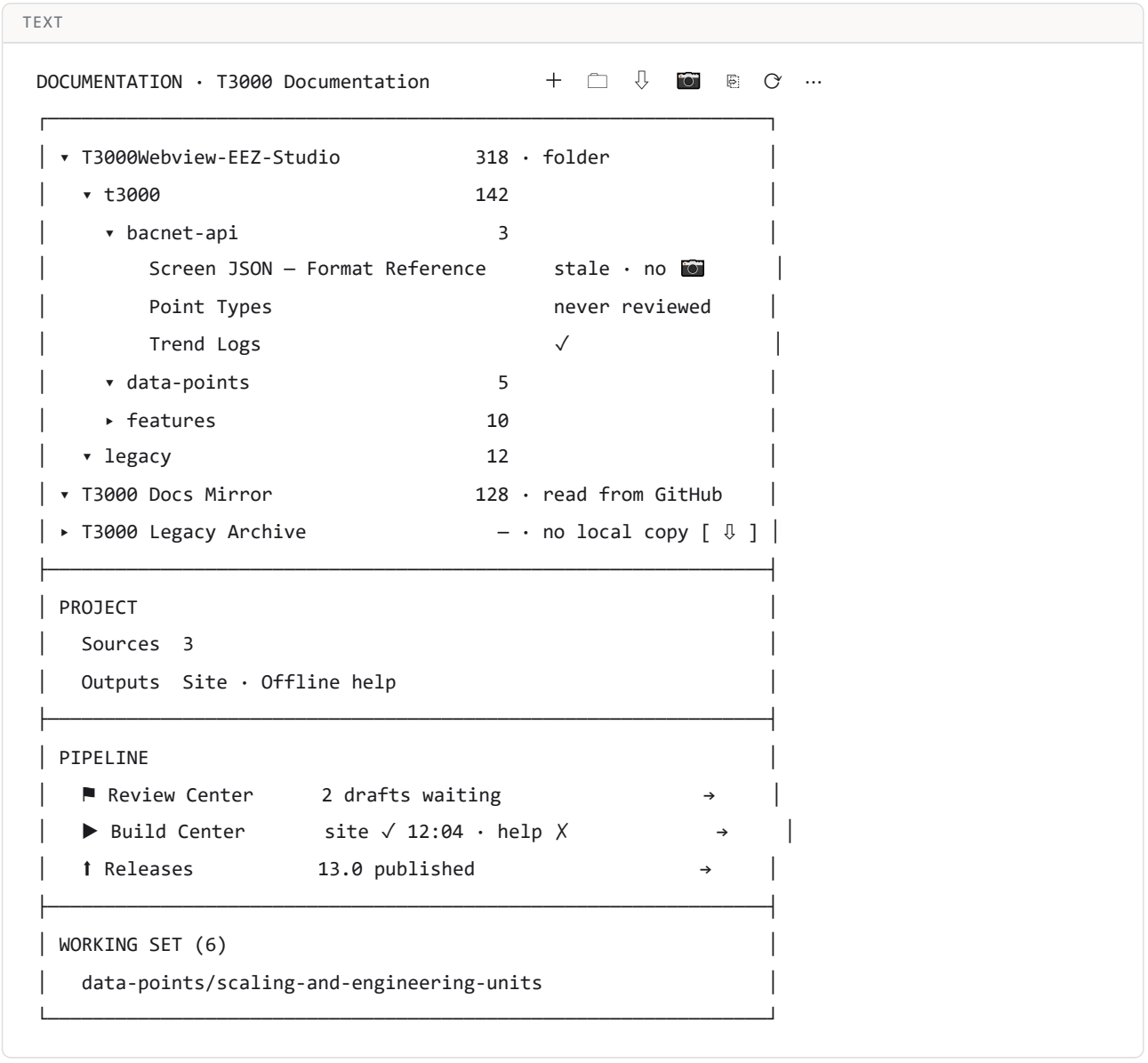
TEXT

```
DASHBOARD                                          ⟳
┌──────────────────────────────────────────────────────────────┐
│ PROJECTS                                              +      │
│   T3000 Documentation          3 sources · built 12:04 · current│
│                                                [ ✎ ] [ ⓘ ]   │
│   My First Docs                1 source · never built        │
│   Legacy Import                1 source · built 09:12        │
├──────────────────────────────────────────────────────────────┤
│ OVERVIEW                                                     │
│   Pages      318       Reviewed  304                         │
│   Stale       14       Issues    0 · 3                       │
│   Sources      3       Outputs   Site · help                 │
│   Build      Build Center                                    │
├──────────────────────────────────────────────────────────────┤
│ NEEDS YOU                                             (7)    │
│   ▸ Screenshots to place (1) · Retake these (4) · Never… (2) │
├──────────────────────────────────────────────────────────────┤
│ RECENT ACTIVITY                                              │
│   Recent Activity                        20 events           │
└──────────────────────────────────────────────────────────────┘
```

`✎` = *Edit the Pages* (`temcoProjects.openProject`) → the project becomes the one being worked on and the Documentation panel shows its tree. `ⓘ` = *Show the Project* (`temcoProjects.showProject`) → the Documentation panel comes forward at its **Project** section.

Documentation — four views, the tree first

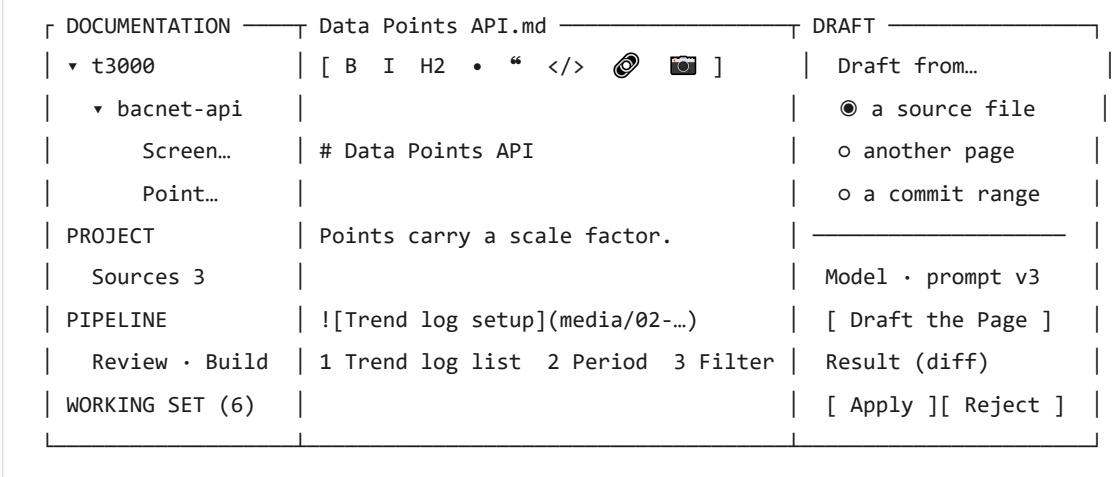
TEXT



Title bar: ＋ New Page · 🗀 New Chapter · ⇩ Add a Source · 📷 Capture · ⎘ Paste · ⟳ Refresh.

The page and the right panel

TEXT



The rule: the right panel holds what you consult while working in the page — **DRAFT** for a documentation page, **SCHEMA** for a SQL query. Everywhere else it is closed.

What the page is, for every row

Left panel row	Page in the middle
a project row	the **project page** — counts, sources, outputs, build buttons
Overview · a fact row	the page it counts: tree · report · Problems · project form · Build Center
Needs You · a group	the **queue page**
Needs You · a capture	the attach prompt: which page takes it
Needs You · retake / never reviewed	the page, with its image
Recent activity · a row	the activity page
a tree page	the page, in the editor, DRAFT on the right
Project · a source	that source's page
Project · Outputs	the project's edit form
Pipeline · Review / Build / Releases	Review Center · Build Center · Releases
a Working Set row	the page
a repository row	the repository page
a model or a waiting draft	the AI page
a SQL table row	SQL, with the statement filled in

The flow

```text
① DASHBOARD · PROJECTS  ＋  → name · source (folder) · output (site)
      └ the row appears, selected
          ├ [ ✎ ] → DOCUMENTATION · tree
          └ [ ⓘ ] → DOCUMENTATION · project + pipeline
② DOCUMENTATION · ＋ New Page → location · kind · title → one file
      ├ the row lands in the tree; F2 renames it
      ├ [ Run the generator ] → it becomes a page
      └ the page opens; Ctrl+Alt+S / Ctrl+Alt+V put a screen in its media/
③ DASHBOARD · Recent activity shows the build, the capture, the review
④ DOCUMENTATION · PIPELINE → Review Center · Build Center · Releases
```

Two decisions worth naming

A row's F2 renames the *file*, and keeps the old id as an alias. The page id *is* the path (`05-content-and-registry.md` , rule 1), so the rename is the named act that rule allows: the file moves, the repository's own

`docs:registry` writes the new entry, and the old id is written into that entry's `aliases` before anything else happens. Shift+F2 renames the title instead, which is the registry value. A project with no generator is refused — without one there is no entry to carry the alias, so the rename would orphan every inbound link.

The right-hand panel is `DRAFT`. It lives in the auxiliary bar and is about the page in front: *Draft a Change* asks what the model may read (this page's history · a source file · another page), and the drafts waiting for that page are applied or rejected there — the same commands the Review Center runs.

Needs You replaced the workbench's Needs Work view. They were the same question asked twice; the queue has real rows (captures with no page, stale pages, never reviewed) and the old view had one link.

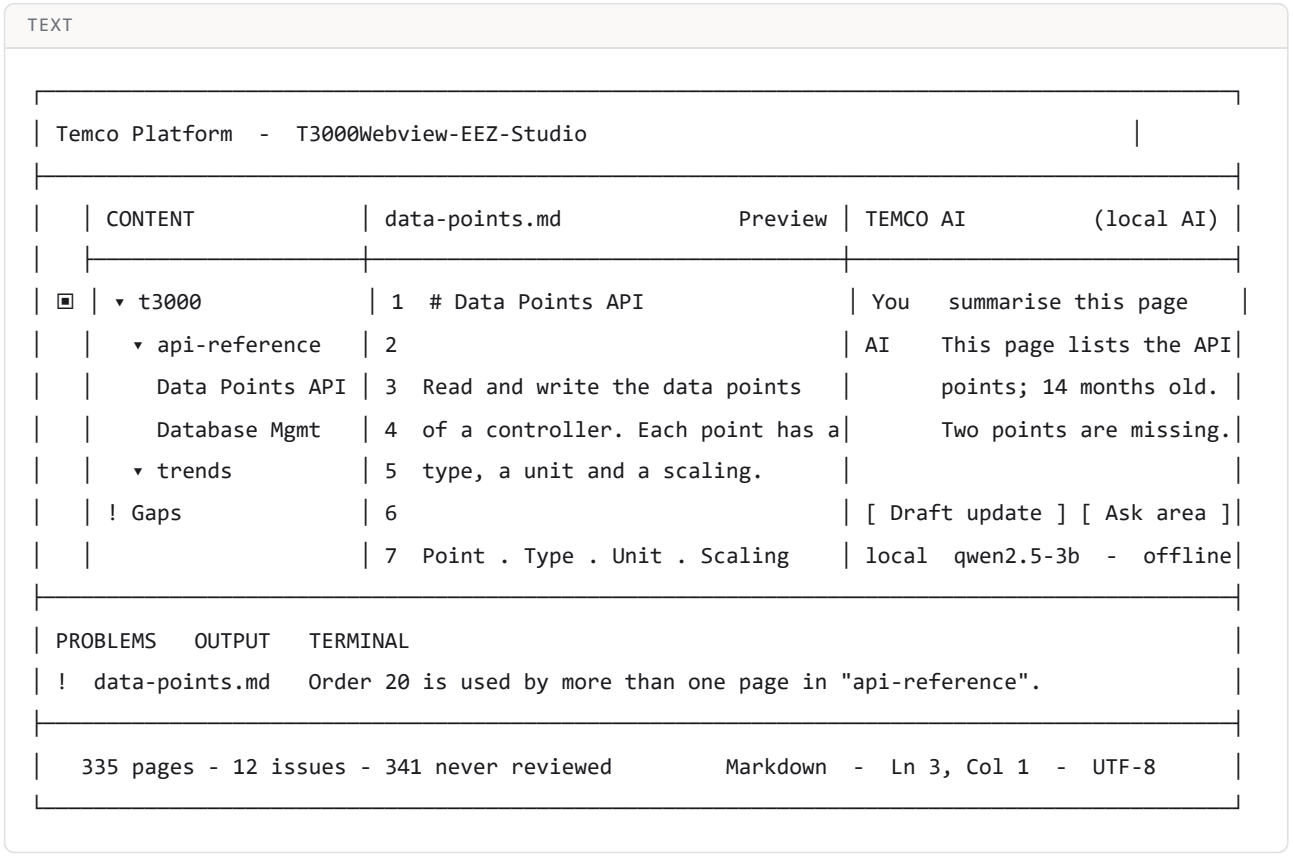
Still open

- **Annotation**, which waits on the callout decision in `authoring-flow.md` (a webview's DOM, Windows UI Automation, or by hand).
- A **`version-report.json` writer** in the product repository's release workflow: this application reads the file, and shows its pages on the project page and in **Needs You**, but nothing writes it yet.

The application window

What the extension looks like inside **Temco Platform** — the shell's panels are unmodified, and only two of the five areas are ours.

TEXT



Panel	What it is	Whose
▣ **Content**	the docs tree: areas → chapters → pages, counts on the right, **Gaps** at the bottom	ours — the only view that does not exist in ordinary VS Code
Editor	the page: plain Markdown, no front matter, Preview beside it	the shell's
Temco Docs	the AI: chat about the open page, `Draft update`, `Ask the area`, and which model is loaded	ours
Problems	structural faults only, each naming the file to fix	ours
Status bar	`pages · issues · never reviewed`	mixed

Three properties of this layout are decisions, not accidents:

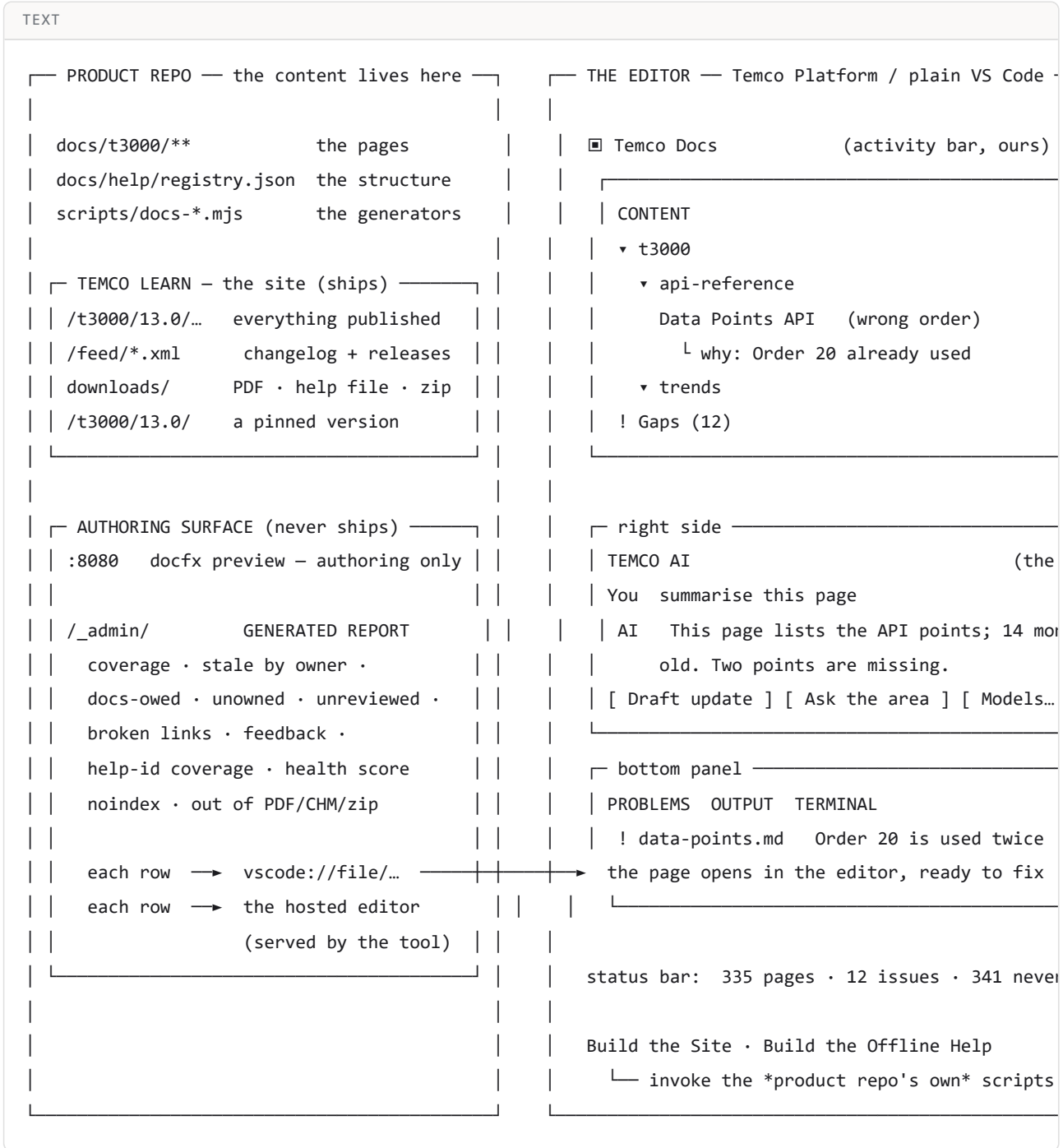
1. **The only new real estate is the Content view and the AI panel.** Search, source control, compare, terminals, Markdown preview and the problem list are the shell's, already there, nothing to build.
2. **The AI is a panel beside the page, not a separate application.** It reads the page you have open and the registry; `Draft update` writes a normal edit you review — so accepting is a diff, rejecting is `Ctrl+Z`.
3. **Both of the module's areas exist now.** The Content view came first and the AI panel after it, and the mock above is the panel as built — same place, same buttons, so the layout never had to move.

Two worlds, and the seam between them

The dashboard is not in this application. `00-architecture.md` puts it on the **site**: a **generated static page** at `/_admin/` — coverage · stale by owner · docs-owed · unowned pages · broken links · feedback · help-id coverage — noindex, excluded from the PDF, the help file and the offline copy, and produced by the **product repository's** own pipeline (the admin report task in `13-implementation-plan.md`). Nothing in this extension generates it, and nothing here writes into it: the design's rule is that those are *generated pages, not documentation*.

What the extension does in that relationship is be the **hand-off target**. A dashboard row deep-links into the editor (`vscode://file/…`), and this is where the page gets fixed:

TEXT



The dashboard answers *"which pages need work across the organisation?"* — on the site, generated by the product repo, page-level health, never shipped
The Content view answers *"which pages need work in the repository I have open?"* — in the editor, one repository, the reasons as children of the page that has them
The same pages, two distances. Neither is a copy of the other, and neither recomputes what the other owns
This extension is where the work happens — a dashboard row is a hand-off, not a destination

Start a Registry appears where it applies rather than in a title bar: as the Content view's empty state, for the folder that has no `docs/help/registry.json` yet. It works on a folder **you have open** — a repository you have not cloned is refused in words, because starting one there would be a commit made through the API on your behalf. Filling the registry it creates is the repository's own generator's job (`npm run docs:registry`).

Where the AI sits, and why that is a user action. It belongs on the **right**, beside the page being worked on. VS Code has no contribution point that places a container there — `contributes.viewsContainers` accepts `activitybar` and `panel` only — so the view is contributed to the panel container and the **workbench docks it on the right**: the secondary side bar accepts any view, remembers the position, and a fresh install needs one drag. If it must open there by itself, the alternative is an editor-area webview panel opened beside the page (`ViewColumn.Beside`), which is a tab rather than a dock. Recorded in `platform-design.md` , not hidden here.

The application: shell, screens and flow

The record of the agreed shape of Temco Platform as an application — the regions of the window, the screens, the way one leads to the next, and the two rules the design is built on. It is the design of record: a surface that disagrees with it is either a bug or a proposal to change this file.

Confirmed 2026-10-27. Decisions and their reasons are in *What was decided*; the work order is in *Order of work*.

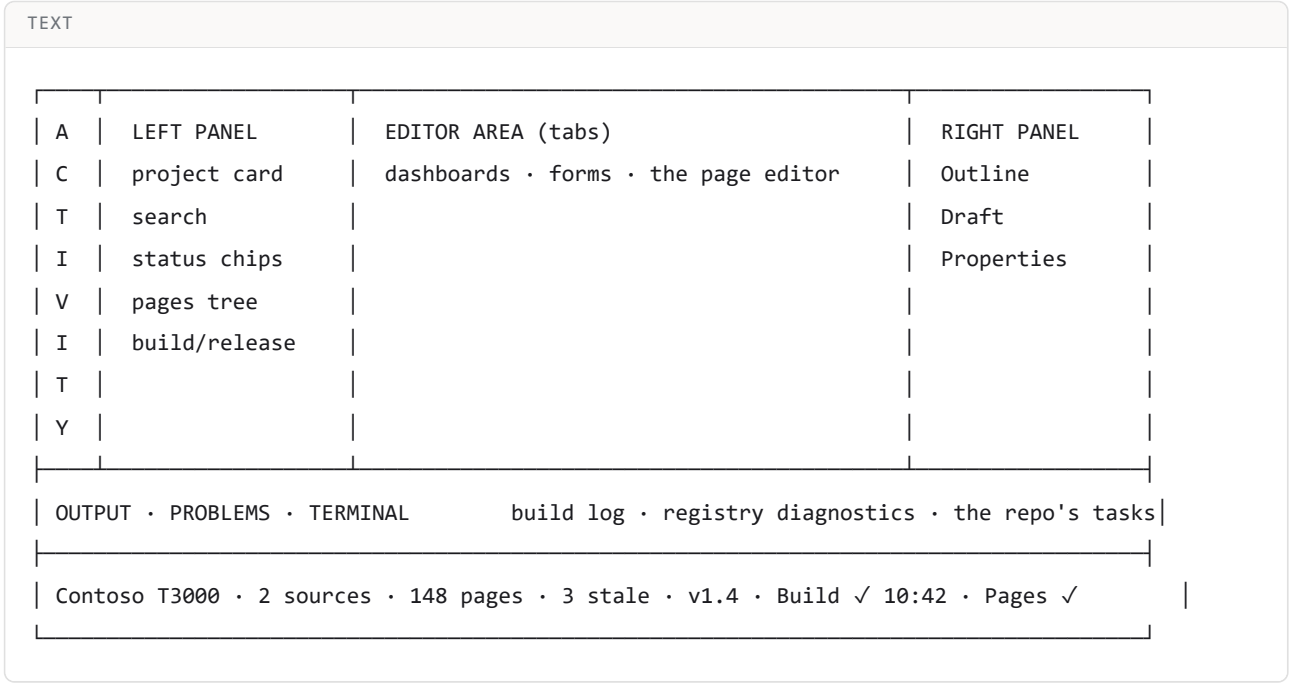
To **use** the application rather than design it, read getting-started.md: the same flow as numbered steps, with the button for every page and the command id behind it.

The two rules

1. **A VS Code view is native.** The workbench draws a tree view; an extension cannot put Fluent inside one. So the left panel is a Fluent **header + chips + footer** (a webview view) above a **native Pages tree**. The tree keeps reveal, drag, multi-select, keyboard and accessibility for free — the parts where re-implementing costs the most and shows the least.
2. **One project is one window state.** Switching project re-reads and lands on the dashboard. Nothing keeps describing the project that was open before.

The regions

TEXT



Activity bar: five entries

	entry	contains	today's views
◉	**Home**	projects, recent, what needs you, Learn	`temcoDashboard` (projects, platform, activity) + `temcoLearn`
▤	**Docs**	the workspace: pages, sources, publish	`temco` (pages, needs-you, sources, publish) + `temcoRepositories.list`
✦	**AI**	chats, models, drafts	`temcoAiHub` + the `temcoAi` secondary sidebar
⚙	**Tools**	SQL Studio, and the modules not written yet	`temcoSql`
⚙	**Settings**	settings, Help	`temcoSettings` + `temcoHelp`

Eight icons became five: three are places to *be* (Home, Docs, AI), two are where you *configure* things. `Repositories` moved into Docs ▸ Sources, `Learn` into Home, `Help` under Settings. No view is deleted.

Implemented (2026-10-27): the five containers exist with those titles, `temcoRepositories.list` is contributed under `temco`, and the `temcoLearn` / `temcoHelp` / `temcoRepos` containers are gone. Container **ids** were kept where they were already used (`temcoDashboard`, `temco`, `temcoAiHub`, `temcoSql`, `temcoSettings`) so saved layout and `onView` activation keep working — only titles and view membership changed. The guard in `build/check-temco-modules.ts` now refuses a view pointed at a container nobody declares, which is the mistake this consolidation can make silently.

Screens

#	screen	center	left	right
S1	**Home — no project**	hero: New project · Open a source · recent	projects tree	—
S2	**Create project**	3-step form: name/slug/version → where → what is in it	—	—
S3	**Home — project selected**	one callout for the one thing that needs a person, the five-stage pipeline with the figure each stage is answerable for, then *what needs a person* and *what happened lately*	project tree	—
S4	**Add source**	—	inline form in Sources	—
S5	**Docs workspace**	the page editor or a dashboard tab	**Fluent panel**: project card · search · chips · pages tree · build/release footer	Outline · Draft · Properties
S6	**Registry review**	two panes: areas ↔ pages, with issues, and *Fix*	as S5	—
S7	**Page editor**	Temco Editor: one-row bar, paper, status strip	as S5	Outline (default) · Draft · Properties
S8	**New page**	kind · title · area · order · ships → creates file + registry row → **S7**	as S5	—
S9	**Preview / Diff**	beside the editor: the published layout, or the diff against HEAD	as S5	—
S10	**Build**	step list (Resolve → Render → Assets → Sitemap), streaming log, result card	as S5	—
S11	**Release**	version, what ships, destination `releases/<version>/`, a confirmation naming every value	as S5	—
S12	**Deploy to Pages**	repo · address the site will have · what will be committed · **Publish**; never force-push	as S5	—
S13	**Export**	format cards: site `.zip` · single-file HTML · Markdown bundle · (`.docx`) · scope	as S5	—
S14	**AI**	conversations, models, the answer with page citations	conversations + runtime	—
S15	**Learn**	the site: documentation · training · tests · products · Q&A · topics, then the reader for one version	the site folder	—

#	screen	center	left	right
S16	**Settings**	profile · platform (theme, AI runtime, build defaults, Pages branch)	sections	—
S17	**Help**	search, topics, support (logs, diagnostics)	topics	—

Screens S1–S13 are the documentation journey; S14–S17 are the platform around it.

Navigation

from	action	to
anywhere, no project	launch	S1
S1	New project	S2 → S6
S2	Create	S6; the status bar switches to the new project
S5 tree / S3 row	a page	S7 (same tab), split on `Ctrl+click`
S6	a row	S7, or the issue in the page's own Problems
S7	Build	S10 → S11 → S12
S11	Release	S12 (toast), S10 keeps the result
S12	Deploy	stays; status bar `Pages ✓` and the URL opens
S13	Export	stays; the file is revealed in the OS
any	a registry error	S6 with the issue selected
any	the project in view is gone	S1

The publish journey

```text
create project ─▶ add source ─▶ registry ─▶ write ─▶ validate ─▶ BUILD ─▶ RELEASE ─▶ DEPLOY ─▶
                                   (S6)      (S7)     (S6)      (S10)     (S11)     (S12)    (S13)
```

- **Build** renders the registry itself: `dist/site/<page>.html` from `marked`, one stylesheet, `sitemap.xml` (only when the GitHub remote gives a real address), `.nojekyll`. A project that already has its own `docs:site` / `docs:build` script keeps using it — the generator is the fallback that makes "a folder of `.md` files" buildable.
- **Release** freezes `releases/<version>/` and records it in the catalog (already built).
- **Deploy** commits the built site to the branch Pages serves and pushes it — a plain, fast-forward push, never a forced one: if the branch holds work this build does not know about, that is reported instead of erased. The commit is made in a throwaway repository whose only parent is the branch's tip, so the project's own working tree is never touched. The one-time Pages switch (`Settings → Pages → Source: branch “gh-pages”, folder “/”`) is attempted through the `gh` CLI when it is installed, and printed as two clicks when it is not. **Why not Actions:** the generator reads the documentation registry, which only exists inside this extension, so a CI job

cannot run it. An Actions workflow would need a second copy of the renderer inside the project — a copy that goes stale the first time either side changes. CI publishing becomes possible again the day a project owns its own renderer; the branch push is what makes publishing work until then.

- **Export** is local and dependency-free: the site as a `.zip`, any page as one self-contained HTML file, the pages and their media as a Markdown bundle. The zip writer is in the module (`src/zip.ts`, store + deflate over a CRC32 table) rather than a dependency, because an export a colleague cannot open is not an export. `.docx` needs a writer package (`mammoth` is import-only) and is still owed.

Temco Learn

Temco Learn is its own module (`extensions/temco-learn`, the tenth), because a reader-facing website is not the same job as keeping a page in step with the code. **The website itself is a separate repository** — `temcocontrols/temco-learn`, Vite · React 19 · Fluent UI v9 · Markdown — and this module is a *client* of that checkout: it runs the site's own servers, builds it, shows it, writes new builds into it, and commits the result. Nothing about rendering the site lives here any more, so the site can be worked on, built and published by someone who never opens the editor.

`temcoLearn.siteRepo` names the clone (empty means `~/Temco/temco-learn`). Everything below is a fact about that folder:

- `content/<project>/site.json` — the project's title, section and summary; `content/<project>/<version>/*.md` — the pages a person writes, with front matter for the title;
- `public/imported/<project>/<version>/` — a project's own build HTML, copied in as it came out;
- `public/catalog.json` — the one file both the website and this module read. `npm run catalog` folds the two sources above into it (order, titles, versions, `builtAt`), so the panel cannot list a page the site does not have. A second reader of a second shape is how a panel and a page start disagreeing;
- `package.json` scripts — `client-dev` (Vite, with reload), `build` (catalog + search index + `tsc` + `vite build`), `serve` (the built pages), `catalog`, `search`, `check`. Neither server script names a port: the module passes `--port`, so there is one port for both and one setting that moves it;
- `.github/workflows/pages.yml` — a push at `main` builds the site and hands `dist` to GitHub Pages. The built site is never committed: what is published is the source and the catalog it generates.

The module runs the site, it does not re-implement it. `src/siteRunner.ts` spawns the repository's own `serve` script through `npm`, tracks one server on one port (1025 by default), and treats "starting" as distinct from "running": it polls the address until it answers, so a frame is never pointed at a port with nothing behind it. `Stop` kills the process tree and waits for the port to go quiet, because `npm` hands the server to a child that can outlive the kill. A `dist` that was never built is built before the built pages are first served, rather than served as an empty folder.

`Preview` is a panel framing the running site (`src/preview.ts`, `temcoLearn.site.preview`), with a bar over the frame that is a **caption, not a toolbar**: a tinted strip (`--vscode-sideBar-background`, 34px, one hairline under it) holding the state as a coloured dot and one word, then the address, then the acts — and every one of those is a **link**, not a box, so nothing in the strip competes with the website below it. No address bar, because the address is not a question: one port, set once, and a person who wants the site is not choosing a URL — but the address *is* clickable, because what a person does with a URL is click it, and it goes where the act beside it goes. The three acts read `Reload · Open in Browser · Stop`: the second one is named for where it goes rather than for the noun `Browser`, which said what the thing is instead of what the button does. `Reload` and `Open in Browser` are inert with no address; `Stop` never is. Closing the panel does not stop the server, the way closing a browser tab does not stop a website; `Stop` does. `Add This Build to the Site` copies a build into `public/imported/<project>/<version>/`,

writes the `site.json` the catalog needs (the title comes from the build's own `<title>`), runs `npm run catalog`, and shows the new version — only the version is asked, and it is asked once, in the version box, never in a notification. `Publish` commits the clone and pushes it at `main`; it refuses a folder that is not a clone instead of initialising one, because an empty history pushed at a real site is worse than a refusal.

The panel is **three views in the container**, the way Home is three — VS Code draws the header, the separator and the collapse state, so the grouping is the one a person already knows from the Explorer, and each section carries its own title-bar actions. A row is a name and a glyph, and what it holds is `TreeItem.description` — the grey words the Explorer writes beside a folder (`3 versions`), never a badge of our own. Sentences live on the Overview page and in tooltips, never in a row.

view	rows	the grey words
Overview	one row, `Temco Learn`, which opens the page	—
Site	**Preview the Site** (a state dot and one word; its heading opens the Site page; `Stop` / `Start`, `Open` and `Open in Browser` `:1025` under it), then **Library** (one row per project, at its newest version)	the state: `Running`, `Starting…`, `Stopped`, `Failed`, `Not set up`; projects served
Artifacts	one row per project built on this machine, opening onto the files its build wrote; `Publish` closes the view (`initialSize: 4`)	**nothing** — the name only, and a hover carries the count and the paths; `Publish` puts its count at the row's right end

Three views, one subject each. *Artifacts* is **content**: the files the builds on this machine wrote, plus the single act that is about all of them at once (`Publish`). Nothing here is a task list and nothing here is a state — a view holds things, and the things are files.

Adding a build is not in the panel at all, because it is a *site* operation: it copies a build into the site repository and asks for its version, exactly like cloning, building and publishing write into that repository. So it lives on the **Site page**, as a section of its own — `Builds not on the site`, one row per build with the reason it waits (`not on the site`, `newer than 13.0`) and one `Add…` button — and every command that means "add a build" (`temcoLearn.addBuilds`, from the Overview callout, its workflow card, the Publishing page's step, the `+` on the Site view) opens that page **with the section first and marked**. That is the whole answer to "what is that row for": it is the step that turns a folder on disk into a version on the website, it needs the version typed, and it belongs beside the other steps on the site.

`Build a project` is not a row either: building happens in Temco Docs, so it appears **only when nothing has been built at all** — `Nothing built on this machine yet` / `Build a Project…` — where it is the honest next step. The ellipsis is VS Code's signal that a click leaves this window, and the tooltip says where it goes.

A label is **navigation** or an **instruction**, and the two are not the same sentence: a button on a page that goes to the other module reads `Open Temco Docs` (the way VS Code says `Open Settings`), while the last row of an empty view reads `Build a Project…`, because a view with one line in it has to answer *what do I do*, not *what opens*.

Site answers two questions, in that order. **Preview the Site** is a group whose *heading* is the page and whose *children* are the only acts a panel is good for: `Stop` while it is running and `Start` while it is not (one row, the word swapping with the state), then the two places the site can be read — `Open`, a tab in this window, and `Open in Browser`, which shows the port alone (`:1025`) — and both start the server first if it is down. The two are named for the difference a

person feels, not for the API behind them: `Open` stays here, `Open in Browser` leaves for the browser they already have. Everything else a clone needs — clone, pull, install, build, change the folder or the port, the log — is the Site page's, because a second copy in the panel is how a panel and a page come to disagree about what is running. A clone that is not there swaps those rows for `Clone temco-learn…` and `Use an Existing Clone…`, and the heading says `Not set up` and still opens the page that explains it.

The heading's glyph is the **state**, not the site: a filled circle in `charts.green` while it serves, `sync~spin` while it comes up, `error` in `charts.red` when it failed, `circle-outline` when it is stopped, `cloud-download` in `charts.blue` when there is no clone — the way VS Code's own Ports view draws a running process, so a glance down the panel answers *is it up?* before a word is read.

A row's facts sit at the row's right-hand end, and in a tree that is a decoration badge and nothing else: `description` is drawn inline, immediately after the name, so it can never be a column — but a badge is a `::after` on the label element, which is a flex row, so it is the last flex item and lands at the edge (`views.css` even gives it a 4px right margin). `RowDecorations` answers one badge per row of the module's own scheme: `Running` / `Stopped` / `Failed` / `Not set up` in the state's colour, `:1025` on the browser row, `2 projects` on the library. So the three rows read as a table without a single space character of padding, and the words are in the tooltips as well, for anyone who has turned badges off. The provider answers **synchronously**: a decoration that arrives as a promise leaves the first paint of its row blank, because `DecorationsService` only has the data when it is already there.

Library is one row per **project, at its newest version** (`T3000 · 13.0`), opening that page on the site at `#/<project>/<version>/` and starting the server first if it is stopped. Not one row per version: a product with twenty versions turned the panel into a scroll bar, the newest is the one a person wants nine times in ten, and the site's own list holds the rest. The name is `Library` because the rows are documentation sets, not pages — a *page* lives inside a version, and a *category* is the catalog's `section` field, a different axis. The row writes the project's **name**, without the "docs" a build's own page title usually ends with (`Test1 docs` reads `Test1 · 1.0`), and it carries no description: the version is already in the label, and a grey `2 pages` beside four rows was the noisiest thing in the view. A site with nothing on it does not report emptiness: its one row is `Add a Build to the Site…`, which runs the same picker the view's title bar does. There is deliberately no group of `dist` builds: the build as written is a file on disk, which is the Artifacts page's business, not something to read here.

The Site page is where the clone is managed (`src/sitePage.ts` , `temcoLearn.site.page`), and it is what `Preview the website` opens when there is no clone — a folder dialog was never an explanation. It reads the folder every time it is drawn (`src/siteFacts.ts` : branch, commit, ahead/behind, changed paths, `node_modules` , `dist`) and offers the act for each answer: clone into `~/temco/temco-learn` , use an existing clone, change the folder or the port, pull (`--ff-only` , never a merge), install, build, start, stop, open a terminal, and clear or open the log (`src/siteLog.ts` , the same bounded ring the publish page keeps). The runner grew the two git steps the page needs (`git` , `pull` , `clone`), so a clone and a pull print into the same log a build and a server do.

The page is a control panel, not an article, and it names itself **`Documentation Site`** — `Site` said nothing the view title did not, and `Temco Learn` above it was a breadcrumb of one, which is a label pretending to be navigation. The title block says **what the site is** (its own repository, the toolchain, one port) and the state (`Running · localhost:1025 · built` , `Stopped` , `Failed`), then **one primary act chosen by that state** — `Clone` → `Install dependencies` → `Build the site` → `Start the site` → `Open` , with `Try again` when a start fails. `Open` goes to a tab in this window; the title carries the `globe` chip, because it is the page of the site and the browser is what it shows.

The site is read in two places, and the page names both the way the panel does: `Open` in the **This machine** row stays here, `Open in Browser` leaves for the browser the reader already has. A button labelled `Open` that opened a browser was the ambiguity worth removing; the tooltip on each says which one it is.

A failure is said **once**, at the top, in the tinted callout — with the reason and the two acts that answer it (`Try again` , `Open the full log`). The title block then says only `Failed` ; saying the reason twice made the reader look for the difference between two identical messages.

The clone's own facts are **not in the sentence**. A meta line under the title block carries them, in mono, and pairs them with the two figures that say how far the copy has drifted — `0 commits behind origin` , `5 paths changed` — so they scan as numbers with left edges in a column rather than hiding at the end of a paragraph. The repository, the commit and the changed paths are **links** (a command, never an `href`): the first two open GitHub at that commit, the last opens the Publishing page, which is where those paths are listed. A clone with nothing changed links nothing — a link to an empty list is a dead end:

```
temcocontrols/temco-learn · main · 577ca57 · committed 3 days ago
0 commits behind origin        5 paths changed
```

Below that, the page is two parts, and both section titles wear the brand's vertical rule (the shape the Overview page uses):

part	what it answers	drawn as
This machine	what is here, and is it running	the server as one line — state in mono, `Repository` | `Terminal` , then `Start` / `Stop` , `Open` and `Open in Browser` — then four **small cards in one row**: Folder · Port · Installed · Built (two across under 760px). A card is a label, a value and the act that changes it, **with no border**: its condition is a 6px dot in `charts.green` when the step is done, `charts.yellow` when it is not, and a hollow one when there is no condition to report
Activity	what it said	a tabbed area, `Output` and `Publish` , plain text on a tint off the card surface: no border, no header bar, no line numbers. The tabs are radios and labels, so they switch with no script and still answer the keyboard

`Log` was the wrong word, and so was `Console` — a log is a file, and this is what the module *did*: a clone, a pull, a build, a server, and the last publish. It used to be two places as well: pressing `Build` opened the preview panel to report on itself, so a build either pushed the site out of the way or the message was lost. Every message now goes to the page that owns the button (`say` / `complain` in `src/extension.ts`), which is why the area has two tabs: `Output` is this window's command lines, `Publish` is the publish page's log, which a person used to have to leave the page to read.

Every line is cleaned before it is drawn (`cleanLine`): ANSI colour codes stripped (an escape sequence arrives as `ΓÇö` in a console that is not a terminal), control characters dropped, the module's own `[site]` / `[git]` prefixes and the `$` prompt removed, and the glyphs a build or a dev server uses to decorate its output (`➜` , `✓` , box drawing) taken out — the words they decorate are kept. The raw lines stay in the output channel, where a terminal is the right place for them.

One port, one way of serving. `serve` listens on 1025 (`temcoLearn.sitePort`, above 1024 so it binds without root on macOS and Linux) and shows `dist/` as it was last built. There is no second mode: `client-dev` — Vite with hot reload — exists for editing the site's own React and Markdown, which is the repository's business and not something this page decides, so a terminal still runs it (`npm run client-dev -- --port 1025`). A page that serves what was built has one way to serve it, and a switch that changes nothing for the reader is a question nobody asked. A change to the port takes effect the next time the server starts, and the log says so.

Publishing is one page and one press (`src/publishPage.ts`, `temcoLearn.publishing`). The row reads `Publish the Site` — the count is already at its right-hand end as a badge, so the label says what is published and the number stays a number. The row opens the page rather than acting on the click, and the page has four sections, each with the 2 px brand bar:

- **The repository** — one card, not a band of four cells. A head line of **links** (the remote opens the repo on GitHub, the commit opens that commit, the branch sits plain between them), then `12 paths in this commit` and the first **five** of them, each a link to the file on GitHub. The rest fold into a native `<details>` — `Show 7 more paths` — so a publish that touches forty files reads as six lines and expands in place, with no script and no round-trip. The foot says whether anything is unpushed and offers `Open the clone` · `Terminal`.
- **What publishing does** — the four step cards, each ending in the act that performs it.
- **Where it goes** — one card for the one subject: `ADDRESS` as the page's biggest **link** (it opens the live site), then `Served from` (main, built by GitHub Actions, handed to Pages), `Last publish` and `Waiting` as label/value pairs.
- **The last attempt** — the lines git printed, unchanged.

The button that does it (`temcoLearn.publishNow`) writes its result where the decision was made, so there is no preview strip to read it in and no second page to be redirected to.

Artifacts has a page (`src/downloadPage.ts`, `temcoLearn.downloads`), opened by the Artifacts view's empty row and by its title bar. Three sections, each with the 2 px brand bar the other pages carry: **What a build writes** (five cards — the website, the help file, the pdf, the archive, the Word document), **Where it lands** (one card, because there is one working area: the folder written out as the card's own **link**, and two lines spelling out `<project>/dist` and the `HelpDocs` folder), and **The files on this machine** (a grid of **file cards** per project, packed **from the left** at the width the content asks for — `minmax(215px, 240px)` with `justify-content: start`, because letting them stretch put one file in a card 900px wide — each with the glyph for its kind, the name, `offline help · 1.1 MB`, and the two acts a file on disk has: `Show in the folder` for every card, and `Save a copy` where a copy can be taken). `Look` used to sit there and drew the viewer into the inventory; looking at a `.pdf` or the built site is the panel's job and the OS's, so the card finds the file instead and gets out of the way.

The page lists only what a build produces **as a whole**. A loose `.html` in `dist` is a page someone was iterating on, not an output: listing it beside `T3000.pdf` made the page read like a folder dump, so it is filtered out — and a project whose only result is such a fragment is not a project here at all, which is why the count in the title block counts the cards rather than the files on disk. The panel's Artifacts view draws the same line: it lists what can be taken away, never a fragment. **Publish** closes that view as `Publish the Site` — the count sits at the row's right-hand end as a badge, and the tooltip carries the last publish. Publishing is about every build at once, which is why it is the one act that sits with them.

Nothing is invented: a project with no `.chm` has no `.chm` row, and the panel says `Nothing built on this machine yet` only when nothing exists at all. Publishing state is read from git (`src/state.ts`) in the clone, so two machines never disagree about what is out.

Answers appear in the **preview's own strip**, under its bar, and never in a notification: adding a build, building, publishing, and every failure say what happened there, with at most one thing to do next. Only the *inputs* stay native — the version box, and the build picker when a row did not name its build, and the folder dialog for a clone that has not been chosen yet.

Settings → General is a bundled Fluent page (`webview/settings/`, React 19 + Fluent UI v9), not markup generated in the extension host: the tokens are Fluent's, the brand is Azure's blue, and every control is a real Fluent control. It is drawn in the house page language rather than a settings template — the brand wash with an eyebrow, a title and a note; one tinted callout; a section heading with the 2px Azure rule before it; an entry row per setting with its control at the right-hand end; a facts band; and links at the foot of the last block. `main.tsx` imports `temco-ui`'s `media/page.css`, so the wash, the rule, the row and the buttons are the same rules Temco Learn's Overview page is drawn by and the two pages cannot drift.

The host's half is narrow: `src/settingsPages.ts` builds the bundle when a source tree has none (a compile wipes `out/`), hands the webview `settings.js` and `settings.css` under a content-security policy that allows nothing else, and answers three messages — `ready` with one state, `setting` with a key to write, `command` to run. A refresh is a new state and never a new document, so a write does not throw away the reader's scroll. `npm run build-webview` builds it; `build-webviews` and `vscode:prepublish` run that too.

Design system

- **Fluent v9** (`@fluentui/react-components`, `@fluentui/react-icons`) with the brand ramp seeded from Azure blue `#0078D4` through `createLightTheme` / `createDarkTheme`, following VS Code's light, dark and high-contrast through `data-vscode-theme-kind`. The package named *Azure theme* (`@fluentui/azure-themes`) is v8/Fabric and does not apply.
- **Tokens**: spacing 4 / 8 / 12 / 16 / 24 · radius 4 / 6 / 10 · type 20 / 16 / 14 / 13 / 11 · row 22 (compact 20) · status colours taken from the VS Code chart colours, so a chip, a dot and the status bar cannot disagree.
- **Row anatomy**, one component reused everywhere: `[icon] title · meta · status dot · hover actions`.
- **Page anatomy**: the title block (one `h1`, one line of why, the ways in), then sections whose titles carry a 2px brand bar to their left, then at most **one** tinted callout, which is the thing a person would do next. Figures live beside the thing they are about — a stage of a pipeline, a row in a list — never in a strip of tiles whose only label is the number itself.
- **A list folds, it does not truncate**: the first five rows are shown and the rest are behind a native `<details><summary>…</summary>`, so the count in the heading stays true.

What was decided

question	decision
activity bar	five: Home · Docs · AI · Tools · Settings
left panel	Fluent header, search, chips and footer in a webview view; the Pages tree stays native
build	our own generator, so a project with only `.md` files builds; a project's own `docs:site` still wins
deploy	build in the editor, push to `gh-pages` with a plain (never forced) push; the one-time Pages switch via `gh` when available
export	site `.zip`, single-file HTML, Markdown bundle; `.docx` optional
order of work	generator → deploy → export → panel and shell

Order of work

1. **Generator** ✅ — `src/siteRender.ts` (pure: slugs, page shell, index, sitemap, one stylesheet), `src/site.ts` (host: read, render, write, copy media), `scripts/check-site.mts` (10 cases); `Build the Site` falls back to it.

2. **Deploy** ✅ — `src/pages.ts` (the remote decides the sitemap's address) and `src/git.ts` shared with the diff command. The push of a site lives in `temco-learn` (`src/publish.ts`: `ensureRepo` refuses a folder that is not a clone, then a plain, never-forced `git push origin HEAD:main`; `scripts/check-publish.mts`, 5 cases against real throwaway repositories); `temcoDocs.publishSite` stays behind as a redirect. Still owed: the status-bar state and the confirmation screen S12.

3. **Export** ✅ — `src/zip.ts` (a real archive writer: store, deflate, CRC32), `src/export.ts` (site zip, single-file page, Markdown bundle; `temcoDocs.export` + three commands), `scripts/check-export.mts` (5 cases that read the archive back). Still owed: `.docx`. Then the panel.

4. **Panel and shell** — *(in progress)* the activity bar is five entries ✅; the **Project** strip above the native Pages tree ✅ (`src/panelView.ts`, `check-panel.mts`); the **status bar** ✅ (`src/statusLine.ts`, `check-status.mts`); **S3** (the project dashboard) ✅ inside the Project Detail page: pages · drafts · stale · issues, a *Needs you* group, and *Last out of the building*; **S13** ✅ as the **Exports** page (`temcoDocs.export` opens it; the three shapes are still their own commands, plus `.docx` from `src/docxWrite.ts` — Word's format is a zip of XML, so it needs no dependency); **S12** ✅ folded into the Releases page as *Where It Publishes*; **S1** ✅ the Projects page opens itself when the window is on no project and has no tab; **S6** ✅ as *The Registry* page — every entry with the reason it might not appear in the site, and one action per reason; **S10** ✅ the generator reports its steps (Resolve → Render → Assets → Sitemap) to the notification and the output channel as they happen; **S11** ✅ publishing a release confirms first, naming version, destination and file count.

   Phase 4 is done, and so is every gap the screen audit found. What remains for the shell is the tabbed *Ship* surface (Build · Release · Deploy · Export as one page) — deliberately not built: after the cross-links and the Exports page, the four surfaces point at each other, so it is polish rather than a gap. The status line deliberately carries the window's facts (project · pages · issues) and not the version or the last build: those need the catalog, and a catalog read on every file save is too high a price for a status bar. The strip in the side bar shows them instead, from a render that already had a reason to read the catalog.

Overview is an application page (`src/overview.ts` , view type `temcoLearn.overview`), drawn with `temco-ui` the way every other page of ours is, and it is never published. It opens with the sentence that says what the module is for — *Temco Learn publishes the documentation website* — continued by the counts it reads from the snapshot (*3 projects are published today, 7 versions and 42 pages between them*) so no project name is ever written into the prose. Then, in this order: one callout for what needs doing (or that nothing does, or that nothing has been built yet); **Publishing workflow**, four cards — *Build · Add · Publish · Download* — one line each, ending in the link that performs it (`Open Temco Docs` , `Choose a build · 1 waiting` , `Publish · 3 changes` , `Show in the folder`); and **Where it stands**, three lines, one per page that acts on the facts they name. The section headings carry the 2 px brand bar the other pages use.

And it stops there — with one line back. The page used to go on with three parts — *Published projects*, *Build outputs* and *Publication* — that restated what the pages which act on those facts already say: the Site page holds the clone, the versions it serves and where it is published; the Artifacts page holds what a build wrote; the Publishing page holds the address, the branch and what a push would carry. A summary page that copies the page it summarises drifts from it, so the **lists** went. What came back is **Where it stands**, and it is deliberately one line per page rather than a section per subject:

line	the fact on it	it opens
`Documentation Site`	`2 projects · 3 versions · 4 pages`	the Site page
`Artifacts`	`3 files · 5.3 MB`	the Artifacts page
`Publishing`	`https://temcocontrols.github.io/docs/ · 3 changes waiting`	the Publishing page

A **count** is the one thing worth repeating: it is computed from the same snapshot on every draw, so it cannot drift, and it answers "is there anything in there?" without a list that has to be kept in step. Each line ends in the standard `Open ›` , so the detail is one click away and lives where it is edited.

Add a build is a section of the **Site page**, not a panel view: it belongs to the page that clones, builds, runs and publishes the site, because it writes into the same repository those do. `Builds not on the site` lists every build on disk whose project is absent from the catalog or whose `dist/site` is newer than the version the catalog lists, each with the reason (`not on the site` , `newer than 13.0`) and one `Add…` button. Adding asks for the version, copies into `public/imported/` , runs `npm run catalog` , and publishes nothing. Every command that means "add a build" opens that page with the section first and marked. Nothing about it is automatic: a test build stays out of the public site until somebody adds it. A link on this page is text with a glyph (`temcoLearn.*` command), never an `href` into the website — that boundary was crossed once already. The module therefore depends on `temcocontrols.temco-ui` , exactly as `temco-docs` does.

A build is not one thing. `temco-docs` writes the site (`dist/docs` or `dist/site`), an offline help file (`.chm` , inside the repository or in a `HelpDocs` folder beside it), often a `.pdf` , and the exports it keeps in `dist/` — a zip of the site, a Word document, single pages. `src/artifacts.ts` reads those three places per project and reports what is there: nothing is invented, so a project with no `.chm` simply has no `.chm` row, and a file in a shared `HelpDocs` folder belongs to the project it is named after. Each artifact is either something to look at (`site` , `chm` , `pdf` , a single page) or something to take away (`zip` , `docx` , and the files that are both), which is the divide the panel draws.

The panel (`src/tree.ts`) is three views in the order the questions are asked — **Overview** (one row: the page above), **Site** (`Preview the Site` with a state dot and the port, and **Library** with one row per project at its newest version), and **Artifacts** (a project, and the files its build wrote — names only, with the size and the path on hover; a row saves a copy and offers *Show in the Folder* inline; `Publish the Site` closes the view, its count at the row's right end). What

a row holds is its `description`: the grey words beside a name, as the Explorer writes them — and where a row deliberately says nothing, the hover carries it.

The authoring flow

> **Superseded for the side bar layout.** The Docs side bar (Pages, Needs You, Sources, Publish) is defined in redesign.md. This page remains the record for the screenshot capture stage only.

The design for **screenshots in the authoring loop**, and for the side bar that had to change because of them. This is the record the next round argues with, so that a redesign starts from a decision rather than from the previous screenshot.

The reference

The bar is **Dr.Explain**, and its loop is not "a documentation editor with an image button":

> capture a *running window* → its controls become numbered callouts → write one line of text per number → export the same topics to CHM, PDF and the web.

Two properties make that work: the screenshot is taken *of a window*, so the callouts can be placed by the machine instead of by hand; and every artifact is built from one set of topics. Before this change we had no capture stage at all, so the side bar had filled the space with destinations that were not places in the documentation — Projects, Areas, Pipeline, Working Set.

The flow

```text
T3000 is running — the thing being documented
    │  Ctrl+Alt+S  capture a screen          Ctrl+Alt+V  paste from the clipboard
    ▼
CAPTURE     the screen becomes a file
    ▼
TOPIC       the page being written; the file lands in that page's own media/
    │       NN-area-slug.png · ≥ 1600 px · ≤ 2 MB · type sniffed, never trusted from the name
    ▼
TEXT        the body, with the reference at the cursor
    ▼
GATE        registry · links · media audit · width · the version a page describes
    ▼
BUILD       the product repo's own scripts → the site, and the offline help
    ▼
NEW VERSION version-report.json → "retake these" rows → Ctrl+Alt+V again
```

Stages 1 and 3–5 exist and are untouched by this change. **Capture was the missing stage**, and it is now built as far as it can be without the callout decision below.

The rule that places every feature

The question a feature answers	Where it lives
"Where in the docs am I?"	the **Pages** tree
"What am I writing right now?"	the **page** and the editor's toolbar
"What is waiting for me?"	**Needs You**
"What am I shipping?"	one Build & Release page, reached from the tree's title bar
none of the above	it does not belong in the product

What changed

The tree gained a home of its own; the Documentation panel kept the groups it had.

Surface	What it is
Documentation panel (`temco`)	Projects · Pages · Areas · Pipeline · Working Set — as before
Documentation Tree (new activity-bar container, `temcoDocsTree`)	**Pages** — the documentation tree, the same provider as the panel's Pages view
	Needs You — the queue below it: screenshots with no page yet, pages the code moved away from, pages never reviewed

One provider behind both tree views, so there is still one answer to "what is the documentation". The menus on the panel's Pages view apply to the tree view as well.

Needs You lists only facts something else already computes, and every row runs a command that already existed:

- screenshots with no page yet, from the capture inbox;
- pages the product repo marks `stale` (read, never recomputed);
- pages never reviewed, as one row with a count and the report behind it.

Screenshots are a stage. `temcoDocs.captureImage` (Ctrl+Alt+S) opens the Windows snip overlay and takes what it puts on the clipboard; `temcoDocs.pasteImage` (Ctrl+Alt+V) takes an image that is already there. Both then:

1. find the page in front — the active editor's page, else the row the tree has selected;
2. write into that page's `media/` as `NN-slug.ext` , the number continuing the folder's own sequence;
3. insert `![title](media/NN-slug.ext)` at the cursor, or at the end of the page when it was not open;
4. warn — not refuse — below 1600 px wide, and refuse outright above 2 MB.

With no page in front the screen is not guessed at: it goes to `<root>/.temco/captures/` and waits in **Needs You** until *Attach the Screenshot to a Page…* puts it where a person says. The inbox is outside every `media/` , so the site only ever holds what a page shows.

`temcoDocs.chooseImage` is the third path, for a file that is already on disk; it is the one that also works outside Windows.

Why reading the clipboard needed no change to the core

The extension API reads the clipboard's **text** and not its image — `docs/temco-platform/ui/editor-design.md` §3 said so, and it is still true. But this application is Windows, and `System.Windows.Forms.Clipboard` reads the image with no native module and nothing installed: what Win+Shift+S, PrintScreen, Snipping Tool and every other capture tool leave behind. So the limit is a platform fact rather than a wall, and it costs one PowerShell child process per capture.

What is not built, and the decision it waits for

Annotation. Crop, arrow, box, text and blur/redact, saved over the capture, with the untouched original kept aside so a retake is diffable. It is a contained piece (a canvas over the captured image), but it contradicts `15-online-editor.md`, *What is not built* — "a screenshot is captured properly or not at all" — so it is a rule change that needs to be taken deliberately, not slipped in with a button.

Automatic callouts, the Dr.Explain core. It depends entirely on what is captured:

What the screen is of	What can be placed automatically
our own webview T3000	callouts from the real DOM elements — this is Dr.Explain's magic, nearly free
the classic Win32 T3000	control bounds from Windows UI Automation — automatic, but coarse
a device screen, or a customer's machine	nothing; manual crop, arrow, box, blur is the whole feature

The answer decides whether the annotator is a callout tool or a drawing tool, which is why it comes before the annotator.

The version report. `docs/help/version-report.json` — version, changed areas, affected pages, images older than the version they describe — written by the release workflow, read by **Needs You** and by Build & Release, consumed by the MCP server. Designed, not built; it needs no UI of its own.

Order

1. **The two views** — done.
2. **Capture, paste and the inbox** — done.
3. **The callout decision**, then annotation in the page.
4. **Build & Release** as one page: the build, review and release commands that exist today, in one place.
5. **`version-report.json`**, then the Action and the MCP wrapper.

The editor

The design for **R6 — editing and publishing**: the toolbar, creating a document, images, drafts, and where the review happens. It is written the same week its first step was built, so it separates what exists from what is proposed, and it puts the one decision that is not mine in front of the person who has to make it.

The content rules are not here. Registry schema, link forms, media rules, review flow and publishing belong to `design/temco-learn/`, which stays the source of truth; this document only says how the desktop application implements them.

---

1. The surface: VS Code's editor, with the project's toolbar on it

`15-online-editor.md` designs the **hosted editor**: Monaco, bundled and self-hosted, with an API behind it, served by the center tool on a host that has a clone. That editor is not this application's editor and building it twice would be the bug.

	the hosted editor	this application
the core	Monaco, bundled by us, ~2 MB, custom build, `<textarea>` fallback	**VS Code is Monaco plus a shell** — nothing to bundle
what it adds	toolbar, preview pane, gate markers, upload, publish, PR	toolbar, a link to a real page, the outline, the gates' markers — of which the markers already exist (`src/diagnostics.ts`)
what it never does	never WYSIWYG: a rich round-trip rewrites the line layout of the whole file	the same rule, and the same reason
publishing	`POST /api/docs/publish` → a real PR	**never.** This application builds and reports; shipping stays a separate act

So the desktop equivalent of "Monaco + the toolbar + completion + markers" is **VS Code, with the toolbar built** — which is what `src/format.ts` now is. An editor page of the project's own, hosting Monaco in a webview, was considered and is refused for the reason `15-online-editor.md`, *What is not built* gives for the website: the integration is the part that matters, and the editor is the part that already exists.

What exists — the toolbar, built

Sixteen commands, all `temcoDocs.format.*`, in `src/format.ts`. Each one edits the buffer in front: the page a person is typing in, undone with `Ctrl+Z`. None of them writes a file, opens a tab, or touches the registry.

Where a person meets it	What is there
the editor's title bar, Markdown files only	**Bold** · *Italic* · *Link to a page* · **Format**
the editor's context menu, and the tab's	the submenu **Temco Docs** — all sixteen, grouped
the palette	every action, by name; **Format** lists them all in one picker

Group	Actions
Inline	Bold `**…**` · Italic `*…*` · Inline code `` `…` ``
Blocks	Heading level (`##` – `####`) · bullet · numbered · task · block quote · code block with a language · table · horizontal rule · callout
Insert	Link · **Link to a page**
Navigate	Go to a heading (this page's outline)

Three things it does on purpose, each of which has an obvious wrong answer:

- **Every button is a toggle.** Bold over `**bold**` removes the marks; the bullet list over a list takes the bullets off; the same heading level again removes it. That is what a button does in every other editor.
- **A block gets a blank line before it when the line above is prose.** Not decoration: `text` followed by `---` is a level-2 heading in Markdown, and a table glued to a paragraph is a paragraph.
- **A callout, a fence and an alert marker start their own line** — mid-line they would glue to the text after the cursor, and `> [!NOTE]` is an alert only at the start of a line.

No keybindings are contributed, deliberately. `Ctrl+B` hides the side bar everywhere else, and this extension has to keep working in an ordinary VS Code. Everything here is a command, so a chord is one personal binding away.

A page that is not a file on disk says so. A page read from a source, or a diff, has a scheme that is not `file`, and every action refuses it in words rather than appearing to work.

---

2. Creating a document

A page is a file, not a pair — and that is the product repository's own doing. This section said the opposite when it was first written, so the correction is worth stating plainly, with where it comes from. `T3000Webview-EEZ-Studio/scripts/docs-registry.mjs` **generates the entries**:

```
TEXT

buildPages(existing)                    walks docs/t3000/** and docs/legacy/**
  id      = the docs-relative path without `.md`      the page id *is* the path
  title   = prev.title  ??  the file's first `#`      ← so a skeleton starts with one
  area    = the top folder      chapter = the second path segment (or `(root)`)
  order   = prev.order  ??  10, 20, 30… inside the chapter
  owner   = 'docs'   ship = true for t3000   tags/keywords/code = []
  every field:  prev.<field> ?? a default             ← an existing (curated) value always wins
and the file itself is written with  JSON.stringify(registry, null, 2) + '\n'
```

So **creating a page writes no entry at all**: the file goes in, the repository's own generator — which this application already runs as one of its allowlisted scripts — adds the entry, derives its place, and takes its title from the first heading. What only a person curates is `order`, `chapter`, `title`, `owner`, `tags` and `code`, and those are the values the generator then leaves alone. Section 5 is that decision, as taken.

Two consequences for the flow below, both now built: a new page is **a file and a generator run**, offered in that order; and there is **no "loose page" kind**, because the `exclude` key the design package's schema describes is not produced by this repository's generator — a Markdown file under `docs/` that no area covers is an `unregistered page` error, not a working note.

The kinds worth offering

Kind	The headings it starts with	Why it is a kind
Article	Overview · Configuration · Related pages	the ordinary case
Task	Before you start · Steps · If it does not work	the commonest documentation shape
Reference	Fields · Notes	the shape an API or a register actually has
Troubleshooting	Symptom · Cause · Fix	matches the `troubleshooting` area the product already has
Release note	What is new · Fixed · Known issues	what changed in a build

Each skeleton is **headings only** — no invented sentences, because a template that writes prose is a template someone has to delete. Every one starts with `# <title>`, which is what the generator reads.

The flow, as built

TEXT

```
New page ──────────────────────────────────────────────────────────────────
1  Where    the row it was started from — a page's own area and chapter, a chapter itself,
            an area's root chapter — or the row you have selected, or:
            which source ▸ which area ▸ which chapter
2  Kind     article · task · reference · troubleshooting · release note
3  Title    Trend Logs
4  File     trend-logs          →  docs/t3000/features/trend-logs.md        (shown, not guessed)
──────────────────────────────────────────────────────────────────────────

[ Write the page ]  ── it starts as “# Trend Logs” and the kind's headings
── then ──
“It becomes a page when the registry names it.”          [ Run the generator ]
```

A repository row cannot answer step 1 — it is not in an area — so it answers the source and the rest is still asked. Nothing else is asked when the kind came from the *New Page of a Kind* submenu: step 2 is the entry that was clicked, and steps 3 and 4 follow as they always do.

New chapter is the same flow with the folder named first, because a chapter is a page's *path*: a folder with nothing in it is not a chapter, not in the registry, and not in the table of contents. The folder arrives with its first page.

---

3. Images

An image is the one action in the toolbar that is missing, and the reason is not the code: the file has to be written into the page's own `media/` folder before `![alt](media/…)` points at anything.

The rules are already written down, in the design package, so they are not ours to invent:

Rule	Source
the file goes into the **page's own** `media/`, with a slugified name (`02-run-screen.png`)	`15-online-editor.md`, *Uploads*
accepted: PNG, JPG, WebP, GIF, SVG — diagrams as SVG, sanitised on the way in	`15-online-editor.md`, *Uploads*
images ≤ 2 MB · a page ≤ 1 MB · content-type sniffed, never trusted from the name	`15-online-editor.md`, *Safety rules*, rule 2
the destination is derived, never supplied	`15-online-editor.md`, *Safety rules*, rule 1
screenshots are captured properly or not at all — no editing, no resizing	`15-online-editor.md`, *What is not built*

What an extension can and cannot take. A file the user **picked** (`showOpenDialog`), a file **dropped** on the page and an image already on the clipboard are all reachable — the last one not through the extension API, which exposes the clipboard's *text* only. On Windows the module reads the bitmap itself (`System.Windows.Forms.Clipboard`, one child process per capture), which is what makes `Ctrl+Alt+V` work; the loop it belongs to, and what is still missing from it, is `authoring-flow.md`.

---

4. Drafts, review and publishing

Not built, and the largest piece of R6. The design is already fixed and is not ours to change:

	where it belongs
editing a **local** source	in place — the page file is the page, which the Workspace view already does
editing a **git** source	on a draft branch in a materialised clone — which is the `fetch` read mode R2 still owes
review	`docs:blocking`, `registry --check`, the link check and the media check, run by the repository's own scripts
publishing	never from this application: the gates, the commit and the PR are the product repository's
a page's fields in the editor	restricted by `15-online-editor.md`, *Safety rules*, rule 4: structure needs the docs owner

So the honest order is: **images and create-a-page first, because both need only a decision**; drafts after, because they need the `fetch` mode and a review flow that lives in the product repository.

---

5. The decision, as taken

The question was how far this application may go into a product repository. The answer, in the three parts it was asked in:

Asked	Taken	So the code does this
The Library's title bar holds twelve buttons	**three inline, the rest in the view's `…`**	`newProject` · `openProjects` · `openNeeds` stay in group `navigation`; everything else moved to named groups, which is how a `view/title` item lands in the overflow menu. **The header starts work; a row and a page operate on a thing** — and the target-less ones ask which, which they already did
Which operations the Workspace tree grows	**all five**: new page · new chapter · rename the title · move (up, down, and by dragging) · retire	`src/pageOps.ts` — six commands, a `TreeDragAndDropController`, and one modal per write
How far into `registry.json`	**one named value, on a confirmation**	`src/registryEdit.ts` — `title`, `order`, `chapter`, `deprecated`, `superseded_by`, and nothing else; never a whole-file rewrite

Where a row offers its operations was settled on the third attempt: **the row menu is the Explorer's, the toolbar mirrors it, and the hover carries nothing.** A **left click** does what it does in the Explorer — opens a page, expands a folder — and a **right click** (or the keyboard's menu key) opens *New Page…* · *New Chapter…* · *New Page of a Kind* ▸ · *Reveal in Explorer* · *Open in Integrated Terminal* · *Copy Path* · *Copy Relative Path* · *Rename the Title…* · *Move the Page Up* · *Move the Page Down* · *Retire…* · *Open the Registry*, each item only where it means something. The view's `…` shows the same list — plus *New page*, *New chapter*, *Refresh*, Home and Validate — acting on the row the tree last had focused, which is also what makes it usable from the palette: **a view title action is run with the tree pane's context**, and every row command falls back to the view's own selection. There are no hover icons on a row: a menu started from a row can only ever be a quick pick in the middle of the top of the window, and that is exactly what the row menu above replaces.

What each operation writes

TEXT

```
operation                          what changes on disk                                state
─────────────────────────────────  ──────────────────────────────────────────────────  ──────────────
Open · Reveal · the outline        nothing                                             built
Create the missing page file       one empty .md (declared, absent)                    built (earlier)
New page in this chapter           one .md from a kind's skeleton                      built
  → then the repo's docs:registry  (its own script; adds the entry)                    offered, not forced
New chapter                        …the same, with the folder named first              built
Rename the title                   title in registry.json (+ the page's first #)       built
Move up / Move down / drag         order in registry.json — two numbers, swapped       built
Drop on another chapter            chapter + order — *structure*, docs owner           built
Retire a page                      deprecated + superseded_by                          built
Rename or move the FILE            the page id — inbound links break                   **never**, by rule
An image into the page's media/    a new file, binary                                  **not built** — see
```

Four rules hold the writes together, and each one is a decision rather than an implementation detail:

- **A move is a swap, never a renumbering.** Placing a page between two others by rewriting every `order` in the chapter is the generator's own job — it hands out 10, 20, 30… to whatever nobody curated. So a move writes two numbers and the diff is two lines.
- **The file is never renamed or moved.** The page id *is* the path (`05-content-and-registry.md`, *Page ids and aliases*), so a page is renamed by changing what it is *called*: one string in the registry, plus the page's own `#` heading when it says the same words — two answers to one question otherwise.
- **A chapter change says the word structure.** `15-online-editor.md`, *Safety rules*, rule 4 marks `chapter` as the docs owner's to change, so the confirmation says so and asks for a line in the pull request.
- **Retirement is not deletion.** The file stays; the registry records the replacement; the redirect for a retired page is generated by the repository's own pipeline (`14-governance.md`, *Deprecation and retirement*).

The evidence for "one value, and nothing else"

`src/registryEdit.ts` renders the file the way the repository's **own generator** writes it — `JSON.stringify` with two spaces, one trailing newline — **and with the file's own line endings**, because a clone checked out on Windows is CRLF while the generator emits LF, and a write that flipped every ending would be a diff of the whole file. With both right, the guarantee is testable, so it was tested:

Check	Result
the real `docs/help/registry.json` — 318 pages, 124 KB — opened, re-rendered, compared	**byte-for-byte identical**
a drag into another chapter, diffed line by line against the original	**two lines changed**: `chapter` and `order`
a drag onto a sibling, same chapter	two lines changed, the two `order` values swapped
a drop on the page's own chapter	**nothing written**, and it says so
a rename, diffed	one line changed, plus the page's own heading
a new page	the file written, **the registry untouched**, then the generator offered

Running the generator afterwards is what keeps the file canonical: the values written here are the ones the generator's own `prev.<field> ?? default` rule preserves, so nothing is lost by it and nothing here duplicates it.

---

6. Acceptance, per step

#	Step	The test that ends it
1	the toolbar	**done** — a Markdown file shows the four buttons; every button toggles; `---` after prose is not a heading; the page picker inserts a link the gate resolves; a page read from a source is refused in words
2	operating on a page	**done** — a page is created from its kind's skeleton and the generator is offered next; a title is renamed and the registry diff is one line; a page is dragged into another chapter and the diff is the two values; retiring one writes `deprecated` and the replacement and keeps the file
3	insert an image	a dropped PNG lands in the page's `media/` under a slugified name and is referenced relatively; a 3 MB PNG and a `.exe` renamed `.png` are both refused, in words, with the limit
4	drafts	a page is corrected, the gates run, the diff is reviewed, and the result is a commit a person made — not one this application made

The platform

The center documentation platform. One internal application — VS Code plus this extension — that takes documentation from **many kinds of source**, lets people manage and edit it, and **builds the final documents**: a site, a CHM, a PDF. A local AI reads the history of those sources and drafts the updates. Everything it knows lives in a local **SQLite catalog**.

This document is the platform's own design. The **content** rules — the registry schema, the checks, the review flow, the publishing engine — are the design package in `design/temco-learn/`, which stays the source of truth.

How Microsoft's own documentation is managed, for comparison

Worth stating, because it is the shape being copied:

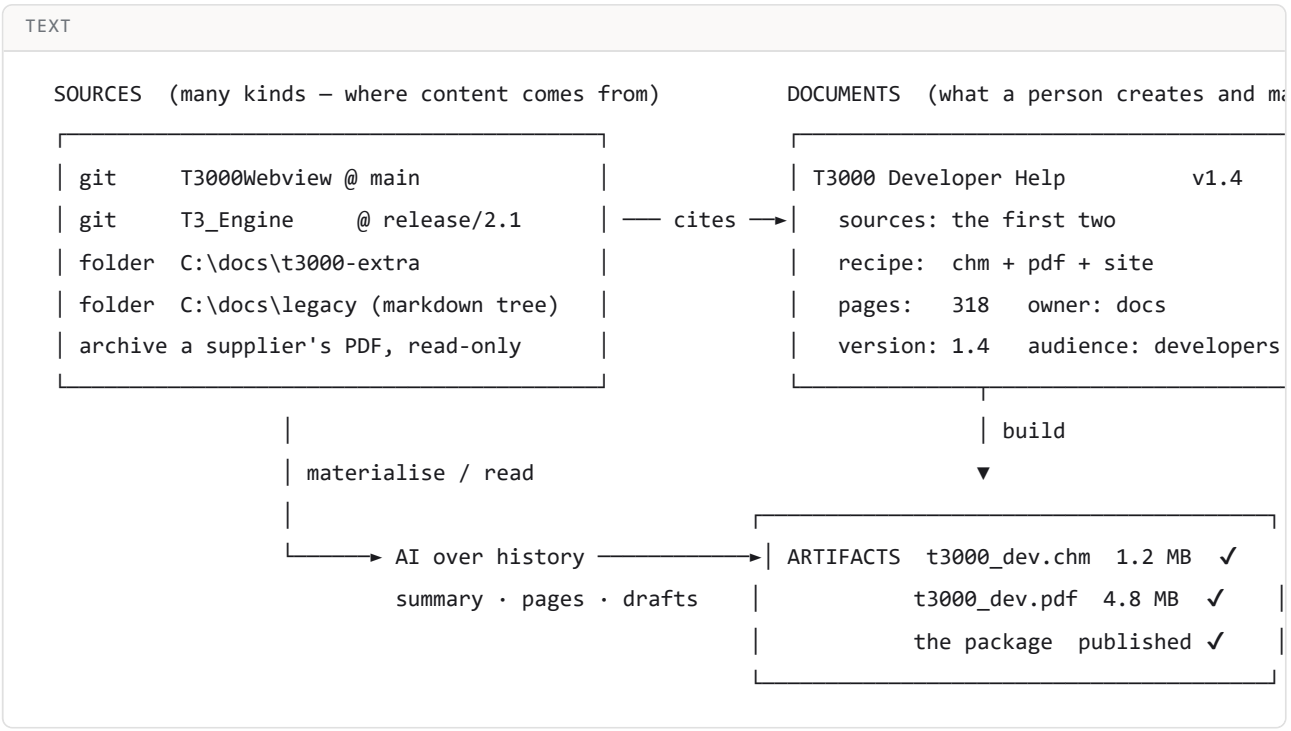
- **A docset** is a set of Markdown pages plus a table of contents, living in a repository (Microsoft's own docs are a private repo of Markdown).
- **Metadata** on each page — `author`, `ms.topic`, `ms.date` — decides where it appears and who owns it.
- **Monikers** are version markers: one page can serve several product versions, and the build emits the right one per version.
- **A build pipeline** turns the docset into the published site, and separately into an offline PDF.
- **The reader** gets one searchable site; the *authoring* side (review, status, ownership) is internal.

The equivalents: docset ≈ a **Document**, moniker ≈ its **version**, pipeline ≈ its **recipe**. The difference is where it runs: theirs is a hosted service, ours is an internal app on the maintainer's machine — and ours must also produce the **CHM**, which theirs does not.

The model: sources → documents → artifacts

The unit of work is **not a repository**. A repository is where content may *start*; the thing a person creates, owns, versions and builds is a **document** — "T3000 Developer Help", shipped as `t3000_dev.chm`; "T3000Webview Guide", shipped as `t3000_webview.pdf`.

TEXT



Source — where content comes from. Not necessarily GitHub, not necessarily a repository, not necessarily Markdown:

Kind	Examples	What the platform does
`git`	any remote: GitHub, GitLab, a bare path — `owner/repo@branch`	reads the tree, the history and the diffs
`folder`	a docfx project, a Markdown tree, loose HTML, images	reads it in place; never moves it
`archive`	a supplier's PDF, a vendor's ZIP of pages	inventories it, read-only, for reference and search

The **branch is chosen per source**, defaulting to the repository's own default (`main`), and reported when the documentation is not on it.

Document — what a person creates: a name, its sources and their branches, a **recipe** (how to build it), a **version**, an audience, an owner, and the outputs it must produce. This is the platform's unit of management, and it is what the generator, the builds, the health and the AI are all *about*.

Artifact — a file a build produced: path, size, time, hash, and which document and source revision it came from.

Build — one run of a document's recipe at a source revision: a job with a log, a status, and the artifacts it left behind. Success is an artifact that did not exist before, never an exit code.

Analysis — what the local AI did over the history: a summary of a commit, the pages it affects, a draft, and what is still owed.

The catalog

The platform's memory: documents, sources, builds, artifacts, commits and what changed since you last looked. Relational, long-lived, queryable offline.

Where: `context.globalStorageUri` of the **store module** — a `catalog.db` and a `sources/` cache of materialised trees. One window, one file, opened by the module that owns it: a caller that opened it with its own storage would find an empty database beside its own code. **Never inside a repository**, never committed, never in a product's git history.

```SQL
sources     id · kind(git|folder|archive) · name · remote_url · path · default_branch · branch
            last_fetch_at · last_commit_sha · last_commit_at · size_bytes · state

documents   id · name · slug · version · audience · owner · recipe(kind) · created_at · updated_at

doc_sources document_id · source_id · include_glob · branch · role(primary|supplement)

recipes     document_id · engine(docfx|own-script|chm|pdf|site) · config_path · script · outputs_js

builds      id · document_id · started_at · finished_at · status · revision · log_path

artifacts   id · build_id · document_id · path · bytes · built_at · sha256 · kind(chm|pdf|site|zip)

commits     sha · source_id · at · author · subject · summary · summary_model · summarised_at

commit_pages  commit_sha · page_id · how(code-glob|path|terms)

pages       id · document_id · source_id · path · title · owner · reviewed_at · stale · counts_json

scans       id · source_id · at · mode(local|fetch|api) · revision · counts_json

events      id · at · kind · subject_kind · subject_id · detail · seen_at
            kind ∈ { new_source, docs_found, registry_found, push, pages_changed,
                     docs_removed, branch_mismatch, build_ok, build_failed, draft_ready }
```

Every view is a query over these rows. Nothing is asked of the network twice, and the last scan is readable with the network unplugged — and says when it was taken.

Reading a source: three ways, and when to use which

The user's requirement — "*load the source code… download to a local temp folder or read it from online*" — is two mechanisms with different costs, and the platform picks per job:

Mode	How	Used for	Cost
local	a folder you have open, or a path in the catalog	everything: files, checks, edits, builds	none
materialised	a shallow fetch into `globalStorage/sources/<slug>` — `git clone --filter=blob:none --depth 1`, or a fetch of the needed range — on a scratch branch the platform owns	drafts (the model needs the page text), builds, file-existence checks	disk, bounded
online	the host's API for one commit's changed files and patches	commit summaries, "what changed" — *diffs* only, no clone	API calls, bounded

The rule that picks between the last two: **summaries need diffs; drafts need files.** A commit summary over an API patch is cheap and needs no clone; a draft needs the real page, so the platform materialises the source once and reuses it. A patch can be truncated for a very large file — the summary records that it was, rather than guessing.

Caps, because this is a maintainer's PC: a source cache budget (default 2 GB) with LRU eviction, a maximum fetch depth, a maximum commits analysed per run, and a *Prune the source cache* command. Nothing is ever fetched without a command or a queue the user started.

The AI over history, per source

TEXT

```
commits since the last scan   ──►  summarise what it did, in the product's words
    (API patches, or the local clone)      │
                                           ├──► which pages it affects
                                           │      code globs from the registry, else path, else t
                                           ├──► draft the page edits  (local model, draft-only)
                                           └──► add to the owed list
▲  nothing here publishes, commits or approves: a named person does that
```

Summaries, their model and their time are stored, so a summary is computed once and is visible offline. The whole pass is a **job with progress** — resumable, bounded per run, and honest about what it skipped.

Creating a document

Create a document asks for the few facts the catalog needs, and nothing more:

```
TEXT
```

```
name        T3000 Developer Help
sources     [x] T3000Webview  branch main        [x] C:\docs\t3000-extra
            [ ] T3_Engine     branch release/2.1
recipe      ( ) site   ( ) pdf   (x) chm   [x] also the site
version     1.4            audience  developers            owner  docs
─────────────────────────────────────────────────────────────────────────

the platform answers with what it can see:  318 pages · 5 sources · 2 sources carry a registry
                                             suggested: run the T3000Webview generator first
```

A document can be created before its sources are perfect — that is the point. The platform records what it found, says what is missing, and offers the next step.

Editing and writing

- **A local source is edited in place** — the page file is the page, as the Content view already does.
- **A git source is edited on a draft branch** in the materialised clone: the AI's diff is applied there, the user reviews it, and *they* commit or open a PR. The platform never commits on anyone's behalf.
- **Nothing under `_admin/` or `_edit/` is ever written** — `00-architecture.md` calls those *generated pages, not documentation.*
- The hosted editor stays the design package's web surface; this is its desktop equivalent.

Telling the user — quietly

Unseen events become **one** grouped notification, never one per event, and `seen_at` is stamped when the group is shown or dismissed so a fact is never announced twice. Default `docs-only`: a daily summary of documentation change, no modals. A *Since you last looked* node heads the view.

Feasibility

Yes. It is a catalog, a few connectors, a build runner and the AI that already exists — no new engine, and no server. The real constraints, each with its answer:

Constraint	Answer
The database — decided `node:sqlite`: it ships with the extension host's Node, so there is no dependency to add	a build whose Node predates 22.5 is refused in words by the store, at the point of use. The `engines.vscode` floor should be raised once the first F5 confirms the built-in is present
Disk — cached source trees grow	a budget (2 GB default), LRU eviction, *Prune the source cache*, and a `.gitignore`-aware size report
The network — a whole-organisation read and commit patches need rate limit	a token in secret storage (5,000 calls/hour), and *online* mode on demand only. The 60-call limit has been reached in practice; it is not an optional convenience
Time — AI summaries over history are slow	a job queue with progress, a per-run cap, resumable, and summaries stored once
Many kinds of local documentation	one ingester per kind, starting with: a Markdown tree, a docfx project, and an inventory-only reader for HTML and PDF
Trust — this is the maintainer's machine	all state in `globalStorageUri`; nothing written into a repository; nothing fetched without a command or a queue the user started

Surfaces

The platform has **two lists** in the sidebar and **rich pages** in the editor area. Nothing else competes for attention: a document is managed from its page, a source is read from its row, and the AI is beside whatever you are working on.

What opens when the platform is activated is the project you are on — its dashboard, the one surface that says what is in it and what next — then the page you were last working on, then the home page when there is neither. `temcoWorkbench.start.open` (`project` · `page` · `home` · `welcome` · `none`) chooses, and `none` opens no tab at all for anyone who would rather nothing opened by itself. The side panel the window is revealed on is a second, separate choice — `temcoWorkbench.start.panel`, whose values are the left panel's own items (Home, Documentation, Repositories, AI, Forum, Tools, Settings) and whose default is `temcoDashboard`.

The Workspace is the first view, and the Library is beneath it. Both sit in the `temco` activity bar: the Workspace tree is the documentation module's, the Library is the workbench's — the one index of everything every module has. The Workspace shows the project being worked on and never the open folders, so the registry checks, the status bar and the faults follow the project too — a tree built from whatever happens to be open would be a second, unrelated answer to the same question. The view is never hidden and never blank: with nothing to show it carries **rows that say why and offer the one act that fills it** (*no project yet* → create one; *no project is being worked on* → choose one in the Library; *this project has nothing to show* → open it and read its sources). Those rows are drawn by the tree itself — `viewsWelcome` was tried first and dropped, because a welcome's `when` clause is resolved when the view is first painted, which is before the refresh has read anything: it announced "nothing recorded yet" over a catalog that held five projects.

With exactly one project recorded there is nothing to choose between, so it selects itself and the tree is ready without anyone picking it; that is not a person choosing, so the sidebar is left where it was. Choosing a project afterwards brings the Workspace forward, and clearing one brings the Library back — the two views are a sequence, and only a click walks it.

Nothing here is specific to a product. The names in the figures below are examples; the code knows no repository, no branch and no product name, and it reads whatever the catalog and the editor are told about.

```
SIDEBAR (activity bar — the two lists)      EDITOR AREA (the pages)          RIGHT SIDE
┌──────────────┐                            ┌──────────────────────┐         ┌──────────────────────
│ DOCUMENTS    │  what you manage           │ Document             │         │ TEMCO AI
│ SOURCES      │  what you read from        │ Create a document    │         │  the AI, beside
└──────────────┘                            │ AI analysis (job)    │         │  what you work on
                                            │ Build log            │         └──────────────────────
                                            └──────────────────────┘
```

The project flow

The flow was reworked after the first attempt at it was judged — correctly — not good: a project's pages existed in two places with two truths, a source could not be acted on from the row that named it, and *Add a source* threw the whole edit form at you. What changed, in the order it matters:

```
selected project ──────────► the Workspace view, the counts, the faults, the AI
      │                       (one world: the view follows the project, not the open folder)
      ├── Library row  ⚙ Read · Open the source · Set the branch · Forget     (actions on the row
      ├── hub          Read every source · Add a source · Build · Edit        (the work, on one pa
      └── hub ▸ Sources  the add form lives here — a source is one decision, not the whole project
```

#	What it was	What it is
A	the Workspace view showed the open folders; a project's pages lived in the catalog, beside it	**one world**: with a project being worked on, the view shows *that project's* sources — a local folder gets the full checks even when the window has it closed, and a source that is not here shows what the read recorded, marked `origin: 'catalog'` so nothing unchecked looks checked. With no project it says why instead of showing the folders
B	a Library row was text; reading was only on the view's title bar	rows carry a context value — **Read** · **Open the source** · **Set the branch** · **Forget** on a source, **Work on** · **Edit** · **Add a source** · **Build** on a project
C	*Read the sources* read **one** source and said "sources"	**Read every source** reads them all, one at a time (a rate-limited host is not asked twice at once), with one progress notice and one report
D	*Add a source* opened the whole edit form	the add form is on the hub's **Sources** tab: repository + branch, or a folder, or an archive
E	a build ran only the output script	a build runs the repository's own `docs:registry` first when it has one — that is what writes the index, the headings and the stale verdict — then the output script, and says which of the two it ran
F	the AI answered about the open folder only, and said nothing about what it was answering	the panel is **about the project**: it names it, and when no editor is in front it falls back to the page the project remembers you were on

The Library's *Projects* row (`extensions/temco-workbench/src/views.ts`) is the list of what this application manages. Each project's row carries what a project is made of and what can be done with it — the four technical facts are on the row and its children, so nothing has to be opened to decide: version · owner · outputs; its sources with how many pages each holds and how it was read; how many pages, how many without an owner, how many artifacts and when the last one was built; and the operations — *Work on it* · *Open* · *Read sources* · *Build* · *Edit* · *Remove*…. The one being worked on is first and marked, because the panel is read top-down.

A page for a project that has been removed closes itself, and the projects page takes its place. The first version kept the tab and put a notice in it — which was wrong twice over. It cleared `document.body` , taking the tabs, the actions and every element the next render writes into with it, so the tab could never become another project's page; and it left a page *about nothing*, with working-looking buttons on it. A tab for something that no longer exists is worse than no tab: the removal says so in one message, closes that page, and opens the list of what is left. Removal from any surface does this, and a page opened for a project that is not there does the same thing rather than rendering.

There is no *Work* view. It was a second copy of the tree's *Gaps* list plus a counter, in a third place, behind one icon that meant five things at once — decided with the user. What replaced it:

- the sidebar's **Library** holds one expandable **Needs attention** row: a count line, the missing-file row, a stale row, an unowned row, then the faulty pages, then **More…**
- **More…** (or the row's title, or the row's title-bar button) opens **What needs doing** — a page with the room to say *why* each thing is listed and *where* the fix is: sources · faults · declared-but-missing · stale · no owner · owed. A missing file is created from there with one click, which is the one write this application is allowed

- each mark means **one** thing: `⊗` a structural fault, `!` a declared page whose file is absent, `↺` the generator's stale verdict, `▲` an unowned page, `⇄` owed
- the **staleness report** moved to the Library's title bar, where the view it belonged to used to be

Owed is never computed here. *Documentation owed* is the product repository's own gate — `scripts/docs-owed.mjs`, `npm run docs:owed`, waivers that expire (`11-review-and-cicd.md`) — so the page runs that script as a task and reports what a run is: where, when, the exit code. Its output, in the terminal, is the verdict, and the page says so. A second opinion about what is owed is the one thing a documentation platform must not invent.

The selection is state, in one place (`extensions/temco-store/src/selection.ts`, in `workspaceState`): which project, and which page of it you were last on. A project belongs to the window, not the machine — two windows on two products must not answer about each other's pages. Opening a project makes it the one being worked on, and the library row, the view's description and the dashboard all say so; a project that is removed while selected clears the selection rather than leaving a view about something that is not there.

On activation, nothing has to be clicked. The one project with nothing chosen selects itself (with two or more nothing is guessed — see *Surfaces*); the Workspace view arrives with the only repository's areas open and their chapters beneath them; the Library opens *Projects*, so the names of what you have are on screen; and the tab that opens is the welcome page (`temcoWorkbench.start.open`, default `welcome`), or — once that checkbox is off — the selected project's pages, then the page you were last on, then the overview. A tree that re-decided itself after every refresh would be worse than one that never opened at all, so a person's own expand or collapse is remembered for the session and beats both defaults — a refresh re-reads the data, never the layout (`extensions/temco-workbench/src/views.ts`).

The window

As built$2 The figures below are the design; what exists is these same surfaces with different names, and the difference is recorded here rather than left to be discovered:

- The sidebar holds **three** sections in one container, not two: **Workspace** (the page tree, the first thing built), **Work** (what needs doing — faults per open folder, and changes not yet shown), **Library** (the projects, the sources with their pages beneath them, and the catalog's own state).
- The unit is called a **project** in everything a person reads. The catalog's table is still `documents`; a project is what that row is, and renaming the table buys nothing but a migration.
- **Four pages, one job each.** The **hub** (`extensions/temco-projects/src/projectHub.ts`) is what one project *is* — Overview · Pages · Sources · AI · Builds, with Read and Build at the top; the list of projects is the Library's *Projects* row, not a page of its own. The **edit page** (`extensions/temco-projects/src/projectPage.ts`) is what it is *made of*. Opening the editor for every look at a project would make reading its pages and editing its fields the same act, so they are separate commands (`openProject` and `editProject`). The **dashboard** (`extensions/temco-workbench/src/home.ts`) is the application itself, and the **AI page** (`extensions/temco-ai/src/aiPage.ts`) is the AI. All four are tabs in one group.
- **The project's pages are tabs in one group**, and so are the documentation pages they open — never a split. The rule lives in `extensions/temco-ui/src/columns.ts` and is stated there once, together with the two wrong answers that led to it: a page opened `Beside` produced a second column a few hundred pixels wide, and "is the tab in front one of ours?" — tried twice — is not a question a webview tab answers reliably. So the file asks nothing about what is in front; it finds the group that already holds a page of ours and opens there, which makes one group the only outcome. `reveal` with that column also *moves* a page a previous layout left in a second

group, so the split heals the next time the page is opened. `ViewColumn` is named in exactly one place in the whole source.

- **The layout survives a narrow column.** Every grid is `minmax(min(Nem, 100%), 1fr)`, every flex child that holds text is `min-width: 0`, and a table wider than the pane scrolls inside its own box. A bare `minmax(Nem, 1fr)` sets a hard floor that pushed the right-hand column out of the pane — the page scrolled sideways instead of reflowing, and it looked like a rendering fault rather than a CSS one.
- The **AI page** (`extensions/temco-ai/src/aiPage.ts`) is the AI as something owned and configured — the four facts a person needs before trusting an answer, what can be downloaded, and which is chosen. It stands beside the AI *panel*: the panel is the conversation, the page is the setup. A project's AI tab carries the same facts plus the suggestions read from that project's pages.
- **Suggestions need no model.** The ones on the AI tab — an unowned page, a source never read, a chapter past about 25 pages, a repeated title — come from the catalog, so they are identical with the AI off. What a model adds is prose over the same facts, never the facts themselves.

TEXT

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ Temco Platform — T3000 Developer Help                          ⌘  ▢  ×       │
├──────────────┬──────────────────────────────────────────────────┬────────────┤
│ ▣ Temco      │  Document — T3000 Developer Help  v1.4   [ Build ▾ ] │ TEMCO AI   │
│              │ ─────────────────────────────────────────────── │            │
│              │ health 82 · 12 stale · 3 unowned · 0 broken links │ You  summarise │
│  DOCUMENTS   │ sources T3000Webview @ main · C:\docs\t3000-extra │      this page │
│   ▸ T3000    │ 318 pages                                        │ AI   This page │
│     Dev Help │ ─────────────────────────────────────────────── │      lists the API │
│   ▸ Webview  │ [ Pages ] [ Builds ] [ AI ] [ Settings ]         │      points; 14 │
│     Guide    │                                                  │      months old. │
│   ▸ Firmware │   ▾ t3000/api-reference      13 pages  stale 4   │            │
│     Help     │     ▾ data-points             5 pages  ok        │ [ Draft update ] │
│              │       Data Points API         1 page   updated 2 days │ [ Ask the area ] │
│  SOURCES     │   ▾ t3000/architecture       37 pages  unowned   │ [ Models… ]  │
│   ● T3000    │                                                  │            │
│     Webview  │                                                  │ local · offline │
│   ● T3_Engine│                                                  │ qwen2.5-3b   │
│   ▸ C:\docs  │                                                  │            │
│     \t3000-  │                                                  │            │
│     extra    │                                                  │            │
├──────────────┴──────────────────────────────────────────────────┴────────────┤
│ PROBLEMS  OUTPUT  TERMINAL  │  ! data-points.md   Order 20 is used twice     │
├──────────────────────────────────────────────────────────────────────────────┤
│ 3 documents · 6 sources · 2 drafts to review · 14 pages owed     Markdown  Ln 3, Col 1 │
└──────────────────────────────────────────────────────────────────────────────┘
```

The AI goes on the right, beside the page being worked on — that is where `app-layout.md` 's mock puts it, and it is where a reader of a page wants a question answered.

The AI sits in the right side bar, and that is a declaration, not a preference. The container is contributed under `contributes.viewsContainers.secondarySidebar` — a location VS Code's own schema has, and which this design document wrongly claimed did not exist. The secondary side bar is therefore its default home: nothing to drag, nothing to pick, no setup step. If someone moves it (the panel, the primary side bar), *Show the AI in the right side bar* (`temcoAi.dockRight` , in the `temco-ai` module) focuses the view and invokes the per-container `<containerId>.resetViewContainerLocation` command VS Code registers, which returns it to its declared location. That is also why the extension never asks *where* a view currently is: the view-location API is not exposed to extensions, and the reset command makes the question unnecessary.

The toolbar

The design package puts the editor on the **website** — `15-online-editor.md` , *The editor core*, picks Monaco, *The screen* gives the toolbar it needs, and *The API* the contract behind it. Two of those three are already true on the desktop without a line of ours, and saying which is which is what keeps this small:

TEXT

```
what 15-online-editor.md asks for            the website has to      this application already has
──────────────────────────────────────────── ─────────────────────── ──────────────────────────────
Monaco, the editing core                      bundle it, ~2 MB        VS Code *is* Monaco plus a s
find, folding, breadcrumb outline, minimap    free once bundled       the same, from the editor
gate findings as squiggles on the line        /api/docs/validate      src/diagnostics.ts, already
a toolbar of formatting buttons               build it in main.js     src/format.ts — this step
link completion from index.json               a completion provider   Link to a page, a command
a WYSIWYG mode                                refused on principle    refused for the same reason
preview identical to the site                 docfx serve in an iframe the repository's own build
publish, and the PR it opens                  POST /api/docs/publish  never: it builds, it does no
```

The one row that *is* work is the toolbar, and it lands in the editorial surfaces a person already uses rather than in a page of ours: four buttons in the editor's title bar for a Markdown file, the same sixteen actions in the submenu on the editor's context menu and on the tab's, and one **Format** entry that lists them all, so the palette is the keyboard path. `15-online-editor.md` , *The screen* asks for the same list, and the set matches it: emphasis, headings, the lists, the quote, the code block, the table, the rule, an image, a link to a page, an outline.

Three decisions inside it are worth recording, because each one had two answers:

Decision	Taken	Why
The editor is VS Code's, with the project's toolbar on it	**not an embedded editor page**	the design package refuses an embedded VS Code for the website (`15-online-editor.md` , *What is not built*); here it would be embedding the editor inside the editor, and re-building the preview, the markers and the file handling that already exist
No Markdown formatting commands ship in VS Code	**so these are ours**	checked, not assumed: no `toggleBold` exists in any built-in extension, so `**…**` had no button before this
No keybindings are contributed	**deliberate**	`Ctrl+B` hides the side bar everywhere else, and this extension must keep working in an ordinary VS Code. Every action is a command named `temcoDocs.format.*` , so a chord is one binding away for whoever wants it

The button that needed a rule rather than an implementation is **Link to a page**. `05-content-and-registry.md` fixes the form — a *relative* `.md` link, resolved and warned on by the build, with `~/` for root-relative — so the picker offers the pages the registry declares and inserts a path the gate can resolve. A page in another source is refused in words, because no relative path reaches it. **Callout** takes its five kinds from DocFX's own alerts (`> [!NOTE]` , `TIP` , `IMPORTANT` , `CAUTION` , `WARNING`), which is the syntax the renderer actually implements.

An image is the one button that is not here, and the reason is the write rule rather than the code: the file has to be written into the page's own `media/` before the reference to it points at anything. That amendment, the kinds of new document, and what each costs are the subject of `docs/editor-design.md` .

Operating on things

The operations were in the sidebar's title bar — twelve buttons on the Library, four of them needing a target nobody had picked — and the tree could only be read. What replaced it is one rule and one place per kind of thing:

TEXT

```
the title bar of a view        starts work, and stands for nothing in particular
the ··· in that title bar       everything that needs a target, or is housekeeping
a row                          operates on the thing the row is
a page (dashboard, hub)        operates on the thing the page is about
```

TEXT

```
LIBRARY HEADER — before

┌──────────────────────────────────────────┐
│ [+][▤][✓][◧][+ᵥ][↓][↓][●][⚙][☰][⌘][▶] │   12 buttons — and four of them have no target to act
└──────────────────────────────────────────┘


LIBRARY HEADER — after

┌──────────────────────────────────┬──┬──┐
│ [+ New project] [Projects] [✓]   │ ··· │   3 buttons, and the ··· holds everything else
└──────────────────────────────────┴──┴──┘

  ···   Read every source · Read a source… · Add a source…
        Work on a project… · Build the project…
        Run a script… · Staleness report · Catalog · Set a token
```

The Workspace tree now carries the page operations, because that is where a page is. `page` and `missingPage` are two context values rather than one, so the menu can offer *Create the missing page file* only where a file is missing, and *Move* only where there is one.

```
TEXT
```

```
▾ T3000  t3000 · 318 pages
  ▾ features  41 pages  ⇄ owed
      Trend Logs      ● 2 issues
      Data Points     ▲ unowned
      System Time     ⚠ missing file
      Trend Log       (read from the source)

  left click a row        a page opens · a folder expands          (the Explorer's own behaviour)
  right click a row       the row's menu — and the keyboard's menu key opens the same one
  on a folder row         New Page… · New Chapter… · New Page of a Kind ▸ · Reveal in Explorer
                          Open in Integrated Terminal · Copy Path · Copy Relative Path
                          Open the Registry
  on a page row           Open the Page · Reveal in Explorer · Open in Integrated Terminal
                          Copy Path · Copy Relative Path · Rename the Title…
                          Move the Page Up · Move the Page Down · Retire… · Open the Registry
  the view's toolbar      [New page] [New chapter] [Refresh] […]  — the ··· is the list above, for
                          row the tree last had focused
  drag a page onto        a chapter (chapter + order) · a sibling (two order values, swapped)
```

The tree behaves the way the Explorer does, because that is the tree everyone already knows. A left click opens a page or expands a folder; a right click opens the row's own menu, which carries only what that row can do — a page has no *New Page…*, a chapter has no *Rename*. Nothing is on hover, because a menu started from a row can only ever be a quick pick in the middle of the top of the window, and the menu above is what replaces it. The one act that is *not* the Explorer's is deliberate: **the view's `…` shows the same list for the row the tree last had focused**, so the buttons in the title bar and the palette reach every act without a right click (a view title action runs with the tree pane's context, and the row commands fall back to the view's selection).

The kinds are the file types. A page is always Markdown — its id is its path under `docs/`, the generator reads its title from its first `#` — so the submenu lists what actually differs between one new page and another: the headings it starts with (Article · Task · Reference · Troubleshooting · Release note).

A drag is a registry write, and it says which values it will change. Dropping a page on a chapter writes its `chapter` and a new `order`, and the confirmation says the word *structure*, because `15-online-editor.md`, *Safety rules*, rule 4 marks that field as the docs owner's. Dropping it on a sibling in the same chapter swaps two `order` values and nothing else. Nothing here renumbers a chapter: placing a page between two others is the generator's own job, and the generator does it for whatever nobody curated.

The projects page is the operations home, which it already was — each card carries *Work on it* · *Open* · *Read sources* · *Build* · *Edit* · *Scripts…* · *Remove…* — with one addition: **Scripts…** lists the repository's own allowlisted scripts that *this* project's folders actually have, and runs the one you pick through the Library's own command. A script belongs to a repository, so it is offered per project only because a project is what cites one; when the project is not the one being worked on, it is made so first, out loud, because the Workspace view changes with it.

Documents — the home page

TEXT

```
DOCUMENTS                                          [ + Create a document ]   [ Scan sources ]
──────────────────────────────────────────────────────────────────────────────────────────────

Since you last looked
  ● T3000 Developer Help      12 pages changed · built 2 days ago · 2 drafts waiting
  ● C:\docs\t3000-extra       new folder found · 34 pages · never built

DOCUMENT               VERSION  SOURCES  PAGES  HEALTH  LAST PUSH          STATE
T3000 Developer Help   1.4      2        318    82      main 2 days ago    ok · 12 stale · 3 unow
  ├ 12 stale pages — the code they claim moved on since the last review
  └ 3 pages have no owner
T3000Webview Guide     2.0      1        54     94      main 3 weeks ago   ok
Firmware Help          0.9      1        96     61      rel/2.1 5 months   stale, 4 unowned, no b
──────────────────────────────────────────────────────────────────────────────────────────────

4 documents · 6 sources · 468 pages · 1 source has no registry yet

(nothing here yet)   No documents yet.  [ Create a document ]   [ Scan for sources ]
```

Sources — what the platform reads from

TEXT

```
KIND     SOURCE                     BRANCH      REGISTRY  DOCS   PAGES  LAST PUSH          READ A
git      T3000Webview               main        —         yes    318    main 2 days ago    fetch
  └ registry is on feature/lvgl-svg-renderer, not on main          ⚠ branch mismatch
git      T3_Engine                  release/2.1 yes       yes     96    rel/2.1 5 months    api
git      T3000-API                  main        —         —       —     main 3 years       api
folder   C:\docs\t3000-extra        —           —         yes     34    —                  local
folder   C:\docs\legacy             —           yes       yes    104    —                  local
archive  vendor\bacnet-stack.pdf    —           —         —       —     —                  read
… 17 more repositories with no documentation found

[ Scan for sources ]  [ Add a source… ]  [ Prune the source cache (1.8 GB of 2 GB) ]
```

Create a document

TEXT

```
┌─ Create a document ──────────────────────────────────────────────────────────┐
│  1  Name        T3000 Developer Help    version 1.4    audience developers    owner docs   │
│                                                                              │
│  2  Sources — pick from what the catalog knows                    [ + Add a source ]  │
│     [x] T3000Webview    branch [ main ▾ ]    318 pages · registry —        ⚠ see below      │
│     [x] C:\docs\t3000-extra                  34 pages · no registry                        │
│     [ ] T3_Engine       branch [ rel/2.1 ▾ ]  96 pages · registry yes                      │
│       the platform says what it found:                                                     │
│         the registry is on feature/lvgl-svg-renderer, not main   → [ use that branch ]     │
│         C:\docs\t3000-extra has 34 pages and no registry         → [ start one ]           │
│                                                                              │
│  3  Recipe      engine [ docfx ▾ ]   config docs/docfx.json                                │
│     outputs     [x] CHM t3000_dev.chm  [x] PDF t3000_dev.pdf  [ ] publish to the site     │
│     it runs the repository's own script, as a task — never a re-implementation             │
│                                                                              │
│  4  Review      T3000 Developer Help v1.4 · 2 sources · 352 pages · chm + pdf              │
│                 first build:  npm run docs:registry  then  npm run docs:site               │
│                                                  [ Back ]  [ Create ]  [ Create & build]│
└──────────────────────────────────────────────────────────────────────────────┘
```

Reading a source — three modes

TEXT

```
local      a folder you have open, or a path in the catalog
           files · checks · edits · builds                                  cost: none

fetch      a shallow clone into  globalStorage/sources/<slug>
           git clone --filter=blob:none --depth 1                           cost: disk (2 GB, LI
           used for: drafts (the model needs the page text), builds, file-existence checks

api        one commit's changed files + patches, over the host's API
           used for: commit summaries, "what changed"                       cost: calls (5 000/h
           rule: summaries need diffs · drafts need files
```

AI analysis — the job, and what it produced

TEXT

```
AI — analysis over history                                  model qwen2.5-3b · local · offline
──────────────────────────────────────────────────────────────────────────────────────────────

Job    18 of 24 commits summarised · 6 remaining      [ Run ]  [ Pause ]  [ Run the rest ]
       T3000Webview @ main · since 2026-09-01 · skipped 1 (patch too large: 1.4 MB)

SUMMARISED
  411c5e55  2 days ago   docs   Clean the docs and move them into the platform repository
                         → moved 3 chapters, rewrote the design hub intro
  f27cf000  5 days ago   docs   Update the docs to add a YAML file for DocFX
                         → affected t3000/architecture (2 pages)

DRAFTS TO REVIEW
  t3000/architecture/overview.md   +12 −4   "the registry now declares a YAML file…"  [ Open diff
  t3000/data-points/README.md      +3  −0   "the Design Studio section moved"         [ Open diff

PAGES OWED   14 pages · oldest 8 months      [ Open the owed list ]

Nothing here is published or committed: applying a draft is a diff you accept, one `Ctrl+Z` away.
```

Which page reads what

TEXT

```
Documents page   ← documents · pages · builds · artifacts · events
Sources page     ← sources · scans · pages · commits
Document page    ← doc_sources · recipes · builds · artifacts · pages · commits
Create a doc     ← sources (what exists)   → documents · doc_sources · recipes
AI analysis      ← commits · commit_pages · pages   → summaries · drafts · owed
Build log        ← builds · artifacts
Notifications    ← events where seen_at is null
```

What every surface obeys

- **Say where the facts came from.** Every row carries its fidelity — local, fetch or api · floor.
- **Empty states act.** A page with nothing on it offers the one thing that applies, never a sentence telling the reader to go and find it.
- **A problem lands where the fix is** — the page for a page problem, its row for a document that has never been built.
- **Nothing is written to a source**, and nothing is published, except by a command the user ran and confirmed.

Decisions

Decision	Made
The AI's dock	docked on the **right by the workbench** — the secondary side bar takes any view and remembers the position. No contribution point can place it there, so a fresh install needs one drag, and that drag is the documented setup step
The catalog's store	`node:sqlite` — built into the Node the extension host runs, so the platform adds no dependency for it. Where that built-in is missing, the store refuses in words at the point of use and activation is unaffected
Branch policy	the default branch, with the mismatch reported and a per-source override
Notification default	`docs-only` — a daily summary of documentation change, no modals
The entry point	**the welcome page opens on activation** (`temcoWorkbench.start.open` , default `welcome` , with the checkbox in its own footer as VS Code has it), falling through to the project's pages, then the page you were last on, then the overview. The one recorded project selects itself, so the tree is ready without anyone picking it
The Workspace, gated by content	**the Workspace stays the first view and the Library sits beneath it**, and the open folders are never read into it. Its empty states are **rows the tree draws itself**, offering the act that fills it — *create a project* · *choose one in the Library* · *open this project and read its sources*. Hiding the view on a context key (`when:`) and phrasing these states as `viewsWelcome` were both tried and rejected: a selected project with no visible tree reads as a fault, and a welcome's `when` is resolved before the refresh has read anything

Build order

Step	What	Done when
R1	**The catalog** — the store, its schema, migrations, and the rows above	**done**: 12 tables, applied idempotently and exercised against a real `node:sqlite` database — inserts, the document↔source join, the unseen-events query. The store lives in `globalStorageUri`, never in a repository
R2	**Sources** — add a git source (any host, chosen branch) or a folder; local, materialised and online modes	**partly built**: a source is added, read in `local` and `api` mode, and remembered across restarts. Not built: the materialised (`fetch`) mode, and the branch mismatch is reported by the Content view rather than by a read
R3	**Documents** — create one, cite sources, choose a recipe, and build it	**partly built**: a project is created, cites sources, chooses a recipe, and its build runs the repository's own script as a task and records the files by path. Not verified: a real artifact (`t3000_dev.chm`) has not been produced through this path yet
R4	**AI over history** — commit summaries, affected pages, drafts, the owed list, as a resumable job	a real commit has a summary and a draft, and a human still approves
R5	**The views** — Documents and Sources: health, fidelity, latest push, *Since you last looked*	**partly built**: the sidebar holds Workspace (the pages of the project being worked on, with a row saying why while there is nothing) first and Library (projects · sources · repositories · catalog · needs attention · the AI) beneath it, and *What needs doing* answers "what do I have, and what changed?" on a page. Not built: the *Since you last looked* digest, and fidelity per source beyond the read mode
R6	**Editing and publishing** — the editor with its toolbar, creating a document, images, draft branches, the design package's hosted editor, PR flow	**partly built**: the **toolbar** is built on VS Code's own editor — emphasis, blocks, a link to a real page, the outline — as the sixteen `temcoDocs.format.*` commands in `src/format.ts`. A page can be **created, named, moved and retired** — `src/pageOps.ts`, six commands and a drag controller — and a project's own scripts can be run from its card on the projects page. Not built: inserting an image, drafts and the PR flow, which need the `fetch` read mode and a review flow that lives in the product repository

R6 is mostly a rule change, and the rule was changed by naming it. The toolbar edits the buffer a person is already typing in, so it adds no writer at all. Creating a page writes **one file** — the repository's own `docs:registry` adds the entry, derives its place from the path and its title from the first heading, so nothing here writes an entry for a page that did not exist. And a page that is *moved* changes **one curated value at a time** in `docs/help/registry.json`, through `src/registryEdit.ts`, which renders the file the way its own generator does — including the file's line endings — so a move is a two-line diff. `docs/editor-design.md` §5 records the decision, the four rules that hold it, and the evidence.

R1–R3 make it a documentation platform: sources in, documents and artifacts out. R4 makes it intelligent. R5 makes it a management system. R6 closes the loop.

What this does not change

- **Content never moves.** A document cites sources; it never relocates their pages.
- **Only artifacts ship.** A customer installation receives the CHM and the F1 map — never this app.

- **The AI proposes; a person approves.**
- **The design package still wins** on content, registry and publishing rules. This document only says what the platform knows, where it keeps it, and what it builds.

Docs center: the redesign

One consolidated design for the documentation center. It supersedes the layout sections of `app-flow.md`, `authoring-flow.md` and `extension-design.md`, which contradict each other. Nothing here is built yet.

1. The model

```text
Sources ─► Temco Platform (authoring) ─► Product package ─► Temco Learn (website)
repos, pdf,   projects · AI · review      products/<id>/      search · downloads · versions
word, chm                                 manifest.json
                                          docs/ api/ guides/ assets/ releases/
```

System	Job	Where
Temco Platform	where people work: projects, sources, AI drafts, review, generate	this repository
Product package	the unit of exchange; a plug-in the portal discovers	`products/<id>/`
Temco Learn	the reader's MSDN-style portal	the `temco-learn` repository
DocFX	one pluggable build engine; CHM and PDF are other exporters	invoked by Build, never a surface

Vocabulary. Project (a `documents` row) → Section (the registry's *area*) → Folder (its *chapter*, nestable) → Page. Versions come from the project's releases. A `products` table is added only if several versions must be open at once.

Truth rule. Markdown and `registry.json` in the repository are the content. `catalog.sqlite` is a register (projects, sources, builds, releases, drafts, caches) and never holds a second copy of page text, except a draft not yet accepted.

2. Journey

```text
A LAUNCH ─► B START PAGE ─► C CREATE PROJECT ─► D EMPTY PROJECT ─► E ADD PAGES
                                                                      │
  I RELEASE ◄─ H PUBLISH ◄─ G NEEDS YOU / AI LOOP ◄─ F EDIT / CAPTURE ◄┘
```

A. First launch

TEXT

```
┌────┬──────────────────────────┬──────────────────────────────────────────────────────────────┐
│ ▤◀ │ DOCS                     │  Welcome to Temco Docs                                       │
│    │  No project yet          │   ┌ Start ──────────────┐  ┌ Bring in existing ──────┐      │
│    │  [ ＋ Create a project ] │   │ ＋ New project      │  │ Word · RTF · CHM · HTML │      │
│    │  [ ⇩ Import a doc set  ] │   │ ▤ From a template   │  │ A folder of Markdown    │      │
│    │  [ ⎇ Open a repository ] │   │ ⎇ From a repository │  │ A git repository        │      │
│    │                          │   └─────────────────────┘  └─────────────────────────┘      │
├────┴──────────────────────────┴──────────────────────────────────────────────────────────────┤
│ catalog.sqlite created · default organisation written                                        │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

B. Start page (projects exist; shown in the editor, not a container)

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ Temco Docs                                                ＋ New project │
│ RECENT PROJECTS                                                          │
│  ▤ T3000 Documentation   v13.0   3 sources   built 12:04 ✓   [ Open ] [ ⋯ ]  │
│  ▤ BACnet Tools          v1.2    1 source    never built     [ Open ] [ ⋯ ]  │
│ NEEDS YOU (all projects)                RECENT ACTIVITY                  │
│  ⚑ 2 drafts waiting       T3000          12:04 build site ✓ T3000        │
│  📷 1 screenshot to place  T3000          11:40 capture placed → Scaling  │
└──────────────────────────────────────────────────────────────────────────┘
```

C. Create a project (4 steps)

TEXT

```
1 ORIGIN   ◉ Blank · ○ From a template · ○ From a repository · ○ Import a document set · ○ Open ex
2 BASICS   Name [T3000 Documentation]  Version [13.0]  Owner [alice ▾]  Audience [developer ▾]
3 SOURCES  ☑ ⎇ t3000webview [main ▾] [src/**]   ☑ ⎇ t3000-esp [main ▾]   ＋ GitHub org · folder
4 OUTPUTS  Engine [DocFX ▾]  ☑ Website ☑ PDF ☑ CHM ☑ Search index ☐ AI index  →  Review  →  [ Cr
```

Create writes `documents`, `doc_sources`, `recipes` and the smallest valid `docs/help/registry.json`. The six default sections are seeded for a blank project: Overview, Guides, Architecture, API, Troubleshooting, Release Notes.

D. First open (empty project)

TEXT

```
┌────┬──────────────────────────┬──────────────────────────────────────────────────────────┐
│ ▤◀ │ DOCS              + 🗀 ⟳ │ T3000 Documentation 13.0                                 │
│    │ ▾ T3000 Docs · 13.0      │ Your project is ready. Next:                             │
│    │     (empty)              │  ① Add the first page            [ New Page ]            │
│    │ ▸ NEEDS YOU         (0)  │  ② Bring in existing material     [ Import ]             │
│    │ ▸ SOURCES           (2)  │  ③ Let the AI draft from sources  [ Draft sections ]     │
│    │ ▸ PUBLISH                │                                                          │
└────┴──────────────────────────┴──────────────────────────────────────────────────────────┘
```

3. The window

TEXT

```
┌────┬──────────────────────────┬──────────────────────────┬──────────────────────────┐
│ ▤◀ │ DOCS              + 🗀 ⟳ │ first-project.md ×  │ Preview │ ASSIST                 │
│    │ ▾ T3000 Docs · 13.0 (pick) │                          │ [Draft][Properties]      │
│    │   ▾ Overview          3  │ markdown editor          │  title · owner · ship    │
│    │   ▾ Guides           14  │  Ctrl+Alt+S inserts image │  keywords · help ids     │
│    │     ▾ Getting Started 4  │                          │  code · reviewed         │
│    │         First project ●  │                          │                          │
│    │   ▸ API · Troubleshooting… │                          │                          │
│    │ ▸ NEEDS YOU (7)  collapsed │                          │                          │
│    │ ▸ SOURCES (3)    collapsed │                          │                          │
│    │ ▸ PUBLISH        collapsed │                          │                          │
├────┴──────────────────────────┴──────────────────────────┴──────────────────────────┤
│ T3000 13.0 · 318 pages · 14 stale · 0 problems        ⚑ 2 drafts        site ✓ · help ✗ │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

Constraints from the VS Code API that shape this layout:

- A tree view cannot hold dropdowns, so **the project is the tree's first row**. Clicking it opens a quick-pick; the view title shows it too.
- The three lower views are independent panes, collapsed by default. There is no accordion behaviour.
- A text editor has no tabs: Edit and Preview use the Markdown side-by-side preview, and **Properties is a tab in the right panel**.
- Dropping on the tree reports only the row dropped on, never the gap between rows.
- The tree is **path-based**. A page id is its path under `docs/`, so `guides/getting-started/install.md` is the page `install` in the folder `getting-started`, inside `guides`. Every folder level is derived from those paths: the registry does not have to name the middle folders, and folders nest to any depth.

4. E. The Pages tree

Rows and hover icons

TEXT

```
 PAGES                                          +  🗀  ⇩  ⟳  …
┌──────────────────────────────────────────────────────────────┐
│ ▾ Guides                                  14                 │
│   ▾ Getting Started                [+][🗀][...]   ← folder hover │
│       Install                                                │
│       First project   ●            [📷][✦][⋯]    ← page hover   │
│       Licensing                                              │
│ ▾ API                                                        │
│   ▾ Data Points API                                          │
│       Scaling         ⚠  📷                                    │
│         └ ⚠ code changed since last review                   │
│         └ 📷 no image on this page                           │
└──────────────────────────────────────────────────────────────┘
```

Folder row: name and page count only. Leaf row: title only.

State is a badge (● draft · ⚠ stale · 📷 no image) or a folded child, never an extra row.

Hover icons: folder = + New Page · 🗀 New Folder · … ; page = 📷 Capture · ✦ Draft with AI · ⋯

Tooltip: id · owner · audience · last change.

New page and new folder: the selection is the location

TEXT

Selected row	New Page lands…	New Folder lands…
nothing / empty space	at the root of the section	the root
a folder	inside it	inside it (a sub-folder)
a page	next to it	next to it

TEXT

```
click + (or Ctrl+N with the tree focused)
┌──────────────────────────────────────────────────────────────┐
│ New page in  Guides / Getting Started                        │
│ [ First project_____________________ ]                       │
│   → getting-started/first-project.md                         │
│                            Enter = create    Esc = cancel    │
└──────────────────────────────────────────────────────────────┘
         │ Enter ─► file written, registry entry added, row selected, page opens
```

Act	You type	Automatic
New Page	title	file name from the slug, kind Article, last position, registry entry, empty template
New Page ▸ Task / Reference / Troubleshooting / Release Note	title	the same, with the kind set
New Folder	folder name	the folder and its first page, since an empty chapter does not exist in the registry

The input box is a quick-input, the closest thing the extension API allows to Explorer's inline edit. Renaming in the row itself would need a change to the core of this fork and is deferred.

Context menus

TEXT

```
 on a FOLDER row              on a PAGE row                    on empty space
┌──────────────────────┐    ┌──────────────────────────────┐    ┌──────────────────┐
│ New Page…            │    │ Open · Open to the Side      │    │ New Page…        │
│ New Folder…          │    │ ──────────────────────────── │    │ New Folder…      │
│ Import Here…         │    │ New Page…  (sibling)         │    │ Import…          │
│ ──────────────────── │    │ New Page of a Kind  ▸        │    │ Refresh          │
│ Rename…        F2    │    │ ──────────────────────────── │    └──────────────────┘
│ Move Up / Down       │    │ Rename File…        F2       │
│ ──────────────────── │    │ Rename Title…  Shift+F2      │
│ Reveal · Copy Path   │    │ Move Up / Down               │
│ Delete…              │    │ ──────────────────────────── │
└──────────────────────┘    │ Capture…  Ctrl+Alt+S         │
                            │ Paste Image  Ctrl+Alt+V      │
                            │ Draft a Change with AI…      │
                            │ Mark Reviewed                │
                            │ ──────────────────────────── │
                            │ Reveal · Copy Path           │
                            │ Delete…                      │
                            └──────────────────────────────┘
```

Keys

TEXT

```
F2 rename file (old id kept as an alias)    Shift+F2 rename title only    Delete  with confirmatio
Ctrl+C / Ctrl+X / Ctrl+V  copy or move rows between folders      Enter / click  open the page
```

F2 moves the file, the repository's own generator writes the new registry entry, and the old id is written into `aliases` first. A project with no generator is refused, because the rename would orphan inbound links.

Drag and drop

```
drag a page   onto a FOLDER row  → into that folder, last position
              onto a PAGE row    → into that page's folder, right after it
              onto empty space   → to the section root
multi-select  Ctrl/Shift+click, then drag together          Ctrl+drag  copies
every move    → file moves, `order` rewritten, old id kept as an alias
refused       → into itself or its own child; across projects
from the OS   → a .md, .png or .docx dropped on a folder is imported there
```

5. F. Editing a page

```
┌──────────────────────────────────────────────────────────────┬──────────────────┐
│ first-project.md        [ side-by-side Preview ]             │ ASSIST           │
│ # First project                                              │ [Draft][Ask][Props]
│ Points carry a scale factor.                                 │ Draft from…      │
│ ![Trend log](media/02-trend.png)  ◄─ Ctrl+Alt+S inserts this │  ◉ source file   │
│                                                              │  ○ another page  │
│                                                              │ RESULT (diff)    │
│                                                              │ [Apply][Reject]  │
└──────────────────────────────────────────────────────────────┴──────────────────┘
Capture: snip overlay → clipboard → media/NN-slug.png (warn < 1600 px, refuse > 2 MB).
No page in front → .temco/captures/ and a row in NEEDS YOU until it is attached.
Apply is one undoable edit. Drafts wait in SQLite until accepted.
```

The **Properties** tab is the form over the registry fields: title, section, order, owner, audience, ship, keywords, help ids, code, reviewed.

6. G. Needs You and the AI loop

```
NEEDS YOU (7)
 📷 Screenshots to place (1)  row → "which page takes it?"
 ⚠ Retake these (4)          row → the page
 ○ Never reviewed (2)        row → the report
 ● Drafts waiting (2)        row → diff, Apply / Reject

source changes ─► commit summarised once (model recorded) ─► mapped to pages (code-glob · path · te
  ─► page marked stale ─► NEEDS YOU ─► Draft a Change ─► draft row ─► diff ─► Apply
```

7. H. Publish

TEXT

```
┌──────────────────────────────────────────────────────────────┐
│ PUBLISH · T3000 Documentation 13.0                           │
│  ① REVIEW ──► ② BUILD ──► ③ RELEASE                          │
│  2 waiting     site ✓ help ✗   not released                  │
│ ① drafts and changed pages: [Open diff] [Approve]            │
│ ② Engine [DocFX ▾] ☑Website ☑PDF ☑CHM ☐ZIP   [▶ Build]       │
│    success is the artifact left behind, never the exit code  │
│ ③ Writes products/t3000/releases/13.0, latest → 13.0  [⬆ Release] │
└──────────────────────────────────────────────────────────────┘
```

8. I. Result on disk, and Temco Learn

TEXT

```
products/t3000/
  manifest.json   {id, title, version}
  docs/ guides/ api/ assets/ toc.yml docfx.json
  releases/13.0 · 12.9 · 12.8      latest → 13.0
                  │
                  ▼
Temco Learn discovers products/*  →  Home · Product page · Doc page · Downloads (Online · PDF · CH
```

The Learn page's top navigation mirrors the project's sections, so what the writer sees in the tree is what the reader sees.

9. SQLite storage (catalog.sqlite)

TEXT

```
organisations ─┬─< repositories           cache of what GitHub listed
               │
sources ───────┤  kind git|folder|archive · remote_url · path · branch · registry_path · last_comm
  │            └─< scans
  ├─< commits ─< commit_pages           sha · summary · summary_model · how: code-glob|path|term
  │ n..n (doc_sources: branch, include_glob, role)
  ▼
documents  (the PROJECT)  name · slug · version · audience · owner · origin · template
  ├── 1:1 ─ recipes          engine · config_path · script · outputs
  ├─< pages                  id · path · title · owner · reviewed_at · stale · counts   (cache of
  ├─< builds ─< artifacts    status · revision · log_path  /  kind · path · bytes · sha256
  ├─< releases               version · destination · files · published_at · build_id
  └─< drafts                 page_id · title · body · state draft|accepted|discarded
events  at · kind · subject · detail · seen_at       meta  key/value schema version
```

10. What the extension does now

The rule this table records: **one list, one owner.** Every kind of list has exactly one surface, and a webview page never lists something a tree or a panel already lists. Everything a webview page *did* became a command on that surface.

Then	Now
views Pages, Project, Pipeline, Working Set, plus the queue declared in `temco-workbench`	Pages, Needs You, Sources, Publish — all declared by `temco-docs`
3-step New Page wizard	the selection is the location; one title box
10 webview pages (projects, project, needs, repository, documents, page, source, review, builds, releases)	2 left: **Project home** and **Release report**. The other eight commands still exist and now focus the surface that owns the list, so every button and menu item in the other modules still lands somewhere
the tree stopped at area → chapter → page	the tree is a path: folders inside folders, any depth, like Explorer
Working Set	removed (use Open Editors)
Properties as an editor panel	the right panel's **Properties** tab, beside **Draft**
publishing copied artifacts	publishing also writes `manifest.json` per product and regenerates the Learn portal
`home.ts`, `views.ts` named in `extension-design.md`	do not exist; that document now points here

11. Where the work stands

Done:

1. The three older documents carry a pointer here instead of a competing layout.
2. The four views, the queue moved out of `temco-workbench`, `Sources` / `Publish` renamed.
3. The project is the tree's first row; a new project is seeded with the six sections.
4. New Page and New Folder take their location from the selection; hover icons; multi-select drag and drop.
5. Properties is a tab in the right panel.
6. `manifest.json` is written on publish, and the Learn portal is generated from every product's manifest.

Not done, and deliberately so:

- The `temco-learn` site itself. The portal is generated by this extension and served from local storage; a real reader site is a separate repository.
- Inline rename in the tree row. It needs a change to the fork's tree widget; an input box is the closest the extension API allows.
- Dropping *between* rows. The tree reports only the row dropped on, so a drop on a page means "after it".

Open decisions

- Whether `products` becomes a table later (versions open side by side).

- Whether the six sections stay fixed, or are defined per project in `manifest.json`.

Shell and pages, v2 — the full layout, drawn

This supersedes the shell, activity-bar and left-panel parts of application-design.md. Every surface below is drawn in full; nothing is summarised as "keeps the same frame".

Counted from the manifests before designing anything:

	today	v2
activity-bar containers	5	**4**
side-bar views	19	**7**
center pages	~44	**17**
commands	250+	250+ (every id unchanged)

Legend: `[ Button ]` is a button · `▸ ▾ •` are tree disclosure and items · `⋯` is a menu · `⬚` is a picture · `‹ ›` are tabs · `◀` marks the selected row · `(•)` marks a radio choice.

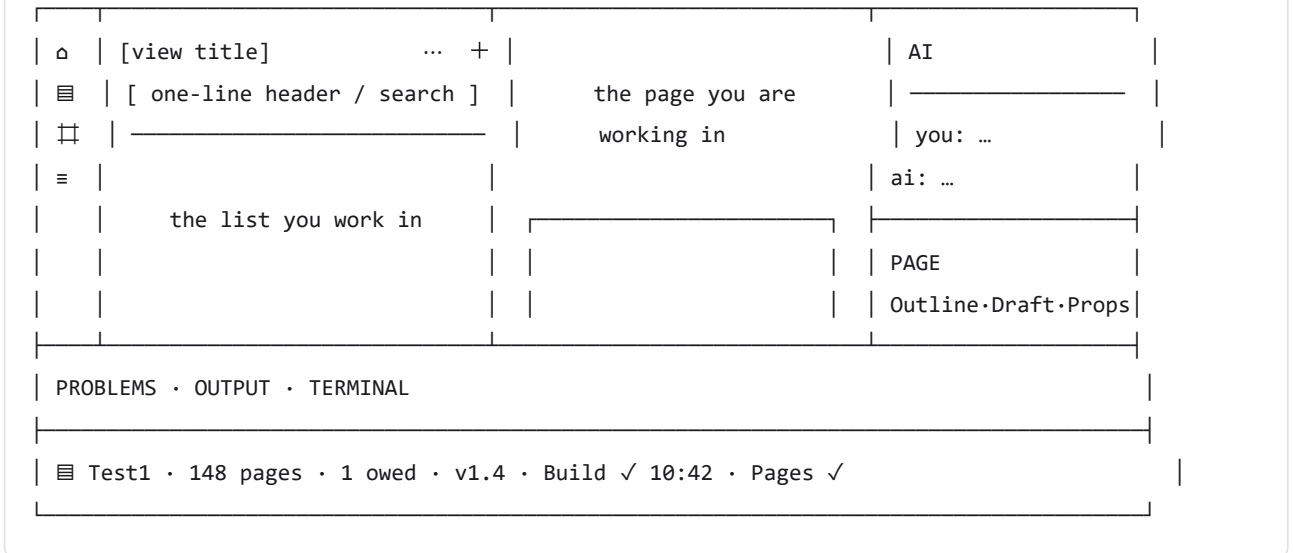
The four rules

1. **A side bar is a list you work in** — not a dashboard, not a status board. Status belongs to the status bar; actions belong to the view's title bar.
2. **A row is a thing, or it is an action — never both.** `test11.md` is a thing. *Build Center* is an action wearing a tree row at the same weight, which is why the column reads as noise.
3. **One name per concept.** Not *Needs You* here and *Review Center* there. Not *Publish* for both "copy a build" and "push to GitHub".
4. **A module with nothing to show gets no icon.** An empty container is a broken promise.

---

1. The window

TEXT



Left one container, one list · **Center** one page · **Right** two secondary side bars that follow the page (§5) · **Bottom** the window's facts, once.

---

2. Activity bar: four icons

The activity bar is the whole application's top-level navigation: **one icon is one container, and a container is one side bar.** That is the only structure VS Code gives an extension, so the bar is where "how many places does this application have?" is decided.

```
TEXT
```

```
┌────┬──────────────┬──────────────────────────────────────────────────────┐
│    │ GLYPH        │ TOOLTIP — what the icon is                           │
├────┼──────────────┼──────────────────────────────────────────────────────┤
│  1 │ house        │ Home — projects, recent activity, repositories, the site  │
│  2 │ book         │ Docs — the documentation workspace                   │
│  3 │ grid         │ Tools — SQL, BACnet, IO, network                     │
│  4 │ sliders      │ Settings — settings and help                         │
├────┼──────────────┼──────────────────────────────────────────────────────┤
│  5 │ account      │ accounts                    (VS Code's own, not ours)  │
│  6 │ gear         │ Manage                      (VS Code's own, not ours)  │
└────┴──────────────┴──────────────────────────────────────────────────────┘
```

Rows 1–4 are ours; 5–6 belong to VS Code and cannot be removed. The whole bar can be collapsed by the user at any time, so nothing here may be the *only* way to reach a page — every icon's contents are also in the command palette.

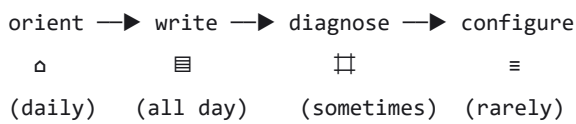
2.1 What each icon opens

#	icon	tooltip	side bar it opens	the job it does	you reach for it when
1	house	**Home**	`Projects` · `Repositories` · `Recent activity`	orient: which project, which Temco code the docs are written from, what happened, what was published	you start a session, or you are not sure what to do next
2	book	**Docs**	`Pages`	write: the tree of pages, the two acts that ship them	you are working on the documentation, which is most of the time
3	grid	**Tools**	`Query & Browse` · `Schema & Maintenance`	diagnose: devices, IO points, BACnet objects, network, SQL	a controller is misbehaving, or you need data out of the system
4	sliders	**Settings**	`Settings`	configure: appearance, modules, models, publishing, help	something is wrong with the installation, or you are changing how it behaves

The lists are unevenly split — **Home 3, Docs 1, Tools 2, Settings 1** — because that is the real number of lists that hold things you click into (rule 1). An uneven bar is the honest bar: a container with one list is fine, a container kept only so the bar looks symmetric is not.

2.2 Why these four, in this order

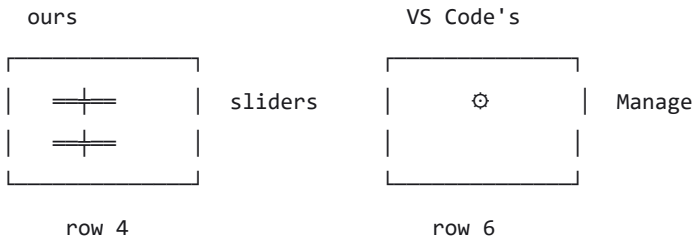
TEXT



- **Order follows a session, not a feature list.** Left to right is the order a person actually moves: find the project, do the work, investigate a problem, change a setting. Icons are muscle memory; the position is the documentation.
- **One icon per kind of activity, not per module.** Nine modules, four activities: that is why the count is four and not nine, and why adding a module does not add an icon.
- **The test for membership.** An icon earns its place only if it opens a list you *work in* and revisit regularly. If it opens one page you visit monthly, it is a row somewhere, not an icon. That test is what removed *Learn* (read-only output → a page under Home) and *AI* (see §2.4).

2.3 The Settings glyph: sliders, not a gear

TEXT



A gear at 16 px is a grey blob; two gears in one column is one meaning too many, and the one at the bottom is not ours. So the Settings container is drawn as **sliders** — the second universal conventions for preferences — and its tooltip reads **Settings** where VS Code's reads *Manage*.

The container id stays `temcoSettings`, so saved layout and `onView` activation are untouched. The whole change is one file: `media/settings.svg`.

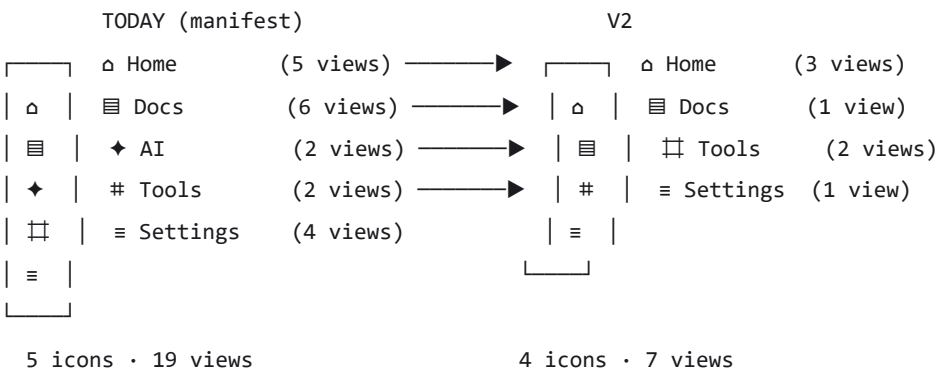
2.4 What is deliberately *not* in the bar

not here	where it is	why
AI	the right-hand secondary side bar (§4.16)	a conversation is not a destination; it is a thing you have *while* writing. Putting it on the left would make it a place you leave the page to visit
Published site / Learn	Home ▸ Published site (§4.6)	read-only output of work done elsewhere — a page, not a place to keep open
Repositories	the **Home** side bar (§4.4)	it is the shelf of **Temco code** that pages are written from — a list of things, opened while orienting, not an activity of its own. The drafting that reads it happens in Docs (§4.8a), and every draft names the file it was given
BACnet · network · device manager	inside Tools	they are jobs inside the diagnostic activity, not activities of their own — and two of them have no manifest yet, so they must not get an icon that opens nothing (rule 4)

2.5 Before and after

Today the manifest declares five activity-bar containers, all from `temco-workbench`:

TEXT



Two things move — nothing disappears:

- **AI leaves the bar** for the right-hand side bar: the *assistant* and *runtime* views regroup there (§4.16), next to the draft panel that is already on that side.
- **Repositories changes container.** It is a view inside **Docs** today (the sixth one), which is why it never had an icon. It moves to **Home**, where "what can I read?" is one of the questions Home answers. Home's five sections today are mostly status boards; under this design it has three real lists — *Projects*, *Repositories*, *Recent activity* — while Docs drops to the single *Pages* tree (§4.4).

Home, Docs, Tools and Settings all stay. Their *contents* change (§3–§7), not their identity.

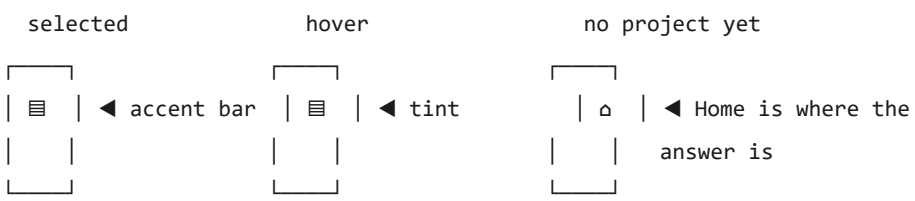
Two other containers exist today but were never in the bar, and this design keeps them out:

container	location	what it is
`temcoAi`	secondary side bar (right)	the AI side bar — already where §2.4 wants AI
`temcoDraft`	auxiliary bar (right, bottom)	the AI draft panel, titled *Page* — it renders the page being written beside the page list

What the eight icons you saw were. The earlier build also carried `temcoLearn`, `temcoHelp` and `temco-repositories` containers. Those three were *removed* in the manifest fix that preceded this document, which is why the bar reads five today: Learn → Home ▸ Published site, Help → Settings ▸ Help, Repositories → Home ▸ Repositories. So the sequence is **8 → 5 (done, manifest) → 4 (this document, AI leaves)**.

2.6 States

TEXT



- **Selected** — the icon for the open container carries VS Code's accent bar; clicking it again collapses the side bar.
- **No badge on any icon.** Counts belong to the status bar and to the view that owns them; a number on an icon is read as "notifications" and gets ignored after a week.
- **No empty container, ever** (rule 4): if a module has nothing to show it is not in the bar at all.

2.7 What implementing this touches

change	risk
5 containers → 4 (remove the AI container)	one manifest, no code — `onView` for its views moves with them
19 views → 7	views are removed only after their content exists elsewhere (§8); *Repositories* is **moved**, not removed
`media/settings.svg` redrawn as sliders	one icon file
container ids unchanged	saved layout, `onView` activation and every `when: view == …` clause keep working
`check-temco-modules` already refuses a view inside a container that does not exist	a half-finished step fails the build instead of showing an empty panel

3. Docs — the one container that changes shape

Before (as you saw it): five stacked regions, the tree got ~120 px

TEXT

```
┌──────────────────────────────┐
│ Project                      │
│  Test1  v1.0                 │  status
│  1 page · no issues          │
│  [Pages 1][Needs you 1]      │
│  [ Find a page…           ] │
├──────────────────────────────┤
│ Pages                        │
│  ▾ Test1                     │  the work — squeezed
│    test11.md                 │
├──────────────────────────────┤
│ Needs You                    │  status
├──────────────────────────────┤
│ Sources                      │  status
├──────────────────────────────┤
│ Publish                      │  actions drawn as rows
│  Review Center               │
│  Build Center                │
│  Releases                    │
└──────────────────────────────┘
```

After: one header line, one tree, everything else one click away

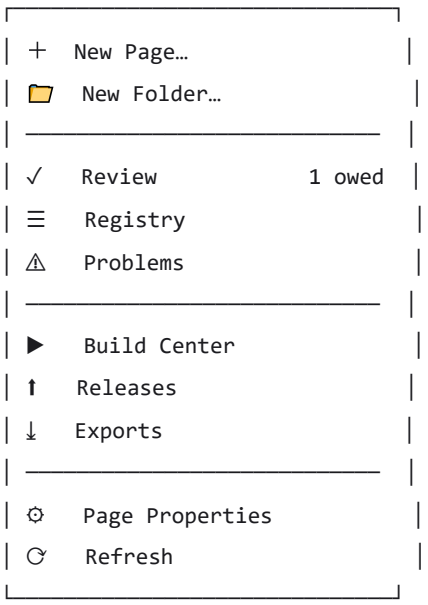
TEXT

```
┌──────────────────────────────┐
│ Test1 · 1 owed   [Build][⇪] │  ← project · what needs you · the two acts
│ [ Find a page…            ] │
├──────────────────────────────┤
│ PAGES                        │
│  ▾ Test1                     │
│    ▾ Overview                │
│      • test11.md             │
│    ▸ media                   │
│    ▸ Commissioning           │
│                              │
│                              │  ← the tree owns the height
│                              │
└──────────────────────────────┘
```

`1 owed` is the link to Review. `[Build]` builds the site; `[⇪]` publishes it.

The ⋯ menu in the view title

TEXT

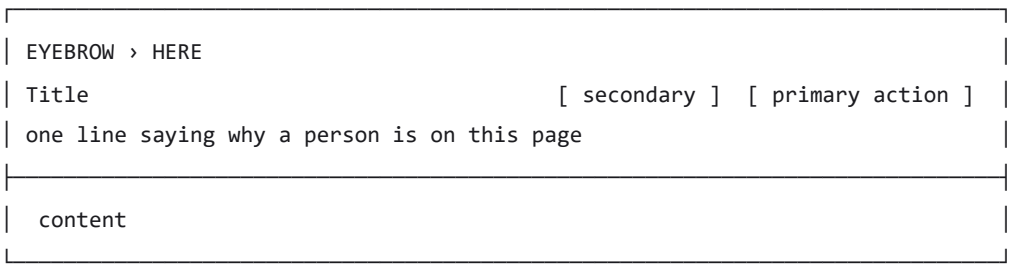


---

4. Every page, drawn in full

All pages share one frame, so a page is recognisable at a glance:

TEXT



The Home side bar — three lists under one icon

Home's six pages (§4.1–§4.6) are opened from this column; each section is a real view, so it collapses, filters and keeps its own ⋯:

TEXT

```
┌──────────────────────────────┐
│ Test1                        │  ← the project everything else is about
│ 148 pages · 2 sources        │
│ [ Find a project…          ] │
├──────────────────────────────┤
│ PROJECTS                  ⋯  │
│  ▸ Test1                ◀    │
├──────────────────────────────┤
│ REPOSITORIES              ⋯  │
│  ▸ temco-docs      main  ✓   │
│  ▸ temco-platform  main      │
│  ▸ docs/help       folder    │
├──────────────────────────────┤
│ RECENT ACTIVITY           ⋯  │
│  ⬆ Test1 v1.0       2 h      │
│  ✎ Commissioning    12 min   │
└──────────────────────────────┘
```

`Repositories` is the view today filed under **Docs**; it belongs here because "what can I read?" is a Home question.

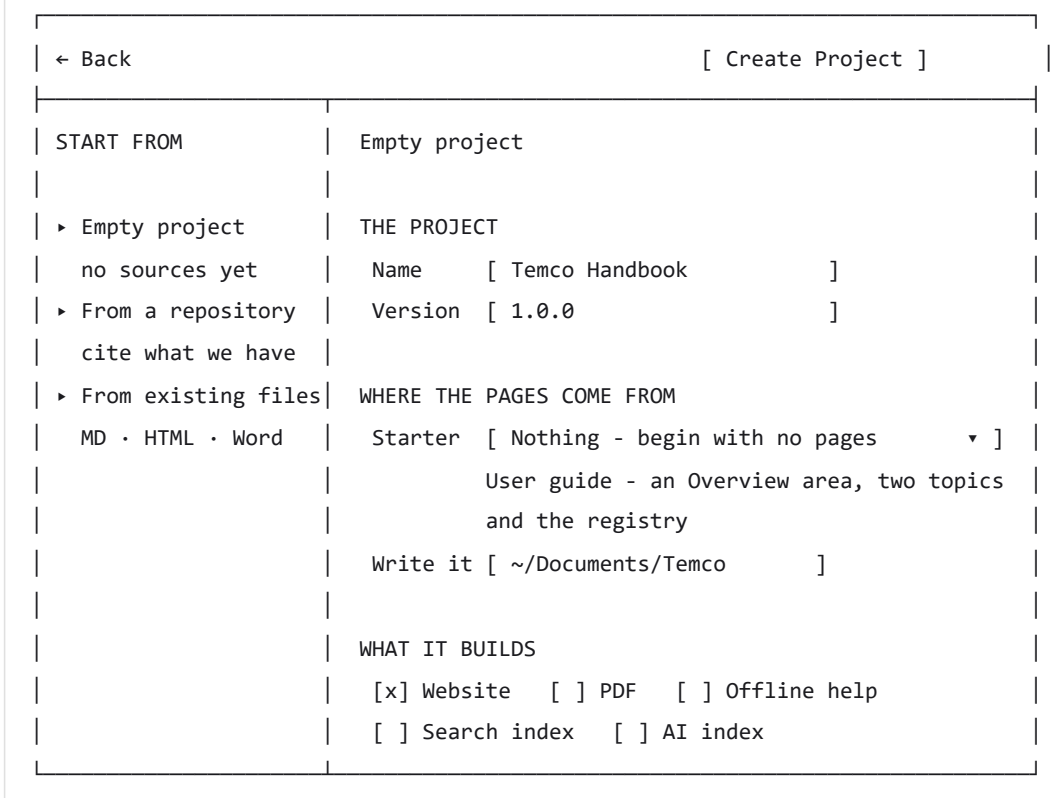
4.1 Home — Projects

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ DASHBOARD › PROJECTS                                                     │
│ Projects                                        [ + New Project ]        │
│ Everything this machine documents. A project is what a release is named after │
│ and what every other page asks about.                                    │
├──────────────────────────────────────────────────────────────────────────┤
│  ┌──────────────────────────┐  ┌──────────────────────────┐              │
│  │ Test1          v1.0      │  │ (empty)                  │              │
│  │ 148 pages · 2 sources    │  │ Start a project to see   │              │
│  │ current            [▸]   │  │ it here                  │              │
│  └──────────────────────────┘  └──────────────────────────┘              │
│                                                                          │
│  First time here?   [ Create a Project ]   [ Create from a Repository… ]  │
└──────────────────────────────────────────────────────────────────────────┘
```

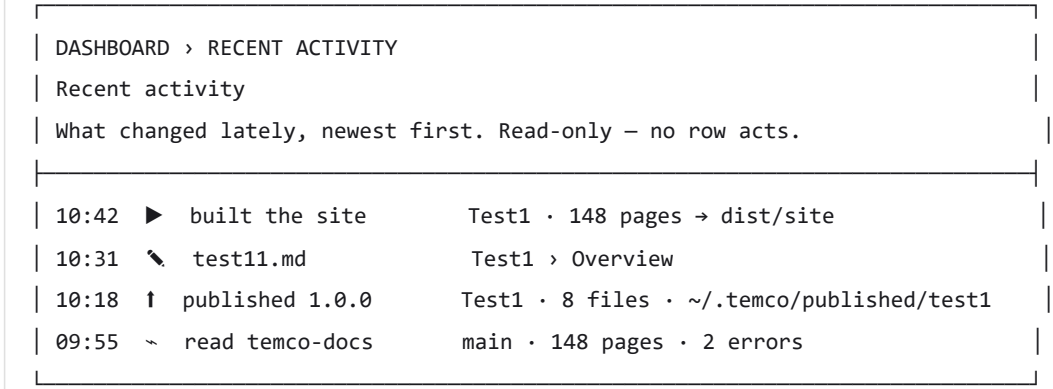
4.2 Home — Create a project *(full-window form; no side bar)*

TEXT



4.3 Home — Recent activity

TEXT



4.4 Home — Repositories *(the Temco code the documentation is written from)*

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ HOME › REPOSITORIES                                                      │
│ Repositories                                    [ + Add a Repository… ]  │
│ The Temco repositories these pages are written from — one list, two acts:│
│ read them to cite what they hold, or tick files and draft from the code. │
├──────────────────────────────────────────────────────────────────────────┤
│ ▽ temcocontrols                                          organisation    │
│   ▸ temco-platform   C:\QN\temcocontrols\temco-platform                   │
│       main · read 12 min ago · 148 pages · 2 errors  [Read] [Draft] […]  │
│   ▸ temco-docs       C:\QN\temcocontrols\temco-docs                       │
│       main · never read · local copy on disk         [Read] [Draft] […]  │
│   ▸ docs/help        D:\Contoso\docs\help                                │
│       a folder · read yesterday · 64 pages            [Read] [Draft] […]  │
├──────────────────────────────────────────────────────────────────────────┤
│ … on a row: Set the branch · Open the local copy · Copy the URL ·        │
│             Remove the local copy · Forget it                            │
└──────────────────────────────────────────────────────────────────────────┘
```

The word on the row is *repository*; inside a project the same thing is called a *source* (§4.7), because that is what it is there — one of the places a project cites. One entity, two scopes, and the list is the scope where you act on it.

button	what it does	what it costs
[Read]	finds `docs/*/registry.json` inside it — or converts a folder once — and records the pages it holds	one API call, or one folder walk; **nothing is ever written to the source**
[Draft]	opens the file ticker (§4.5) so code files can be ticked as material for a new page (§4.8a)	nothing until *Draft* is pressed; a page exists only when the draft is accepted

`[ Draft ]` is the button that makes this page part of writing rather than a library: the code in these repositories is the raw material of every page that is not already written, and the material a page was given is recorded on the page.

Why both a view and a page. The view in the Home side bar is the everyday list — arrow keys, filter, open one. This page is the same sources drawn wide, for what a tree row cannot hold: the organisation they came from, `[ + Add a Repository… ]`, in bulk. One law on both: a row is a thing, an action is in its buttons or `…` (rule 2).

4.5 Home — Repository detail *(the file ticker: what a page can be written from)*

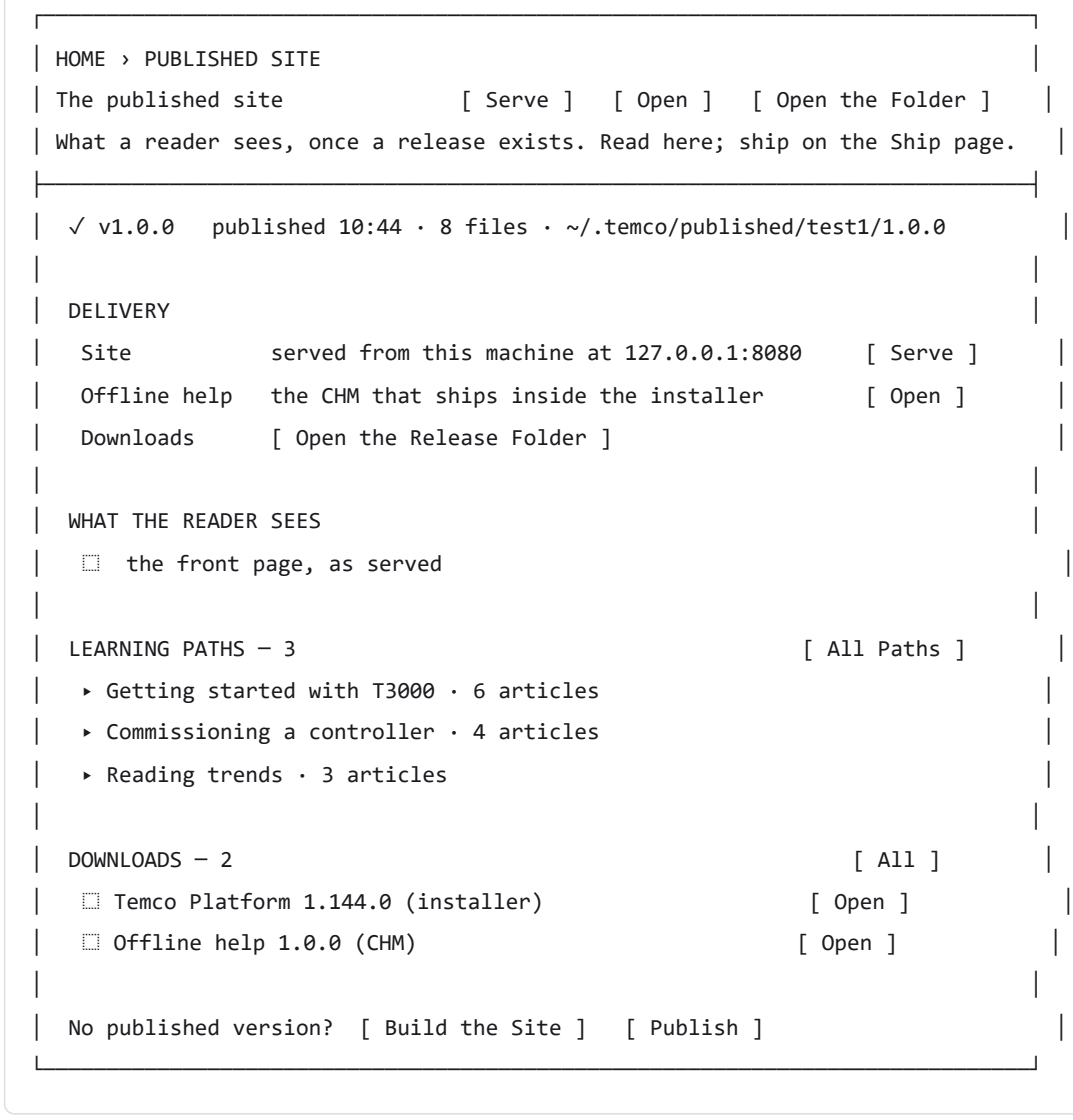
TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ REPOSITORY                                                               │
│ temco-platform          [ Copy the Clone URL ] [ Open on GitHub ] [ Refresh ]│
│ What this repository holds that documentation can be written from.       │
├──────────────────────────────────────────────────────────────────────────┤
│ [ main ]  [ 148 pages ]  [ 2 errors ]  [ read 12 minutes ago ]           │
├──────────────────────────────────────────────────────────────────────────┤
│ [ Download ]  [ Pull ]  [ Remove the Local Copy ]                        │
├──────────────────────────────────────────────────────────────────────────┤
│ CODE TO DRAFT FROM                                [ Filter by path… ]    │
│ 3 ticked · 41 kB of 12,000     [ Code only ]  [ Select All ]  [ Clear ]  │
├──────────────────────────────────────────────────────────────────────────┤
│ ☑ src/commissioning/wiring.ts     12 kB   the order a technician follows  │
│ ☑ src/commissioning/torque.ts      9 kB   limits and units                │
│ ☑ src/safety/interlocks.ts        20 kB   what stops a run                │
│ ☐ src/ui/panel.tsx                61 kB   too large — would be skipped    │
│ ☐ docs/help/registry.json          —      already read as 148 pages       │
│ ☐ package.json                     —      generated, never material       │
├──────────────────────────────────────────────────────────────────────────┤
│ [ Draft a Page from These 3 Files ]    [ Create a Project from These Files ] │
└──────────────────────────────────────────────────────────────────────────┘
```

This is the step the user asked for, made explicit: **code in, page out.** The list is honest about what it will cost — generated, vendored and build output are never offered, and a file over the size budget is shown greyed with the reason rather than silently dropped. What is ticked here is exactly what the draft is given (§4.8a), and the page that results says so.

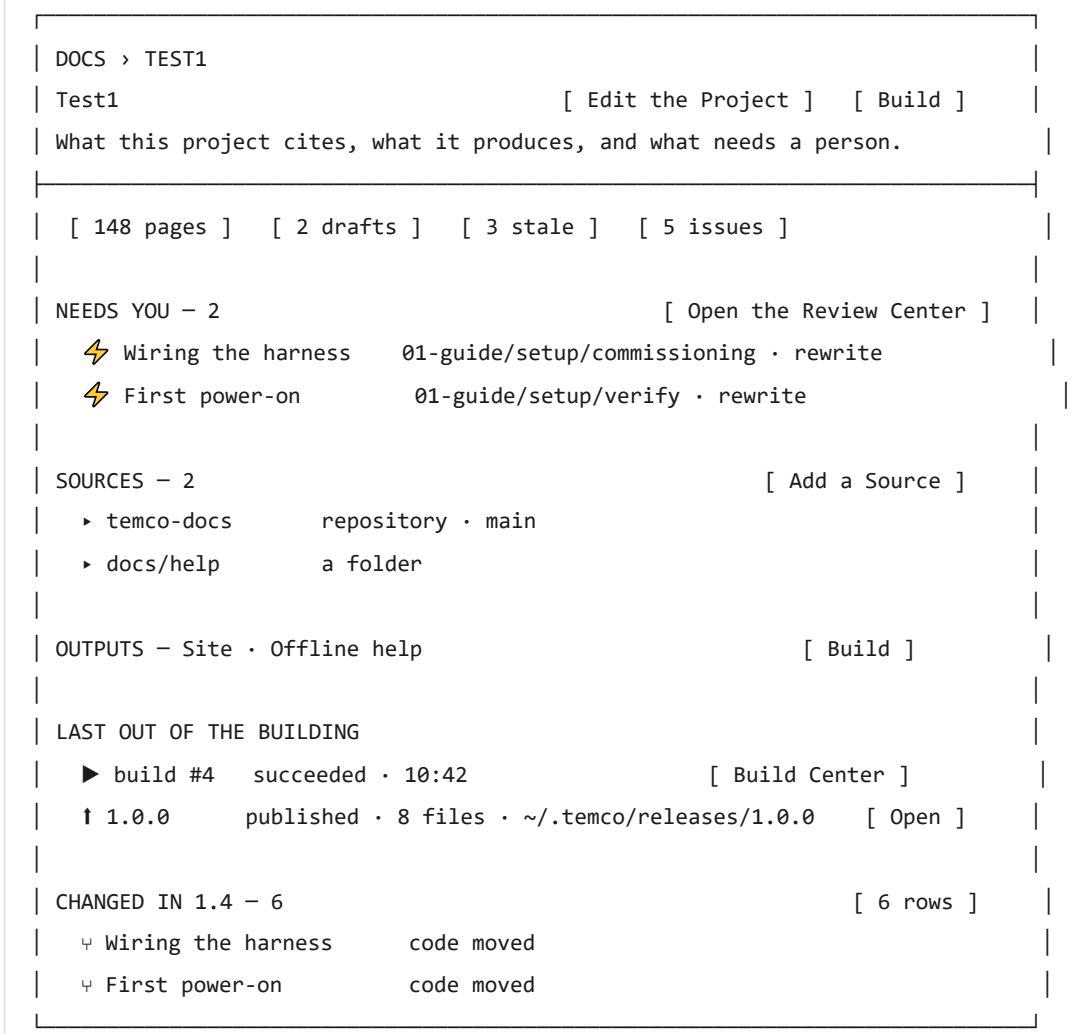
4.6 Home — Published site *(Learn's 5 pages + 2 views → one page; lives in Home)*

TEXT



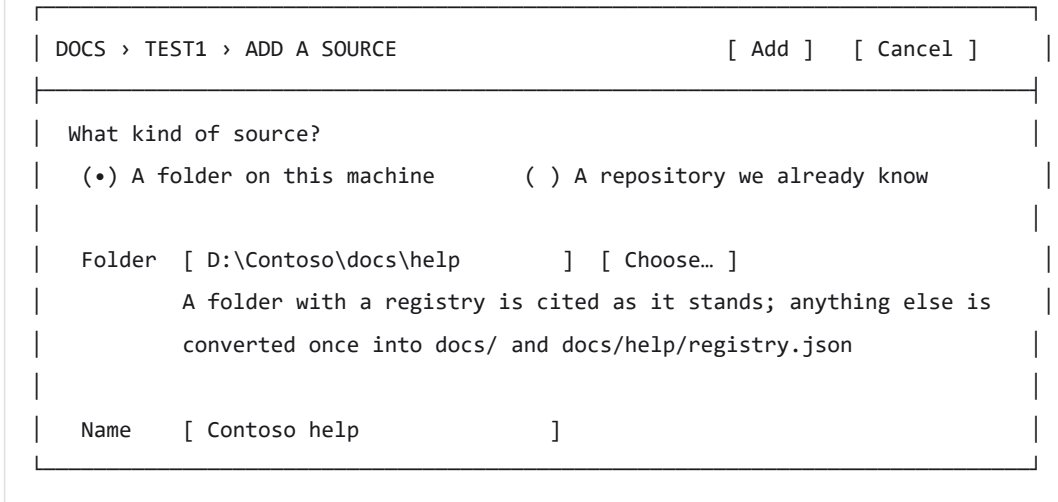
4.7 Docs — Project *(S3, the project dashboard)*

TEXT



4.8 Docs — Add a source *(a form in place of the list)*

TEXT



4.8a Docs — New page *(material first — this is where code becomes prose)*

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ DOCS › TEST1 › NEW PAGE                          [ Draft ]   [ × ]       │
├──────────────────────────────────────────────────────────────────────────┤
│  Goes in      Overview/ ▾       File name   commissioning.md              │
│  Title        Wiring the harness                                         │
├──────────────────────────────────────────────────────────────────────────┤
│  WHAT IS IT WRITTEN FROM?                                                │
│   ( ) Nothing — a blank page                                             │
│   (•) The pages already here    ids, titles and headings · 148 pages     │
│   ( ) A repository's code                                                │
│        temco-platform ▾   [ main ]   [ Tick the files… ]  3 files · 41 kB │
│        ☑ wiring.ts   ☑ torque.ts   ☑ interlocks.ts                       │
├──────────────────────────────────────────────────────────────────────────┤
│  WHAT SHOULD IT SAY?                                                     │
│  [ Explain how the harness is wired, in the order a technician follows. ] │
├──────────────────────────────────────────────────────────────────────────┤
│  12,000 characters of room · 3 files in · 1 file too large, left out     │
└──────────────────────────────────────────────────────────────────────────┘
```

Three things are decided on this page and nowhere else:

- **What the draft may read.** Three choices, and code is only one of them. The budget is stated, not discovered: 12,000 characters, largest files first, and everything left out is named. Generated, vendored and build output are never offered.
- **What the draft is for.** One sentence in the person's words — the note, not a template.
- **What it is.** Where it goes, the file name and the title, so the page lands in the right place rather than being moved afterwards.

`[ Draft ]` writes nothing. It opens the proposal in the PAGE side bar (§4.18); the page exists when **Accept** is pressed, and from that moment it carries one line of provenance — `temco-platform · src/commissioning/wiring.ts + 2 · 3f2a1c` — shown on Properties, on **Review**, and in **Registry**.

4.9 Docs — the page editor (Temco Editor)

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ ↶ ↷ │ Style ▾ │ Lists ▾ │ B I U S ⌨ ▮ │ ❝ ⌨ – ▦ │ ⬚ │ ··· │     🔍 ☰ 🕮      │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│                    Commissioning                                         │
│                    ─────────────────────────                             │
│                    Wire the harness as shown.                            │
│                                                                          │
│                    ⬚  the harness   (drag a handle to resize)            │
│                                                                          │
│                    | Step | Done |                                        │
│                    | ---- | ---- |                                        │
│                    | Wire | Yes  |                                        │
│                                                                          │
│                    Then read [ the next page ](./verify.md).             │
│                                                                          │
├──────────────────────────────────────────────────────────────────────────┤
│ 1,204 words · 6 min read · saved 10:44 · Markdown · UTF-8 · LF           │
└──────────────────────────────────────────────────────────────────────────┘
```

4.10 Docs — Preview / Diff *(beside the editor, same tab group)*

TEXT

```
┌──────────────────────────────┬──────────────────────────────────────────┐
│ the page you are writing     │ PREVIEW                  [ Diff ]  [ × ] │
│ (Temco Editor)               │ the published layout, from the same renderer │
├──────────────────────────────┼──────────────────────────────────────────┤
│ # Commissioning              │  Commissioning                           │
│ Wire the harness as shown.   │  ─────────────                           │
│ | Step | Done |              │  Wire the harness as shown.              │
└──────────────────────────────┴──────────────────────────────────────────┘
```

4.11 Docs — the grid editor (CSV / TSV)

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ points.csv                                              saved · 42 rows  │
├─────┬──────────┬──────────┬──────────────┬──────────────────────────────┤
│  #  │ point    │ units    │ range        │ description                  │
├─────┼──────────┼──────────┼──────────────┼──────────────────────────────┤
│  1  │ AI-1     │ °C       │ -40 … 120    │ supply air temperature       │
│  2  │ AO-7     │ V        │ 0 … 10       │ valve position               │
└─────┴──────────┴──────────┴──────────────┴──────────────────────────────┘
```

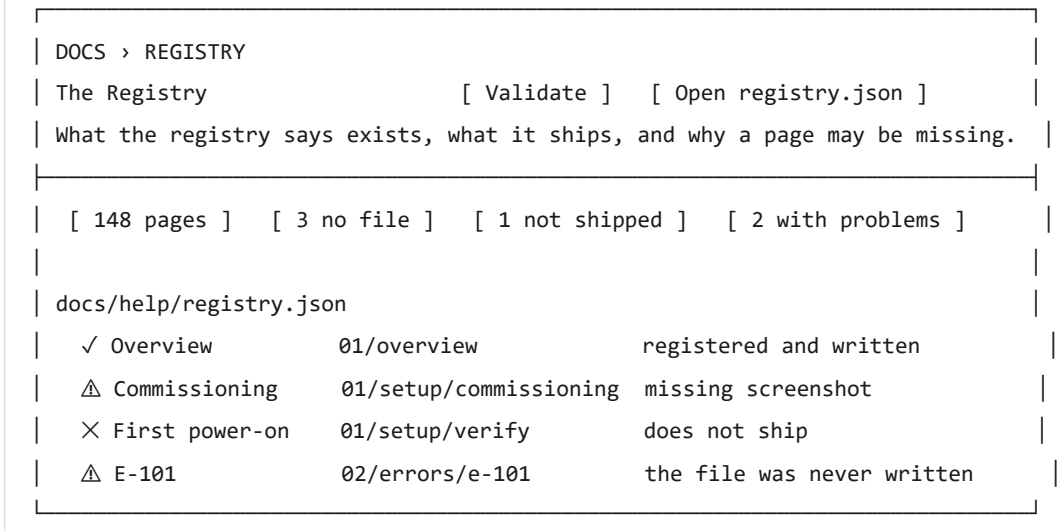
4.12 Docs — Review *(absorbs the Needs You view, the Review Center and the Needs You page)*

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ DOCS › REVIEW                                                            │
│ Review                                          [ Draft a Change ]       │
│ Drafts the local model wrote that no person has settled. One at a time.  │
├──────────────────────────┬───────────────────────────────────────────────┤
│ DRAFTS — 2               │  Wiring the harness                           │
│   ▸ Wiring the harness ◀ │  01-guide/setup/commissioning · rewrite       │
│   ▸ First power-on       │                                               │
│                          │  ┌ the draft ──────────────────────────┐      │
│ CAPTURES — 1             │  │ Add a torque note to the table.     │      │
│   ⬚ 2026-10-27-1042.png ◀ │  └─────────────────────────────────────┘      │
│                          │  ┌ the page today ─────────────────────┐      │
│                          │  │ Wire the harness as shown.          │      │
│                          │  └─────────────────────────────────────┘      │
│                          │  [ Accept ]  [ Discard ]  [ Attach Capture ]  │
└──────────────────────────┴───────────────────────────────────────────────┘
```

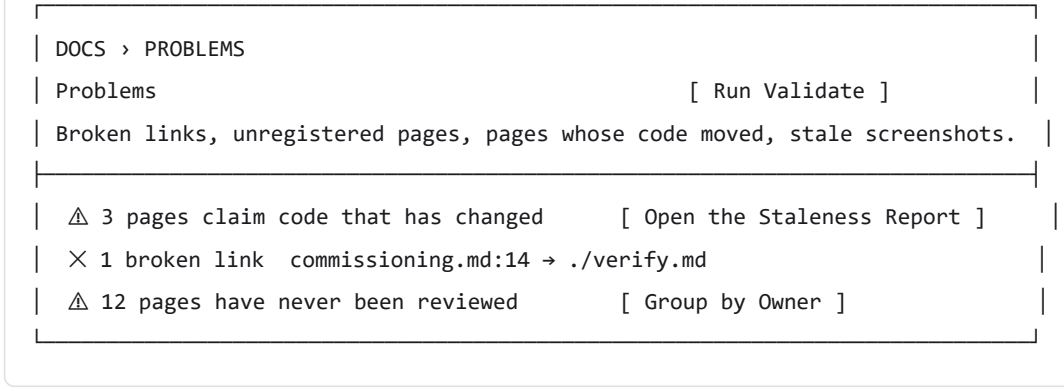
4.13 Docs — Registry *("why is this page not in the site?")*

TEXT



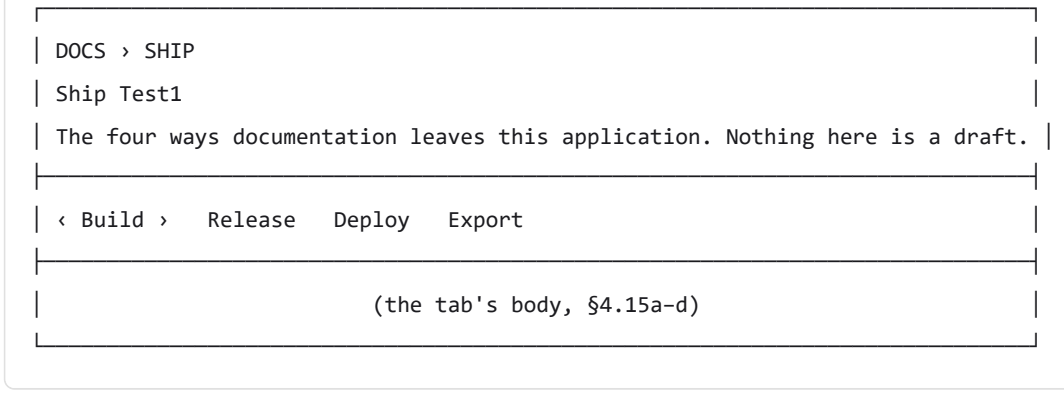
4.14 Docs — Problems *(validate + staleness in one list)*

TEXT



4.15 Ship — one page, four tabs

TEXT



4.15a ‹ Build ›

TEXT

```
│  [ Build Site ]   [ Build Offline Help ]   [ Refresh ]                        │
│                                                                               │
│  ✓ Resolve   148 pages to publish, 148 addresses                              │
│  ✓ Render    148 pages written beside their media/                            │
│  ✓ Assets    index.html, assets/site.css                                      │
│  ✓ Sitemap   148 addresses under https://owner.github.io/temco/              │
│                                                                               │
│  BUILDS — 4                                                                   │
│   #4  succeeded · 10:42 · 8 files                    [ Open dist/site ]       │
│   #3  failed · 10:30 · no artifact                   [ Open the Output ]      │
│                                                                               │
│  ARTIFACTS — 8                                                                │
│   index.html · site.css · 6 pages                                 [ Open ]    │
```

4.15b Release

TEXT

```
│  [ ⬆ Publish ]        version 1.0.0 → ~/.temco/published/test1/1.0.0          │
│      a confirmation names version · destination · file count before copying   │
│                                                                               │
│  PUBLISHED VERSIONS — 1                                                       │
│   1.0.0   build #4   published 10:44 · 8 files         [ Serve ]   [ Open ]   │
│                                                                               │
│  WHAT PUBLISHING DOES                                                         │
│   · copies the files a build wrote, into a folder the platform owns           │
│   · records version, build, destination and date                              │
│   · never rewrites a version that already exists                              │
```

4.15c Deploy

TEXT

```
│  WHERE IT PUBLISHES — owner/temco                                             │
│                                                                               │
│   🌐 https://owner.github.io/temco/   pushed to the gh-pages branch           │
│   ⚙ One time only                     Settings → Pages → branch gh-pages, /   │
│   ⚠ Never forced                      a branch with unknown commits is        │
│                                      reported, never overwritten              │
│                                       [ Publish to GitHub Pages ]             │
│                                                                               │
│  No GitHub remote?  this folder is not a git repository with an origin, so    │
│  there is nowhere to publish to until one is added.                           │
```

4.15d Export

TEXT

```
│  [ 148 pages ]   [ 4 shapes ]   [ every file lands in dist/ ]                 │
│                                                                              │
│  THE SITE, AS A ZIP          index.html, site.css, pages, media/   [ Export ]  │
│  ONE PAGE, AS ONE FILE       a .html with the pictures inside      [ Export ]  │
│  ONE PAGE, AS WORD           a .docx a reviewer can mark up        [ Export ]  │
│  THE PAGES, AS MARKDOWN      .md + media/ + registry.json          [ Export ]  │
```

4.16 AI — chat, in the right-hand side bar

TEXT

```
┌──────────────────────────────┐
│ AI                       ⋯  │
├──────────────────────────────┤
│ you: why is commissioning    │
│      missing a screenshot?   │
│                              │
│ ai:  The page refers to      │
│      media/wiring.png, which │
│      is not in media/.       │
│      ┌──────────────────┐    │
│      │ 01-guide/setup/… │    │  ← citations, clickable
│      │ registry.json:112 │   │
│      └──────────────────┘    │
│      [ Take me there ]       │
│      [ Draft the fix ]       │
├──────────────────────────────┤
│ [ Ask about this page…   ] ⏎ │
└──────────────────────────────┘
       ⋯ : Actions · Citations · Models · Knowledge · Configure
```

4.17 AI — the configuration page *(6 pages → one page, four tabs)*

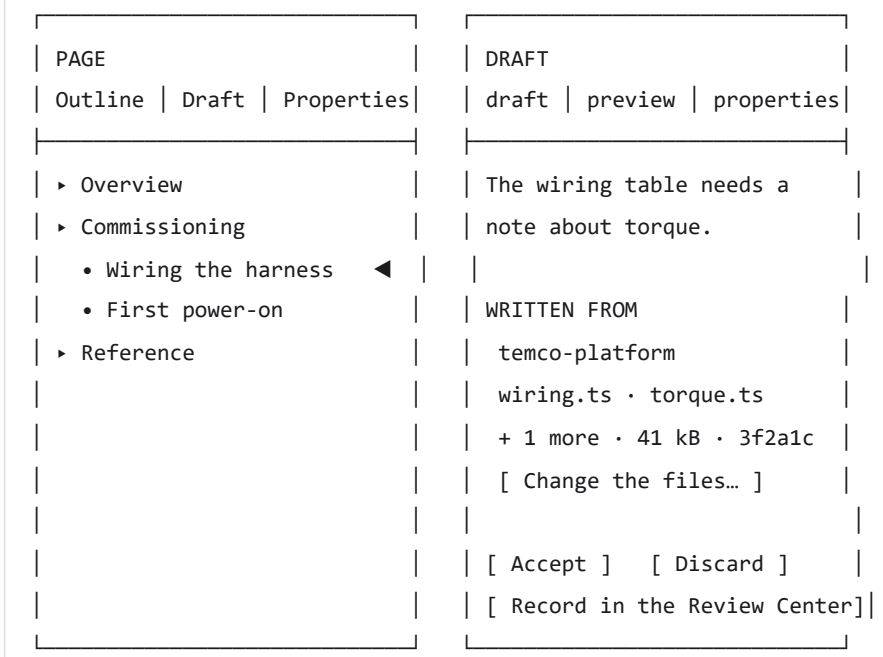
TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ AI › SETTINGS                                                            │
│ AI Models                        [ local · qwen2.5-3b ]   [ Open the Folder ] │
│ What the model may do, what it read, and what runs on this machine.      │
├──────────────────────────────────────────────────────────────────────────┤
│ ‹ Actions ›   Citations   Models   Knowledge                             │
├──────────────────────────────────────────────────────────────────────────┤
│  WHAT IT CAN DO                                                          │
│   ✓ Summarise a page            drafts, as a diff         [ Run ]        │
│   ✓ Draft a change to a page    drafts, as a diff         [ Run ]        │
│   ✓ Answer about a page         the AI bar, with citations [ Run ]       │
│   ✕ Edit a page directly        never                      blocked       │
│                                                                          │
│  WHERE OUTPUT GOES                                                       │
│   ▸ Drafts        Review Center, as a diff            ✓ never silent     │
│   ▸ Answers       the AI bar, with citations          ✓ never silent     │
│   ▸ Suggestions   the Gaps list of an area            ✓ never silent     │
│                                                                          │
│  GUARD RAILS — fixed on purpose, not settings                            │
│   ✓ Never write a page without a draft                                   │
│   ✓ Always cite the page or source used                                  │
│   ✓ Refuse when the local model is not loaded                            │
│   ✓ Keep prompts and answers out of the repository                       │
└──────────────────────────────────────────────────────────────────────────┘


‹ Citations ›  every answer's sources, kept with the answer that used it
‹ Models ›     2 downloaded, 1 loaded · Runtime llama.cpp · CPU, 6 threads ·
               Storage ~/Temco/models · Privacy: no network once loaded
‹ Knowledge ›  what the index holds — Chunks · Areas · Dimensions · Index age,
               and [ Rebuild Index ]
```

4.18 The PAGE side bar (right): Outline · Draft · Properties

TEXT

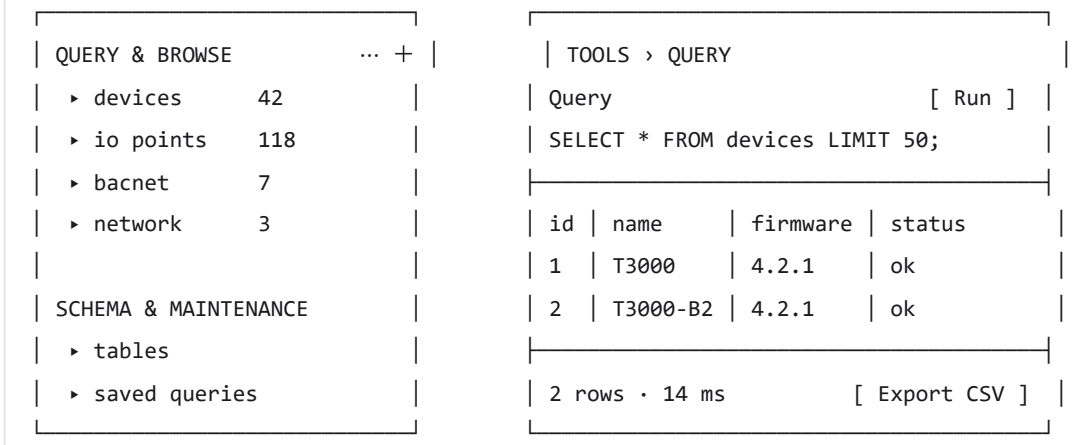


`WRITTEN FROM` is the answer to "*where did this text come from?*" — one line, every time: which repository, which files, which commit, and how much of the budget they took. It is on the draft, on the finished page's `Properties`, on **Review**, and on every row of **Registry**, so nobody has to trust a memory. `[ Change the files… ]` reopens the ticker (§4.5) without asking the question again.

`Properties` is the page's registry values — title, owner, audience, ship, and that same `written from` line — edited in place.

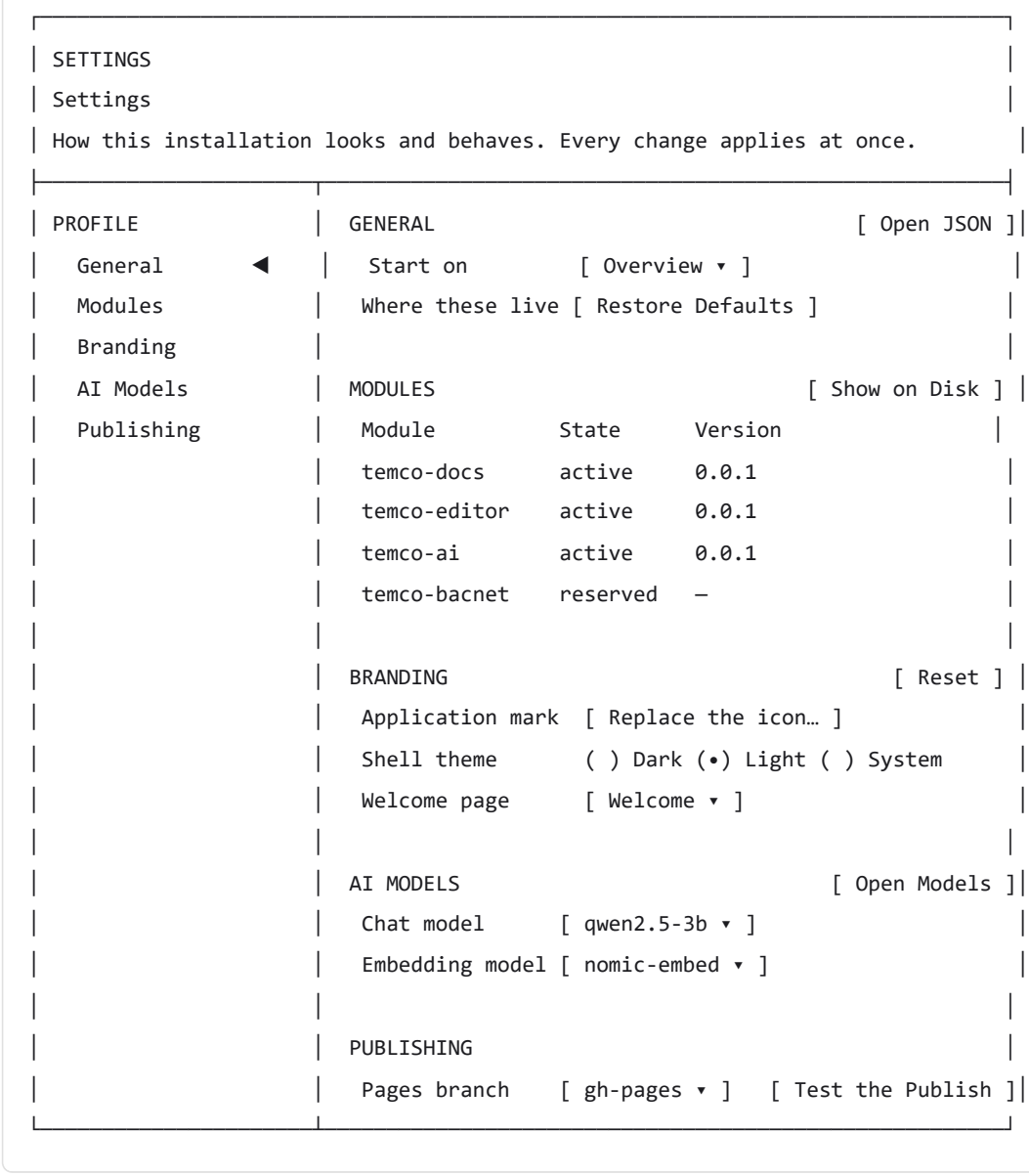
4.19 Tools — SQL and diagnostics *(unchanged in shape; drawn so the whole window is accounted for)*

TEXT



4.20 Settings — one page, five sections, left rail

TEXT



4.21 Settings — Help *(6 pages + 2 views → one page, six sections)*

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ SETTINGS › HELP                                                          │
│ Help                                        [ Report an Issue ]          │
│ What this build can do, where its logs are, and how to report a problem.  │
├──────────────────────┬───────────────────────────────────────────────────┤
│ WELCOME              │ START HERE                                        │
│   Start here    ◀    │  6 ways in · 3 walkthroughs                       │
│   Keyboard           │  [ Keyboard Shortcuts ]  [ Release Notes ]        │
│   Logs               │                                                   │
│   Report an issue    │ THIS WINDOW — 3 facts                             │
│   About              │  Version 1.144.0 · commit 72a5048 · 9 modules active │
│                      │                                                   │
│                      │ KEYBOARD SHORTCUTS              [ Command Palette ] │
│                      │  Ctrl+Shift+P   the command palette               │
│                      │  Ctrl+Alt+P     preview the page                  │
│                      │  Ctrl+Alt+D     diff with HEAD                    │
│                      │                                                   │
│                      │ LOGS                                 [ Open Log Folder ]│
│                      │  main.log        42 KB · last 9 lines        [ Open ]│
│                      │  renderer.log    12 KB · last 9 lines        [ Open ]│
│                      │                                                   │
│                      │ REPORT AN ISSUE                                   │
│                      │  What goes with it — 4 of 6              [ Copy ]  │
│                      │  ✓ version · ✓ modules · ☐ logs · ☐ settings snapshot │
│                      │                                                   │
│                      │ ABOUT                                 [ Check Updates]│
│                      │  Version · Modules · Active modules               │
└──────────────────────┴───────────────────────────────────────────────────┘
```

---

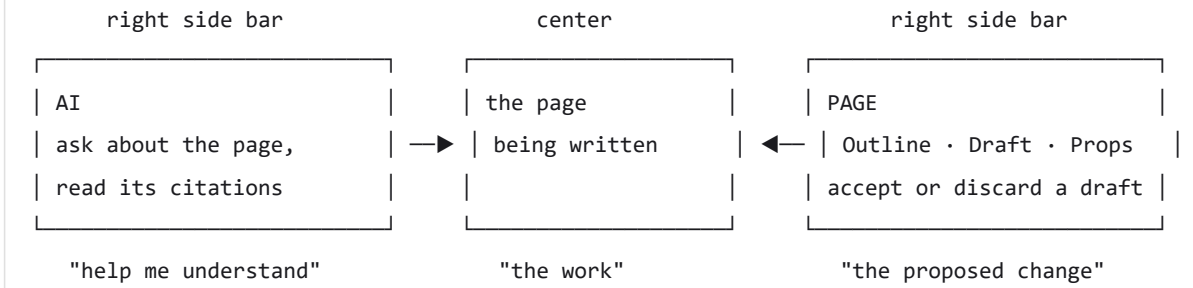
5. How code, AI, the page and the draft fit together

Four things, four jobs. Only one of them is a place you navigate to; the rest happen where you already are:

TEXT

```
  Repositories (§4.4, §4.5)      the shelf: Temco code, read or ticked
        │  tick files
        ▼
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ MATERIAL §4.8a   │   │ DRAFT §4.18      │   │ PAGE §4.9        │
│ 41 kB of 12,000  │──▶│ + WRITTEN FROM   │──▶│ *.md in Pages    │
└──────────────────┘   └──────────────────┘   └──────────────────┘
                                          AI proposes; Accept writes
```

TEXT



A draft arrives in the PAGE side bar, is settled there, and also appears on **Review** while it waits. Nothing is written until **Accept**; no draft is applied without a person. A draft that came from code never loses the thread back to it — **Repo.** is one click from the draft, and one page of the same code file is one click from the repository (§4.5).

---

6. Where every action lives

you want to…	you go to	not to
start a project	Home ▸ Projects ▸ **+ New Project**	a second "Projects Overview" page
read code from a repository	Home ▸ Repositories ▸ **[Read]**	the raw registry.json
write a page by hand	Docs ▸ Pages ▸ double-click, or `···` ▸ **New Page…** ▸ *Nothing*	a section called *Needs You*
write a page from code	Docs ▸ `···` ▸ **New Page…** ▸ *A repository's code* ▸ tick files	a separate *Drafting* page or container
settle a draft	the `1 owed` link, Ship ▸ Review, or the PAGE side bar	three different lists
build / release / deploy / export	`[Build]` `[⇪]` in the Docs header, or Ship	four separate pages
see why a page is not in the site	Docs ▸ `···` ▸ **Registry**	the raw registry.json
read what was published	Home ▸ Published site	a *Site* view
change a setting	Settings ▸ Settings (left rail)	a container with four views
find a shortcut or the logs	Settings ▸ Help	a *Help* container with two views
ask about a page	the AI side bar (right)	an activity-bar icon

Every command id is unchanged, so the palette still reaches all 250.

---

7. What is removed, and what absorbs it

removed	absorbed by
Project side-bar section	the one-line header in Docs
Needs You view + Needs You page	**Review** page + the `1 owed` link
Sources view	the **Project** page (already lists sources with branches)
Publish view (3 action rows)	`[Build]` `[⇪]` in the header, and `⋯ ▸ Build Center / Releases / Exports`
Repositories view	**moves** from Docs to Home (still a view; the page stays for the wide work)
Overview view	**Home** ▸ **Recent activity** + the project cards
Site / *Delivery* views	**Home** ▸ **Published site** page
Profile / *Platform* / *Learn* / *Support* views	**Settings** page + **Help** page
AI activity-bar container	the AI secondary side bar (§4.16)
Build Center · Releases · Exports pages	**Ship** tabs (bodies moved, not rewritten)
5 AI pages	**AI** ▸ **Settings** with four tabs (§4.17)
4 registry commands	**Registry** page (the raw file stays one click away)

19 side-bar views → **7**. ~44 pages → **17**. Nothing is deleted before its content exists elsewhere.

---

8. Order of work

1. **Docs panel** — 5 regions → header + tree + `⋯`. No page touched; reversible.
2. **Home** — three lists (Projects · Repositories · Recent) and their pages, including Published site (§4.3, §4.4, §4.6). The *Repositories* view changes container only; no code moves. On the page, `[ Draft ]` is added beside `[ Read ]`.
3. **Drafting loop** — the ticker on the repository page (§4.5), the material picker on *New Page* (§4.8a) and `WRITTEN FROM` on the draft (§4.18). The parts exist today (`newPagePage.ts` , `codeFiles` / `readMaterials`); this step names them in the UI and records the line on the accepted page.
4. **Settings + Help** — two pages with rails (§4.20, §4.21); four views retire.
5. **Ship** — one page, four tabs (§4.15); the three bodies move as they are.
6. **Quality** — Review and Registry (§4.12, §4.13); retire the *Needs You* view and four registry commands.
7. **AI** — the side bar and one settings page (§4.16, §4.17); the activity-bar icon goes.

After each step `npm run check-temco` refuses a view inside a container that no longer exists, so a half-finished step fails the build instead of showing an empty panel.

---

9. Decisions taken here

- **Published site lives in Home** (§4.6) — decided.

- **Four activity-bar icons, kept as designed** — Home · Docs · Tools · Settings. The Settings container keeps its own icon but is **drawn as sliders instead of a gear**, because VS Code's own *Manage* gear sits at the bottom of the same column; two gears is one meaning too many. Id unchanged, so saved layout survives.
- **Tools stays in the activity bar** with both views untouched, because 94 commands and two of its modules have no manifest yet; hiding it would make existing work unreachable.
- **Settings and Help are pages, not containers** (§4.20, §4.21) — decided.
- **Repositories stays a view, under Home** (§2.5, §4.4) — it is a list you work in, so it keeps the free filter, arrow keys and collapse of a native view; its page adds the two per-row acts. **It is the shelf of Temco code the documentation is written from**, not a library of finished docs: `[ Read ]` cites what a repository already holds, `[ Draft ]` turns ticked code files into material for a new page (§4.5 → §4.8a → §4.18).
- **Drafting always starts on one page** — *New Page* (§4.8a) — where the material is chosen before a word is proposed. The budget (12,000 characters) and everything left out are stated on that page, and the accepted page keeps a one-line `written from` record (repository · files · commit) wherever it is shown. Provenance is part of the page, not a note about it.
- **Ship replaces Build Center, Releases and Exports in one move** (§4.15) — decided. The three bodies move into the four tabs unchanged; the three commands stay registered so anything bound to them keeps working, and each opens Ship on its own tab.

Nothing in this document has been implemented. Step 1 is the Docs panel, and it waits for a go-ahead.

---

10. Look and feel

> **Superseded by a proposal awaiting confirmation:** welcome-v3-design.md — Fluent 2 tokens with Azure's purchase-options anatomy (three option cards, a brand-tinted wash, numbers, one callout, questions). Read that first; the section below describes the version now in the code, which v3 replaces.

The look is the one a developer reads every day (platform.openai.com/docs): **a big quiet title, one line of grey, pill buttons, and content held in soft panels.** What it replaced was a coloured band with a gradient and a glow; it made the first screen shout and left every page under it looking unfinished, which is exactly what was wrong with it.

Implemented. The rules are in `extensions/temco-ui/media/page.css` (the "document page" layer), the markup in `extensions/temco-ui/src/components.ts`, and Welcome · Projects · Overview · Documents use it today.

TEXT

```
Documentation
Write the pages of a product, draft them from the code that runs it,
and publish a site a customer can open.

[ Search ]  Find a page, a file or a command…            Ctrl P
( Create a project )   ( Read a source )

┌──────────────────────────────────────────────────────────────────────────┐
│ Start with a project                │ TypeScript                         │
│ A project holds the pages, the      │  1  // a page, drafted from the code │
│ sources they are read from, and     │  2  const page = await temco.draft({ │
│ the artifacts a build produces.     │  3    from: 'src/wiring.ts',       │
│                                     │  4    into: 'guide/wiring.md',     │
│ ( New project )   ( Add a source ) │  5  });                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────┐
│ Draft a page from the repository it documents    ( Read a Source )       │
└──────────────────────────────────────────────────────────────────────────┘

Build paths                                          [ All Commands ]
┌────────────┐ ┌────────────┐ ┌────────────┐ ┌────────────┐
│ ▣ Create… │ │ ▣ Read…    │ │ ▣ Open…    │ │ ▣ Review… │
└────────────┘ └────────────┘ └────────────┘ └────────────┘

                                                    ( ✦ Ask AI )

a 34px title, a 15px grey line, pill buttons, a soft panel, a code card,
a banner, and four dense tiles
```

10.1 Near-monochrome, and why

surface	colour	why
the page	the theme's own background	a page is not a poster
the panel and the banner	`--bg-inset` / `--bg-card`	one step in from the page, no fill colours
borders	`--border`	a hairline, never a shadow
the one filled button	**the theme's foreground** — black pill in a light theme, white in a dark one	it is the thing to press, and it survives both themes without a second rule
the accent (`--az-blue` `#0078d4`)	a link, the 3px rule on the leading figure, a focus ring	three places on a page, or it stops meaning anything
syntax in the code card	four tones mixed against the page	reads in a dark theme, and is the only saturated colour on the screen

No gradient, no glow, no brand band. The check fails the build if a gradient reappears in this layer.

10.2 Type does the work

role	size / weight	note
page title	34px / 600, `-0.028em`	27px under 900px wide
panel heading	20px / 600	the `h2` a section is told apart by
body	15px / 1.7 for the page's line, `--fs-sm` for supporting text	still follows the workbench font
code	12.5px / 1.75, tabular line numbers	numbers drawn, never written into the text
small caps, letter-spacing, chips	**none**	the one thing the previous version over-used

10.3 The eight components, and the gate that holds them to the stylesheet

`hero()` (the title block: eyebrow · `h1` · line · field · buttons · figures) · `panel()` (soft box + the code card beside it) · `code()` (numbered, four tones) · `banner()` (one sentence + one action) · `kpi()` · `section()` · `cards(items, { dense })` · `floatAction()` . All are published by `temco-ui` and reached through each module's `ui.ts` , the same way `page()` and `icon()` are.

`scripts/check-components.mts` runs in `npm run compile` (`check-temco`) and fails the build when:

- a class the components emit has **no rule** in `page.css` ;
- a value a person typed reaches the document **unescaped**;
- an option left out still renders an empty element;
- **the code card edits the sample** — the gate parses the rendered card back into plain text and compares it with the source lines, because line numbers and colour are drawn, not written;

- a surface stops taking its background from the theme, or the filled button stops being the theme's foreground;
- a **gradient** appears in the page layer.

10.4 Where it is used now

page	title block	panel	banner	figures	sections
Welcome (`temcoWorkbench.help.welcome`)	yes — with the field and two buttons	yes — "Start with a project" + the drafting sample	yes — the draft-from-repository line	—	*Build paths* (4 tiles) · *Pick up where you left off* · *Guides* (3 walks) · floating **Ask AI**
Projects (`temcoDocs.projects`)	yes — with two buttons and four figures	—	—	in the title block	*All Projects*
Overview (`temcoWorkbench.platformView`)	yes — two buttons, four figures	—	—	in the title block	*Documentation Pipeline*, feeds, cards
Documents (`temcoDocs.documents`)	— (page head kept)	—	—	yes — pages · read · missing · problems	—

Next surfaces to take it: the AI dashboard, the Tools pages, the Ship tabs.

10.5 Translating pages, and translating the interface

Two different jobs, kept apart on purpose.

Pages — `Temco Docs: Translate This Page…` turns the page in front into a sibling file (`setup.md` → `setup.de.md`) using an Azure AI Translator resource. The service is given *sentences*, never the file: `translate.ts` cuts the page into segments whose concatenation reproduces it exactly, sends only the prose, and puts everything back — code fences, inline code, table rules, link targets, image paths, front matter, line markers. That invariant is a build gate, not a comment. `scripts/check-translate.mts` proves it with a fake transport that echoes its input: the page comes back byte for byte, a fenced sample never appears in what was sent, and each failure a person can meet (401, 403, 429, unreachable, a short answer) is a sentence about what to do. Proved by negative test: removing the front-matter rule fails the gate by name.

The key is a secret in the OS keychain (`context.secrets`); the endpoint and region are settings — where the resource lives is configuration, who may use it is not.

Interface — strings on the new surfaces go through `vscode.l10n.t()` , so a bundle can be dropped in without touching code. The rest of the application is still plain English; the rule from here is that a string a person reads is marked.

The front page — the Azure portal's palette and scale

Status: implemented, corrected after review. The page is the **platform description** — titled `Temco Platform`, with no breadcrumb — in the **Azure portal's own palette and type scale** (24px page title, 18px section, 14px card, 13px text; `#0078d4` blue, `#1f2327` ink, `#faf9f8` page, `#edebe9` hairline), not the marketing scale of §2. The rules are in `extensions/temco-ui/media/page.css`; the markup in `extensions/temco-ui/src/components.ts`.

---

0. What went wrong twice, so it does not go wrong a third time

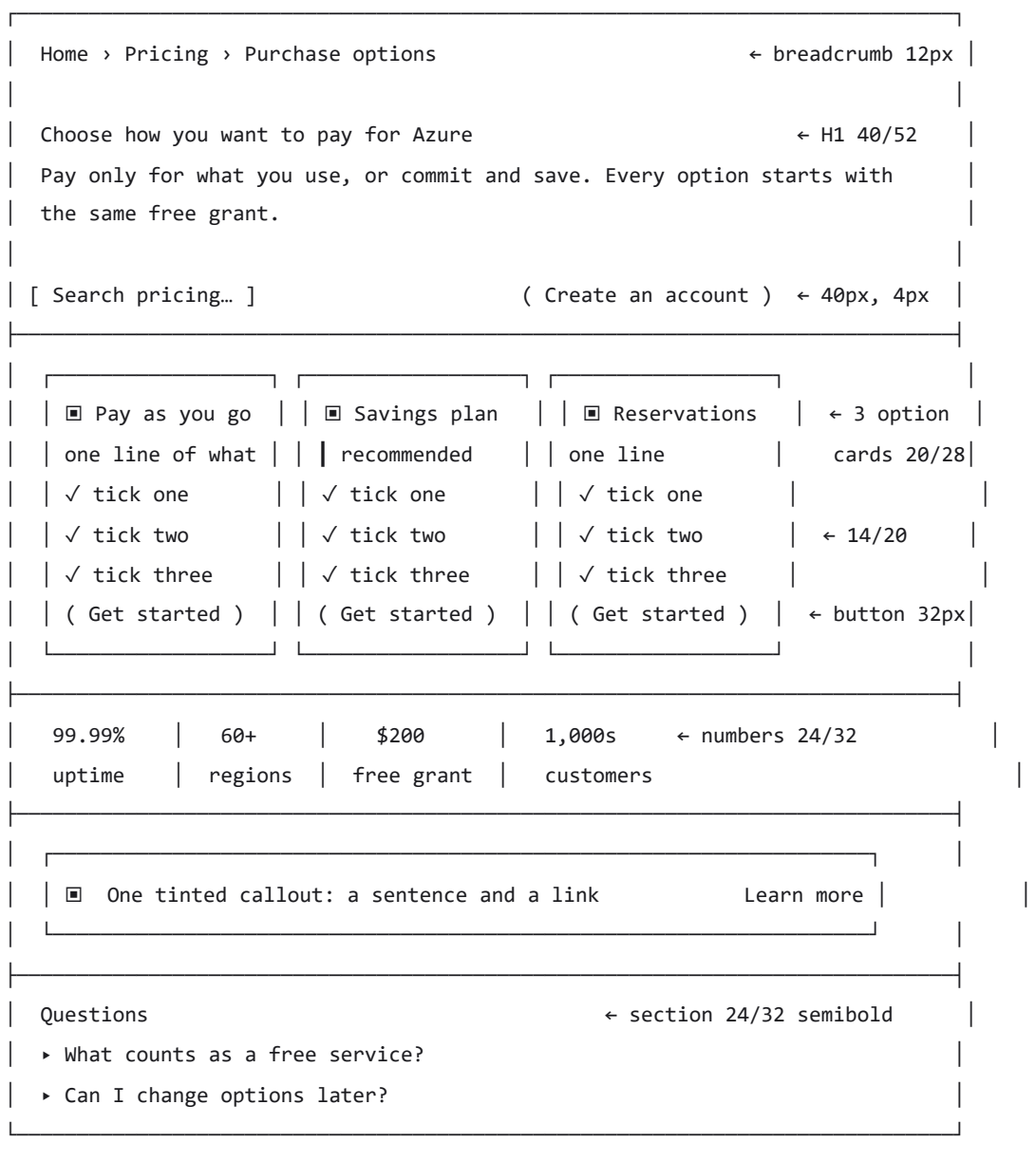
attempt	what it was	why it was wrong
v1	a navy→blue gradient band with a cyan glow and a grid	that is a marketing *poster*. Nothing on the Azure pricing page looks like it, and it made the first screen shout
v2	the OpenAI documentation page from the screenshot	calm and readable, but it turned the front page into a *reading* page and dropped the one thing the Azure page is built on: **three options side by side, each with ticks and a button**

v3 is the Azure account / purchase-options page read as five bands, built out of Microsoft's own Fluent 2 numbers (taken from the Fluent 2 design-system stylesheet, not invented): a tinted wash behind the title, three option cards, a numbers strip, one tinted callout, and questions at the bottom.

---

1. The reference, as five bands

TEXT



What makes it *that* page, in order of importance:

1. **The wash.** A soft brand tint behind the title, fading into the page — not a saturated band, not a photo.
2. **Three cards in a row, the same height, each ending in a button.** The middle one is marked, not coloured in.
3. **Ticks, not paragraphs.** Three short lines per card.
4. **Numbers in a row** under the cards.
5. **One tinted callout** in the whole page.
6. **Questions at the bottom**, in rows, not in a dialog.

---

2. The tokens — Microsoft's own numbers

Taken from the Fluent 2 design-system stylesheet (`--fontSizeHero800` , `--colorNeutralBackground1` , …), so the page is the system rather than an imitation of it.

Type

role	token	size / line	weight
page title (H1)	Hero900	**40 / 52**	600
panel & section title (H2)	Base600	**24 / 32**	600
card title (H3)	Base500	**20 / 28**	600
intro paragraph	Base400	**16 / 22**	400
body & card copy	Base300	**14 / 20**	400
caption, breadcrumb, label	Base200	**12 / 16**	400
code	—	**12.5 / 20**	400 mono

Font family: the workbench font in the app (as now), with **Segoe UI** named first in the fallback chain.

Space, radius, elevation

	values
spacing	2 · 4 · 6 · 8 · 10 · 12 · 16 · 20 · 24 · 32 · 40 · 48
radius	2 · 4 (controls) · 8 (cards) · 12 (the one big panel) · pill
shadow	`2` : `0 0 2px rgba(0,0,0,.12), 0 1px 2px rgba(0,0,0,.14)` · `4` · `8` · `16` — the reference uses **4** on a resting card and **8** on hover

Colour — light

role	value	used for
brand	`#0f6cbd`	the primary button, links, the recommended card's stroke
brand hover / pressed	`#115ea3` / `#0c3b5e`	states
brand tint	`#ebf3fc`	the wash behind the title, the callout, the recommended card's fill
text 1 / 2 / 3	`#242424` / `#616161` / `#707070`	headings / body / captions
surface page / card / subtle	`#fafafa` / `#ffffff` / `#f5f5f5`	page / cards / inset bars
stroke	`#d1d1d1`	hairlines; `#616161` when it must be seen
success / warning / danger	`#0e700e` / `#bc4b09` / `#d13438`	state only, never decoration

Colour — dark (Windows dark)

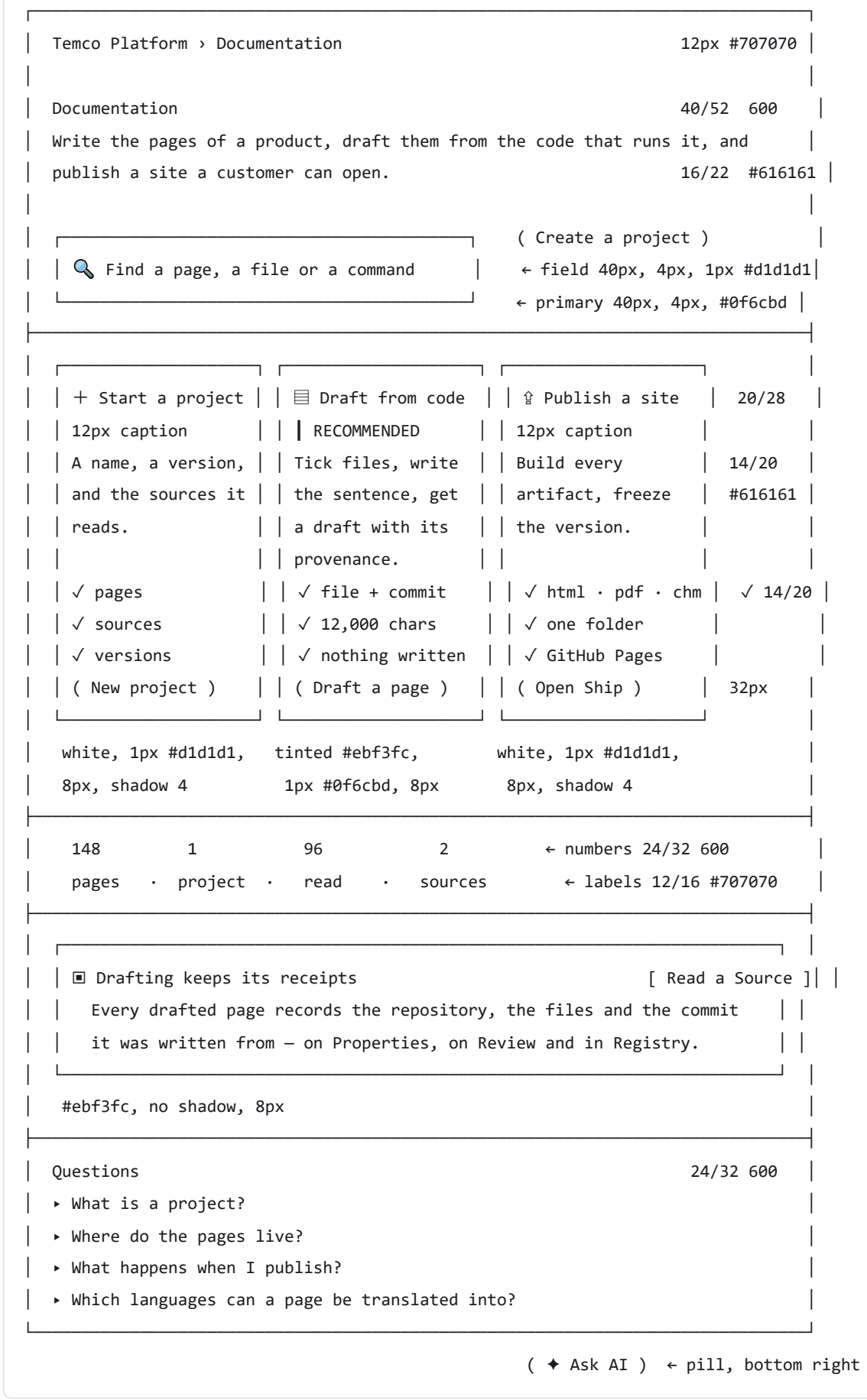
role	value
brand / hover / tint	`#115ea3` / `#479ef5` / `#082338`
text 1 / 2 / 3	`#ffffff` / `#d6d6d6` / `#adadad`
surface page / card / subtle	`#1f1f1f` / `#292929` / `#333333`
stroke	`#666666`

What the theme owns, and what Azure owns

	owner	why
page background, card background, strokes, text	**the VS Code theme** (`--vscode-*`), with the Fluent value as the fallback	a person's editor theme must not be argued with
the brand blue, its tint, the type ramp, spacing, radius, shadows, the wash	**Fluent 2 / Azure**, fixed	this is the identity; it is the same in both themes
the wash	brand tint → transparent, 160px tall, behind the title only	it is the one thing that makes the top of the page read as Azure

3. Welcome, drawn (light, in the app's window)

TEXT



Notes on the drawing:

- **The wash** sits behind the title band only, 160px tall, `#ebf3fc → transparent`, and it is the *only* decoration on the page.
- **The three cards are the page.** They are the Azure purchase-option cards with our three acts.
- The **middle card is recommended** — filled with the tint and stroked with brand, because drafting from code is the thing this product does that others do not.
- **No panel with a code sample** on the front page: the code moved to where it belongs (the Drafting page, `§4.8a`), and the front page stopped competing with it. This is the change I most want confirmed.

---

4. Dark theme — the same page

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│  Temco Platform › Documentation                          12px #adadad    │
│  Documentation                                           40/52  #fff     │
│  Write the pages of a product…                           16/22  #d6d6d6  │
│  ┌──────────────────────────────────────┐    ( Create a project )         │
│  └──────────────────────────────────────┘    ← #115ea3, white text       │
├──────────────────────────────────────────────────────────────────────────┤
│  ┌────────────────────┐ ┌────────────────────┐ ┌────────────────────┐      │
│  │ #292929  1px #666  │ │ #082338  1px #479ef5│ │ #292929  1px #666  │      │
│  └────────────────────┘ └────────────────────┘ └────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────┘
   the wash is #082338 → transparent: the same shape, eleven shades darker
```

---

5. The option card, up close

TEXT

```
   resting                  hover                    recommended
┌────────────────────┐   ┌────────────────────┐   ┌────────────────────┐
│ #ffffff            │   │ #ffffff            │   │ #ebf3fc            │
│ 1px #d1d1d1        │   │ 1px #0f6cbd        │   │ 1px #0f6cbd        │
│ shadow 4           │   │ shadow 8           │   │ shadow 4           │
│                    │   │                    │   │ ▎ RECOMMENDED 11px │
└────────────────────┘   └────────────────────┘   └────────────────────┘
```

- The whole card is not a button: the **button inside it** is the action (rule 2 from the shell document).
- Hover moves the stroke and the shadow, never the layout.
- A tick is a 16px check glyph in `#0f6cbd` and a 14/20 line — the same construction as the Azure page.

---

6. The other pages inherit the same language

page	title	what sits under it
Projects	32/40	the same three-band rhythm: figures strip → list → tinted callout when there is nothing yet
Overview	32/40	figures strip → pipeline → feeds → cards
Documents	32/40	figures strip → areas + pages table
Ship (later)	32/40	four tabs, then the same card grid

One rule keeps them from drifting: **a page has at most one tinted callout, and its title is one size (32/40) unless it is the front page (40/52).**

---

7. Translation, in the design

Because the translator exists, the design has to say where it appears — background and content both:

TEXT

```
Documentation                                  ← 40/52
 ┌──────────────┐
 │ EN ▾ · 11 languages │   ← a chip row under the title, 12px, on the wash
 └──────────────┘
   the page in front is English; the chip says so and opens "Translate this page…"

a translated page (guide/wiring.de.md)
┌──────────────────────────────────────────────────────────────────────┐
│  ▣ German · written from temco-platform@3f2a1c       12px #616161  │
└──────────────────────────────────────────────────────────────────────┘
    #f5f5f5 bar, 8px, inside the page's own top — it is not a callout
```

Translate is a **chip and one bar**, never a banner: a banner would make every page look like it is asking a question.

---

8. The content (the actual words)

where	words
breadcrumb	`Temco Platform › Documentation`
H1	**Documentation**
intro	Write the pages of a product, draft them from the code that runs it, and publish a site a customer can open.
field	`Find a page, a file or a command` · `Ctrl P`
primary	**Create a project**
card 1	**Start a project** — A name, a version, and the sources it reads. · ticks: pages, sources, versions · `New project`
card 2	**Draft from code** — Tick the files, write one sentence, and the draft arrives with its provenance. · ticks: file + commit, 12,000 characters, nothing written until accepted · `Draft a page`
card 3	**Publish a site** — Build every artifact, then freeze the version for readers. · ticks: html · pdf · chm, one folder, GitHub Pages · `Open Ship`
numbers	148 pages · 1 project · 96 read · 2 sources
callout	**Drafting keeps its receipts** — Every drafted page records the repository, the files and the commit it was written from. · `Read a Source`
questions	What is a project? · Where do the pages live? · What happens when I publish? · Which languages can a page be translated into?
floating	`Ask AI`

9. What changes in code, when this is agreed

#	file	change
1	`extensions/temco-ui/media/page.css`	the layer is rewritten to these tokens: Hero900/Base600/Base500/Base400/Base300, spacing 2–48, radius 2/4/8/12, shadow 4/8, brand `#0f6cbd` + tint `#ebf3fc`, and **one** sanctioned wash
2	`extensions/temco-ui/src/components.ts`	`hero()` gains the wash + breadcrumb; `optionCards()` replaces `cards()` on the front page; `callout()` replaces `banner()`; `questions()`; `chipRow()`; `numbers()`
3	`extensions/temco-workbench/src/helpPages.ts`	Welcome rebuilt to §3 and §8
4	`extensions/temco-docs/src/docsPages.ts`, `dashboard.ts`	title 32/40, figures strip, one callout where it earns it
5	`check-components.mts`	the gradient ban becomes "exactly one sanctioned wash, on `.az-wash`, brand tint → transparent"; the rest of the checks stay
6	`docs/temco-platform/ui/shell-v2.md` §10	replaced by this document once approved

Risk: low — it is CSS plus one page's markup. Everything else on the page (`kpi`, `section`, feeds, tables) keeps its current markup and only changes clothes.

---

10. What I need confirmed

#	decision	my recommendation	the alternative
1	**The three option cards** on the front page	yes — it is the Azure page's whole idea	keep the panel-with-code from v2
2	**Title size**	40/52 on the front page, 32/40 elsewhere	32/40 everywhere
3	**The wash** behind the title	yes, 160px, `#ebf3fc → transparent`	no wash, flat page
4	**Recommended card**	the middle one, tinted	none recommended
5	**Questions** at the bottom	yes, four rows	omit
6	**Floating Ask AI**	yes	omit

Say which of the six to change; everything else is as drawn. Then I implement it in one pass and show the render.